

Привет! Ниже представлены максимально подробные, развернутые ответы на все твои вопросы по Java. Текст составлен академично, емко и глубоко, с разбором внутренних механизмов работы платформы, чтобы у тебя сформировалось целостное и фундаментальное понимание.

Java Fundamentals

1. JVM-JDK-JRE. Что это такое? Кто кого включает и как взаимодействуют.

Для понимания экосистемы Java необходимо четко разделять три базовых компонента платформы: **JVM (Java Virtual Machine)**, **JRE (Java Runtime Environment)** и **JDK (Java Development Kit)**. Они представляют собой иерархическую матрешку, где каждый последующий уровень расширяет возможности предыдущего.

- **JVM (Виртуальная машина Java):** Это ядро платформы. JVM представляет собой виртуальный компьютер со своим собственным процессором, оперативной памятью и системой команд (байт-кодом). Основная задача JVM — интерпретировать или компилировать байт-код (файлы с расширением `.class`) в инструкции конкретного процессора операционной системы, на которой она запущена. Именно JVM обеспечивает выполнение знаменитого принципа Java: «*Write Once, Run Anywhere*» (Напиши один раз, запускай везде). JVM абстрагирует код от особенностей конкретного «железа» и ОС. Помимо выполнения кода, JVM отвечает за автоматическое управление памятью (включая `Garbage Collection`), верификацию байт-кода на безопасность и обработку системных ошибок. JVM поставляется в готовом (скомпилированном) виде под каждую конкретную операционную систему (`Windows`, `Linux`, `macOS`).
- **JRE (Среда времени выполнения Java):** Это минимально необходимый набор ПО для запуска уже скомпилированных Java-приложений обычными пользователями. JRE включает в себя саму виртуальную машину **JVM**, а также набор базовых библиотек классов Java (`Java Class Library`, таких как `java.lang`, `java.util`, `java.io`), которые необходимы для работы программы. JRE не содержит инструментов для разработки: в ней нет компилятора, отладчика или профилировщика. Если пользователю нужно просто запустить Java-программу или игру, ему достаточно иметь JRE.
- **JDK (Комплект разработчика Java):** Это полноценное рабочее окружение для программиста. JDK включает в себя **JRE** (а значит, и JVM с библиотеками), а также весь инструментарий разработчика: компилятор (`javac`), архиватор (`jar`), утилиту для генерации документации (`javadoc`), отладчик (`jdb`) и другие консольные утилиты для мониторинга производительности и диагностики. Разработчики всегда устанавливают именно JDK.

Взаимодействие компонентов:

Процесс начинается с того, что разработчик пишет исходный код в текстовом файле с расширением `.java`. С помощью компилятора `javac`, входящего в состав **JDK**, этот код преобразуется в промежуточный байт-код и сохраняется в файл `.class`. Затем, когда

приложение запускается на исполнение, в дело вступает **JRE**: она находит скомпилированные файлы и подгружает необходимые стандартные библиотеки классов. Внутри JRE запускается **JVM**, которая берет этот байт-код, проверяет его безопасность через подсистему Bytecode Verifier, переводит в машинные инструкции целевой ОС на лету и выполняет.

2. Как скомпилировать и запустить класс, используя консоль?

Процесс компиляции и запуска Java-кода вручную через терминал состоит из двух базовых шагов с использованием утилит из JDK: `javac` и `java`. Однако сценарий усложняется, если код разбит по пакетам.

Сценарий 1: Простой класс без пакета

Предположим, у нас есть файл `Main.java` в текущей директории со следующим содержимым:

```
public class Main {  
    public static void main          {  
        System.out.println("Hello from Console!");  
    }  
}
```

1. **Компиляция:** Открываем консоль в папке с файлом и выполняем команду:
`javac Main.java`
Если в коде нет синтаксических ошибок, компилятор выполнит свою работу молча. В этой же директории появится файл `Main.class`, содержащий байт-код.
2. **Запуск:** Для запуска мы вызываем утилиту выполнения и передаем ей *имя класса*, а не имя файла:
`java Main`
Обрати внимание, что расширение `.class` указывать не нужно, иначе JVM попытается искать класс с именем `Main.class`, что приведет к ошибке `ClassNotFoundException`.

Сценарий 2: Класс находится внутри пакета (Package)

Если код организован правильно, наверху файла будет объявлен пакет, например: `package com.example;`. Файловая структура должна строго соответствовать имени пакета, то есть файл должен лежать в папке `com/example/Main.java`.

1. **Компиляция с сохранением структуры:** Рекомендуется компилировать класс, находясь в корневом каталоге проекта (в папке, где лежит папка `com`), используя флаг `-d` (`destination`), который указывает, куда складывать скомпилированные файлы и заставляет компилятор автоматически создавать структуру папок для пакетов:

```
javac -d . com/example/Main.java
```

Точка . означает текущую директорию. Будет создана структура com/example/Main.class.

2. **Запуск из корня:** Запуск производится также из корневой папки, при этом указывается полное квалифицированное имя класса (Fully Qualified Name):

```
java com.example.Main
```

Примечание: Начиная с Java 11, для простых программ, состоящих из одного файла, можно выполнять запуск напрямую без явной предварительной компиляции одной командой: `java Main.java`. В этом случае JVM скомпилирует код в оперативную память и сразу выполнит его, не создавая физический .class файл на диске.

3. JIT-компилятор.

JIT (Just-In-Time) компилятор — это важнейший компонент JVM, который превращает Java из чисто интерпретируемого (и из-за этого медленного) языка в высокопроизводительную платформу, сопоставимую по скорости выполнения с C и C++. Изначально, при старте приложения, JVM работает в режиме **интерпретации**.

Интерпретатор берет каждую строку байт-кода по очереди, переводит ее в машинный код процессора и тут же выполняет. Это позволяет программе запуститься мгновенно без долгих предварительных задержек, но сама скорость выполнения такого кода невысока, так как перевод происходит прямо во время работы, многократно для одних и тех же инструкций.

Здесь на сцену выходит JIT-компилятор, работающий «на лету» (Just-In-Time — как раз вовремя).

1. **Профилирование кода:** Пока программа работает в режиме интерпретации, JVM непрерывно собирает статистику (проводит профилирование). Она ищет так называемые **«горячие точки» (Hot Spots)** — участки кода, которые выполняются чаще всего. Обычно это циклы, часто вызываемые методы или цепочки валидации.
2. **Компиляция в нативный код:** Как только счетчик вызовов определенного метода превышает установленный порог, JIT-компилятор подхватывает этот метод, компилирует весь его байт-код целиком непосредственно в оптимизированный машинный код целевого процессора и сохраняет этот результат в специальной области памяти (Code Cache).
3. **Подмена выполнения:** При следующем вызове этого метода JVM больше не использует медленный интерпретатор. Она сразу передает управление аппаратному процессору для выполнения нативного машинного кода. Это ускоряет работу данного участка кода в десятки раз.

В современных продвинутых JVM (таких как HotSpot) используется многоуровневая компиляция (Tiered Compilation):

- **C1 (Client Compiler):** Фокусируется на быстрой компиляции с базовыми оптимизациями, чтобы код как можно скорее стал нативным, уменьшая время прогрева приложения.
- **C2 (Server Compiler):** Подключается позже для самых «горячих» мест. Он тратит больше времени и ресурсов процессора на анализ, но применяет агрессивные

глобальные оптимизации: инлайнинг методов (встраивание мелких методов вместо их вызова), развертывание циклов (loop unrolling), исключение избыточных проверок на null и оптимизацию побега (Escape Analysis), которая позволяет выделять память под объекты на стеке вместо кучи, если объект не выходит за рамки метода.

4. Какие области памяти использует java для размещения простых типов, объектов, ссылок, констант, методов, пул строк и т.д.

Базовая внутренняя структура памяти JVM.

Память, которой управляет JVM, строго разделена на несколько логических областей в соответствии со спецификой хранящихся данных и особенностями доступа к ним.

Основное разделение идет на память, индивидуальную для каждого потока (Thread-safe/Thread-local), и общую разделяемую память (Shared memory).

1. Stack (Стек потока выполнения)

Стек создается для каждого потока индивидуально в момент его старта. Он работает по принципу LIFO (Last In, First Out). Каждому вызову метода внутри потока соответствует отдельная структура — **фрейм стека (Stack Frame)**.

- **Что там хранится:** Внутри фрейма хранятся **локальные переменные примитивных типов** (int, double, boolean и т.д.) и **значения ссылок на объекты** (адреса в памяти кучи, указывающие, где именно лежит реальный объект).
- **Особенности:** Стек работает невероятно быстро. Память выделяется и освобождается автоматически по мере входа в методы и выхода из них. Когда метод завершает работу, его фрейм полностью уничтожается, и все локальные примитивы стираются. Размер стека фиксирован (его можно настроить через параметр -Xss), и если глубина рекурсии или количество локальных переменных превысят лимит, возникнет ошибка StackOverflowError.

2. Heap (Куча)

Куча — это общая область памяти, доступная для всех потоков приложения. Она создается при старте JVM и динамически управляется Сборщиком Мусора (Garbage Collector).

- **Что там хранится:** Здесь размещаются абсолютно **все экземпляры классов (объекты)** и **все массивы** (даже массивы примитивов). Если вы пишете MyClass obj = new MyClass();, сам объект new MyClass() создается в куче, а переменная obj (ссылка на него) лежит в стеке.
- **Внутренняя структура кучи:** Исторически куча делится на поколения (Generational Heap) для оптимизации работы сборщика мусора, так как большинство объектов живут недолго:
 - **Young Generation (Молодое поколение):** Делится на зону **Eden** (сюда попадают все вновь созданные объекты) и две зоны выживания **Survivor (S0, S1)**. Здесь происходит частая и быстрая очистка (Minor GC).
 - **Old Generation (Старое/Тенуарное поколение):** Сюда перемещаются объекты,

которые пережили определенное количество циклов сборки мусора в Young Generation. Здесь хранятся долгоживущие объекты (например, синглтоны, конфигурации, сервисы). Очистка тут происходит реже, но она более тяжелая (Major/Full GC).

- **Пул строк (String Pool):** Начиная с Java 7, специальная область для хранения уникальных строковых литералов расположена именно **внутри Кучи (Heap)**. Это позволяет эффективно собирать мусором строки, которые больше никому не нужны.

3. Metaspaces (Метапространство)

Пришла на смену старой области Permanent Generation (PermGen) в Java 8. В отличие от PermGen, Metaspaces выделяется не из общего объема памяти JVM, а напрямую из системной оперативной памяти (Native Memory) компьютера, что снижает риск получить OutOfMemoryError.

- **Что там хранится:** Метаданные классов, структура методов (байт-код методов), таблицы виртуальных методов (vtable), **статические переменные** (обрати внимание: сами статические ссылки лежат в Metaspaces, но если они указывают на объекты, эти объекты все равно лежат в Heap), а также **пул констант (Runtime Constant Pool)** класса.

5. Пакеты в java. Зачем применяются? Принцип именования пакетов.

Пакеты (Packages) в Java — это механизм группировки связанных по смыслу классов, интерфейсов, перечислений и аннотаций в единое пространство имен. Физически на уровне операционной системы пакеты представляют собой обычную иерархию папок (директорий).

Зачем применяются пакеты?

1. **Разрешение конфликтов имен (Namespace Management):** В крупных проектах или при использовании сторонних библиотек легко создать два класса с одинаковым названием (например, два класса User или Connection). Разнесение их по разным пакетам (например, com.myproject.database.Connection и com.myproject.network.Connection) позволяет JVM однозначно понимать, какой именно класс требуется использовать.
2. **Управление доступом (Access Control):** Пакеты тесно связаны с инкапсуляцией. Модификатор доступа по умолчанию (package-private/default) и модификатор protected используют границы пакета как защитный барьер, скрывая внутреннюю логику пакета от внешнего мира и предоставляя доступ только «своим» классам из той же папки.
3. **Модульность и организация кода:** Пакеты позволяют структурировать проект логически. Разработчику гораздо проще ориентироваться в кодовой базе, когда классы разложены по полочкам: отдельно контроллеры, отдельно сервисы, отдельно сущности базы данных.

4. **Поиск классов (Class Searching):** JVM использует структуру пакетов для быстрого нахождения файлов байт-кода на диске или внутри JAR-архивов.

Принцип именования пакетов

Существует общепринятое соглашение (Java Code Conventions) по именованию пакетов, цель которого — обеспечить абсолютную уникальность имен во всем мире.

- **Реверсивное имя домена:** Имя пакета всегда начинается с доменного имени организации, развернутого задом наперед. Если сайт компании — `company.com`, то корневой пакет ее проектов будет начинаться с `com.company`. Для open-source проектов часто используются `org.apache`, `net.sf` и т.д.
- **Только строчные буквы:** Все элементы имени пакета должны быть написаны в нижнем регистре (lowercase). Использование заглавных букв запрещено, чтобы избежать проблем со спецификацией файловых систем различных ОС (например, Windows регистронезависима, а Linux регистрозависим).
- **Разделение точками:** Иерархия уровней разделяется точками. После домена идет название проекта, а затем функционального модуля:
`com.company.projectname.modulename.submodule`.
- **Запрещенные символы:** В именах пакетов нельзя использовать дефисы или спецсимволы. Если в названии компании есть дефис (например, `my-company.com`), его заменяют на подчеркивание: `com.my_company`. Если сегмент названия совпадает с ключевым словом Java (например, `new` или `switch`), к нему также принято добавлять подчеркивание (`com.company.project._new`).

6. Отличия между ссылочными и примитивными типами.

В Java все типы данных строго разделены на две фундаментальные категории: 8 примитивных типов и бесконечное множество ссылочных типов.

Критерий сравнения	Примитивные типы (int, boolean, char, double...)	Ссылочные типы (Классы, Интерфейсы, Массивы, String, Integer)
Что представляют	Чистые, атомарные значения данных. Не являются объектами.	Объекты, инкапсулирующие состояние (поля) и поведение (методы).
Место хранения	Всегда хранятся в Стеке (Stack) (если это локальные переменные внутри метода) либо внутри самого объекта в Heap (если это поле	Сам объект всегда создается в Куче (Heap) . В стеке или внутри другого объекта хранится лишь ссылка (адрес памяти).

	класса).	
Выделение памяти	Фиксированный, заранее известный размер памяти (например, int — 4 байта, byte — 1 байт, double — 8 байт).	Размер ссылки фиксирован (4 или 8 байт в зависимости от разрядности JVM), а размер самого объекта динамический и зависит от его полей.
Дефолтные значения	Никогда не бывают пустыми. При инициализации полей класса получают дефолтные значения: 0 для чисел, false для boolean, \u0000 для char.	По умолчанию имеют значение null, что означает «ссылка никуда не указывает».
Методы и операции	Не имеют никаких методов. К ним нельзя применить оператор . (вызов метода). Операции выполняются встроенными арифметическими/логическими знаками (+, -, ==, &&).	Имеют методы (как минимум унаследованные от Object). Сравнение через == сравнивает адреса памяти, а не логическое содержимое. Логическое сравнение требует метода equals().
Передача в метод	Передаются строго по значению (копируется само число). Изменение внутри метода никак не влияет на оригинал снаружи.	Передаются по значению ссылки (копируется адрес памяти). Метод может изменить внутреннее состояние объекта по этой ссылке, и изменения будут видны снаружи.
Производительность	Работают молниеносно, не требуют накладных расходов на создание объектов и очистку памяти сборщиком	Требуют выделения памяти в куче, разыменования ссылок, создают нагрузку на

	мусора.	Garbage Collector.
--	---------	--------------------

7. Модификаторы доступа. Что такое package level access. Пример использования.

Модификаторы доступа в Java — это ключевые слова, регулирующие уровень видимости классов, их конструкторов, методов и полей для других частей программы. Они обеспечивают важнейший принцип ООП — инкапсуляцию.

В Java существует 4 уровня доступа, задаваемые тремя ключевыми словами:

1. **public:** Элемент доступен абсолютно из любого места программы — из текущего класса, текущего пакета, пакетов-наследников и любых внешних пакетов.
2. **protected:** Элемент доступен внутри текущего класса, внутри текущего пакета, а также классам-наследникам (subclasses) в других, внешних пакетах.
3. **default (package-private / package level access):** Модификатор по умолчанию. Он применяется, если перед элементом **не указано** вообще никакого ключевого слова модификатора доступа.
4. **private:** Элемент доступен строго внутри того класса, где он объявлен. Ни наследники, ни другие классы пакета его не видят.

Что такое Package Level Access (Доступ на уровне пакета)?

Это и есть тот самый модификатор default (по умолчанию). Элемент, объявленный как package-private, виден и доступен **только для классов, находящихся в том же самом пакете (в той же папке)**. Для любых классов из внешних пакетов (даже если эти внешние классы являются наследниками данного класса) этот элемент полностью скрыт.

Пример использования Package Level Access

Представь, что мы пишем модуль для работы с базой данных внутри пакета com.company.db. Нам нужен вспомогательный класс, который будет парсить сырые строки ответов от СУБД, но мы не хотим, чтобы программисты, использующие нашу библиотеку из пакета com.company.ui, имели к нему доступ и ломали его логику.

```
package com.company.db;
```

```
// Класс имеет package-private доступ (нет слова public)
```

```
class DbResponseParser {
```

```
    // Метод также имеет package-private доступ
```

```

void parseRawData {
    System.out.println("Парсинг данных внутри пакета db");
}
}

```

Теперь создаем основной класс в этом же пакете, который будет доступен всем снаружи:

```

package com.company.db;

public class DatabaseService {
    public void executeQuery {
        // Успешно создаем и используем парсер, так как мы в одном пакете
        DbResponseParser parser = new DbResponseParser();
        parser.parseRawData("{id: 1}");
    }
}

```

Если кто-то в пакете `com.company.ui` попытается написать `DbResponseParser parser = new DbResponseParser();`, компилятор выдаст ошибку: «*DbResponseParser is not public in com.company.db; cannot be accessed from outside package*». Это позволяет делать архитектуру модульной и защищенной от нецелевого использования.

8. Почему метод `main()` объявлен как `public static void`? Как `main` выглядит в Java 25?

Классическая сигнатура главной точки входа в Java-приложение выглядит строго следующим образом:

```

public static void main

```

Каждое слово в этом объявлении имеет фундаментальное значение для работы JVM:

- **public:** JVM должна иметь возможность вызвать этот метод из внешней среды при

старте программы. Если сделать метод `private` или `protected`, подсистема запуска JVM не сможет получить к нему доступ из-за правил инкапсуляции, и запуск завершится ошибкой.

- **static:** Статический метод принадлежит классу, а не его объекту. На момент старта приложения в памяти еще не создано ни одного экземпляра вашего класса. Если бы метод `main()` был нестатическим, JVM пришлось бы сначала создать объект этого класса с помощью конструктора. Но у класса может не быть конструктора по умолчанию, или конструктор может требовать сложных параметров, о которых JVM ничего не знает. Объявление метода как `static` позволяет JVM вызвать его напрямую, просто загрузив класс в память.
- **void:** Метод ничего не возвращает. После завершения выполнения метода `main()` Java-программа (точнее, её главный поток) завершает свою работу. Код возврата операционной системе (успешно/ошибка) передается не через возвращаемое значение метода, а через системный вызов `System.exit(code)` при необходимости. Если программа завершилась штатно, JVM сама вернет ОС код 0.
- **String[] args:** Это массив строковых аргументов, которые пользователь может передать программе из консоли при запуске (например, `java Main arg1 arg2`). Даже если аргументы не переданы, массив создается пустым, но его наличие в сигнатуре обязательно для сопоставления метода с точкой входа.

Как `main()` может выглядеть в Java 25?

В рамках проекта **Amber** (начиная с превью-версий в Java 21 и постепенной стабилизации в последующих релизах, включая Java 25) в язык были внедрены **неявные классы и методы экземпляра `main` (Implicitly Declared Classes and Instance Main Methods)**. Эта фича создана для снижения порога входа новичков и избавления от шаблонного кода (boilerplate).

В современных версиях Java, включая Java 25, валидный файл `HelloWorld.java` может выглядеть просто вот так:

```
void main {  
    println("Hello, Modern Java!");  
}
```

Что изменилось под капотом:

1. **Больше нет обязательного `public static`:** JVM теперь умеет сама находить и запускать методы `main`, которые имеют уровень доступа `protected` или `package-private`, а также методы, которые являются *экземплярными*

(нестатическими). Если метод нестатический, JVM неявно сама создаст объект класса перед вызовом.

2. **Аргументы `String[] args` опциональны:** Если вашей программе не нужны параметры командной строки, вы можете просто опустить их в объявлении метода.
3. **Неявный класс (`Implicit Class`):** Вам даже не нужно писать `public class HelloWorld { ... }`. Компилятор Java автоматически создаст неявный класс верхнего уровня, имя которого будет соответствовать имени файла на диске.
4. **Автоматический импорт методов ввода-вывода:** Методы вроде `println()` доступны напрямую благодаря автоматическому статическому импорту из базовых утилит логирования/взаимодействия.

При этом классический вариант `public static void main(String[] args)` остается полностью рабочим и легитимным для обратной совместимости и больших промышленных систем.

9. Массив. Класс или примитивный тип?

В Java массив всегда является объектом ссылочного типа, независимо от того, какие элементы он в себе хранит — примитивы или другие объекты.

Доказательства и механизмы работы массивов:

1. **Выделение памяти в куче:** Переменная массива — это ссылка. Сам массив всегда создается с использованием ключевого слова `new` (явного или скрытого за синтаксическим сахаром инициализации вроде `int[] arr = {1, 2, 3};`) и размещается исключительно в **Куче (Heap)**.
2. **Наследование от `Object`:** Массивы наследуются напрямую от класса `java.lang.Object`. Это означает, что у любого массива можно вызвать методы `toString()`, `hashCode()`, `getClass()` или `equals()`. Правда, метод `toString()` у массивов не переопределен красиво и возвращает внутренний маркер типа и хэш-код (например, `[I@2a139a55` для массива `int[]`).
3. **Динамическое создание классов JVM:** Классы для массивов не имеют физического исходного кода `.java`. Они создаются динамически самой виртуальной машиной JVM прямо во время выполнения программы. Имена этих скрытых классов формируются по особому шаблону: `[` означает массив, а последующая буква указывает на тип элементов. Например, класс для массива целых чисел `int[]` называется `[I`, а для массива строк `String[]` — `[Ljava.lang.String;`.
4. **Наличие поля и специфика:** У массива есть встроенное финальное поле `length`, возвращающее количество его элементов. Обрати внимание: это именно поле, а не метод (пишется `arr.length`, а не `arr.length()`), что является уникальной встроенной особенностью синтаксиса массивов на уровне байт-кода.

Контрпример для закрепления: если мы создаем массив примитивов `int[] arr = new int[5];`, то элементы внутри массива хранятся компактно как примитивные 4-байтовые числа, но сам контейнер `arr` — это полноценный объект в куче, ссылка на который лежит в стеке.

10. Классы-оболочки. `Integer cache` (пул чисел). Отличия от примитивных типов.

Для каждого из 8 примитивных типов данных в Java существует соответствующий ему ссылочный тип — **класс-оболочка (Wrapper Class)**:

- byte $\rightarrow$ Byte
- short $\rightarrow$ Short
- int $\rightarrow$ Integer
- long $\rightarrow$ Long
- char $\rightarrow$ Character
- float $\rightarrow$ Float
- double $\rightarrow$ Double
- boolean $\rightarrow$ Boolean

Все эти классы являются **immutable (неизменяемыми)** — после создания объекта его значение поменять нельзя. Они также объявлены как `final`, от них нельзя унаследоваться.

Главные отличия классов-оболочек от примитивов:

1. **Возможность хранения null:** Переменная типа Integer может хранить значение null. Примитив int обязан всегда хранить какое-то число. Это критически важно при работе с базами данных или сетевыми запросами, где отсутствие значения (NULL в SQL) — это штатная ситуация.
2. **Использование в Generics (обобщениях):** Коллекции Java (например, ArrayList, HashMap) умеют работать исключительно с объектами. Написать ArrayList<int> нельзя, компилятор выдаст ошибку. Приходится использовать оболочки: ArrayList<Integer>.
3. **Накладные расходы по памяти:** Примитив int занимает ровно 4 байта памяти в стеке. Объект Integer в куче занимает значительно больше: 8-12 байт на заголовок объекта (object header), 4 байта на само значение типа int, плюс байты выравнивания (padding) до кратности 8 байтам. В итоге один объект Integer на 64-битной JVM обычно занимает 16 или 24 байта, что в несколько раз больше примитива.

Integer Cache (Пул чисел)

Для оптимизации потребления памяти и повышения быстродействия в классе Integer реализован механизм кэширования объектов.

Когда мы создаем объект Integer через метод Integer.valueOf() (который вызывается неявно при автоупаковке), Java не создает каждый раз новый объект в куче, а сначала проверяет, не находится ли число в диапазоне кэша. По умолчанию этот диапазон составляет **от -128 до 127** включительно.

Пример поведения:

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a == b); // Выведет true, так как оба ссылаются на один объект из кэша
```

```
Integer c = 200;  
Integer d = 200;  
System.out.println(c == d); // Выведет false, так как 200 вне диапазона кэша, созданы два  
разных объекта в куче
```

Нижняя граница кэша всегда фиксирована (-128), а вот верхнюю границу (127) можно увеличить при старте JVM с помощью специального флага конфигурации:

-XX:AutoBoxCacheMax=<size>.

Подобные пулы кэширования также реализованы для Byte, Short, Long (все от -128 до 127), Character (от 0 до 127) и Boolean (кэширует два готовых статических объекта TRUE и FALSE). В классах Float и Double кэширование отсутствует, так как вещественных чисел даже в малом диапазоне бесконечно много.

11. Autoboxing и unboxing. Принцип действия на примерах.

Autoboxing (Автоупаковка) и **Unboxing (Автораспаковка)** — это синтаксический сахар, внедренный в Java 5, который автоматически преобразует примитивные типы в их соответствующие классы-оболочки и наоборот. Этот механизм избавляет разработчика от необходимости писать громоздкий код явных конвертаций вручную.

Autoboxing (Автоупаковка)

Это процесс автоматического оборачивания примитивного типа в объект класса-оболочки, когда объект ожидается, а передается примитив.

Пример в коде:

```
Integer number = 10; // Автоупаковка примитива int в объект Integer  
List<Integer> list = new ArrayList<>();  
list.add(5); // Автоупаковка int 5 в Integer для сохранения в коллекцию
```

Что делает компилятор под капотом: На этапе компиляции Java заменяет этот синтаксический сахар на вызов статического фабричного метода `Integer.valueOf()`:

```
Integer number = Integer.valueOf(10);  
list.add(Integer.valueOf(5));
```

Благодаря использованию `Integer.valueOf()`, автоупаковка автоматически задействует Integer Cache, описанный в предыдущем вопросе.

Unboxing (Автораспаковка)

Это обратный процесс: автоматическое извлечение примитивного значения из объекта класса-оболочки, когда выражение требует примитивного типа данных.

Пример в коде:

```
Integer wrapper = Integer.valueOf(50);  
int primitive = wrapper; // Автораспаковка объекта в примитив int  
  
int sum = wrapper + 20; // Распаковка wrapper для выполнения арифметической операции
```

Что делает компилятор под капотом: Компилятор преобразует этот код, вставляя вызов метода экземпляра класса оболочки, в данном случае — `intValue()` (для других типов соответственно `doubleValue()`, `booleanValue()` и т.д.):

```
int primitive = wrapper.intValue();  
int sum = wrapper.intValue() + 20;
```

Главная опасность: `NullPointerException`

Поскольку объект-оболочка может ссылаться на null, попытка выполнить автораспаковку для пустой ссылки неизбежно приведет к критической ошибке во время работы программы.

Пример ошибки:

```
Integer brokenWrapper = null;  
int val = brokenWrapper; // Кажется, что компилируется нормально, но...
```

Под капотом компилятор превратит это в `int val = brokenWrapper.intValue();`. Попытка вызвать метод `intValue()` у ссылки, равной null, выбросит `NullPointerException`. Об этом нужно помнить при работе с коллекциями и условными операторами.

12. Нововведения Java 8.

Вышедшая в 2014 году версия Java 8 стала самым революционным и масштабным обновлением за всю историю языка. Она полностью изменила парадигму написания кода, добавив элементы функционального программирования в классический объектно-ориентированный язык.

Основные ключевые нововведения:

1. **Лямбда-выражения (Lambda Expressions):** Позволили передавать блоки кода (функции) как аргументы в методы. Лямбды представляют собой анонимные функции с лаконичным синтаксисом (параметры) `-> { тело_метода }`. Они помогли радикально сократить объем шаблонного кода, заменив громоздкие анонимные классы.
2. **Функциональные интерфейсы (Functional Interfaces):** Это интерфейсы, содержащие ровно **один абстрактный метод**. Лямбда-выражения могут быть приведены только к типу функционального интерфейса. Для поддержки этой концепции была добавлена аннотация `@FunctionalInterface` и целый пакет стандартных интерфейсов в `java.util.function` (`Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `Function<T, R>` и др.).
3. **Stream API (java.util.stream):** Мощный инструмент для декларативной обработки коллекций данных в функциональном стиле. Позволяет строить конвейеры (pipelines) обработки данных, состоящие из промежуточных операций (`filter`, `map`, `sorted`) и терминальных операций (`collect`, `forEach`, `reduce`). Stream API также поддерживает легкое распараллеливание вычислений через `parallelStream()`.
4. **Дефолтные (default) и статические (static) методы в интерфейсах:** Сняли жесткое ограничение, что интерфейсы могут содержать только абстрактные методы. Дефолтные методы содержат реализацию по умолчанию, что позволило

расширять старые интерфейсы (например, добавить метод `forEach` в `Iterable`) без нарушения обратной совместимости с миллионами написанных ранее классов-реализаций.

5. **Класс `Optional<T>`:** Специальный контейнер, который может содержать или не содержать внутри себя значение отличной от `null` ссылки. Создан как альтернатива возврату `null`, призванная на уровне дизайна API заставить разработчика обработать потенциальное отсутствие значения и снизить вероятность возникновения `NullPointerException`.
6. **Новое `Date/Time` API (`java.time`):** Полностью заменяет старые, потокобезопасные и крайне неудобные классы `Date` и `Calendar`. Новые классы (`LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Duration`) созданы по принципу неизменяемости (`immutable`) и потокобезопасны.
7. **Ссылки на методы (`Method References`):** Способ компактно передать существующий метод класса в качестве лямбда-выражения с помощью синтаксиса с двойным двоеточием, например `String::toUpperCase` или `System.out::println`.

Java Strings

13. В чем разница между созданием строки как `new String()` и литералом (при помощи двойных кавычек)? Метод `intern()`.

Строки в Java — очень специфичный тип данных, для которого на уровне JVM реализованы жесткие механизмы оптимизации памяти. Разница в способе создания строки определяет, где именно в памяти окажется итоговый объект.

Вариант 1: Использование строкового литерала (двойные кавычки)

```
String s1 = "Hello";  
String s2 = "Hello";
```

При таком объявлении JVM проверяет специальную внутреннюю область памяти в куче — **Пул строк (`String Pool`)**.

- Если строки "Hello" еще нет в пуле, JVM создает объект строки в пуле и возвращает ссылку на него переменной `s1`.
- Когда компилятор доходит до создания `s2`, JVM видит, что точно такая же строка "Hello" уже присутствует в пуле. Вместо создания нового объекта, она просто возвращает переменной `s2` адрес уже существующей строки.
- В итоге `s1 == s2` вернет `true`, потому что обе переменные указывают на один и тот же физический адрес в памяти.

Вариант 2: Использование оператора `new String()`

```
String s3 = new String("Hello");
```

```
String s4 = new String("Hello");
```

Оператор `new` всегда принудительно заставляет JVM выделить новую область памяти в стандартной **Куче (Heap)** (вне String Pool) и создать там новый уникальный экземпляр класса String.

- При выполнении этого кода будут созданы два абсолютно независимых друг от друга объекта в куче.
- В итоге `s3 == s4` вернет `false`, так как они ссылаются на разные адреса памяти, несмотря на то, что текстовое содержимое у них полностью идентично.

Метод `intern()`

Метод `intern()` позволяет вручную управлять перемещением строк в Пул строк во время работы программы.

Когда у объекта строки вызывается метод `s.intern()`, JVM делает следующее:

1. Ищет в String Pool строку с точно таким же текстовым содержимым, как у исходного объекта.
2. Если такая строка найдена в пуле, метод возвращает ссылку на этот уже существующий пул-объект.
3. Если такой строки в пуле нет, исходное текстовое содержимое добавляется в Пул строк, и возвращается ссылка на вновь созданную пул-строку.

Пример:

```
String s3 = new String("Hello");
String s5 = s3.intern(); // s5 теперь указывает на строку внутри String Pool

String s1 = "Hello";
System.out.println(s1 == s5); // Выведет true! Они указывают на один объект в пуле.
```

14. Как реализуется класс String, какие поля там есть?

Класс `java.lang.String` в Java является **неизменяемым (Immutable)** и объявлен как `final`, что гарантирует, что ни один сторонний класс не сможет унаследоваться от него и нарушить внутреннюю логику безопасности работы со строками.

Внутренняя структура полей класса String

Внутреннее устройство класса претерпело серьезные изменения в истории Java. Долгое время (до Java 8 включительно) строки хранили символы внутри массива типа `char[]`. Однако символ `char` в Java занимает 2 байта (UTF-16), и для хранения обычного английского текста или цифр половина памяти тратилась впустую (старший байт всегда был равен нулю).

Начиная с **Java 9** и далее (включая все современные версии), в рамках проекта **Compact Strings**, внутренняя структура класса была оптимизирована. Сегодня основные приватные

поля класса String выглядят так:

1. **private final byte[] value;** — Это массив байт, где физически хранится текстовое содержимое строки. Переход с char[] на byte[] позволил экономить до 50% памяти для строк, которые содержат только однобайтовые символы (латиница, цифры).
2. **private final byte coder;** — Поле-маркер кодировки строки. Оно принимает два возможных значения:
 - LATIN1 (значение 0): если строка состоит целиком из символов, укладываемых в кодировку ISO-8859-1 (латиница, базовые знаки). Каждый символ занимает ровно **1 байт** в массиве value.
 - UTF16 (значение 1): если в строке появляется хотя бы один символ, требующий больше 1 байта (например, кириллица, иероглифы, эмодзи). В этом случае каждый символ кодируется старыми двумя байтами, и размер массива value удваивается.
3. **private int hash;** — Поле для кэширования хэш-кода строки. По умолчанию оно равно 0. Хэш-код строки вычисляется лениво — только при первом вызове метода hashCode(). После вычисления результат записывается в это поле. Поскольку строка неизменяема (массив value финальный и закрыт снаружи), хэш-код никогда больше не изменится. Это обеспечивает огромную скорость работы строк в качестве ключей в HashMap.

Все методы изменения строк (concat(), substring(), toLowerCase()) на самом деле не меняют массив value текущего объекта, а внутри себя создают и возвращают абсолютно новый объект String с новым массивом байт.

15. Понятие Unicode. ASCII-символы. UTF-8, описание кодировки. Отличие от UTF-16.

Для понимания текстовых данных необходимо разделять абстрактные стандарты символов и их физическое представление в памяти компьютера.

- **ASCII (American Standard Code for Information Interchange):** Старинная 7-битная кодировка, созданная в США. Она способна закодировать всего 128 символов: английский алфавит в обоих регистрах, цифры, базовые знаки препинания и управляющие символы. Каждый символ занимает ровно 1 байт (8-й бит оставался неиспользованным). В ней абсолютно нет места для других национальных алфавитов.
- **Unicode (Юникод):** Это не кодировка, а **международный стандарт (таблица)**, который присваивает абсолютно каждому существующему в мире символу (буквам всех языков, математическим знакам, историческим иероглифам, эмодзи) уникальный числовой идентификатор — **кодovou точку (Code Point)**. Диапазон кодовых точек огромен — от 0 до 0x10FFFF (более 1.1 миллиона символов). Юникод решил проблему несовместимости шрифтов и кодировок, но сам по себе он не говорит, как эти числа хранить в памяти компьютера в байтах. Для этого существуют кодировки семейства UTF.

UTF-8 (Unicode Transformation Format)

Это кодировка с **переменной длиной символа (Variable-width encoding)**. Один символ Юникода может занимать в UTF-8 от 1 до 4 байт.

- **Принцип работы:** Символы стандартной таблицы ASCII кодируются в UTF-8 ровно **1 байтом**, при этом сам байт полностью идентичен ASCII. Это обеспечивает идеальную обратную совместимость. Европейские алфавиты, кириллица, арабские буквы кодируются **2 байтами**. Иероглифы и сложные азиатские языки — **3 байтами**. Эмодзи, редкие исторические символы — **4 байтами**.
- **Плюсы:** Невероятно экономна для западных текстов, где преобладает латиница и цифры. Является стандартом де-факто для интернета.

Отличие UTF-8 от UTF-16

- **UTF-16** — это также кодировка с переменной длиной, но ее базовый шаг — **2 байта**. Символы кодируются либо **2 байтами (16 бит)**, либо **4 байтами (32 бита)**.
- **Расход памяти:** Если текст написан на английском, UTF-8 потратит по 1 байту на символ, а UTF-16 — по 2 байта (в два раза больше памяти). Если текст написан на китайском, UTF-8 потратит 3 байта на иероглиф, а UTF-16 — 2 байта (UTF-16 будет выгоднее).
- **Внутреннее использование в Java:** Исторически среда выполнения Java использует UTF-16 как внутреннее представление символов в памяти для манипуляций (типы char основаны на 16 битах). Однако, как упомянуто в ответе про структуру String, современные версии Java оптимизируют хранение латиницы до уровня 1 байта (фактически приближаясь по эффективности к UTF-8/Latin1), переключаясь на полноценный UTF-16 только при появлении сложных символов (например, кириллицы).

16. String, StringBuilder, StringBuffer. Отличия. Почему эти классы объявлены как final?

Все три класса предназначены для работы с текстовыми строками, но они имеют кардинальные различия в плане модифицируемости и безопасности при многопоточном использовании.

1. **String:** Неизменяемый (**Immutable**) класс. Любая операция модификации текста порождает новый объект в памяти.
2. **StringBuilder:** Изменяемый (**Mutable**) класс. Внутри него находится расширяемый массив байт. При операциях вроде append() или insert() текст дописывается прямо в существующий массив без создания промежуточных объектов-пустышек. Данный класс **не является потокобезопасным**. Его методы не синхронизированы, поэтому он работает максимально быстро. Рекомендуется использовать для любых манипуляций со строками внутри одного потока (например, сборка SQL-запроса или парсинг строк в цикле).
3. **StringBuffer:** Полный изменяемый функциональный аналог StringBuilder, но **потокобезопасный (Thread-safe)**. Практически все его методы помечены ключевым словом synchronized. Это означает, что несколько потоков могут одновременно и

безопасно модифицировать одну строку. Из-за постоянных накладных расходов на блокировки потоков (locks), он работает значительно медленнее, чем `StringBuilder`. В современном коде используется редко, так как задачи конкурентной сборки одной строки силами разных потоков встречаются нечасто.

Почему эти классы объявлены как `final`?

Запрет наследования для классов `String`, `StringBuilder` и `StringBuffer` продиктован тремя критически важными причинами:

- **Безопасность (Security):** Строки повсеместно используются в качестве параметров при открытии сетевых соединений, подключении к базам данных (логины, пароли, URL), открытии файлов на диске. Если бы класс `String` не был `final`, злоумышленник мог бы создать свой собственный класс-наследник `MyEvilString`, переопределить в нем логику (например, метод `equals()` или внутреннее чтение массива) и незаметно подменить данные авторизации или пути к файлам в системных методах Java.
- **Потокобезопасность неизменяемости:** Неизменяемость класса `String` гарантирует, что объект можно безопасно передавать между потоками без всякой синхронизации. Если бы от строки можно было унаследоваться, дочерний класс мог бы нарушить инвариант неизменяемости, добавив изменяемые поля состояния.
- **Оптимизация производительности и Пул строк:** JVM рассчитывает на то, что строка с определенным содержимым никогда не изменится. Это позволяет кэшировать хэш-код и использовать Пул строк. Если бы существовали изменяемые подклассы `String`, целостность Пула строк была бы полностью разрушена. Наличие `final` также позволяет JIT-компилятору применять агрессивные оптимизации (например, замену вызовов методов прямым обращением к данным — виртуализация методов отключается).

Java Classes

17. Базовый класс в java. Методы класса.

В Java используется жесткая концепция единого объектного корня. Абсолютно любой класс (включая массивы и пользовательские классы), явно или неявно, унаследован от корневого класса `java.lang.Object`. Если вы пишете `class MyClass {}`, компилятор автоматически превращает это в `class MyClass extends Object {}`.

Класс `Object` содержит 11 методов, которые определяют базовое фундаментальное поведение любого Java-объекта:

1. **`public final Class<?> getClass()`** — Возвращает метаинформацию о классе объекта во время выполнения (объект типа `Class`). Используется в рефлексии (`Reflection`). Не может быть переопределен.
2. **`public int hashCode()`** — Возвращает целочисленное представление (хэш-код) объекта. Используется хэш-таблицами (такими как `HashMap`). По умолчанию часто возвращает значение, основанное на внутреннем адресе объекта в памяти.
3. **`public boolean equals(Object obj)`** — Сравнивает два объекта на логическое равенство. По умолчанию реализация проверяет строгое равенство ссылок (`this ==`

obj), то есть возвращает true, только если обе ссылки ведут на один и тот же объект в памяти.

4. **protected Object clone() throws CloneNotSupportedException** — Создает и возвращает побитовую поверхностную копию (шаблонный дубликат) текущего объекта. Требуется реализации интерфейса Cloneable.
5. **public String toString()** — Возвращает строковое представление объекта. По умолчанию выдает строку вида: ИмяКласса@хэшкод_в_шестнадцатеричном_виде. Настоятельно рекомендуется переопределять во всех пользовательских классах ради удобства логирования.
6. **public final void wait(), public final void wait(long timeout), public final void wait(long timeout, int nanos)** — Семейство методов многопоточной синхронизации. Переводит текущий поток в состояние ожидания (блокировки) до тех пор, пока другой поток не вызовет методы оповещения для этого же объекта-монитора. Могут вызываться только из синхронизированного контекста (synchronized).
7. **public final void notify()** — Пробуждает один случайный поток, который ранее был переведен в состояние ожидания на мониторе данного объекта через метод wait().
8. **public final void notifyAll()** — Пробуждает абсолютно все потоки, ожидающие на мониторе данного объекта.
9. **protected void finalize() throws Throwable** — Старинный метод очистки ресурсов перед тем, как сборщик мусора полностью удалит объект из памяти. **Категорически устарел (Deprecated)** начиная с Java 9 и полностью удален/не рекомендуется к использованию в современных версиях из-за непредсказуемости времени вызова и падения производительности. Вместо него используют интерфейс AutoCloseable и блоки try-with-resources.

18. Принципы ООП. Abstraction.

Объектно-ориентированное программирование базируется на четырех фундаментальных столпах: **Энкапсуляция** (скрытие внутреннего устройства за публичным интерфейсом), **Наследование** (передача свойств от родителя к потомку), **Полиморфизм** (способность кода работать с объектами разных типов единообразно, если они имеют общий предок) и **Абстракция**.

Разбор принципа Abstraction (Абстракция)

Абстракция — это выделение главных, наиболее значимых характеристик и сущностей реального объекта или процесса, с одновременным игнорированием и отбрасыванием всех несущественных, второстепенных деталей.

В программировании абстракция помогает бороться со сложностью систем. Мы создаем ментальную модель объекта, содержащую только то, что важно для решения конкретной задачи в нашей программе.

Пример концепции: Представь, что мы моделируем объект «Автомобиль».

- Если мы пишем систему для автошколы или компьютерной игры-симулятора, нам важны такие характеристики автомобиля, как текущаяСкорость, уголПоворотаРуля, передача, а также методы нажатьГаз(), повернутьРуль().

- Если мы пишем систему учета для налоговой инспекции или автостоянки, нам абсолютно неважно, как машина поворачивает. Нам важны поля: госНомер, владелец, налоговаяМощность и годВыпуска.

Это и есть абстракция — мы смотрим на один и тот же реальный объект под разными углами, отсекая все лишнее для конкретного контекста.

Как Абстракция реализуется в Java?

В Java главными инструментами построения абстракций являются **Интерфейсы (interface)** и **Абстрактные классы (abstract class)**.

- **Интерфейсы** задают чистую абстракцию поведения (контракт). Они говорят: *«Неважно, КТО ты и КАК ты это делаешь, важно то, что ты УМЕЕШЬ делать вот эти действия»*. Например, интерфейс List — это абстракция списка. Нам, как пользователям, неважно, как он устроен внутри (как массив в ArrayList или как связный список в LinkedList), мы просто вызываем абстрактный метод .add() и доверяем контракту.
- **Абстрактные классы** создают абстракцию сущности, которая может содержать как абстрактное описание, так и частичную общую базовую реализацию для группы будущих подклассов.

19. Правила переопределения метода boolean equals(Object o).

Метод equals() проверяет объекты на логическое равенство (эквивалентность содержимого). При его переопределении разработчик обязан строго соблюдать математический контракт, описанный в документации класса Object. Нарушение этих правил приведет к непредсказуемому поведению стандартных коллекций Java.

Согласно спецификации, отношение эквивалентности должно быть:

1. **Рефлексивным (Reflexive):** Объект должен быть равен самому себе. Для любой непустой ссылки x, вызов x.equals(x) обязан всегда возвращать true.
2. **Симметричным (Symmetric):** Если объект А равен объекту Б, то и объект Б должен быть равен объекту А. Для любых непустых ссылок x и y, x.equals(y) должно возвращать true тогда и только тогда, когда y.equals(x) возвращает true.
3. **Транзитивным (Transitive):** Если А равно Б, а Б равно В, то А должно быть равно В. Для любых непустых ссылок x, y и z, если x.equals(y) возвращает true и y.equals(z) возвращает true, то вызов x.equals(z) обязан вернуть true.
4. **Согласованным / Постоянным (Consistent):** Повторные вызовы метода не должны менять результат, если поля объектов, используемые в проверке, не изменялись. Для любых непустых ссылок x и y множественные вызовы x.equals(y) должны стабильно возвращать либо только true, либо только false.
5. **Проверка на null:** Ни один объект не равен null. Для любой непустой ссылки x, вызов x.equals(null) обязан всегда возвращать false. Метод не должен выбрасывать NullPointerException.

Идеальный шаблон написания метода equals():

```

@Override
public boolean equals      {
    // 1. Проверяем равенство ссылок (быстрая оптимизация)
    if (this == o) return true;

    // 2. Проверяем на null и проверяем строгое совпадение классов
    if (o == null || getClass() != o.getClass()) return false;

    // 3. Приводим тип аргумента к нашему классу
    MyClass that = (MyClass) o;

    // 4. Поэлементно сравниваем все значимые поля класса
    // Для примитивов используем ==, для объектов - Objects.equals()
    return this.id == that.id &&
        Objects.equals(this.name, that.name);
}

```

Важное замечание: Использование проверки `getClass() != o.getClass()` гарантирует строгое соблюдение симметричности. Замена её на оператор `instanceof` часто ломает контракт симметричности и транзитивности, если в системе появляются подклассы-наследники, добавляющие новые поля.

20. Зачем переопределять методы `hashCode` и `equals` одновременно?

В Java существует незыблемый контракт между методами `equals()` и `hashCode()`. Он звучит так: **Если два объекта равны с точки зрения метода `equals()`, они обязаны возвращать абсолютно одинаковые значения хэш-кода через метод `hashCode()`.**

Обратное утверждение неверно: если хэш-коды одинаковы, объекты не обязательно равны (это называется *коллизией*).

Что произойдет, если переопределить `equals()`, но оставить дефолтный `hashCode()`?

Дефолтная реализация `hashCode()` в классе `Object` генерирует число, основанное на адресе объекта в памяти. Представим, что мы создали два объекта класса `User` с одинаковым `id`:

```
User user1 = new User(1, "Misha");  
User user2 = new User(1, "Misha");
```

Поскольку мы переопределили `equals()`, вызов `user1.equals(user2)` вернет `true`. Но так как `hashCode()` мы не трогали, они вернут абсолютно разные числа (например, 12345 и 67890).

Последствия на примере структуры `HashMap` / `HashSet`

Когда вы кладете объект в `HashMap` в качестве ключа (`map.put(user1, "Admin")`), карта делает следующее:

1. Вызывает `user1.hashCode()`, получает хэш (например, 12345), вычисляет на его основе индекс ячейки (бакета) в массиве и кладет туда значение.

Когда вы пытаетесь извлечь значение по точно такому же логическому ключу (`map.get(user2)`):

1. Карта вызывает `user2.hashCode()`, получает хэш 67890.
2. На основе этого хэша вычисляется совершенно другой индекс бакета. Карта идет в эту другую ячейку, видит, что там пусто, и возвращает вам `null`.

В итоге: объект в карте физически лежит, логически эквивалентный ключ у вас на руках есть, но найти и достать данные вы никогда не сможете. Коллекция полностью ломается. Именно поэтому одновременное и согласованное переопределение этих методов является строжайшим правилом разработки в Java.

21. Правила переопределения метода `int hashCode()`. Можно ли в качестве результата возвращать константу?

При переопределении метода `hashCode()` необходимо руководствоваться следующими официальными правилами контракта:

1. **Стабильность во времени:** Если поля объекта, участвующие в логике сравнения `equals()`, не менялись, то метод `hashCode()` при многократном вызове в рамках одного сеанса работы приложения обязан возвращать одно и то же целое число.
2. **Зависимость от `equals` (Равенство $\rightarrow$ Одинаковый хэш):** Если два объекта равны по результатам работы `equals(Object)`, их хэш-коды должны быть строго одинаковыми.
3. **Желательность уникальности при неравенстве:** Если два объекта не равны по `equals()`, они не обязаны возвращать разные хэш-коды. Однако разработчик должен стремиться к тому, чтобы для разных объектов генерировались разные хэши, так как это напрямую влияет на производительность хэш-таблиц.

Можно ли в качестве результата возвращать константу?

Да, с точки зрения синтаксиса и формального контракта языка, написать следующий код абсолютно **легально**:

```
@Override
public int hashCode {
    return 42; // Возвращаем константу всегда
}
```

Этот код **не нарушает** контракт равенства: если два объекта равны, они оба вернут 42 (правило №2 соблюдено). Код будет компилироваться и успешно работать.

Почему так делать категорически нельзя на практике?

Если ваш метод возвращает константу, вы полностью уничтожаете смысл использования хэш-таблиц (таких как HashMap или HashSet).

- Внутри HashMap все объекты будут получать один и тот же хэш-код 42, а значит, все они будут складываться в одну и ту же ячейку (бакет) памяти.
- Структура данных выродится из эффективной хэш-таблицы в обычный **связный список** (или сбалансированное дерево в современных версиях).
- Скорость поиска, вставки и удаления элементов упадет со стремительной константной сложности $O(1)$ до катастрофической линейной сложности $O(N)$.

При больших объемах данных приложение просто зависнет.

Для генерации качественного хэш-кода рекомендуется использовать утилиту `Objects.hash(поле1, поле2, ...)`.

22. Правила переопределения метода `clone()`.

Метод `clone()` в Java предоставляет механизм создания дубликата объекта. Однако историческая реализация этого механизма признана экспертами (включая Джошуа Блоха) неудачной и сложной в использовании из-за путаницы с интерфейсами и исключениями. Если вам необходимо переопределить метод `clone()`, вы обязаны следовать следующим правилам:

1. **Реализация интерфейса `Cloneable`:** Класс, объект которого мы хотим клонировать, обязан явно реализовывать маркерный интерфейс `java.lang.Cloneable`. Если этого не сделать, вызов метода `super.clone()` во время работы программы выбросит проверяемое исключение `CloneNotSupportedException`.
2. **Вызов `super.clone()`:** Внутри переопределенного метода необходимо обязательно вызывать `super.clone()`. Это гарантирует, что иерархия вызовов дойдет до базового метода класса `Object`, который умеет корректно выделять память под нужный тип

класса и копировать поля на низком уровне (побитово). Написание `new MyClass()` вместо вызова `super.clone()` является нарушением конвенции.

3. **Повышение видимости:** Метод `clone()` в классе `Object` объявлен с модификатором `protected`. При переопределении его видимость принято повышать до `public`, чтобы клонирование можно было вызывать свободно на объектах класса снаружи.
4. **Управление исключениями:** Метод в оригинале выбрасывает проверяемое исключение `CloneNotSupportedException`. В переопределенном методе рекомендуется обернуть этот вызов в блок `try-catch` или убрать `throws` из сигнатуры метода, чтобы пользователям вашего класса не приходилось постоянно обрабатывать это исключение.

Поверхностное (Shallow) против Глубокого (Deep) клонирования

- **Поверхностное клонирование:** Это то, что делает `super.clone()` по умолчанию. Он просто копирует значения полей. Если полями являются примитивы, они скопируются отлично. Но если полем является ссылка на другой изменяемый объект (например, `List` или класс `Address`), в клоне скопируется лишь *адрес* этой ссылки. В итоге оригинал и клон будут делить один внутренний объект на двоих. Изменение адреса в клоне изменит его и в оригинале.
- **Глубокое клонирование:** Если у класса есть ссылочные изменяемые поля, после вызова `super.clone()` разработчик обязан **вручную скопировать каждый внутренний объект** и явно переприсвоить новые ссылки в полученный объект-клон.

Из-за этих сложностей в современной разработке вместо метода `clone()` предпочитают использовать **конструкторы копирования** (например, `public MyClass(MyClass original)`) или методы фабричного копирования.

23. Чем отличаются `finally` и `finalize`? Для чего используется ключевое слово `final`?

Несмотря на схожесть в написании, эти три понятия имеют абсолютно разные роли в языке Java.

1. Блок `finally`

Это ключевое слово, используемое в конструкции обработки исключений `try-catch-finally`.

- **Назначение:** Блок `finally` гарантирует, что содержащийся в нем код будет выполнен в любом случае, независимо от того, произошло ли исключение в блоке `try` или нет, и было ли оно поймано в блоке `catch`.
- **Зачем нужен:** Применяется для гарантированного закрытия внешних ресурсов (файловые потоки, сетевые сокеты, соединения с БД), чтобы избежать утечек памяти.
- **Исключение из правил:** Единственный случай, когда `finally` не выполнится — если внутри блока `try/catch` работа JVM будет принудительно аварийно прервана вызовом команды `System.exit(0)` или произойдет критический крах ОС / отключение питания.

2. Метод finalize()

Это защищенный метод класса java.lang.Object (подробно разобран в вопросе 17).

- **Назначение:** Вызывается сборщиком мусора перед тем, как объект будет окончательно стерт из кучи.
- **Текущий статус:** Полностью объявлен устаревшим (**Deprecated**) и удаляется из платформы. Его использование считается плохой практикой из-за непредсказуемости момента вызова (GC запускается, когда захочет, ресурсы могут висеть открытыми часами).

3. Ключевое слово final

Это универсальный модификатор, который означает «**запрет на изменение/расширение**». Его поведение кардинально зависит от того, к какому элементу структуры кода он применен:

- **Для переменной:** Превращает переменную в константу. Ее значение можно присвоить только один раз (при объявлении или в конструкторе). Попытка изменить значение позже вызовет ошибку компиляции. Если переменная ссылочная (объект), то final запрещает переписывать саму *ссылку* на другой объект, но внутреннее состояние самого объекта по этой ссылке менять можно.
- **Для метода:** Запрещает переопределение (overriding) данного метода в классах-наследниках. Полиморфизм для этого метода отключается.
- **Для класса:** Запрещает наследование от этого класса. Создать класс-наследник (subclass) через ключевое слово extends становится невозможно. Все методы final-класса неявно становятся final. Примеры: String, классы-оболочки примитивов.

24. JavaBeans: основные требования к классам Bean-компонентов, соглашения об именах.

JavaBeans — это исторический, но до сих пор живой стандарт и концепция создания переиспользуемых, инкапсулированных программных компонентов (классов) в Java. По сути, это строгое соглашение о том, как должен быть оформлен класс данных, чтобы сторонние фреймворки (например, Spring, Hibernate или инструменты визуального проектирования UI) могли автоматически управлять этими объектами через рефлексию. Основные обязательные требования к классу-компоненту JavaBean:

1. **Наличие открытого конструктора без аргументов:** Класс обязан иметь явный или неявный public конструктор по умолчанию (без параметров). Фреймворки создают экземпляры таких классов динамически, вызывая именно Class.getDeclaredConstructor().newInstance(), и только потом наполняют его данными.
2. **Инкапсуляция полей (Приватность):** Все свойства (поля) класса обязаны быть объявлены как private или protected. Прямой доступ к полям объекта снаружи запрещен.
3. **Наличие публичных геттеров и сеттеров:** Доступ к свойствам класса должен

осуществляться строго через специальные методы доступа, соответствующие соглашению об именах (get/set методы).

4. **Поддержка сериализации (Опционально, но крайне желательно):** Класс должен реализовывать интерфейс `java.io.Serializable`. Это позволяет сохранять состояние бина на диск или передавать его по сети.

Соглашения об именах (Naming Conventions) для JavaBeans

Имена методов доступа жестко завязаны на имя приватного поля:

- Если поле называется `private String firstName`;, то:
 - Метод чтения (Getter) должен называться строго: `public String getFirstName()`
 - Метод записи (Setter) должен называться строго: `public void setFirstName(String name)`
- **Специфика для логических (boolean) полей:** Если поле имеет тип `boolean`, например `private boolean active`;, то для метода чтения вместо префикса `get` принято использовать префикс `is`:
 - Getter: `public boolean isActive()`
 - Setter: `public void setActive(boolean active)`

Первая буква имени поля после префиксов `get/set/is` всегда переводится в верхний регистр. Нарушение этих правил именования приведет к тому, что технологии автоматического связывания данных (например, мапперы JSON или ORM Hibernate) не смогут обнаружить свойства вашего класса.

25. Ассоциации. Композиция и агрегация.

В ООП для описания взаимодействия объектов используются связи. **Ассоциация** — это наиболее общая, базовая форма связи, обозначающая, что объекты разных классов просто знают друг о друге и как-то взаимодействуют. Отношение ассоциации выражается принципом **«HAS-A» (Имеет / Содержит внутри себя)**.

Ассоциация разделяется на две специфические разновидности, определяющие степень зависимости жизненных циклов объектов: **Агрегацию** и **Композицию**.

1. Агрегация (Слабая связь)

Это отношение, при котором один объект является частью другого объекта, но при этом оба объекта имеют **независимые жизненные циклы**. Если уничтожить главный (контейнерный) объект, вложенный объект останется жить и может спокойно использоваться в других частях программы.

Пример концепции: Отношение между «Университетом» (University) и «Профессором» (Professor). Университет состоит из профессоров. Но если университет обанкротится и закроется (объект уничтожится), профессора не исчезнут физически. Они могут перейти работать в другой университет.

Реализация в Java: Объект-часть создается где-то снаружи и передается в конструктор главного класса в готовом виде:

```

class Professor { String name; }

class University {
    private Professor professor;
    // Профессор передан извне, его жизнь не принадлежит университету
    public University          { this.professor = prof; }
}

```

2. Композиция (Сильная связь)

Это жесткая, неразрывная форма ассоциации, при которой вложенный объект **не может существовать без главного объекта**. Жизненный цикл части полностью управляется и принадлежит контейнеру. Если уничтожить главный объект, все его композитные части будут уничтожены автоматически вместе с ним.

Пример концепции: Отношение между «Человеком» (Human) и его «Сердцем» (Heart). Сердце является частью человека. Сердце не может существовать само по себе в отрыве от конкретного человека. Если уничтожить объект Человек, объект Сердце уничтожается одновременно.

Реализация в Java: Объект-часть обычно создается непосредственно внутри конструктора или полей главного класса, скрыто от внешнего мира:

```

class Heart { double beatRate; }

class Human {
    private final Heart heart;
    // Сердце зарождается и умирает строго вместе с Человеком
    public Human { this.heart = new Heart(); }
}

```

26. Чем отличается абстрактный класс от интерфейса? Какие

идентификаторы по умолчанию имеют поля интерфейса?

Различия между абстрактными классами и интерфейсами в Java лежат как в концептуальной плоскости (дизайн архитектуры), так и в синтаксических возможностях реализации. С развитием Java (особенно после Java 8 и 9) техническая разница сократилась, но концептуальная осталась неизменной.

Сравнительный анализ (Таблица различий):

Критерий	Абстрактный класс (abstract class)	Интерфейс (interface)
Множественность	Класс может наследоваться только от одного абстрактного класса (в Java запрещено множественное наследование классов).	Класс может реализовывать сколько угодно много интерфейсов одновременно.
Поля состояния	Может содержать любые переменные: изменяемые, приватные, protected, статические, обычные поля экземпляра.	Может содержать только константы . Полноценного состояния у интерфейса быть не может.
Конструктор	Может иметь конструкторы. Они вызываются подклассами через super() для инициализации полей базового класса.	Не имеет конструкторов. Нельзя создать экземпляр или инициализировать поля через конструктор.
Методы	Может содержать методы с абсолютно любыми модификаторами доступа (private, protected, public) и любой реализацией.	Исторически — только абстрактные. В современной Java: public абстрактные, default методы, static методы, а также private методы (с Java 9).
Смысловая роль	Выражает сущность: отношение «IS-A»	Выражает контракт/возможность:

	(Является кем-то). Создает жесткую иерархию кровного родства.	отношение « CAN-DO » (Умеет делать что-то). Объединяет совершенно разные классы по возможностям.
--	---	--

Какие идентификаторы по умолчанию имеют поля интерфейса?

Любая переменная, объявленная внутри интерфейса, автоматически, неявно на уровне компилятора получает строго три модификатора: **public static final**.

Это означает, что любое поле интерфейса является **публичной глобальной константой**.

```
interface MyInterface {
    int MAX_COUNT = 100; // Компилятор неявно превратит в: public static final int MAX_COUNT = 100;
}
```

Поскольку поле является `final`, вы обязаны инициализировать его значением сразу в момент объявления в интерфейсе. Изменить это значение во время работы программы невозможно. Модификаторы `private` или `protected` для полей интерфейса использовать запрещено.

27. Когда применять интерфейс логичнее, а когда абстрактный класс?

Выбор между интерфейсом и абстрактным классом — это классическая задача архитектурного проектирования.

Когда логичнее применять Абстрактный класс?

1. **Наличие общего кода и состояния:** Если у вас есть группа тесно связанных классов, у которых много общего функционала (одинаковые готовые методы) и, главное, у них есть общие внутренние изменяемые поля (состояние). Интерфейс не может хранить обычные поля класса, а абстрактный класс — может.
2. **Построение иерархии родственников («IS-A»):** Когда вы можете четко сказать, что дочерние классы являются разновидностью родительского.
 - *Пример:* Абстрактный класс `Animal`, от него наследуются `Dog`, `Cat`, `Lion`. У них есть общие изменяемые поля (`age`, `weight`) и общие методы (`sleep()`). Собака является

животным.

3. **Необходимость ограничения доступа:** Если вы хотите скрыть часть внутренней базовой логики от внешнего использования, объявив методы как `protected` или `package-private`. В интерфейсах большинство методов по-прежнему ориентированы на публичный контракт.

Когда логичнее применять Интерфейс?

1. **Подключение роли или способности («CAN-DO»):** Когда вам нужно описать определенный контракт поведения для классов, которые в остальном могут быть абсолютно не связаны друг с другом кровным родством в иерархии.
 - *Пример:* Интерфейс `Runnable` (умеет выполняться в потоке) или `Serializable` (умеет превращаться в байты). Этот интерфейс могут реализовывать абсолютно разные классы: и класс `User`, и класс `NetworkConnection`, и класс `HeavyMatrixCalculator`. Между ними нет родства, но они все обладают способностью выполнять контракт.
2. **Обеспечение гибкости через множественное наследование:** Если вы предвидите, что вашему классу придется совмещать в себе множество разных функций. Поскольку наследоваться можно только от одного класса, выбрав абстрактный класс, вы тратите этот единственный шанс. Реализуя интерфейсы, класс может быть одновременно и списком, и закрываемым ресурсом, и сравниваемым объектом (`class MyCollection implements List, AutoCloseable, Comparable`).
3. **Создание чистых API (Decoupling):** Для максимального разделения компонентов системы, чтобы тестировать их через подмену (Mock-объекты).

28. Бывают ли интерфейсы без методов? Для чего?

Да, в Java существуют интерфейсы, внутри которых не объявлено ни одного метода (они полностью пустые). Такие интерфейсы называются **маркерными интерфейсами (Marker Interfaces)** или интерфейсами-пометками.

Для чего они используются?

Главная цель маркерного интерфейса — **присвоить классу определенный тип (метку/метаданные)** во время компиляции и выполнения, чтобы виртуальная машина JVM или сторонние библиотеки знали, что с объектами этого класса разрешено совершать определенные специфические системные операции.

Наиболее известные стандартные маркерные интерфейсы в Java Core:

1. **`java.io.Serializable`:** Сигнализирует JVM, что объекты данного класса можно превращать в поток байт (сериализовать) для сохранения на диск или передачи по сети. Если попытаться запустить стандартную сериализацию для класса, не помеченного этим интерфейсом, среда выполнения выбросит ошибку `NotSerializableException`.
2. **`java.lang.Cloneable`:** Разрешает вызов метода `clone()` у объекта (как подробно описано в вопросе 22). Без этой метки метод `super.clone()` выбросит исключение.

3. **java.util.RandomAccess:** Используется коллекциями. Он сигнализирует, что данная структура данных поддерживает быстрый произвольный доступ к элементам по индексу за константное время $O(1)$. Например, ArrayList реализует этот интерфейс, а LinkedList — нет. Алгоритмы утилитного класса Collections.binarySearch() проверяют эту метку через оператор instanceof: если метка есть, применяется быстрый цикл перебора по индексам, если метки нет — итераторы, что предотвращает катастрофическое падение производительности.

Современный статус: В современной Java для создания метаданных вместо маркерных интерфейсов чаще предпочитают использовать механизм **Аннотаций** (например, @MyMarker), так как аннотации более гибкие, могут содержать дополнительные параметры и не засоряют иерархию типов класса. Однако старые системные маркеры остаются фундаментальной частью языка.

29. Что такое Перегрузка и переопределение. Можно ли менять модификатор доступа метода, если да, то каким образом?

Перегрузка (Overloading) и **Переопределение (Overriding)** — это два фундаментальных проявления полиморфизма в Java, но они работают на разных этапах жизни программы.

- **Перегрузка (Overloading):** Создание внутри одного и того же класса нескольких методов с одинаковым именем, но **разными сигнатурами аргументов** (разное количество, типы или порядок параметров). Это полиморфизм компиляции (Compile-time / Early binding).
- **Переопределение (Overriding):** Изменение реализации метода родительского класса внутри *класса-наследника*. Имя метода и сигнатура параметров остаются абсолютно идентичными. Это полиморфизм времени выполнения (Runtime / Late binding).

Изменение модификатора доступа при Переопределении (Overriding)

Да, модификатор доступа менять **можно**, но строго по одному фундаментальному правилу: **При переопределении метода уровень видимости можно только расширять (делать более доступным), но ни в коем случае нельзя сужать (делать более закрытым).**

Иерархия видимости от закрытого к открытому выглядит так:

private $\rightarrow$ default (package-private) $\rightarrow$ protected $\rightarrow$ public.

- Если метод в родительском классе был объявлен как protected, то в классе-наследнике вы имеете право переопределить его как protected или сделать шире — public. Но вы не имеете права сделать его default или private.
- Если метод в родителе был public, у наследника он обязан оставаться строго public.

Почему это так? (Принцип подстановки Барбары Лисков — Liskov Substitution Principle): Код программы должен иметь возможность работать с объектом класса-наследника точно так же, как с объектом родительского класса, не подозревая о подмене. Если бы дочерний класс мог сузить доступ до private, то старый рабочий код, вызывавший этот метод через родительскую ссылку, внезапно сломался бы из-за нарушения прав доступа, что

разрушило бы стабильность полиморфизма.

Изменение модификатора доступа при Перегрузке (Overloading)

При перегрузке методов внутри класса вы можете ставить **абсолютно любые модификаторы доступа** без каких-либо ограничений. Один метод может быть public, а его перегруженная копия с другими параметрами — private. Для компилятора это совершенно разные, независимые методы.

30. Перегрузка и переопределение. Можно ли менять возвращаемый тип метода, если да, то как? Можно ли менять тип передаваемых параметров?

Разберем ограничения на изменение типов данных для обоих сценариев.

1. Сценарий Переопределения (Overriding)

- **Можно ли менять возвращаемый тип?** Да, можно, но только в рамках концепции **ковариантности возвращаемых типов (Covariant Return Types)**. Класс-наследник имеет право изменить возвращаемый тип метода, но новый тип обязан быть **дочерним подклассом** исходного возвращаемого типа из родительского класса.
 - *Пример:* Если метод родительского класса возвращает Number, то переопределенный метод в наследнике имеет полное право возвращать Integer или Double, так как они наследуются от Number. Но возвращать String он не может.
- **Можно ли менять тип передаваемых параметров? Категорически нельзя.** Если вы измените тип хотя бы одного передаваемого параметра (например, вместо int напишете double), то с точки зрения компилятора это автоматически перестанет быть переопределением. Это превратится в обычную *перегрузку* метода. Аннотация @Override над таким методом выдаст ошибку компиляции.

2. Сценарий Перегрузки (Overloading)

- **Можно ли менять возвращаемый тип?** Да, в перегруженных методах возвращаемые типы могут быть абсолютно любыми и никак не связанными друг с другом. Однако есть строжайшее правило: **Изменение только одного возвращаемого типа при полностью идентичных параметрах не является перегрузкой.** Если у вас есть метод void process(int x), вы не можете создать в этом же классе метод int process(int x). Компилятор выдаст ошибку «метод уже задекларирован», так как при вызове process(5) он не сможет понять, какую именно функцию вы хотите выполнить.
- **Можно ли менять тип передаваемых параметров? Да, это главная цель перегрузки.** Перегрузка как раз и создается за счет изменения типов, количества или порядка передаваемых аргументов.

31. Каким образом передаются переменные в методы, по значению или по ссылке?

Это один из самых частых вопросов с подвохом на собеседованиях. Ответ здесь однозначный и жесткий: **В Java абсолютно все переменные всегда передаются в методы строго ПО ЗНАЧЕНИЮ (Pass-by-value)**. Концепции передачи параметров по ссылке (как ключевое слово `ref` или `out` в C# или указатели в C++) в Java не существует. Однако путаница у разработчиков возникает из-за разного понимания того, что именно является «значением» для примитивов и для объектов.

1. Передача примитивных типов

Когда в метод передается примитив (`int`, `boolean` и т.д.), в стек вызова метода **копируется само физическое значение** (само число).

```
void changePrimitive int { x = 10; }
```

```
int a = 5;
```

```
changePrimitive(a);
```

```
System.out.println(a); // Выведет 5. Оригинал не изменился, метод крутил копию.
```

2. Передача ссылочных типов (Объектов)

Когда в метод передается объект, в метод передается **копия значения ССЫЛКИ** (копия адреса памяти, указывающая на объект в куче). Сам объект при этом не копируется. В итоге внутри метода у вас есть локальная переменная, которая хранит точно такой же адрес памяти, что и внешняя переменная.

- **Эффект 1: Модификация состояния объекта.** Через локальную копию ссылки мы можем залезть в кучу и изменить внутренние поля объекта. Эти изменения **будут видны снаружи**, так как объект в куче один.
- **Эффект 2: Попытка переприсвоить ссылку.** Если мы попытаемся внутри метода присвоить нашей переменной-параметру ссылку на совершенно новый объект через `new`, это изменение **не будет видно снаружи**. Мы лишь перезапишем нашу локальную копию адреса в стеке, а внешняя переменная продолжит указывать на старый объект.

Пример для демонстрации:

```

void modifyObject      {
    user.setName("Misha"); // Изменение поля будет видно снаружи!
    user = new User(2, "Ivan"); // Переприсвоение ссылки - снаружи НЕ ВИДНО!
}

```

32. Что такое конструктор по умолчанию?

Конструктор по умолчанию (Default Constructor) — это специальный явный или неявный конструктор класса, который не принимает никаких входных параметров (аргументов).

Главная особенность конструктора по умолчанию заключается в механизме его автоматической генерации:

- Если разработчик пишет класс и **не указывает вообще ни одного конструктора вручную**, компилятор Java автоматически, скрыто встраивает в скомпилированный байт-код публичный конструктор без параметров.
- Единственная задача этого неявного автоматического конструктора — вызвать пустой конструктор базового суперкласса с помощью неявной команды `super()`.

Когда конструктор по умолчанию исчезает?

Как только разработчик объявляет в классе **хотя бы один свой собственный конструктор с параметрами**, компилятор полностью **отключает автоматическую генерацию** конструктора по умолчанию.

Пример граблей:

```

class Person {
    String name;
    public Person      { this.name = name; } // Наш конструктор
}

```

// Попытка создать объект вот так:

```

Person p = new Person(); // Ошибка компиляции! Конструктора по умолчанию больше нет.

```

Если в такой ситуации вам по-прежнему необходим пустой конструктор (например, для требований спецификации JavaBeans или Hibernate), вы обязаны явно прописать его руками в теле класса: `public Person() {}`.

33. Свойства конструктора. Способы его вызова.

Конструктор — это специальный блок кода, внешне похожий на метод, предназначенный для инициализации начального состояния объекта в момент его создания в памяти.

Главные свойства конструктора:

1. **Совпадение имени:** Имя конструктора должно в точности, с учетом регистра букв, совпадать с именем самого класса.
2. **Отсутствие возвращаемого типа:** У конструктора нет и не может быть возвращаемого типа данных (даже слова `void` писать нельзя). Если случайно написать `public void MyClass()`, компилятор воспримет это не как конструктор, а как обычный метод класса с очень странным именем, и при создании объекта через `new` он вызываться не будет.
3. **Не наследуется:** Конструкторы родительского класса не наследуются классами-потомками.
4. **Автоматический вызов родителя:** Первая строчка любого конструктора обязана содержать явный или скрытый вызов конструктора родительского класса через ключевое слово `super()`. Если этого не написать, компилятор сам вставит неявный вызов `super()`.

Способы вызова конструктора

Конструктор невозможно вызвать у уже существующего готового объекта как обычный метод (нельзя написать `obj.MyClass()`). Существует три легитимных способа вызова:

1. **Использование оператора `new`:** Самый распространенный способ создания объекта на куче: `MyClass obj = new MyClass("аргумент");`.
2. **Вызов альтернативного конструктора этого же класса через `this()`:** Используется для цепочечного связывания конструкторов внутри одного класса (constructor chaining), чтобы избежать дублирования кода инициализации. Должен быть строго первой строчкой кода в конструкторе.
3. **Вызов конструктора родительского класса через `super()`:** Используется внутри конструктора класса-наследника для передачи параметров наверх, родителю. Также обязан быть строго первой строчкой кода.
4. **Через механизм Рефлексии (Reflection API):** Системный вызов, используемый фреймворками: `Class.forName("MyClass").getDeclaredConstructor().newInstance()`.

34. Логические блоки. Как работают и какие бывают?

Логические блоки (или блоки инициализации) в Java представляют собой участки кода, заключенные в фигурные скобки `{ ... }`, находящиеся непосредственно внутри тела класса, но вне каких-либо методов или конструкторов. Они предназначены для предварительной настройки данных объекта или класса.

Логические блоки разделяются на два типа: **Статические** и **Нестатические (блоки инициализации экземпляра)**.

1. Статические блоки инициализации (static { ... })

- **Как работают:** Помечаются ключевым словом static. Выполняются ровно **один раз за все время работы программы** — в тот момент, когда класс впервые загружается в память JVM подсистемой ClassLoader (при первом создании объекта этого класса или первом обращении к его статическим полям/методам).
- **Зачем нужны:** Используются для сложной инициализации статических полей класса (например, наполнение статической неизменяемой карты начальными базовыми значениями, чтение системных настроек из файлов конфигурации при старте).
- **Ограничение:** Внутри статического блока можно обращаться только к другим статическим полям и методам класса. Доступа к ключевому слову this или обычным полям объекта у него нет.

2. Нестатические блоки инициализации (Блоки экземпляра { ... })

- **Как работают:** Представляют собой просто фигурные скобки без всяких ключевых слов. Они выполняются **каждый раз, когда создается новый объект (экземпляр) класса**.
- **Порядок выполнения:** Нестатический блок выполняется непосредственно *перед* тем, как начнет работать сам код выбранного конструктора класса.
- **Зачем нужны:** Позволяют вынести общий код инициализации, который должен гарантированно выполняться для абсолютно всех конструкторов класса, избавляя от дублирования кода.

Жесткий порядок выполнения при создании объекта:

Если у нас есть класс-наследник, то при создании его первого объекта память инициализируется в строго фиксированной последовательности:

1. Выполняются статические блоки родительского класса.
2. Выполняются статические блоки дочернего класса.
3. Выполняются нестатические блоки родительского класса, а затем конструктор родительского класса.
4. Выполняются нестатические блоки дочернего класса, а затем финально сам конструктор дочернего класса.

35. Mutable и Immutable классы. Привести примеры из java core. Как создать класс, который будет immutable. Класс record.

Классы по признаку изменяемости своего внутреннего состояния делятся на две категории:

- **Mutable (Изменяемые):** Классы, объекты которых позволяют изменять свои внутренние поля после создания (через сеттеры или публичные методы).
 - *Примеры из Java Core:* StringBuilder, ArrayList, java.util.Date, любые стандартные сущности (POJO).

- **Immutable (Неизменяемые):** Классы, объекты которых после создания и инициализации в конструкторе **принципиально не могут изменить свое внутреннее состояние**. Если нужно что-то поменять, создается совершенно новый объект с новыми данными.
 - *Примеры из Java Core:* String, все классы-оболочки примитивов (Integer, Double и т.д.), java.time.LocalDate, класс java.math.BigDecimal.

Как вручную создать свой собственный Immutable класс? (Правила Блоха)

Чтобы сделать класс абсолютно неизменяемым, нужно соблюдать жесткий алгоритм:

1. Объявить сам класс как **final**, чтобы никто не мог унаследоваться от него и переопределить методы на изменяемые.
2. Сделать все поля класса **private**, закрыв прямой доступ снаружи.
3. Объявить все поля как **final**. Это гарантирует, что их значения можно присвоить только один раз в конструкторе, и защищает от случайного изменения внутри класса.
4. Не создавать в классе никаких методов-сеттеров.
5. **Принцип глубокого копирования (Оборонительное копирование):** Если среди полей вашего класса есть ссылки на другие *изменяемые* объекты (например, массив или List), то:
 - В конструкторе нужно принимать данные, делать их полную независимую копию и сохранять в поле именно копию.
 - В геттерах никогда не возвращать прямую ссылку на внутренний изменяемый объект, а возвращать либо его копию, либо оборачивать в немодифицируемый контейнер (например, Collections.unmodifiableList()).

Класс record (Рекорды)

Внедренные на постоянной основе в Java 16, **Рекорды (record)** — это специальный глубоко оптимизированный тип декларации классов, предназначенный исключительно для лаконичного создания **неизменяемых контейнеров данных (Data Carrier Classes)**. Вместо написания огромного количества шаблонного кода, рекорд создается одной строчкой:

```
public      User int      {}
```

Компилятор Java автоматически, неявно под капотом сгенерирует для этого рекорда:

- Финальный класс `public final class User extends java.lang.Record.`
- Приватные финальные поля: `private final int id; и private final String name;`.

- Конструктор со всеми параметрами для полной инициализации.
- Методы чтения (геттеры), причем без префикса get: они называются просто по имени полей — id() и name().
- Идеально выверенные, готовые реализации методов equals(), hashCode() и toString(), учитывающие все объявленные поля.

Рекорды идеально подходят для создания сущностей DTO (Data Transfer Object) благодаря своей лаконичности и встроенной архитектурной гарантии полной неизменяемости (Immutable).

36. static - что такое? Что будет, если значение атрибута изменить через объект класса? Всегда ли static поле содержит одинаковые значения для всех его объектов?

Ключевое слово **static** в Java переводит элемент (поле, метод или внутренний класс) из плоскости конкретных индивидуальных объектов в глобальную плоскость самого класса. Статические элементы принадлежат самому шаблону класса, загруженному в Metaspace, и существуют в единственном экземпляре, независимо от того, сколько объектов этого класса создано в куче (или даже если объектов нет вовсе).

Что будет, если значение статического атрибута изменить через объект класса?

С точки зрения синтаксиса, Java позволяет обратиться к статическому полю через переменную конкретного объекта, например:

```
MyClass obj1 = new MyClass();
obj1.staticField = 50; // Изменение через объект
```

На самом деле, под капотом компилятор проигнорирует переменную obj1. Он посмотрит на её тип (MyClass) и превратит этот вызов в прямое обращение к классу:

```
MyClass.staticField = 50;.
```

Последствие: Поскольку это поле общее и единое для всего приложения, изменение значения через один объект obj1 приведет к тому, что это новое значение мгновенно отразится на абсолютно всех остальных объектах данного класса. Если мы создадим MyClass obj2 = new MyClass(); и прочитаем obj2.staticField, мы увидим там число 50. Обращение к статическим переменным через ссылки объектов считается плохим стилем программирования (warning от IDE), всегда следует писать имя класса напрямую: MyClass.staticField.

Всегда ли static поле содержит одинаковые значения для всех его объектов?

Да, физически статический атрибут — это одна и та же ячейка памяти для всех объектов, загруженных конкретным загрузчиком классов (ClassLoader). Все объекты видят одно и то же глобальное состояние.

Однако есть два тонких нюанса, когда поведение может казаться иным:

1. **Локальное затенение переменных (Shadowing):** Если внутри метода объекта вы объявите локальную переменную с точно таким же именем, как и статическое поле класса (например, `int staticField = 10;`), то внутри этого конкретного метода имя будет указывать на локальную переменную стека, скрывая глобальное статическое поле. Но само поле в классе при этом не изменится.
2. **Разделение на уровне ClassLoader:** Если один и тот же класс будет загружен двумя разными изолированными загрузчиками классов (ClassLoader), в памяти JVM возникнут два независимых мета-описания класса. У каждого из них будет свое собственное, независимое статическое поле. Но в рамках объектов, созданных внутри одного ClassLoader, значения всегда строго одинаковы.

37. Может ли статический метод быть перегружен нестатическим? Переопределен?

Разберем оба сценария взаимодействия статического контекста с обычным экземплярным.

1. Может ли статический метод быть ПЕРЕГРУЖЕН нестатическим?

Да, статический метод может быть абсолютно спокойно перегружен нестатическим методом внутри одного и того же класса.

Главное и единственное условие для успешной перегрузки (как описано в вопросе 29) — у них должны быть **разные сигнатуры принимаемых аргументов**.

```
public class MyClass {  
    // Статический метод  
    public static void printData(int) { ... }  
  
    // Успешная перегрузка обычным нестатическим методом  
    public void printData() { ... }  
}
```

Компилятор без проблем различает их при вызове по типу передаваемого параметра.

2. Может ли статический метод быть ПЕРЕОПРЕДЕЛЕН в классе-наследнике?

Нет, статические методы принципиально не могут быть переопределены (Overridden). Если вы попытаетесь в классе-наследнике написать статический метод с абсолютно такой же сигнатурой и именем, как в родительском классе, и поставите аннотацию `@Override`, компилятор выдаст ошибку.

Если убрать аннотацию `@Override`, код скомпилируется, но это будет не переопределение, а механизм, который называется **Сокрытием методов (Method Hiding)**.

Почему переопределение не работает для static:

Переопределение методов базируется на механизме **динамического полиморфизма времени выполнения (Late Binding)**. JVM решает, какой метод вызвать, заглядывая в таблицу виртуальных методов конкретного физического объекта, лежащего в куче. А вызов статических методов компилируется как **статическое связывание (Early Binding)** — компилятор еще на этапе сборки жестко прописывает вызов метода конкретного класса, основываясь исключительно на типе переменной-ссылки, указанной в коде.

Пример сокрытия (Method Hiding):

```
class Child extends Parent { public static void say() { System.out.println("Child"); } }
```

```
Parent p = new Child();  
p.say(); // Выведет "Parent", а не "Child"! Потому что тип ссылки - Parent. Полиморфизм  
не сработал.
```

38. Внутренние классы. Какие бывают и для каких целей используются? Области видимости данных при определенных ситуациях.

В Java класс может быть объявлен внутри тела другого класса. Такие классы обобщенно называют вложенными, и они делятся на четыре четкие категории в зависимости от модификаторов и места объявления.

1. Статические вложенные классы (Static Nested Classes)

- **Что такое:** Класс, объявленный внутри другого класса с модификатором `static`.
- **Специфика связи:** Не имеет никакой неявной связи с объектами внешнего класса. Его можно создать без создания внешнего объекта: `Outer.Nested nested = new`

Outer.Nested());

- **Область видимости:** Имеет доступ **только к статическим полям и методам** внешнего класса (включая приватные). К обычным нестатическим полям внешнего объекта прямого доступа не имеет.
- **Зачем нужны:** Используются как вспомогательные служебные классы, жестко логически привязанные к контексту внешнего класса, но не требующие его состояния (например, класс Builder внутри сущности или класс узла дерева Node внутри структуры данных).

2. Внутренние (Истинные) классы (Inner / Non-static Member Classes)

- **Что такое:** Класс, объявленный внутри другого класса без ключевого слова `static`.
- **Специфика связи:** Каждый объект такого класса **жестко и неявно связан с конкретным объектом внешнего класса**, который его породил. Внутри объекта inner-класса компилятор скрыто хранит ссылку на внешний объект (`Outer.this`). Создать его можно только на базе существующего внешнего объекта: `Outer outer = new Outer(); Outer.Inner inner = outer.new Inner();`
- **Область видимости:** Имеет полный, ничем не ограниченный доступ к абсолютно **всем полям и методам внешнего класса, включая любые private элементы** и нестатические переменные. При совпадении имен полей доступ к внешнему полю осуществляется через синтаксис `Outer.this.fieldName`.
- **Зачем нужны:** Используются для создания тесно связанных компонентов, выражающих внутреннюю часть логики внешнего объекта (например, класс Iterator внутри структуры коллекции).

3. Локальные классы (Local Classes)

- **Что такое:** Классы, объявленные внутри локального блока кода, обычно **прямо внутри тела метода**.
- **Область видимости:** Видят всё, что видит метод. Огромная специфика: локальный класс имеет право читать переменные самого метода (аргументы или локальные переменные) только при условии, что эти переменные являются **final** или **effectively final** (то есть их значение не меняется после инициализации).
- **Зачем нужны:** Применяются крайне редко, для специфических сиюминутных вычислений или создания адаптеров локального уровня.

4. Анонимные классы (Anonymous Classes)

- Подробно разбираются в следующем вопросе.

39. Анонимные классы. Практическое применение. Для каких классов нельзя создать анонимный класс?

Анонимный класс — это локальный класс без имени, объявление которого совмещено с одновременным созданием одного-единственного экземпляра этого класса в памяти с помощью оператора `new`.

Синтаксис:

```
Runnable r = new Runnable() {  
    @Override  
    public void run() { System.out.println("Выполнение в анонимном классе"); }  
};
```

Практическое применение

До появления лямбда-выражений в Java 8 анонимные классы были единственным способом передать поведение «на лету». Сегодня они используются в следующих сценариях:

1. **Создание обработчиков событий (Listeners) в UI:** Быстрая привязка логики клика к кнопке в графических фреймворках (Swing, JavaFX).
2. **Интерфейсы со многими методами:** Если интерфейс или абстрактный класс содержит более одного абстрактного метода, лямбда-выражение применить нельзя. Приходится использовать анонимный класс, чтобы переопределить все методы разом в месте вызова.
3. **Быстрая кастомизация объектов на лету:** Переопределение одного метода базового класса без создания полноценного нового файла-класса в проекте.

Для каких классов НЕЛЬЗЯ создать анонимный класс?

Создание анонимного класса — это всегда скрытое наследование от указанного класса (или реализация указанного интерфейса). Исходя из этого, существуют четкие ограничения:

1. **Для final классов:** Поскольку анонимный класс под капотом пытается стать наследником (extends), попытка создать анонимный класс от final-класса (например, `new String() { ... }`) завершится жесткой ошибкой компиляции. Наследоваться от final запрещено.
2. **Для классов, у которых отсутствуют доступные конструкторы:** Если у базового класса все конструкторы помечены модификатором `private`, анонимный класс не сможет скомпилироваться, так как его собственный скрытый конструктор обязан вызвать `super()` родителя, к которому у него не будет доступа.
3. **Для Перечислений (enum):** Перечисления в Java неявно являются final-структурами и закрыты для классического расширения извне через `new`.
4. **Для Интерфейсов/Классов, находящихся вне зоны видимости (недоступные модификаторы доступа).**

Также технически невозможно объявить анонимный класс «просто так» сам по себе: ему обязательно нужен явный родительский класс или интерфейс, контракт которого он будет реализовывать.

40. Generics. Что это такое и для чего применяются. Во что превращается во время компиляции и выполнения?

Использование wildcards.

Generics (Обобщения) — это механизм языка, появившийся в Java 5, который позволяет параметризовать классы, интерфейсы и методы типами данных. По сути, это возможность передавать типы данных в качестве параметров, точно так же, как мы передаем значения переменных в методы.

Для чего применяются Generics?

1. **Строгая типизация во время компиляции (Strong Type Safety):** До Java 5 коллекции хранили внутри себя просто объекты типа `Object`. В `ArrayList` можно было одновременно положить и строку, и число. При извлечении приходилось делать ручное приведение типов (`(String) list.get(0)`). Если в коллекции случайно оказывался объект другого типа, программа падала во время выполнения с ошибкой `ClassCastException`. Generics заставляют компилятор проверять типы на этапе сборки: `ArrayList<String>` физически не позволит положить туда число, гарантируя безопасность кода.
2. **Избавление от ручного приведения типов (Casting):** Код становится чище, так как компилятор сам знает, какой тип лежит в контейнере, и неявно приводит его.

Во что превращаются Generics во время компиляции и выполнения?

Механизм Type Erasure

В Java внедрен механизм, называемый **Стиранием типов (Type Erasure)**. Это было сделано ради обеспечения идеальной обратной совместимости со старым кодом, написанным на Java 1.4 и ранее.

- **Во время компиляции:** Компилятор использует информацию о дженериках (например, `<T>` или `<String>`) исключительно для проверки корректности типов.
- **После компиляции (в байт-коде и во время выполнения):** Информация о конкретных типах внутри угловых скобок **полностью стирается**. Все универсальные типы параметров `<T>` заменяются на базовый класс **Object** (или на тип его верхней границы, если использовался ограничитель, например `<T extends Number>` сотрется в `Number`).
- В местах извлечения данных компилятор сам автоматически подставляет инструкции явного приведения типов к нужному классу.

В итоге, внутри запущенной JVM объект `ArrayList<String>` и объект `ArrayList<Integer>` превращаются в абсолютно одинаковые структуры типа `ArrayList`, оперирующие сырыми объектами `Object`. Сама JVM во время выполнения не знает, какими дженериками был ограничен этот список изначально.

Использование Wildcards (Символов подстановки ?)

Символ вопросительного знака ? в дженериках означает **неизвестный тип данных**. Он необходим, так как дженерики по умолчанию являются **инвариантными** (например, класс List<Integer> ни в коем случае не является подтипом класса List<Number>, несмотря на то, что Integer наследуется от Number).

Для управления гибкостью используются ограниченные вайлджарды, подчиняющиеся принципу **PECS (Producer Extends, Consumer Super)**:

1. **? extends T (Upper Bounded Wildcard)**: Ограничение сверху. Означает «тип T или любой его подкласс-наследник». Используется, когда структура данных выступает в роли **Продюсера (Producer)** — то есть вы собираетесь только *читать* данные из этого контейнера.
 - *Пример*: void printValues(List<? extends Number> list) позволит передать в метод список List<Integer>, List<Double> и т.д. Читать из него безопасно, так как любой элемент гарантированно является Number. Но положить туда ничего нельзя (кроме null), так как компилятор не знает, какой именно подтип там лежит.
2. **? super T (Lower Bounded Wildcard)**: Ограничение снизу. Означает «тип T или любой его родительский суперкласс». Используется, когда структура выступает в роли **Консьюмера (Consumer)** — то есть вы собираетесь только *записывать* данные в этот контейнер.
 - *Пример*: void addNumbers(List<? super Integer> list) позволит передать список List<Integer>, List<Number> или List<Object>. В такой список можно гарантированно безопасно записывать числа типа Integer. Но читать из него безопасно можно только в переменную типа Object, так как истинный тип родителя неизвестен.

41. Что такое enum? Область применения. Какое использование перечисления некорректно? Привести примеры.

enum (Перечисление) — это специальный тип класса в Java, представляющий собой фиксированный, строго ограниченный набор именованных константных значений.

Внутреннее устройство и Область применения

Под капотом любой enum компилируется в полноценный класс, неявно унаследованный от абстрактного системного класса java.lang.Enum. Все объявленные элементы перечисления являются **публичными статическими финальными объектами (public static final)** этого самого класса enum.

Пример:

```
public enum Role { ADMIN, USER }
```

Компилятор превратит это примерно в:

```
public final class Role extends Enum<Role> {  
    public static final Role ADMIN = new Role("ADMIN", 0);  
    public static final Role USER = new Role("USER", 1);  
    private Role          int          { super(name, ordinal); }  
}
```

Область применения: Используется везде, где системе требуется оперировать упорядоченными фиксированными списками значений, известными еще на этапе написания кода: дни недели, статусы заказов (NEW, IN_PROGRESS, DELIVERED), роли пользователей, стороны света, типы протоколов соединения. Перечисления обеспечивают стопроцентную type-safety (типобезопасность): вы физически не сможете передать в метод статус, которого нет в enum.

Какое использование перечисления НЕКОРРЕКТНО? (Антипаттерны и примеры)

- 1. Попытка использовать Enum для динамически изменяющихся данных:**
Перечисление статично. Если список элементов может измениться во время работы программы или должен подтягиваться из базы данных (например, список категорий товаров, который менеджер может расширять через админку), использовать enum **категорически некорректно**. Любое изменение потребует полной перекомпиляции и перезапуска всего приложения.
- 2. Привязка к порядковому номеру (ordinal()):** У каждого элемента enum есть метод ordinal(), возвращающий его индекс в порядке объявления (0, 1, 2...). Использовать этот индекс для сохранения бизнес-логики в базу данных или логических проверок — грубейшая ошибка.
 - *Пример некорректного кода:* `if (user.getRole().ordinal() == 0) { ... }`
 - *Почему это плохо:* Если завтра другой разработчик решит для красоты отсортировать элементы перечисления по алфавиту или вставит новую роль в начало списка, все индексы съедут, и бизнес-логика приложения сломается самым непредсказуемым образом.
- 3. Создание изменяемых (Mutable) перечислений:** Внутри enum можно добавлять поля, конструкторы и методы. Однако делать эти поля изменяемыми (без ключевого

слова `final` и с наличием сеттеров) — очень плохая практика, порождающая трудноуловимые баги в многопоточной среде, так как константы `enum` глобальны и делятся между всеми потоками. Поля внутри `enum` обязаны быть `private final`.

42. Отличия в применении интерфейсов `Comparator` и `Comparable`?

Оба интерфейса используются для реализации логики сравнения и последующей сортировки объектов, но они кардинально различаются по месту внедрения и степени гибкости.

1. Интерфейс `Comparable<T>`

- **Концепция:** Задает объекту **естественный порядок сортировки (Natural Ordering)**. Он говорит: «Я, как объект, сам изначально знаю, как меня правильно сравнивать с другими объектами моего типа».
- **Реализация:** Класс обязан реализовывать этот интерфейс **внутри своего собственного исходного кода**. Интерфейс содержит всего один метод: `compareTo(T o)`.
- **Возвращаемое значение:**
 - Положительное число (> 0), если текущий объект (`this`) больше объекта `o`.
 - Отрицательное число (< 0), если текущий объект меньше объекта `o`.
 - Ноль ($= 0$), если объекты абсолютно равны.
- **Применение:** Большинство стандартных классов (`String`, `Integer`) реализуют `Comparable` (строки по алфавиту, числа по возрастанию). Метод вызывается автоматически при вызове `Collections.sort(list)` без дополнительных параметров.

2. Интерфейс `Comparator<T>`

- **Концепция:** Представляет собой **внешний обособленный инструмент сравнения (Сравнитель)**. Он говорит: «Сам объект может вообще не знать, как себя сортировать, или мне нужно отсортировать объекты по совершенно другой, специфической логике».
- **Реализация:** Создается как отдельный класс, анонимный класс или лямбда-выражение вне исходного кода целевого класса. Содержит метод `int compare(T o1, T o2)`.
- **Применение:** Используется, когда:
 - Вам нужно иметь **несколько разных стратегий сортировки** (например, класс `User` по умолчанию через `Comparable` сортируется по `id`, но в одном конкретном отчете на UI нам нужно отсортировать пользователей по `age`, а в другом — по `lastName`).
 - У вас нет доступа к исходному коду класса (он пришел из сторонней закрытой JAR-библиотеки), и вы физически не можете заставить его реализовать `Comparable`.
- Вызов осуществляется передачей компаратора вторым аргументом:
`list.sort(myAgeComparator)`.

43. Класс `Optional`. Как помогает бороться с `null`? Пример хорошего

и некорректного использования. Отличия `orElse()` от `orElseGet()`.

Класс `java.util.Optional<T>` появился в Java 8 как объект-обертка (контейнер), который может либо содержать внутри себя значение целевого типа, либо оставаться пустым (`Optional.empty()`).

Как он помогает бороться с `null`?

Сам по себе `Optional` не уничтожает `null` на уровне языка. Его главная цель — **изменение дизайна API**. Когда метод возвращает объект типа `Optional<User>`, он на уровне сигнатуры жестко и честно предупреждает вызывающего программиста: *«Внимание! Объекта внутри может не оказаться, ты обязан обработать этот случай»*. Это избавляет от необходимости угадывать, может ли метод вернуть `null`, и защищает от забытых проверок, манифестируя потенциальное отсутствие данных.

Пример ХОРОШЕГО (идиоматичного) использования

`Optional` спроектирован для использования в цепочках в функциональном стиле:

```
// Идеальный вариант использования в качестве возвращаемого типа метода поиска
public          findUserById int    {
    User user = database.execute(id);
    return Optional.ofNullable(user);
}

// Хорошая и красивая обработка результата на вызывающей стороне
findUserById(10)
    .map(User::getName)
    .map(String::toUpperCase)
    .ifPresent(name -> System.out.println("User found: " + name));
```

Пример НЕКОРРЕКТНОГО использования (Антипаттерны)

1. **Вызов `.get()` без предварительной проверки:** Прямой вызов `optional.get()` без вызова `isPresent()` в случае пустого контейнера выбросит исключение `NoSuchElementException`, что ничем не лучше обычного `NullPointerException`. Теряется весь смысл обертки.
2. **Использование в качестве полей класса или параметров методов:** `Optional` не реализует интерфейс `Serializable`. Его использование в полях класса увеличивает расход памяти (дополнительный объект-пустышка в куче для каждого поля) и ломает сериализацию. Параметры методов также не стоит делать `Optional` — это заставляет писать уродливый код при вызовах. Для полей и параметров используйте классический `null`-чек.
3. **Оборачивание коллекций:** Никогда не пишите `Optional<List<User>>`. Если пользователей нет, метод должен просто вернуть пустой список.

`Collections.emptyList()`.

Отличия `orElse()` от `orElseGet()`

Оба метода используются для задания дефолтного значения в случае, если `Optional` пуст, но они кардинально отличаются по механизму вычисления этого дефолта:

- **`orElse(T other)` (Жадное вычисление):** Принимает готовое значение напрямую. Метод, передаваемый в качестве аргумента внутрь `orElse()`, **будет выполняться ВСЕГДА**, независимо от того, пустой `Optional` или заполненный.
 - *Пример:* `optional.orElse(createNewUser());` — метод `createNewUser()` выполнится, создаст объект на куче или сделает тяжелый запрос в базу, даже если внутри `optional` уже лежал готовый пользователь. Это может сильно ударить по производительности.
- **`orElseGet(Supplier<? extends T> other)` (Ленивое вычисление):** Принимает функциональный интерфейс (лямбду). Код внутри лямбды **вызовется и выполнится ТОЛЬКО тогда, когда `Optional` действительно пуст**. Если значение внутри есть, лямбда проигнорируется, сберегая ресурсы системы.
 - *Пример:* `optional.orElseGet(() -> createNewUser());` — тяжелый метод сработает только в случае реальной пустоты.

44. Принципы SOLID, Yagni, Kiss, Dry.

Это фундаментальные методологии и принципы чистого программирования, направленные на снижение стоимости поддержки и расширения программного обеспечения.

SOLID (Пакет из 5 принципов ООП)

1. **S - Single Responsibility Principle (Принцип единственной ответственности):** У каждого класса должен быть **ровно один повод для изменения**. Это значит, что класс должен решать ровно одну изолированную бизнес-задачу. Если класс одновременно умеет считать зарплату, форматировать отчет в HTML и сохранять его в базу данных, этот принцип нарушен.
2. **O - Open/Closed Principle (Принцип открытости/закрытости):** Программные сущности (классы, модули) должны быть **открыты для расширения, но закрыты для модификации**. Вы должны иметь возможность добавлять новое поведение в систему (например, новые типы оплат), создавая новые классы-наследники или реализуя интерфейсы, а не переписывая старый, уже протестированный рабочий код внутри базовых классов.
3. **L - Liskov Substitution Principle (Принцип подстановки Барбары Лисков):** Объекты в программе должны быть заменяемыми на экземпляры их подклассов без изменения правильности работы программы. Наследник не должен ломать поведение и ожидания, заложенные в родительском интерфейсе (подробно упоминалось в вопросе 29 про модификаторы доступа).
4. **I - Interface Segregation Principle (Принцип разделения интерфейса):** Много специализированных интерфейсов лучше, чем один универсальный «толстый».

Классы не должны вынужденно реализовывать методы интерфейсов, которые им не нужны для работы. Лучше разбить один огромный интерфейс на несколько мелких контрактов.

5. **D - Dependency Inversion Principle (Принцип инверсии зависимостей):** Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от **абстракций**. Абстракции не должны зависеть от деталей, детали должны зависеть от абстракций. В Java это выражается правилом: переменные должны объявляться с типами интерфейсов, а не конкретных классов-реализаций (`List list = new ArrayList()` вместо `ArrayList list = new ArrayList()`).

Дополнительные важные принципы:

- **KISS (Keep It Simple, Stupid / Делай проще, дурачок):** Принцип, призывающий делать код максимально простым, понятным и прямолинейным. Не нужно городить сложные паттерны проектирования там, где достаточно обычного цикла или условия. Простой код легче читать, тестировать и поддерживать.
- **DRY (Don't Repeat Yourself / Не повторяйся):** Каждая порция знаний или логики должна иметь единственное, однозначное представление в рамках системы. Избегайте бездумного копирования блоков кода (Copy-Paste). Дублирующийся код необходимо выносить в общие утилитные методы или базовые классы, иначе при изменении бизнес-логики вам придется искать и менять ее в десятках мест.
- **YAGNI (You Aren't Gonna Need It / Вам это не понадобится):** Принцип, запрещающий писать код про запас, под воображаемые будущие нужды, которых на данный момент нет в техническом задании. Программисты часто тратят недели на закладывание мега-гибкой архитектуры «на будущее», которая в итоге никогда не используется, но делает текущую систему переусложненной и хрупкой. Пишите только то, что нужно здесь и сейчас.

45. Что такое ClassLoader? Если изменить static переменную в классе, загруженном одним ClassLoader, что будет видно в том же классе, загруженном другим. Возможно ли синглтоны создавать несколько раз?

ClassLoader (Загрузчик классов) — это особый компонент виртуальной машины JVM, входящий в подсистему выполнения, задача которого — динамически, по мере необходимости, находить файлы байт-кода (.class), загружать их бинарное содержимое в оперативную память, верифицировать и создавать на их основе структуры метаданных классов в области Metaspace.

Загрузка работает по иерархическому принципу делегирования (Delegation Model).

Стандартно в JVM есть три базовых загрузчика:

1. *Bootstrap ClassLoader* (загружает базовые системные классы Java core).
2. *Platform ClassLoader* (панее Extension; загружает расширения платформы).
3. *Application ClassLoader* (загружает классы вашего приложения из classpath).

Анализ ситуации с изоляцией загрузчиков

В JVM уникальность и идентичность класса определяются не только его полным квалифицированным именем (например, com.example.User), но и **тем конкретным экземпляром ClassLoader, который этот класс загрузил**.

Если один и тот же физический файл MyClass.class загружается двумя разными, независимыми (изолированными) загрузчиками ClassLoader A и ClassLoader Б, то для виртуальной машины JVM это превращается в **два абсолютно разных, никак не связанных друг с другом класса**, несмотря на то, что текстовый код у них совпадает символ в символ. С точки зрения JVM, они живут в параллельных вселенных (разных пространствах имен).

- **Ответ на вопрос про статические переменные:** Если вы измените статическую переменную в классе, загруженном ClassLoader A, это изменение **никак не повлияет и не будет видно** в классе, загруженном ClassLoader Б. Каждый загрузчик создал свое собственное мета-описание класса в Metaspace со своим собственным изолированным выделенным участком памяти под статические поля.
- **Возможно ли синглтоны создавать несколько раз? Да, абсолютно возможно.** Классический паттерн Singleton (Синглтон) гарантирует уникальность объекта только в рамках **одного пространства имен одного ClassLoader**. Если ваше приложение работает в контейнере, где активно используются кастомные загрузчики (например, плагины в Tomcat, OSGi-системы или Spring DevTools), каждый изолированный ClassLoader загрузит класс синглтона независимо, создаст свою копию статического поля instance, и в итоге у вас в памяти JVM одновременно будут прекрасно существовать несколько независимых экземпляров «единственного» синглтона.

Привет! Разбираем следующую большую порцию вопросов. Как ты и просил, ответы будут максимально развернутыми, сплошным подробным текстом, глубоким погружением в детали работы JVM и наглядным форматированием, чтобы всё легко усваивалось перед собеседованием.

Java I/O

46. Какие бывают потоки ввода/вывода?

В Java вся система ввода-вывода (I/O) построена на концепции потоков данных (Streams), которые представляют собой упорядоченные последовательности данных. Их принято разделять на две магистральные группы в зависимости от того, с какими единицами информации они оперируют: байтовые и символьные.

Байтовые потоки обрабатывают сырые бинарные данные по 8 бит. Они являются универсальными, так как любой файл (будь то картинка, аудиозапись, исполняемый файл или текст) на диске хранится в виде байтов. Базовыми абстрактными классами для всех байтовых потоков являются `InputStream` (для чтения) и `OutputStream` (для записи). Их типичные реализации — это `FileInputStream` и `FileOutputStream`, которые работают напрямую с файловой системой, а также `ByteArrayInputStream/ByteArrayOutputStream`, оперирующие массивами байтов в оперативной памяти.

Символьные потоки появились в Java 1.1 специально для текстовой информации. Они оперируют 16-битными символами (Unicode) и автоматически берут на себя конвертацию байтов в символы с учетом заданной или системной кодировки (например, UTF-8 или UTF-16). За это отвечают абстрактные классы `Reader` и `Writer`. Их реализации, такие как `FileReader` и `FileWriter`, внутри себя используют байтовые потоки, но надстраивают над ними логику декодирования.

Кроме того, потоки делят по уровню абстракции на **узловые** (Node Streams) и **обрабатывающие** (Filter/Processing Streams). Узловые потоки подключаются непосредственно к источнику данных (файлу, сокету). Обработывающие потоки принимают в конструктор другой поток и добавляют ему новую функциональность. Яркий пример — буферизация. `FileInputStream` читает файл побайтово, что приводит к огромному числу дорогостоящих системных вызовов к диску. Если обернуть его в `BufferedInputStream`, то данные будут вычитываться с диска большими блоками в память, а приложение станет брать их из быстрого буфера, что колоссально поднимает производительность.

47. Что такое сериализация, для чего нужна, когда применяется?

Ключевое слово `transient`, для чего нужно? Сериализация `static`-полей. Зачем нужен `serialVersionUID`.

Сериализация — это встроенный в Java механизм перевода состояния живого объекта, находящегося в оперативной памяти (в куче), в последовательность байт. Обратный процесс восстановления объекта из байтов называется **десериализацией**. Этот процесс необходим, когда объект нужно "законсервировать" для долгосрочного хранения или

транспортировки: например, сохранить состояние игры на диск, передать объект по сети через сокеты в распределенной системе (RMI) или передать сессию пользователя между серверами в кластере. Чтобы объект можно было сериализовать, его класс обязан реализовывать интерфейс-маркер `java.io.Serializable`. У этого интерфейса нет методов, он лишь служит сигналом для JVM, что объект разрешено разбирать на байты.

Модификатор **transient** применяется к полям класса, которые необходимо исключить из процесса сериализации. Это критически важно в двух случаях. Во-первых, из соображений безопасности: нельзя сохранять в открытом бинарном виде чувствительные данные, такие как пароли, токены доступа или номера кредитных карт. Во-вторых, из-за технической невозможности: некоторые поля могут ссылаться на контекстозависимые ресурсы операционной системы, которые не имеют смысла после перезапуска программы (например, открытый поток `FileInputStream`, запущенный `Thread` или сетевой `Socket`). При десериализации объекта все поля, помеченные как `transient`, инициализируются значениями по умолчанию для их типов данных (`null` для ссылок, `0` для числовых типов, `false` для логических).

Что касается **статических (static) полей**, то они вообще не участвуют в сериализации. Причина кроется в самой природе статичности: эти поля принадлежат классу, а не конкретному экземпляру объекта. Сериализация же сохраняет состояние *конкретного* объекта. Когда вы десериализуете объект, статические поля нового экземпляра будут иметь те значения, которые сейчас загружены в самом классе текущей JVM, а не те, что были у класса в момент сохранения объекта.

Поле **serialVersionUID** — это уникальный 64-битный хэш-идентификатор версии класса, используемый при десериализации. Он служит для контроля совместимости. Если вы явно не укажете `private static final long serialVersionUID` в своем классе, JVM при компиляции рассчитает его автоматически на основе структуры класса (методов, полей, интерфейсов). Однако, если вы **बाद में** измените класс — например, добавите новое не-`transient` поле или измените тип существующего, — автоматически сгенерированный хэш изменится. При попытке десериализовать старый файл в обновленный класс JVM сравнит идентификаторы, обнаружит несовпадение и выбросит `InvalidClassException`. Явное объявление `serialVersionUID` позволяет разработчику вручную управлять совместимостью версий и гарантировать, что старые сохраненные данные смогут успешно прочтаться новой версией кода.

48. Возможно ли сохранить (сериализация) объект не в байт-код, а в XML-файл?

Да, это абсолютно возможно. Стандартная встроенная сериализация Java пишет данные в свой проприетарный бинарный формат, который нечитаем для человека и жестко привязан к платформе Java. Однако объект можно сериализовать в текстовые форматы, такие как XML или JSON.

Для этого в стандартной библиотеке Java (в пакете `java.beans`) существуют классы `XMLEncoder` и `XMLDecoder`. Они позволяют превратить состояние Java-боба (класса, соответствующего конвенции `JavaBean`: с пустым конструктором и геттерами/сеттерами)

в валидный XML-документ.

Тем не менее в современной промышленной разработке стандартный XMLEncoder практически не используется из-за громоздкости и ограничений. Вместо него применяются мощные сторонние спецификации и библиотеки:

- **JAXB (Jakarta XML Binding):** Стандартный инструмент корпоративного уровня, который с помощью аннотаций (таких как @XmlRootElement, @XmlElement) позволяет гибко мапить Java-классы в XML-схемы.
- **Jackson Extension (jackson-dataformat-xml):** Популярнейшая библиотека Jackson, которая умеет превращать объекты в XML одной строчкой кода через XmlMapper.
- **XStream:** Еще одна специализированная библиотека, которая может сериализовать в XML даже те объекты, которые не имеют пустых конструкторов или геттеров, работая напрямую через Reflection API.

Процесс перевода объекта в текстовое представление в контексте этих библиотек чаще называют терминами **маршаллинг** (marshalling) или **сериализация данных**, а обратный процесс — **демаршаллинг** (unmarshalling).

Java Exceptions

49. Опишите иерархию исключений.

Вся система обработки ошибок в Java построена на объектно-ориентированном подходе. На вершине этой иерархии находится класс java.lang.Throwable. Только объекты этого класса (или его наследников) могут быть переданы оператору throw или перехвачены блоком catch.

Класс Throwable разделяется на две фундаментально разные ветви:

3. **Класс Error:** Описывает фатальные, системные проблемы, возникающие на уровне виртуальной машины Java или операционной системы. Эти ошибки связаны с исчерпанием физических ресурсов среды выполнения или сбоями самой JVM. Примеры: OutOfMemoryError (кончилась куча), StackOverflowError (переполнился стек потока), NoClassDefFoundError (в рантайме не найден файл класса, который был доступен при компиляции). Обычное пользовательское приложение не должно пытаться перехватывать эти ошибки в блоках catch, так как после их возникновения среда не может гарантировать стабильную и предсказуемую работу.
4. **Класс Exception:** Описывает нештатные ситуации, которые возникают в логике работы самой программы или при ее взаимодействии с внешним миром. Это те ошибки, которые приложение способно предвидеть, обработать и продолжить свое выполнение в штатном режиме. Эта ветвь, в свою очередь, делится на две категории:
 - **RuntimeException (и его наследники):** Представляют собой так называемые *непроверяемые (unchecked)* исключения. Обычно они сигнализируют об ошибках программирования, плохой логике кода или нарушении контрактов использования методов (например, NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException).
 - **Все остальные наследники Exception (напрямую):** Представляют собой

проверяемые (*checked*) исключения. Они описывают ситуации, которые не зависят напрямую от качества кода, но могут произойти во внешних системах — например, проблемы со связью, отсутствие файла на диске, ошибки в SQL-запросах (IOException, SQLException, ClassNotFoundException).

50. Что такое **checked** и **unchecked** Exception? Их отличия.

Разделение исключений на проверяемые (Checked) и непроверяемые (Unchecked) — это архитектурное решение создателей языка Java, призванное заставить разработчиков явно обрабатывать потенциально опасные участки кода.

Checked (Проверяемые) исключения — это те, которые компилятор Java проверяет на этапе сборки проекта. Если метод содержит код, который может выбросить checked-исключение (например, чтение файла через FileReader), компилятор **обязет** вас сделать одно из двух: либо обернуть этот вызов в блок try-catch непосредственно на месте, либо честно прокинуть его выше по стеку вызовов, добавив в сигнатуру метода ключевое слово throws (например, public void readFile() throws IOException). Они используются для моделирования внешних непредвиденных ситуаций, которые программа обязана обработать (например, если файл удален, нужно вежливо попросить пользователя указать другой путь, а не падать с грохотом).

Unchecked (Непроверяемые) исключения — это те, выполнение которых компилятор полностью игнорирует при проверке исходного кода. К ним относятся все классы, наследующиеся от RuntimeException, а также ветка Error. Вам не нужно писать throws в сигнатуре метода или обязательно оборачивать код в try-catch. Они возникают непосредственно в процессе выполнения программы (Runtime). Как правило, появление unchecked-исключения — это симптом логического бага: вы попытались вызвать метод у ссылки, которая равна null (NullPointerException), или разделили число на ноль (ArithmeticException). Правильный способ борьбы с ними — не ловить их в catch, а писать код аккуратно (делать проверки на null, проверять границы массивов).

Критерий сравнения	Checked Exceptions	Unchecked Exceptions
Базовый класс	Наследуются от Exception (кроме RuntimeException)	Наследуются от RuntimeException
Проверка компилятором	Да, обязательная на этапе компиляции	Нет, компилятор их полностью игнорирует
Обязанность обработки	Необходимо обработать (try-catch) или объявить (throws)	Обработка остается исключительно на усмотрение разработчика
Философский смысл	Ошибки внешней среды, которые нужно	Ошибки в логике кода программы (баги)

	предусмотреть	разработчика)
Типичные примеры	IOException, SQLException, ParseException	NullPointerException, IllegalArgumentException

51. Опишите работу блока try-catch-finally. Может ли работать данный блок без catch? Порядок блоков catch. Multicatch.

Конструкция try-catch-finally является основным инструментом для управления потоком выполнения программы в условиях возникновения исключительных ситуаций.

4. **Блок try:** Очерчивает область кода, которая находится под наблюдением. Если внутри этого блока (или в глубине вызванных из него методов) происходит выброс исключения, нормальное выполнение кода в try мгновенно прерывается, и JVM начинает искать подходящий обработчик.
5. **Блок catch:** Принимает в качестве параметра объект исключения. Если тип выброшенного исключения совпадает с типом в параметре catch (или является его наследником), управление передается в этот блок. Здесь пишется логика восстановления: логгирование, отправка дефолтного ответа, закрытие транзакции.
6. **Блок finally:** Содержит код, который гарантированно должен быть выполнен, независимо от того, как завершился блок try. Отработал ли try идеально до конца, перехватил ли ошибку catch, или случилось непредвиденное checked-исключение, которое улетело выше к вызывающему методу — код в finally все равно выполнится. Единственные исключения, когда finally не срабатывает — это физическое падение операционной системы, отключение питания, зависание потока намертво или принудительный жесткий останов JVM из кода с помощью System.exit(0).

Может ли блок работать без catch?

Да, может. Конструкция try-finally абсолютно валидна с точки зрения синтаксиса. Она применяется в сценариях, когда текущий метод не умеет или не должен обрабатывать возникшую ошибку (он позволяет ей беспрепятственно пролететь дальше к вызывающему коду), но при этом метод обязан гарантированно освободить занятые ресурсы (например, закрыть открытый сетевой сокет или отпустить захваченный лок).

Порядок блоков catch:

Если у одного try есть несколько блоков catch, они должны располагаться строго по принципу **от частного к общему** (от классов-потомков к классам-предкам). Например, сначала должен идти catch (FileNotFoundException e), затем catch (IOException e) и только в самом конце catch (Exception e). Если нарушить этот порядок и поставить общего предка Exception первым, то он будет перехватывать абсолютно все ошибки, делая нижележащие блоки специфичных исключений недостижимым кодом, из-за чего компилятор выдаст ошибку и откажется собирать проект.

Механизм Multicatch (появился в Java 7):

Позволяет группировать обработку нескольких разнородных исключений в одном блоке

catch, если логика их обработки идентична. Это сильно сокращает дублирование кода.

```
try {  
    // Код, работающий с сетью и базой данных  
} catch (SQLException | IOException e) {  
    logger.error("Ошибка при работе с внешними ресурсами", e);  
    // Важная деталь: переменная 'e' в мультикетче неявно имеет модификатор final,  
    // ей нельзя присвоить новое значение внутри блока.  
}
```

Важное ограничение мультикетча: Нельзя перечислять через вертикальную черту исключения, которые находятся в отношениях "предок-потомок" (например, catch (FileNotFoundException | IOException e) компилятор запретит, так как первое уже включено во второе).

52. Блок try with resources.

Конструкция try-with-resources была добавлена в Java 7 как элегантная и безопасная замена классическому закрытию ресурсов в блоке finally. До её появления код освобождения ресурсов (например, закрытие файлов или соединений с БД) превращался в громоздкий кошмар с вложенными try-catch внутри finally, поскольку сам метод .close() тоже может выбросить IOException.

Чтобы объект можно было использовать в try-with-resources, его класс обязан реализовывать интерфейс java.lang.AutoCloseable (или его подмножество java.io.Closeable), содержащий один метод close().

Синтаксис и особенности работы:

```
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"));  
    FileWriter fw = new FileWriter("out.txt")) {  
    // Работа с файлами  
    System.out.println(br.readLine());  
} catch (IOException e) {
```

```
// Обработка ошибок  
}
```

Ресурсы объявляются внутри круглых скобок сразу после ключевого слова `try`. Вы можете объявить несколько ресурсов, разделив их точкой с запятой.

Главные фишки механизма:

- **Автоматическое закрытие:** Независимо от того, завершился ли блок `try` успешно или выбросил ошибку, JVM гарантированно вызовет метод `close()` для всех объявленных ресурсов.
- **Порядок закрытия:** Ресурсы закрываются в порядке, строго **обратном** их объявлению. В примере выше сначала закроется `fw`, а затем `br`. Это логично, так как поздние ресурсы могут зависеть от более ранних.
- **Момент закрытия:** Закрытие происходит **до** того, как управление перейдет в блоки `catch` или `finally`. Это значит, что если в блоке `catch` вы попытаетесь обратиться к базе данных, которая была открыта в ресурсах, она уже будет гарантированно закрыта.
- **Подавленные исключения (Suppressed Exceptions):** Если исключение возникает в теле `try`, а затем еще одно исключение стреляет во время автоматического вызова `.close()`, то основное исключение из `try` не теряется (в отличие от старого подхода с `finally`). Оно летит вверх, а ошибка закрытия ресурса прикрепляется к нему в качестве "подавленного" исключения. Их можно извлечь из основного объекта ошибки с помощью метода `e.getSuppressed()`.

53. Что такое **Error**. Перехват **Error**: есть ли смысл в перехвате, в каких случаях возникает и что с ним делать.

Как мы уже выяснили в вопросе об иерархии, класс **Error** является прямым наследником **Throwable**, но кардинально отличается от **Exception** по своей философии. Он сигнализирует о катастрофических, фатальных сбоях на системном уровне, которые делают невозможным дальнейшее безопасное выполнение программы.

В каких случаях возникает **Error**:

- **OutOfMemoryError** — виртуальная машина исчерпала всю выделенную ей память кучи (Heap) и сборщик мусора (GC) больше не может высвободить ни байта для создания новых объектов.
- **StackOverflowError** — поток израсходовал всю память своего стека из-за бесконечной цепочки вызовов методов.
- **NoClassDefFoundError / LinkageError** — критическая ошибка связывания, когда класс существовал на этапе компиляции, но бесследно исчез из `classpath` во время работы приложения.

Есть ли смысл в перехвате **Error**?

В прикладном, бизнес-коде ловить **Error** в блоках `catch` **практически никогда не имеет смысла**. Если у вашей JVM закончилась оперативная память, то попытка поймать эту ошибку внутри приложения ни к чему не приведет: для обработки самого блока `catch`

(создания лога, отправки сообщения по сети) тоже требуются объекты и память, которой физически нет, поэтому программа всё равно упадет, но уже в непредсказуемом месте.

Что делать, если Error все-таки нужно обработать?

Ловить Error (обычно перехватывают верхнеуровневый Throwable или конкретный тип ошибки) допускается только на уровне системных фреймворков, серверов приложений (например, Tomcat) или корневых потоков логирования с одной-единственной целью: **Graceful Shutdown** (красивое аварийное завершение). Это делается для того, чтобы перед неминуемой смертью процесса успеть записать критическую строку в лог-файл (чтобы разработчики поняли причину падения), попытаться экстренно сбросить важные буферы данных на диск или закрыть транзакции, после чего корректно завершить процесс. Исправлять причину Error внутри кода в рантайме невозможно — проблему нужно решать снаружи (менять параметры запуска JVM, оптимизировать алгоритмы, чинить зависимости).

54. Чем отличается OutOfMemoryError от StackOverflowError. Как их избежать?

Оба эти исключения являются системными ошибками (Error) и связаны с нехваткой памяти, но они поражают совершенно разные области оперативной памяти JVM.

OutOfMemoryError (OOM):

Эта ошибка означает истощение области памяти **Heap (Куча)**. В куче хранятся все динамически создаваемые объекты приложения (всё, что создается через ключевое слово `new`). Если ваша программа непрерывно плодит новые объекты, а старые не могут быть удалены сборщиком мусора (потому что на них всё еще существуют живые ссылки), память кучи заполняется до предела.

- *Как избежать:* 1. Тщательно искать и устранять утечки памяти (Memory Leaks) — ситуации, когда объекты (например, элементы кэша или слушатели событий) больше не нужны, но ссылки на них остаются активными. Для этого используют профайлеры (VisualVM, JProfiler, Eclipse MAT).
- 2. Использовать потоковую обработку данных (Streams/Scanners) вместо выгрузки огромных объемов информации (например, многогигабайтных файлов или миллионов строк из БД) целиком в память.
- 3. При реальной нехватке физического объема увеличить размер кучи при старте приложения с помощью JVM-флагов `-Xms` (стартовый размер) и `-Xmx` (максимальный размер), например: `-Xmx4g`.

StackOverflowError (SOE):

Эта ошибка означает переполнение памяти **Stack (Стек потока)**. У каждого потока в Java есть свой собственный изолированный стек. Каждый раз, когда вызывается любой метод, в стек этого потока помещается специальная структура — фрейм (Stack Frame), содержащий локальные переменные метода, параметры вызова и адрес возврата. Когда метод завершает работу, его фрейм удаляется. Если глубина вложенности вызовов методов превышает размер выделенного стека, выделять место под новый фрейм становится негде. В 99% случаев это происходит из-за некорректной, заикливившейся

рекурсии без условия выхода.

- *Как избежать:*
 - Внимательно проверять условия выхода (базовые случаи) во всех рекурсивных алгоритмах.
 - Если бизнес-логика требует огромной глубины вложенности, переписать алгоритм с рекурсивного на итеративный (использовать циклы while/for и структуры данных типа Deque в куче, так как куча физически намного больше стека).
 - В редких случаях, когда глубина вызовов легитимна (например, сложные парсеры или компиляторы), можно увеличить размер стека для каждого потока через флаг JVM -Xss, например: -Xss2m.

55. Оператор throw. Как работает? Какие свойства? Задача.

Оператор **throw** используется в Java для явного, принудительного выброса исключения в любой точке кода. Конструкция принимает один аргумент — объект класса Throwable (или любого его наследника): `throw new IllegalArgumentException("Возраст не может быть отрицательным");`.

Как это работает и свойства оператора:

Когда выполнение программы доходит до строки с `throw`, нормальный линейный ход выполнения программы мгновенно останавливается. Ближайшие строки кода ниже `throw` внутри этого метода выполнены не будут (компилятор даже выдаст ошибку `Unreachable statement`, если написать код сразу за `throw` без условий). JVM начинает сканировать стек вызовов в обратном порядке в поисках ближайшего подходящего блока `catch`. Если в текущем методе обработчика нет, метод аварийно завершается, управление передается вызывающему методу, и поиск продолжается там. Если дойти до самого корня (метода `main`) и не найти `catch`, поток исполнения погибнет, выведя в консоль `Stack Trace`.

Разбор задачи:

```
try {  
    a = 5;  
    throw new Exception("1");  
} catch (Exception e) {  
    a = 10;  
    throw new Exception("2");  
} finally {  
    a = 15;  
    throw new Exception("3");  
}
```

Давай пошагово проэмулируем логику выполнения этого куска кода JVM:

5. В блоке try переменной a присваивается значение 5. Следующей строкой принудительно выбрасывается исключение `Exception("1")`.
6. Выполнение блока try прерывается. JVM ищет подходящий catch. Блок catch (`Exception e`) ловит это исключение.
7. Внутри блока catch переменной a присваивается новое значение 10. Следом выбрасывается новое исключение `Exception("2")`.
8. В этот момент управление должно было бы покинуть метод и прокинуть `Exception("2")` вверх. Однако в коде присутствует блок `finally`. По законам Java, перед тем как исключение из блоков try или catch покинет границы метода, **управление обязано передаться в блок finally**.
9. В блоке finally переменная a перезаписывается значением 15. Затем выполняется команда `throw new Exception("3")`.
10. Выброс нового исключения внутри блока finally обладает разрушительным свойством: оно полностью **поглощает (маскирует/затирает)** любое предыдущее исключение, которое собиралось вылететь из блока try или catch. Исключение `Exception("2")` бесследно исчезает из стека.

Итоговый ответ: Исключение будет сгенерировано с сообщением **"3"**, а переменная a в итоге примет значение **15**.

JUnit 5 & Mockito

56. Механизм `assert`. Когда возникает `AssertionError`? В чем его смысл?

В Java понятие `"assert"` (утверждение) существует в двух разных ипостасях: на уровне самого языка программирования и на уровне библиотек для тестирования (JUnit).

- **Ключевое слово `assert` в языке (Java Keyword):** Синтаксис выглядит как `assert condition` : "Сообщение об ошибке";. Оно предназначено для внутреннего контроля качества кода во время разработки (Design by Contract). Разработчик заявляет: "Я гарантирую, что в этой точке программы переменная X стопроцентно положительна". Если во время работы `condition` оказывается ложным, JVM мгновенно выбрасывает ошибку **`AssertionError`**. По умолчанию в продакшене эти проверки полностью отключены и игнорируются JVM ради быстродействия. Чтобы они заработали, приложение нужно запустить со специальным флагом виртуальной машины `-ea` (`-enableassertions`).
- **Ассерты в фреймворках тестирования (JUnit Assertions):** Класс `org.junit.jupiter.api.Assertions` предоставляет статические методы типа `assertEquals(expected, actual)`, `assertTrue(condition)` и т.д. Внутри себя эти методы устроены просто: они сравнивают переданные значения, и если они не совпадают, метод принудительно генерирует и выбрасывает всё тот же `AssertionError`.

В чем смысл `AssertionError`:

Его фундаментальный смысл — зафиксировать критическое расхождение между тем, как

программа *должна* работать по задумке автора, и тем, как она отработала по факту. `AssertionError` является наследником класса `Error` (а не `Exception`), подчеркивая, что если утверждение нарушено, то продолжать выполнение теста или программы бессмысленно — логика сломана. Тест-раннер ловит этот `AssertionError`, окрашивает тест в красный цвет и показывает детальный отчет о том, где именно ожидание разошлось с реальностью.

57. JUnit. Что это такое? Принципы написания.

JUnit 5 — это самый популярный фреймворк для написания модульных (юнит) тестов на языке Java. Он позволяет автоматизировать проверку работоспособности отдельных минимальных изолированных кусочков кода (классов, методов), избавляя от необходимости тестировать приложение вручную. Современный JUnit 5 (также называемый JUnit Jupiter) состоит из модульной архитектуры, разделяющей движок запуска тестов, платформу и сам API для написания кода.

При написании качественных тестов принято руководствоваться набором академических и практических принципов. Самый главный из них зашит в акроним **F.I.R.S.T.**:

5. **F – Fast (Быстрые)**: Юнит-тесты должны выполняться за считанные миллисекунды. В проекте их могут быть тысячи, и если они будут работать медленно, разработчики перестанут запускать их перед каждым коммитом, что убьет всю идею автоматизации.
6. **I – Independent (Изолированные/Независимые)**: Тесты не должны зависеть друг от друга. Результат работы одного теста не должен быть входным условием для другого. Они должны без проблем запускаться в случайном порядке или параллельно в несколько потоков. Любое состояние (база данных, статические поля) должно сбрасываться перед каждым тестом.
7. **R – Repeatable (Повторяемые)**: Тест должен выдавать абсолютно идентичный результат при каждом запуске. Он не должен зависеть от внешней среды, текущего времени (используйте моки для часов), часового пояса или наличия подключения к интернету. Если тест падает один раз из десяти без изменения кода (такие тесты называют "flaky tests"), он бесполезен.
8. **S – Self-Validating (Самопроверяемые)**: Тест должен иметь четкий автоматический критерий успешности. Никакого ручного анализа логов в консоли после прогона. Тест либо полностью зеленый (прошел), либо красный (упал с конкретным ассертом).
9. **T – Timely (Своевременные)**: Тесты должны писаться вовремя — параллельно с основным кодом, а в идеальной методологии TDD (Test-Driven Development) — строго до написания самого продакшн-кода.

Еще один важный структурный паттерн при написании тела теста — **AAA (Arrange, Act, Assert)** или **GWT (Given, When, Then)**:

- **Arrange (Given)**: Подготовка окружения, создание объектов, настройка моков.
- **Act (When)**: Непосредственный вызов одного тестируемого метода.
- **Assert (Then)**: Проверка результата с помощью методов `Assertions.assertEquals()`.

58. Stub vs Mock: В чем разница?

В тестировании все фальшивые объекты-заглушки называют общим термином **Test Doubles**, но Мартин Фаулер разделяет их на категории по характеру их использования. Две самые часто путаемые категории — это стабы (Stubs) и моки (Mocks). Главное отличие лежит в плоскости того, как именно мы проверяем результат работы теста: через состояние или через поведение.

Stub (Заглушка):

Это пассивный объект, задача которого — просто подыгрывать тестируемому классу, возвращая заранее заготовленные "захардкоженные" данные при вызове его методов. Стаб создается для того, чтобы тест вообще мог выполняться и не падал по `NullPointerException`.

5. *Пример:* Тестируемый сервис запрашивает курс валют из внешнего API банка. Мы делаем стаб этого API, который на любой запрос всегда возвращает число 85.0.
6. *Как проверяем тест:* Через **состояние** (State Verification). Мы смотрим, что наш сервис правильно посчитал итоговую стоимость товара в рублях, используя эту цифру. Нам абсолютно не важно, сколько раз сервис вызывал стаб и с какими именно параметрами — важен конечный результат вычислений самого сервиса.

Mock (Макет/Имитация):

Это активный объект, который не просто отдает данные, но и **записывает внутри себя историю обращений**. Он фиксирует: какие его методы вызывались, сколько раз, в какой последовательности и с какими именно аргументами.

5. *Пример:* Наш сервис после успешной оплаты должен отправить пользователю email-уведомление через класс `EmailSender`. У метода отправки нет возвращаемого значения (`void`), состояние системы внешне не меняется.
6. *Как проверяем тест:* Через **поведение** (Behavior Verification). Мы не можем проверить возвращаемое значение сервиса, поэтому мы спрашиваем у Мока: "Эй, Мок-`EmailSender`, подтверди, что тестируемый сервис вызвал у тебя метод `send()` ровно 1 раз, и в параметрах был передан именно адрес `user@test.com?`". Если вызова не было или адрес был другой — тест провален.

59. Аннотации @Mock, @Spy и @InjectMocks. В чем разница между @Mock и @Spy, и как работает @InjectMocks?

Эти три аннотации предоставляются библиотекой Mockito и служат для декларативного управления заглушками в тестах, избавляя от ручного создания объектов через `Mockito.mock()`.

- **@Mock:** Создает стопроцентно виртуальный, пустой объект-заглушку. Реальный код класса, над которым стоит эта аннотация, **никогда не выполняется**. Все методы этого мока по умолчанию являются "пустышками": если метод возвращает объект, он вернет `null`, если примитив — `0` или `false`, если это `void`-метод — он просто ничего не сделает. Чтобы мок начал возвращать осмысленные данные, его поведение нужно явно запрограммировать (заставить) с помощью конструкции `Mockito.when(mock.method()).thenReturn(value)`.
- **@Spy:** Создает так называемую частичную заглушку (прокси) над **реальным, живым**

объектом. Для этого у класса обязательно должен быть доступен пустой конструктор (или объект нужно создать вручную и передать в спай). Если вы вызываете метод у @Spy-объекта, и его поведение не было предварительно переопределено через Mockito, то **выполнится настоящий, реальный код этого метода.** Переопределение поведения для спая пишется в специфичном синтаксисе `doReturn(value).when(spy).method()`, чтобы предотвратить вызов реального метода во время самой настройки.

- **@InjectMocks:** Эта аннотация ставится над тем самым классом, логику которого вы **непосредственно тестируете** (System Under Test). Она даёт команду Mockito: "Создай реальный экземпляр этого класса, найди внутри него зависимости (поля) и автоматически засунь (заинжект) в них все те объекты, которые помечены аннотациями @Mock или @Spy в этом тестовом классе". Mockito пытается внедрить их сначала через конструктор, если конструктора нет — через сеттеры, если и их нет — напрямую через Reflection в приватные поля.

60. Жизненный цикл Mockito в JUnit 5. Как использовать Mockito с JUnit 5, чтобы аннотации @Mock заработали?

Если вы просто расставите аннотации @Mock и @InjectMocks над полями тестового класса, при запуске теста вы гарантированно получите NullPointerException. Сами по себе аннотации — это просто маркеры, им нужен обработчик, который просканирует класс и инициализирует эти поля.

В JUnit 5 для интеграции со сторонними инструментами используется мощная концепция расширений (Extensions). Чтобы связать жизненный цикл JUnit 5 и Mockito, тестовый класс необходимо пометить аннотацией **@ExtendWith(MockitoExtension.class)**.

```
@ExtendWith(MockitoExtension.class) // Подключаем расширение Mockito
class UserServiceTest {

    @Mock
    private UserRepository userRepository; // Будет инициализирован как мок перед каждым
    тестом

    @InjectMocks
    private UserService userService; // Сюда автоматически внедрится userRepository

    @Test
    void testGetUser {
```

```

        // Тест работает, поля не null
    }
}

```

Как устроен жизненный цикл этой интеграции под капотом:

4. JUnit 5 находит аннотацию `@ExtendWith(MockitoExtension.class)` и передает управление коду расширения Mockito перед созданием экземпляра тестового класса.
5. **Перед каждым тестовым методом** (аналог фазы `@BeforeEach`) `MockitoExtension` автоматически вызывает внутренний метод `MockitoAnnotations.openMocks(this)`. В этот момент сканируются все поля, создаются прокси-объекты для `@Mock` и `@Spy`, и они подкладываются в `@InjectMocks`.
6. Запускается сам тестовый метод.
7. **После завершения тестового метода** (фаза `@AfterEach`) расширение автоматически закрывает созданный сессионный контекст моков (вызывает `close()`), очищая память и сбрасывая историю вызовов, чтобы обеспечить полную изоляцию для следующего теста.

Альтернативный способ без аннотации класса: Можно отказаться от `@ExtendWith` и вручную в методе с аннотацией `@BeforeEach` вызывать `AutoCloseable closeable = MockitoAnnotations.openMocks(this);`, а в `@AfterEach` вызывать `closeable.close()`. Но использование `@ExtendWith` является стандартом индустрии, так как защищает от забытых закрытий ресурсов.

61. Пирамида тестирования.

Пирамида тестирования — это концептуальная модель, визуализирующая идеальное процентное соотношение и распределение различных типов автоматизированных тестов в проекте. Она помогает команде грамотно распределять ресурсы на автоматизацию, чтобы не свалиться в антипаттерны вроде "леденца на палочке" (когда юнит-тестов нет, а есть только тяжелые UI-тесты).

Классическая пирамида состоит из трех основных уровней:

8. **Unit-тесты (Основание пирамиды):** Тестируют минимальные изолированные компоненты (классы, методы) с использованием моков.
 - *Особенности:* Их должно быть **подавляющее большинство** (70-80% от общего числа тестов в проекте). Они выполняются мгновенно, стоят очень дешево в разработке и поддержке. Если они падают, разработчик сразу видит конкретную строчку кода с ошибкой.
9. **Integration (Интеграционные) тесты (Средний уровень):** Проверяют, как несколько изолированных модулей работают в связке друг с другом или со сторонними системами (БД, брокеры сообщений, внешние REST-сервисы). Здесь часто используют реальные базы данных (например, в Docker-контейнерах через `Testcontainers`).
 - *Особенности:* Их существенно меньше, чем юнит-тестов. Они работают заметно медленнее (требуется время на поднятие контекста Spring или запуск БД) и стоят

дороже в поддержке.

10. **End-to-End (E2E) / UI тесты (Вершина пирамиды):** Тестируют приложение целиком, имитируя реальные действия конечного пользователя. Например, скрипт запускает браузер, кликает по кнопкам, проходит путь от авторизации до покупки товара, прогоняя запросы через всю цепочку от фронтенда до базы данных.
- **Особенности:** Их должно быть **меньше всего** (единицы или десятки). Они чрезвычайно медленные, тяжелые и "хрупкие" (любое изменение дизайна фронтенда или задержка сети могут их сломать, даже если бэкенд работает идеально).

Главный вывод пирамиды: Чем ниже тест находится в пирамиде, тем он быстрее, дешевле и специфичнее. Чем выше — тем он точнее эмулирует реальный пользовательский опыт, но при этом он медленнее, дороже и сложнее в локализации ошибки.

Многопоточность (Multithreading)

62. Что такое процесс? Что такое поток? Состояния потока.

Понимание разницы между процессом и потоком лежит в основе эффективного проектирования конкурентных приложений.

Процесс — это операционная единица, запущенный экземпляр программы, которому операционная система выделила изолированные системные ресурсы: адресное пространство в оперативной памяти, дескрипторы файлов, пулы ввода-вывода. Процессы изолированы друг от друга на уровне ОС. Программа А не может залезть в память Программы Б напрямую — для этого нужны сложные механизмы межпроцессного взаимодействия (IPC: сокет, пайпы).

Поток (Thread) — это наименьшая единица исполнения внутри одного процесса. Один процесс может порождать множество потоков, которые выполняются параллельно или псевдопараллельно (путем переключения квантов времени процессора). Все потоки внутри одного процесса делят между собой **общие ресурсы этого процесса**, в первую очередь — область памяти **Heap (Куча)**. Однако у каждого потока есть своя изолированная область памяти — **Stack (Стек)** для локальных переменных и собственный счетчик команд (PC). Из-за общего пространства памяти потоки могут общаться друг с другом мгновенно, но из-за этого же возникает опасность одновременной порчи данных (состояния гонки).

В Java жизненный цикл потока описывается шестью состояниями, зафиксированными в перечислении Thread.State:

4. **NEW:** Поток создан как объект в памяти (Thread t = new Thread()), но метод start() еще не вызывался. Он еще не исполняется ОС.
5. **RUNNABLE:** Поток запущен (start() вызван). Он либо уже выполняется процессором в данный момент, либо находится в очереди готовых к выполнению потоков и ждет, когда планировщик задач ОС (Thread Scheduler) выделит ему квант времени (процессорное время).
6. **BLOCKED:** Поток приостановлен и ждет освобождения монитора (блокировки)

объекта, чтобы войти в synchronized блок или метод.

7. **WAITING:** Поток ушел в режим бесконечного ожидания действий от другого потока. Это происходит при вызове методов `Object.wait()` без параметров, `Thread.join()` или `LockSupport.park()`. Поток проснется только тогда, когда другой поток вызовет `notify()` или метод завершит работу.
8. **TIMED_WAITING:** Поток спит или ждет в течение строго определенного времени. Состояние активируется методами `Thread.sleep(ms)`, `Object.wait(ms)`, `Thread.join(ms)`. По истечении таймера поток автоматически вернется в состояние `RUNNABLE`.
9. **TERMINATED:** Поток полностью завершил свою работу (метод `run()` выполнен до конца или внутри него вылетело непойманное исключение). Поток мертв, его нельзя запустить повторно.

63. Как создать поток? Какими способами можно запустить поток?

В Java создание потока (логики выполнения) и его непосредственный запуск разделены на уровне API.

Способы создания логики потока:

2. **Наследование от класса Thread:** Создаем свой класс, пишем `extends Thread` и переопределяем метод `public void run()`.
 - *Минус:* В Java запрещено множественное наследование классов. Если ваш класс уже унаследован, например, от какого-нибудь UI-компонента, наследоваться от `Thread` вы не сможете.
3. **Реализация интерфейса Runnable:** Создаем класс, реализующий `Runnable`, пишем логику в методе `run()`. Затем передаем экземпляр этого класса в конструктор стандартного `Thread`. Этот способ предпочтительнее, так как разделяет данные и логику, а также оставляет класс свободным для наследования. Сегодня это чаще всего пишется в виде лаконичных лямбда-выражений: `Thread t = new Thread(() -> System.out.println("Hello"));`.
4. **Реализация интерфейса Callable<V>:** Похож на `Runnable`, но его метод `call()` может **возвращать результат** выполнения определенного типа `V` и имеет право легитимно выбрасывать checked-исключения. Самостоятельно класс `Thread` работать с `Callable` не умеет, его запускают через пулы потоков (`ExecutorService`), а результат получают через объект-обещание `Future<V>`.

Как запустить поток:

Запуск созданного потока осуществляется **строго одним способом** — вызовом метода **`start()`** у объекта класса `Thread`.

Важнейший нюанс для собеседований: Что будет, если вместо `start()` вызвать метод `run()` вручную?

Если вы напишете `myThread.run()`, компилятор не выдаст ошибку, но **никакого нового потока в операционной системе не создается**. Код метода `run()` выполнится синхронно, как обычный метод обычного класса, прямо в текущем потоке (например, в потоке `main`), заблокировав его выполнение. Метод `start()` выполняет под капотом сложную нативную работу: обращается к операционной системе, запрашивает выделение нового системного потока, выделяет ему стек и запускает в нем метод `run()`.

64. Как корректно завершить (убить поток) работу потока?

В ранних версиях Java у класса Thread существовал прямой метод `stop()`, который мгновенно убивал поток извне. Однако он уже очень давно признан устаревшим (deprecated) и категорически запрещен к использованию. Принудительный останов потока через `stop()` деструктивен: поток умирает мгновенно в случайной точке выполнения, не успевая корректно закрыть файлы, сетевые соединения и, что самое страшное, он автоматически отпускает все захваченные им мониторы объектов (synchronized). Это приводит к тому, что объекты остаются в разорванном, несогласованном состоянии (Data Corruption), и другие потоки начинают читать испорченные данные.

Корректный, безопасный способ завершения потока — это механизм кооперативного прерывания (Interruption). Мы не убиваем поток жестко, а вежливо просим его завершиться, выставя специальный внутренний флаг-сигнал.

Для этого используются три метода:

- `workerThread.interrupt()` — вызывается из управляющего потока. Он устанавливает внутренний логический флаг прерывания в потоке `workerThread` в состояние `true`. Если поток в этот момент спал (в методах `sleep`, `wait`, `join`), то сон прерывается и выбрасывается исключение `InterruptedException`.
- `Thread.currentThread().isInterrupted()` — вызывается внутри самого выполняющегося потока. Метод проверяет, не попросил ли кто-то наш поток завершиться. Он просто возвращает `true` или `false`, не сбрасывая состояние флага.
- `Thread.interrupted()` — статический метод, который проверяет флаг прерывания для текущего потока и, в отличие от предыдущего, **автоматически сбрасывает его обратно в false**.

Реализация логики внутри потока:

Поток должен сам регулярно проверять этот флаг в цикле и аккуратно завершать свою работу:

```
public void run {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            // Выполняем полезную работу
            Thread.sleep(100); // Точка возможного прерывания
        } catch (InterruptedException e) {
            // Важно! При выбросе InterruptedException флаг прерывания сбрасывается в false.
            // Чтобы корректно выйти из цикла, мы должны либо сделать break,
```

```

        // либо восстановить статус прерывания:
        Thread.currentThread().interrupt();
    }
}
// Здесь пишется код закрытия ресурсов перед выходом
}

```

65. Как выполнить набор команд в отдельном потоке?

Чтобы выполнить произвольный блок команд в отдельном потоке, эти команды необходимо инкапсулировать в объект, реализующий функциональный интерфейс Runnable, а затем передать этот объект исполнителю.

В современном синтаксисе Java (начиная с версии 8) это делается элегантно через лямбда-выражения:

```

// Вариант 1: Через создание единичного ручного потока
Thread newThread = new Thread() -> {
    // Набор команд, выполняющихся в фоне
    System.out.println("Команда 1 в потоке: " + Thread.currentThread().getName());
    System.out.println("Команда 2...");
};
newThread.start(); // Запуск

```

Однако создавать потоки вручную через `new Thread()` в промышленной разработке считается плохой практикой (антипаттерном). Создание потока на уровне ОС — это дорогая, ресурсоемкая операция. Если под каждую задачу создавать новый поток, система быстро захлебнется от перегрузки памяти и затрат на переключение контекста (Context Switching).

Для выполнения набора команд правильно использовать **пулы потоков (Thread Pools)** из каркаса `ExecutorService`:

```
// Вариант 2: Использование пула потоков (промышленный стандарт)
ExecutorService executor = Executors.newFixedThreadPool(4); // Создаем пул из 4-х
постоянных потоков

executor.submit(() -> {
    // Набор команд отправляется в очередь пула
    System.out.println("Команда выполняется внутри пула потоков");
});

// После завершения работы приложения пул нужно обязательно закрыть
executor.shutdown();
```

При таком подходе потоки создаются один раз при старте пула, а затем многократно переиспользуются для выполнения поступающих задач, что экономит ресурсы.

66. Как работают методы wait() и notify()/notifyAll()?

Эти методы предназначены для организации сложного межпоточного взаимодействия (сигналикации). Они позволяют одному потоку приостановить свое выполнение до тех пор, пока другой поток не сообщит, что произошло определенное событие.

Главная архитектурная особенность: эти методы объявлены в самом корневом классе **Object**, а не в классе **Thread**. Это связано с тем, что ожидание и сигнализация завязаны на конкретный ресурс — **Монитор объекта**, который есть у абсолютно любого объекта в Java.

Строгое правило использования: Методы wait(), notify() и notifyAll() могут быть вызваны потоком **только внутри синхронизированного контекста** (внутри synchronized блоков или методов), причем синхронизация должна выполняться строго по тому самому объекту, у которого эти методы вызываются. Если попытаться вызвать их вне синхронизации, рантайм мгновенно выбросит **IllegalMonitorStateException**.

Алгоритм работы:

- **wait():** Когда поток вызывает object.wait(), он совершает три последовательных действия:
 - Добровольно **отпускает (освобождает) монитор** этого объекта, давая возможность другим потокам захватить его.
 - Приостанавливает свое выполнение и переходит в состояние ожидания WAITING.
 - Помещается в специальный пул ожидания (Wait Set) именно этого конкретного объекта.
- **notify():** Вызывается другим потоком, который ранее захватил этот же монитор. Метод выбирает **один случайный поток** из пула ожидания этого объекта и переводит его в состояние RUNNABLE. Поток просыпается, но он не может продолжить работу мгновенно — ему сначала нужно дождаться, пока текущий поток выйдет из своего synchronized блока (отпустит монитор), а затем на общих основаниях побороться за захват этого монитора с другими потоками.
- **notifyAll():** Работает точно так же, как notify(), но будит абсолютно **все потоки**,

которые в данный момент находились в пуле ожидания этого объекта. Этот метод считается гораздо более безопасным в использовании, так как исключает ситуацию, когда проснувшийся случайный поток не смог обработать условие и уснул обратно, оставив остальные потоки дремать вечно.

67. Чем отличается работа метода `wait()` с параметром и без параметра?

Разница заключается в условиях, при которых поток может выйти из состояния ожидания и вернуться к исполнению.

- **`wait()` без параметров (или с аргументом 0):** Поток переходит в бессрочное состояние ожидания (WAITING). У него нет внутреннего будильника. Он будет находиться в пуле ожидания объекта вечно, пока не наступят форс-мажорные обстоятельства: либо другой поток явно вызовет `notify()/notifyAll()` на этом объекте, либо потоку прилетит команда прерывания `interrupt()`, которая выбросит `InterruptedException`.
- **`wait(long timeout)` с параметром времени (в миллисекундах):** Поток переходит в состояние ограниченного ожидания (TIMED_WAITING). Для него включается внутренний таймер. Поток выйдет из ожидания и вернется в очередь на захват монитора в двух случаях: либо произойдет явный вызов `notify()` со стороны другого потока (до истечения времени), либо указанный `timeout` просто закончится. Если тайм-аут истек, поток просыпается самостоятельно, без чужой команды.

Важная рекомендация по проектированию: Из-за возможности так называемых "спонтанных пробуждений" (Spurious Wakeups), когда поток может проснуться сам по себе из-за особенностей реализации ОС, вызов `wait()` как с параметром, так и без, **всегда должен находиться внутри цикла `while`**, проверяющего нужное бизнес-условие, а не внутри условного оператора `if`.

68. Чем отличаются методы `Thread.sleep()` и `Thread.yield()`?

Оба метода являются статическими методами класса `Thread` и служат для управления текущим потоком, но они имеют принципиально разное семантическое значение для планировщика задач.

`Thread.sleep(long millis):`

Этот метод заставляет текущий поток принудительно приостановить выполнение на строго указанный промежуток времени. Поток переходит из состояния RUNNABLE в состояние TIMED_WAITING. Планировщик ОС гарантированно убирает этот поток с процессора на это время, отдавая ресурсы другим потокам. При этом важно помнить: поток засыпает, но **продолжает удерживать в своих руках все захваченные им блокировки и мониторы объектов (synchronized)**. Никто другой в эти блоки кода зайти во время его сна не сможет. По истечении времени поток возвращается в RUNNABLE.

`Thread.yield():`

Это метод-альтруист. Он не переводит поток в состояние ожидания и не заставляет его спать. Поток остается в состоянии RUNNABLE. Вызывая `yield()`, поток отправляет вежливое

уведомление планировщику задач JVM: "Я сейчас выполняю не самый критичный кусок работы. Если в очереди есть другие готовые потоки, которым нужен процессор, я не против уступить им свое текущее место". Планировщик может прислушаться к этой подсказке, приостановить текущий поток и дать квант времени другому потоку с таким же приоритетом. А может полностью проигнорировать этот вызов и сразу же продолжить выполнять исходный поток, если других претендентов нет или у них низкий приоритет. Метод используется в основном для оптимизации высоконагруженных алгоритмов ожидания (Spin-locks).

69. Чем отличаются методы `sleep(long millis)` и `wait(long millis)`?

Это один из самых излюбленных вопросов на собеседованиях по Java Concurrency, проверяющий понимание базовых механизмов блокировок.

Критерий	<code>Thread.sleep(long millis)</code>	<code>Object.wait(long millis)</code>
Принадлежность	Статический метод класса <code>java.lang.Thread</code>	Нестатический метод класса <code>java.lang.Object</code>
Отношение к блокировкам	НЕ освобождает мониторы. Поток спит, но лок держит.	Освобождает монитор объекта, на котором вызван, на время ожидания.
Требование к контексту	Можно вызывать в абсолютно любом месте программы.	Можно вызывать строго внутри <code>synchronized</code> блока/метода.
Способ пробуждения	Просыпается автоматически по истечении времени.	Просыпается по таймеру ИЛИ раньше, если вызван <code>notify()</code> .
Состояние потока	<code>TIMED_WAITING</code>	<code>TIMED_WAITING</code>

70. Как работает метод `join()`?

Метод `join()` принадлежит классу `Thread` и используется для синхронизации порядка выполнения потоков. Если текущий поток выполнения (например, поток `main`) вызывает метод `t.join()` у объекта другого потока `t`, это означает: "Поток `main` приостанавливает свою работу и обязуется покорно ждать до тех пор, пока поток `t` полностью не завершит свое выполнение (не перейдет в состояние `TERMINATED`)".

Принцип работы под капотом:

Внутри Java метод `join()` реализован очень хитро — через обычный цикл и метод `wait()`. Когда вы вызываете `t.join()`, текущий поток заходит в синхронизированный блок по монитору объекта `t` и выполняет внутри команду `wait()`. Поток переходит в состояние `WAITING`.

Как же этот поток просыпается, если мы нигде в своем коде не пишем `notify()`? Дело в том, что внутри самой JVM на уровне нативного кода зашит жесткий контракт: в момент, когда любой поток завершает свою работу и умирает, JVM автоматически вызывает метод `notifyAll()` на объекте этого потока. В этот момент все потоки, которые "джойнились" к нему и спали в `wait()`, автоматически просыпаются и продолжают свое выполнение.

71. Что такое синхронизация? Зачем она нужна? Для чего нужно ключевое слово `synchronized`? Какие способы синхронизации вы знаете?

Синхронизация — это механизм управления доступом множества потоков к разделяемым ресурсам (переменным, файлам, объектам), обеспечивающий правильную последовательность их выполнения и исключающий одновременное искажение данных.

Зачем она нужна:

Когда несколько потоков одновременно пытаются читать и модифицировать одну и ту же переменную памяти, возникает критическая уязвимость — **состояние гонки (Race Condition)**. Поскольку неатомарные операции (например, простой инкремент `count++`) на уровне процессора разбиваются на три отдельных шага (считать из памяти в регистр, увеличить на 1, записать обратно), потоки могут прервать друг друга на середине процесса, затерев результаты работы соседа. Также существует проблема **видимости памяти (Memory Visibility)**: ядра процессора кэшируют переменные в свои локальные быстрые регистры, и изменения, сделанные Поток 1, могут быть долго не видны Потoku 2, который читает устаревшее значение из своего кэша.

Зачем нужно слово `synchronized`:

Это базовый, встроенный в язык Java инструмент синхронизации, реализующий концепцию взаимного исключения (Mutual Exclusion). Оно гарантирует:

10. **Атомарность:** Блок кода, помеченный как `synchronized`, в один момент времени может выполняться **только одним потоком**. Все остальные потоки выстраиваются в очередь и блокируются (`BLOCKED`).
11. **Видимость:** При входе потока в `synchronized`-блок все кэши процессора для этого потока сбрасываются, и данные гарантированно перечитываются из общей оперативной памяти (Main Memory). При выходе из блока измененные данные мгновенно форсированно сбрасываются обратно в общую память.

Какие способы синхронизации существуют в Java:

- **Ключевое слово `synchronized`** (методы и блоки).
- **Ключевое слово `volatile`** (обеспечивает видимость и запрещает перестановку инструкций, но без взаимного исключения).
- **Явные блокировки из пакета `java.util.concurrent.locks`** (`ReentrantLock`, `ReentrantReadWriteLock`).

- **Атомарные переменные (AtomicInteger, AtomicReference и др.)** — используют неблокирующие алгоритмы на уровне процессора (CAS инструкции).
- **Высокоуровневые синхронизаторы** (Semaphore, CountDownLatch, CyclicBarrier, Exchanger).
- **Потокобезопасные коллекции** (ConcurrentHashMap, CopyOnWriteArrayList и т.д.).

72. По каким объектам синхронизируются статические и нестатические методы?

Когда вы помечаете метод ключевым словом `synchronized` в его объявлении, вам не нужно вручную передавать объект блокировки. Java автоматически определяет монитор, по которому будет идти синхронизация, в зависимости от модификаторов метода:

- **Нестатические (Instance) методы:** Синхронизация идет по объекту **this** — текущему конкретному экземпляру класса, у которого вызван метод. Если у вас есть два разных объекта класса `User`, Поток 1 может одновременно выполнять синхронизированный метод у Объекта 1, а Поток 2 — у Объекта 2. Но два потока не смогут одновременно зайти в этот метод у одного и того же объекта.

```
public synchronized void doSomething { // Блокировка по монитору текущего объекта (this)
    // код
}
```

- **Статические (Static) методы:** Статические методы не привязаны к конкретным объектам кучи, они принадлежат самому классу. Поэтому статические синхронизированные методы блокируются по **объекту класса (Class Object)**, который загружен в JVM ClassLoader. Для класса `MyService` этим объектом будет `MyService.class`. Это глобальная блокировка на уровне всего приложения: сколько бы объектов этого класса вы ни создали, в данный статический метод во всей программе сможет зайти только один поток.

```
public static synchronized void doStaticSomething { // Блокировка по монитору MyClass.class
    // код
}
```

73. Отличия работы `synchronized` от `Lock`?

Интерфейс `Lock` появился в Java 5 в пакете `java.util.concurrent.locks` как продвинутая альтернатива стандартному `synchronized`. Несмотря на то, что они решают одну задачу, между ними есть глубокие различия.

Критерий сравнения	<code>synchronized</code>	Объекты <code>Lock</code> (например, <code>ReentrantLock</code>)

Природа	Встроенное ключевое слово языка, поддерживается на уровне байт-кода JVM.	Обычные Java-классы из стандартной библиотеки.
Захват и освобождение	Происходит неявно. Лок берется при входе в блок и автоматически отпускается при выходе (даже при ошибках).	Происходит явно вручную: метод <code>lock.lock()</code> для захвата и <code>lock.unlock()</code> в блоке <code>finally</code> для освобождения.
Гибкость структуры	Блоки кода должны быть строго вложенными (структура скобок <code>{}</code>).	Можно захватить лок в одном методе, а освободить совершенно в другом.
Попытка захвата (<code>tryLock</code>)	Нет. Поток встает в очередь <code>BLOCKED</code> и будет стоять там вечно, пока лок не освободится.	Да, метод <code>tryLock()</code> пытается взять лок. Если он занят, поток не блокируется, а сразу возвращает <code>false</code> (можно уйти делать другую работу).
Ожидание с тайм-аутом	Отсутствует.	Да, метод <code>tryLock(timeout, timeUnit)</code> позволяет ждать лок в течение строго определенного времени.
Прерываемость ожидания	Поток в очереди <code>synchronized</code> невозможно прервать через метод <code>interrupt()</code> .	Да, метод <code>lockInterruptibly()</code> позволяет прервать поток, пока он стоит в очереди за локом.
Честность (Fairness)	Нет. Очередь не гарантирует порядок: лок захватит тот поток, который успел подбежать в момент освобождения.	Можно включить флаг честности (<code>new ReentrantLock(true)</code>). Тогда лок гарантированно получит тот поток,

		который ждал дольше всех.
--	--	---------------------------

74. Есть ли у Lock механизм, аналогичный механизму wait/notify у synchronized?

Да, такой механизм есть. Поскольку методы wait() и notify() жестко завязаны на внутренний монитор ключевого слова synchronized, использовать их совместно с объектами Lock нельзя. Вместо них используется специальный интерфейс **java.util.concurrent.locks.Condition**.

Объект условия создается напрямую из экземпляра лока: `Condition condition = lock.newCondition();`.

У него есть точные семантические аналоги для методов ожидания и оповещения:

6. **condition.await()** — аналог `object.wait()`. Текущий поток временно отпускает этот Lock, уходит в ожидание и засыпает. Также есть вариации с тайм-аутами.
7. **condition.signal()** — аналог `object.notify()`. Пробуждает один случайный поток из очереди этого конкретного условия.
8. **condition.signalAll()** — аналог `object.notifyAll()`. Будит все потоки, привязанные к этому условию.

Огромное преимущество Condition перед wait/notify:

У одного объекта Lock вы можете создать **сколько угодно разных объектов Condition**.

Это позволяет разделять потоки ожидания по разным логическим условиям.

Классический пример — реализация блокирующей очереди (Bounded Buffer). Вы можете создать два условия: `Condition notFull = lock.newCondition();` и `Condition notEmpty = lock.newCondition();`. Потоки-производители (Producers) будут засыпать на условии `notFull.await()`, если очередь заполнена, а потоки-потребители (Consumers) — на `notEmpty.await()`. Когда потребитель забирает элемент, он делает точечный вызов `notFull.signal()`, пробуждая именно производителей, и не беспокоя коллег-потребителей. В классическом synchronized все потоки варились бы в одном общем котле ожидания, что приводило бы к лишним пробуждениям.

75. Что такое deadlock и livelock, race condition и starvation?

Написать реализацию deadlock.

Это четыре классические проблемы конкурентного программирования, возникающие при неправильном проектировании многопоточности.

2. **Race Condition (Состояние гонки):** Ситуация, когда два или более потоков одновременно работают с разделяемым ресурсом, и итоговый результат вычислений зависит исключительно от случайного порядка переключения потоков планировщиком ОС. Данные затираются, приложение ведет себя непредсказуемо.
3. **Starvation (Голодание потока):** Проблема, при которой поток полностью исправен и готов выполнять полезную работу, но планировщик задач систематически

обделяет его процессорным временем или доступом к блокировке. Это часто происходит, если у потока выставлен слишком низкий приоритет, или если другие "наглые" потоки постоянно перехватывают нечестный лок прямо перед его носом.

4. **Livelock:** Ситуация, когда потоки не заблокированы мертвым грузом, они активно выполняют код, меняют свои состояния и реагируют на действия друг друга, но при этом программа **не совершает никакой полезной работы** (топчется на месте).
Метафора: два вежливых человека встречаются в узком коридоре, оба одновременно делают шаг вправо, чтобы уступить дорогу, затем оба синхронно делают шаг влево, и так до бесконечности.
5. **Deadlock (Взаимная блокировка):** Ситуация-тупик, когда два или более потоков циклически блокируют друг друга, бесконечно ожидая освобождения ресурсов, удерживаемых соседом.

Реализация классического Deadlock:

Чтобы гарантированно получить дедлок, нам нужно создать два потока и два объекта-блокировки. Поток 1 должен захватить Лок 1 и попытаться взять Лок 2. Поток 2 наоборот: должен сначала захватить Лок 2 и попытаться претендовать на Лок 1.

```
public class DeadlockDemo {
    private static final Object lock1 = new Object();
    private static final Object lock2 = new Object();

    public static void main            {
        // Поток 1
        new Thread() -> {
            synchronized (lock1) {
                System.out.println("Поток 1: Захватил lock1");
                try {
                    Thread.sleep(50); // Пауза, чтобы Поток 2 успел перехватить lock2
                } catch (InterruptedException ignored) {}

                System.out.println("Поток 1: Ожидаю lock2...");
                synchronized (lock2) {
                    System.out.println("Поток 1: Захватил lock2");
                }
            }
        }, "Thread-1").start();

        // Поток 2
        new Thread() -> {
            synchronized (lock2) {
                System.out.println("Поток 2: Захватил lock2");
                try {
```

```

        Thread.sleep(50); // Пауза, чтобы Поток 1 успел перехватить lock1
    } catch (InterruptedException ignored) {}

    System.out.println("Поток 2: Ожидаю lock1...");
    synchronized (lock1) {
        System.out.println("Поток 2: Захватил lock1");
    }
}
}, "Thread-2").start();
}
}

```

При запуске этого кода программа выдаст первые четыре строчки лога и зависнет намертво. Потоки заблокируют друг друга навсегда. Способ предотвращения дедлоков — всегда захватывать множественные блокировки в строго одинаковом порядке во всех потоках приложения.

76. Semaphore, CyclicBarrier, CountdownLatch. Чем похожи на Lock и чем от него отличаются?

Все три утилиты входят в пакет `java.util.concurrent` и относятся к категории высокоуровневых синхронизаторов.

Чем они похожи на Lock:

Под капотом все они (как и `ReentrantLock`) базируются на одном общем мощном фундаменте — абстрактном фреймворке **AQS (AbstractQueuedSynchronizer)**. Они используют состояние (state) и очередь заблокированных потоков для управления доступом к коду.

Чем они отличаются от Lock:

Обычный `Lock` реализует простую бинарную логику исключения: лок либо полностью свободен, либо занят строго одним потоком. Эти три утилиты предлагают гораздо более богатую, специфичную бизнес-логику координации:

3. **Semaphore (Семафор):** Хранит внутри себя счетчик разрешений (permits). Вместо одного потока он позволяет зайти в критическую секцию одновременно **целой группе потоков** (например, не более 5 потоков). Поток вызывает `semaphore.acquire()` (счетчик уменьшается), а при выходе — `semaphore.release()` (счетчик увеличивается). Идеален для ограничения доступа к дефицитным ресурсам, например, для создания пула подключений к базе данных.
4. **CountDownLatch (Защелка со счетчиком):** Это одноразовый синхронизатор-таймер. Он инициализируется числом `$N$` (количеством событий). Потоки, вызывающие метод `latch.await()`, блокируются на этой линии. Другие потоки по мере выполнения задач вызывают `latch.countDown()`, уменьшая счетчик. Как только он дойдет до нуля, защелка распаивается, и все стоящие на ожидании потоки мгновенно продолжают бег. Переиспользовать его нельзя. Пример:

дождаться, пока запустятся все микросервисы системы, перед тем как открывать шлюз для пользователей.

5. **CyclicBarrier (Циклический барьер):** Заставляет фиксированную группу потоков (например, ровно 4 потока) собраться в одной географической точке. Каждый поток выполняет свою часть работы, доходит до барьера и вызывает `barrier.await()`, останавливаясь. Пока все 4 потока не вызовут этот метод, никто дальше не пройдет. Как только последний поток прибыл, они все вместе одновременно разблокируются и бегут дальше. В отличие от `Latch`, он является **многоразовым (циклическим)**: после открытия он автоматически сбрасывает счетчик в исходное состояние и готов координировать следующий шаг алгоритма.

77. Для чего применяется `volatile`? Чем отличается от `AtomicInteger`? Другие классы пакета `java.util.concurrent.atomic`.

Ключевое слово `volatile` и классы атомиков призваны решать проблемы синхронизации с минимальными накладными расходами памяти и процессора, без использования тяжелых блокировок операционной системы.

Для чего применяется `volatile`:

Модификатор `volatile` ставится над переменной класса и решает исключительно проблему **видимости памяти (Memory Visibility)** и перестановки инструкций (Instruction Reordering). Он сообщает компилятору и процессору: "Эту переменную нельзя кэшировать в регистрах ядер процессора. Любое чтение этой переменной должно выполняться напрямую из общей оперативной памяти, а любая запись должна немедленно выталкиваться в общую память". Также он выставляет барьеры памяти (Memory Barriers), запрещающие оптимизаторам менять порядок строк кода вокруг этой переменной.

4. *Главное ограничение:* `volatile` **не гарантирует атомарность** сложных составных операций. Если вы сделаете `public volatile int count = 0;` и запустите из 10 потоков операцию `count++`, вы получите Race Condition, так как инкремент состоит из трех шагов, и `volatile` защищает только финальную запись, но не спасает от прерывания потока между чтением и записью.

Чем отличается от `AtomicInteger`:

`AtomicInteger` обеспечивает и видимость, и **полную атомарность** операций над целым числом. Он не использует ключевое слово `synchronized`. Вместо этого он полагается на низкоуровневые, неблокирующие аппаратные инструкции современных процессоров — **CAS (Compare-And-Swap)**. Метод инкремента `atomicInteger.incrementAndGet()` работает в цикле: он считывает текущее значение, вычисляет новое, а затем одной атомарной инструкцией процессора пытается записать его обратно, проверяя, не изменил ли кто-то переменную за эту микросекунду. Если изменил, CAS возвращает неудачу, и поток делает попытку по кругу еще раз, без ухода в тяжелое состояние `BLOCKED`.

Другие классы пакета `java.util.concurrent.atomic`:

- `AtomicBoolean`, `AtomicLong` — аналоги для логического типа и длинного целого.
- `AtomicReference<V>` — позволяет атомарно обновлять ссылку на любой

произвольный объект.

- `AtomicIntegerArray`, `AtomicReferenceArray` — атомарные массивы, где каждая отдельная ячейка защищена CAS-механизмом.
- `LongAdder` / `DoubleAdder` — современные высокопроизводительные аналоги атомиков для ситуаций безумной конкуренции (когда сотни потоков непрерывно инкрементируют счетчик). Внутри они разделяют переменную на массив внутренних ячеек (stripes), снижая конкуренцию за одну ячейку памяти, а при итоговом запросе значения просто суммируют результаты.

Java. Collections

1. Основные интерфейсы коллекций и их иерархия. Какие бывают коллекции? В чём особенности разных видов коллекций? Когда какие стоит применять? Если в коллекцию часто добавлять элементы, и удалять, какую лучше использовать?

Java Collections Framework (JCF) — это унифицированная архитектура для хранения и манипулирования группами объектов.

На вершине иерархии (для одиночных элементов) находится интерфейс `java.util.Collection`.

От него ответвляются три ключевых суб-интерфейса:

5. **List (Список):** Упорядоченная последовательность элементов. Главная особенность — элементы хранятся в строгом порядке добавления, каждый элемент имеет целочисленный индекс, разрешено хранение любых дубликатов. Основные реализации: `ArrayList` (массив под капотом) и `LinkedList` (связный список).
Применение: Когда критически важно сохранять точный порядок поступления данных и иметь возможность быстро доставать элементы по их порядковому номеру.
6. **Set (Множество):** Коллекция, моделирующая математическое множество. Главная особенность — она **категорически запрещает дубликаты**. Двух одинаковых элементов (с точки зрения контракта методов `equals()` и `hashCode()`) в сете быть не может. Реализации: `HashSet` (хаотичный порядок, супербыстрый), `LinkedHashSet` (помнит порядок добавления), `TreeSet` (хранит элементы в отсортированном виде).
Применение: Задачи фильтрации уникальных объектов (например, список уникальных IP-адресов логга).
7. **Queue / Deque (Очередь / Двухнаправленная очередь):** Спроектированы для удержания элементов перед их непосредственной обработкой. Реализуют стратегии FIFO (First-In-First-Out — первый пришел, первый ушел) или LIFO (Last-In-First-Out). Реализации: `ArrayDeque`, `PriorityQueue` (сортирует элементы по приоритету).

Что выбрать при частом добавлении и удалении элементов?

В академической теории (Big O) считается, что для таких операций идеально подходит `LinkedList`, поскольку вставка или удаление узла из середины списка — это просто переброска двух указателей, занимающая фиксированное время $O(1)$. В то время как у

ArrayList при удалении из середины приходится сдвигать все последующие элементы массива в памяти на одну ячейку влево, что занимает $O(n)$.

Однако **на практике в современной Java всё обстоит иначе**. Чтобы удалить элемент из середины LinkedList, до него сначала нужно *дойти* (сделать поиск по цепочке ссылок от начала списка), а поиск занимает $O(n)$. Кроме того, LinkedList создает под каждый элемент тяжелый объект-ноду, размазанный по всей оперативной памяти. ArrayList хранит данные в сплошном, непрерывном куске памяти. Современные процессоры копируют данные из сплошных массивов в свои кэши (L1/L2) целыми блоками с невероятной скоростью через системные инструкции. Поэтому **в 95% реальных задач ArrayList работает быстрее, чем LinkedList**, даже при регулярных вставках и удалениях. Исключение — если вы работаете строго с краями коллекции (добавление/удаление только в самое начало или самый конец). Для работы с краями идеальным и самым быстрым выбором будет **ArrayDeque**.

2. Что такое интерфейс Map? Представляет ли он коллекцию?

Интерфейс `java.util.Map` представляет собой структуру данных, предназначенную для хранения информации в формате пар "**Ключ — Значение**" (Key — Value), часто называемую ассоциативным массивом или картой/мапой. Каждый ключ в мапе обязан быть абсолютно уникальным, и он может указывать строго на одно конкретное значение. Значения при этом могут спокойно дублироваться.

Представляет ли он коллекцию?

С архитектурной точки зрения — **нет, интерфейс Map не является коллекцией** в строгом смысле этого слова, поскольку он **не наследуется от интерфейса Collection**. У них кардинально разная структура интерфейсных методов. Интерфейс Collection оперирует одиночными элементами (метод `add(E e)`), а Map оперирует парами (метод `put(K key, V value)`).

Тем не менее, Map является полноправной составной частью экосистемы *Java Collections Framework*. Мапа предоставляет удобные механизмы "коллекционных представлений" (Collection Views), которые позволяют смотреть на нее через призму стандартных коллекций:

- `map.keySet()` — возвращает полноценный Set всех уникальных ключей мапы.
- `map.values()` — возвращает коллекцию Collection всех значений мапы (там могут быть дубликаты).
- `map.entrySet()` — возвращает Set пар объектов внутреннего интерфейса `Map.Entry<K, V>`, который уже можно итерировать стандартными циклами.

3. Сравнить ArrayList и LinkedList.

Это классическое сравнение двух принципиально разных подходов к организации списков в памяти.

ArrayList (Список на основе массива):

Внутри себя содержит самый обычный стандартный массив Java (`Object[] elementData`).

- **Плюсы:** Невероятно быстрое чтение и обновление элементов по индексу — операция занимает фиксированное время **$O(1)$** , так как адрес ячейки в памяти вычисляется мгновенно по простой математической формуле. Памяти потребляет

самый минимум (только полезные данные + небольшой запас пустого массива).
Кэш-локален.

- *Минусы:* Если массив заполняется, происходит дорогая операция `grow()` — выделение нового массива большего размера и полное копирование туда всех старых элементов. Вставка или удаление элементов из начала или середины списка заставляют сдвигать сотни элементов в памяти ($O(n)$).

LinkedList (Двусвязный список):

Не имеет внутри никаких массивов. Он состоит из цепочки независимых внутренних объектов-контейнеров (нод) класса `Node<E>`. Каждая нода хранит в себе три вещи: полезный объект, ссылку на следующую ноду и ссылку на предыдущую ноду.

- *Плюсы:* Добавление или удаление элементов в начало или конец списка происходит мгновенно за $O(1)$, так как достаточно просто перетереть ссылки у крайних нод. Коллекция никогда не переполняется и не требует глобальных перевыделений памяти.
- *Минусы:* Катастрофически медленный доступ по индексу ($O(n)$). Чтобы достать 500-й элемент, программе придется буквально с самого начала (или с конца) пройти по цепочке ссылок через 499 предыдущих нод. Огромный расход оперативной памяти: под каждый чих создается объект `Node`, содержащий кучу служебных ссылок-указателей (в 64-битных системах это приводит к сильному раздуванию памяти). Плохо дружит с кэшем процессора из-за фрагментации нод в куче.

4. Сравнить HashMap и Hashtable.

Оба класса реализуют интерфейс `Map` и служат для хранения пар ключ-значение на основе хэш-таблиц, но они принадлежат к разным эпохам развития языка Java.

Hashtable:

Это раритетный, устаревший (legacy) класс, который присутствует в Java с самой первой версии 1.0.

- **Потокобезопасность:** Он потокобезопасен "в лоб". Все его методы (такие как `put`, `get`, `remove`) помечены ключевым словом `synchronized`. Это означает, что при любом обращении к мапе блокируется вся таблица целиком для всех остальных потоков, что порождает дикие просадки по производительности в многопоточной среде.
- **Отношение к null:** Категорически **запрещает** использовать `null` в качестве ключа или в качестве значения. Попытка положить туда нол вызовет `NullPointerException`.

HashMap:

Появился в Java 1.2 как современная, облегченная и быстрая замена `Hashtable`.

5. **Потокобезопасность:** Он полностью **не потокобезопасен**. В его методах нет никакой синхронизации. За счет этого он работает в разы быстрее, так как не тратит ресурсы на захват мониторов. В многопоточной среде его использовать напрямую нельзя.
6. **Отношение к null:** Спокойно и легитимно **разрешает хранить один ключ null** (он всегда принудительно помещается в самый нулевой бакет массива) и сколько угодно значений `null`.

Промышленный стандарт: Класс Hashtable сегодня официально признан устаревшим, его использовать запрещено. Если вам нужна быстрая и потокобезопасная мапа, вместо него используют ConcurrentHashMap. Если многопоточность не нужна — используют HashMap.

5. Особенности интерфейса Set. Сравнить HashSet и TreeSet. Как добавляются объекты в HashSet и TreeSet?

Главная и фундаментальная особенность интерфейса **Set** — обеспечение уникальности данных. Он не предоставляет никакого доступа к элементам по индексам (в отличие от List) и служит исключительно как контейнер, гарантирующий, что внутри нет дубликатов.

Сравнение HashSet и TreeSet:

- **HashSet:** Хранит элементы в хаотичном, абсолютно непредсказуемом порядке. Порядок элементов может внезапно измениться даже при добавлении одной новой записи. Работает невероятно быстро: операции добавления, удаления и проверки наличия (contains) занимают фиксированное время $O(1)$.
- **TreeSet:** Хранит элементы в строго упорядоченном, **отсортированном виде**. Порядок сортировки определяется либо естественным порядком элементов (интерфейс Comparable), либо переданным в конструктор кастомным объектом Comparator. Работает медленнее, чем HashSet: основные операции занимают время $O(\log n)$, так как требуют обхода дерева.

Как добавляются объекты под капотом (Принцип работы):

Обе эти коллекции внутри себя не имеют уникального кода — они являются просто обертками над соответствующими мапами.

- **Добавление в HashSet:** Внутри него живет обычный HashMap. Когда вы вызываете `hashSet.add(object)`, этот объект кладется в скрытую мапу в качестве **Ключа**, а в качестве значения подсовывается невзрачный объект-заглушка (`new Object()`). Соответственно, добавление происходит по законам хэш-таблицы: у объекта вычисляется `hashCode()`, находится нужная корзина (бакет) в массиве. Если в корзине уже кто-то есть, они сравниваются через `equals()`. Если совпадение найдено, старое значение перезаписывается (дубликат не создается), если нет — объект дописывается в бакет. Для корректной работы в HashSet у объектов **обязательно должны быть правильно переопределены методы `hashCode()` и `equals()`**.
- **Добавление в TreeSet:** Внутри него живет скрытый TreeMap, устроенный как сбалансированное красно-черное дерево. Когда вы вызываете `treeSet.add(object)`, объект также ложится туда в качестве ключа. При этом TreeSet **абсолютно плевать на методы `hashCode()` и `equals()`** — он их даже не вызывает. Вместо этого он берет объект и начинает сравнивать его с другими узлами дерева с помощью метода `object1.compareTo(object2)` (или через Comparator). Если метод сравнения возвращает число 0, TreeSet делает вывод: "Эти объекты идентичны, это дубликат!", и отказывается добавлять элемент. Из-за этого может возникнуть баг: если ваш метод `compareTo` написан некорректно и возвращает 0 для разных по логике объектов, TreeSet посчитает их дубликатами и сотрет данные, даже если их `equals`

возвращает false.

6. Принцип работы и реализации HashMap. Изменения HashMap в Java 8.

Класс HashMap устроен как хэш-таблица на основе метода цепочек. Внутри него находится массив бакетов (корзин), где элементами являются объекты внутреннего класса Node<K,V> (или Entry). Каждая такая нода хранит в себе четыре поля: int hash, K key, V value и ссылку Node<K,V> next на следующий элемент.

Алгоритм работы метода put(key, value):

- У ключа берется его hashCode(). Чтобы минимизировать количество плохих хэш-функций, Mockito/Java прогоняет его через дополнительный внутренний метод битового сдвига (хеш-преобразование) для лучшего распределения бит.
- Вычисляется индекс бакета в массиве по формуле: $\text{index} = (n - 1) \& \text{hash}$, где n — текущая длина массива (размер массива всегда кратен степени двойки, что позволяет заменить дорогое деление по модулю % быстрой побитовой операцией &).
- Мы переходим к бакету по вычисленному индексу:
 - Если бакет пуст, туда сразу записывается наша новая нода.
 - Если там уже лежит элемент, произошла **коллизия** (разные ключи попали в одну ячейку массива). HashMap начинает итерироваться по связанному списку, который сформировался в этом бакете. На каждом шаге ключи сравниваются: сначала проверяется равенство хэшей (==), а затем вызывается метод `key.equals(currentNode.key)`.
 - Если совпадение по equals найдено, значение value в этой ноде перезаписывается на новое. Если мы дошли до конца списка и совпадений не нашли, новая нода прикрепляется в самый конец этого связанного списка.
- Если количество элементов в мапе превышает пороговое значение ($\text{Capacity} * \text{Load Factor}$, по умолчанию $16 \times 0.75 = 12$), массив бакетов увеличивается в 2 раза, и происходит полное перераспределение (рехэширование) всех элементов.

Изменения в Java 8 (Революционное улучшение):

До версии Java 8, если в одном бакете из-за плохой хэш-функции выстраивался огромный связный список из сотен элементов, скорость поиска и вставки в мапу катастрофически падала со штатных $O(1)$ до ужасных $O(n)$. Этим даже пользовались хакеры для проведения DoS-атак на сервера (Hash Collision Attack).

Начиная с Java 8, этот алгоритм оптимизировали с помощью **Treeification** (древовидности):

2. Если в одном конкретном бакете количество элементов в связанном списке превышает жесткий порог **TREEIFY_THRESHOLD = 8**, и при этом общий размер всей таблицы бакетов в мапе больше или равен 64, этот бакет автоматически трансформируется.
3. Связный список внутри этой корзины превращается в **сбалансированное красно-черное дерево** (класс нод меняется с Node на TreeNode).
4. Благодаря этому скорость поиска элементов внутри забитого бакета падает с

линейной $O(n)$ до логарифмической **$O(\log n)$** , что гарантирует стабильную скорость работы карты даже при умышленных коллизиях.

5. Если впоследствии в результате удалений количество элементов в этом дереве падает до значения **UNTREEIFY_THRESHOLD = 6**, дерево автоматически разваливается обратно в простой линейный связный список, чтобы сэкономить оперативную память.

7. Чем отличается ArrayList от Vector?

Оба класса реализуют интерфейс List и представляют собой списки, построенные на основе динамически расширяемых массивов. Различия между ними носят исторический и технический характер:

4. **Потокобезопасность:** Класс Vector — это legacy-компонент из Java 1.0. Все его методы до единого синхронизированы с помощью ключевого слова synchronized. Из-за этого он потокобезопасен, но обладает ужасающей производительностью в обычных сценариях, так как одиночный поток постоянно тратит время на захват и освобождение внутренних блокировок. ArrayList появился в Java 1.2, он полностью несинхронизирован и работает максимально быстро.
5. **Стратегия расширения памяти (Resize):** Когда внутренний массив переполняется и требуется выделить новый кусок памяти, они ведут себя по-разному. Vector увеличивает размер своего внутреннего массива ровно **в 2 раза** (на 100%). ArrayList увеличивает размер массива более экономно — **в 1.5 раза** (на 50%). Расчет в ArrayList идет через битовый сдвиг: `int newCapacity = oldCapacity + (oldCapacity >> 1);`.
6. **Способ итерации:** Vector — единственный список, который помимо стандартного Iterator поддерживает древний интерфейс обхода элементов Enumeration.

Резюме: Использовать Vector в современном коде не рекомендуется. Если нужна потокобезопасность списков, берут специализированные классы из пакета concurrent.

8. Коллекции из пакета concurrent. Их особенности.

Пакет java.util.concurrent содержит высокопроизводительные коллекции, специально спроектированные для работы в условиях жесткого многопоточного доступа со стороны сотен параллельных потоков. Стандартные синхронизированные обертки (вроде Collections.synchronizedMap) блокируют всю коллекцию целиком на любое действие, превращая многопоточность в узкое бутылочное горлышко. Коллекции из пакета concurrent используют продвинутые неблокирующие алгоритмы или точечные умные локи.

Основные представители и их фишки:

2. **ConcurrentHashMap:** Король многопоточных коллекций. В Java 8+ он полностью отказался от грубой блокировки сегментов. Чтение (get()) в нем происходит вообще без каких-либо блокировок операционной системы, оно абсолютно свободное и сверхбыстрое. При записи (put()) блокировка вешается не на всю карту, а **строго на один первый головной узел конкретного бакета**, в который пишется элемент, с использованием CAS-инструкций и точечных synchronized. Потоки, пишущие в

разные корзины, вообще не мешают друг другу.

3. **CopyOnWriteArrayList:** Реализует стратегию "копирования при записи". Внутренний массив этой коллекции объявлен как `volatile`. Любые операции чтения происходят без блокировок и работают молниеносно. Но как только любой поток пытается добавить, изменить или удалить элемент, коллекция делает **полную физическую копию всего массива в памяти**, вносит изменения в копию и затем атомарно перебрасывает ссылку на новый массив. *Особенность:* Идеально подходит для сценариев, где чтений терабайты, а записи происходят крайне редко (например, список зарегистрированных плагинов или слушателей событий системы). При частых записях коллекция сожрет всю память.
4. **ConcurrentSkipListMap / ConcurrentSkipListSet:** Потокобезопасные аналоги сортированных коллекций `TreeMap/TreeSet`. Поскольку сбалансированные деревья практически невозможно эффективно синхронизировать точно без блокировки всего дерева, эти классы используют альтернативный алгоритм — **SkipList (Список с пропусками)**. Это многоуровневый связный список, который позволяет искать элементы за $O(\log n)$ и отлично масштабируется под CAS-инструкции многопоточности.

9. Что происходит при добавлении в `ArrayList` нового элемента и как это реализовано.

Давай детально разберем внутреннюю кухню метода `add(E e)` класса `ArrayList`.

4. Вы вызываете метод `list.add(element)`.
5. Внутренний счетчик изменений коллекции `modCount` инкрементируется на 1. Это нужно для механизма *Fail-Fast* итераторов (если другой поток начнет менять список во время вашего перебора, итератор увидит изменение `modCount` и сразу выкинет `ConcurrentModificationException`).
6. Метод вызывает внутреннюю подпрограмму проверки вместимости, передавая туда текущий размер списка + 1 элемент (`size + 1`).
7. Проверяется условие: если `size + 1` превышает физическую длину текущего массива `elementData.length`, запускается механизм расширения — метод **`grow()`**.
8. **Внутри метода `grow()` происходит следующее:**
 - Вычисляется новый размер массива по формуле: `int newCapacity = oldCapacity + (oldCapacity >> 1);`. Битовый сдвиг `>> 1` — это быстрое деление числа на 2 на уровне процессора. То есть новый массив будет ровно на 50% больше предыдущего. (Если список был пуст, ему дается дефолтный стартовый размер **10**).
 - Проверяется, не превысил ли новый размер максимально допустимый предел для массивов в JVM (`Integer.MAX_VALUE - 8`).
 - Создается абсолютно **новый физический массив** в куче с типом `Object[]` и размером `newCapacity`.
 - Происходит копирование всех старых элементов из заполненного массива в новый. Это реализовано через сверхбыстрый нативный метод `System.arraycopy()` (или `Arrays.copyOf()`), который копирует куски памяти напрямую на уровне ОС.

- Старая ссылка `elementData` перенаправляется на новый массив, а старый массив отправляется на съедение сборщику мусора (Garbage Collector).
- 9. После того как проверка вместимости завершена (и массив при необходимости расширен), элемент гарантированно записывается в ячейку по индексу:
`elementData[size] = element;`
- 10. Внутренний логический размер списка инкрементируется: `size++;`
- 11. Метод возвращает `true`, сигнализируя об успешном добавлении.

10. Методы для преобразования потоко-небезопасной коллекции в потоко-безопасную.

Если у вас уже есть готовая стандартная коллекция (например, обычный `ArrayList` или `HashMap`), и вам нужно экстренно сделать ее доступной для работы из нескольких потоков, утилитарный класс **`java.util.Collections`** предоставляет семейство фабричных методов-декораторов:

- `Collections.synchronizedList(List<T> list)`
- `Collections.synchronizedMap(Map<K,V> map)`
- `Collections.synchronizedSet(Set<T> set)`
- `Collections.synchronizedCollection(Collection<T> c)`

Как это реализовано изнутри:

Эти методы не переписывают структуру вашей коллекции. Они возвращают специальный объект-обертку (прокси), который реализует тот же самый интерфейс. Внутри этой обертки хранится ссылка на вашу исходную коллекцию, а абсолютно **все методы обернуты в блоки `synchronized`** по монитору самой этой обертки (или специального внутреннего объекта-мутекса).

```
// Пример внутренней реализации метода get() в синхронизированной обертке:
public get int    {
    synchronized (mutex) { // Блокировка перед любым действием
        return list.get(index); // Вызов реальной небезопасной коллекции
    }
}
```

Важнейший подводный камень для собеседования: Несмотря на то, что методы обертки синхронизированы, **итерирование (перебор через цикл `for-each` или `Iterator`) такой коллекции всё равно остается абсолютно потоко-небезопасным!** Если один поток бежит по итератору такого списка, а второй в этот момент вызовет синхронизированный метод `add()`, программа упадет с `ConcurrentModificationException`. Поэтому при итерации

по синхронизированным коллекциям разработчик обязан оборачивать весь цикл в блок `synchronized` вручную:

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
// ...
synchronized (syncList) { // Ручная блокировка всего процесса перебора!
    for (String item : syncList) {
        System.out.println(item);
    }
}
```

Functional Programming & Stream API

11. Функциональные интерфейсы. Определение. Default & static методы. Область применения.

Функциональный интерфейс — это интерфейс, который содержит ровно один абстрактный метод (Single Abstract Method, SAM). При этом он может содержать любое количество дефолтных (default) и статических (static) методов, а также переопределения методов класса `Object` (например, `equals()` или `hashCode()`), которые не увеличивают счетчик абстрактных методов. Чтобы компилятор мог проверить интерфейс на соответствие этому правилу, применяется аннотация `@FunctionalInterface`, хотя её использование не является обязательным.

Дефолтные методы были введены в Java 8 для обеспечения обратной совместимости. Они позволяют добавлять новую функциональность в уже существующие интерфейсы без необходимости переписывать все классы, которые эти интерфейсы реализуют. Классы-наследники автоматически получают реализацию дефолтных методов, но при желании могут их переопределить. Статические методы в интерфейсах работают точно так же, как статические методы в обычных классах, и привязаны к самому интерфейсу, а не к его экземплярам. Они используются как утилитарные методы, помогающие организовать логику, тесно связанную с данным интерфейсом.

Область применения функциональных интерфейсов — это прежде всего функциональное программирование, создание лямбда-выражений и ссылок на методы. Они служат целевыми типами для лямбд, позволяя передавать поведение как аргументы в методы, конструировать цепочки обработки данных в Stream API и реализовывать паттерны проектирования (такие как Стратегия или Команда) с минимальным количеством шаблонного кода.

12. Лямбда-выражение. Замыкания. Синтаксис. Характеристики.

Лямбда-выражение представляет собой анонимную функцию, то есть блок кода, который можно передавать в качестве аргумента или сохранять в переменную. Лямбда-выражения

не существуют сами по себе, они всегда компилируются в реализацию конкретного функционального интерфейса.

Синтаксис лямбда-выражения состоит из трех частей: списка параметров, стрелки `->` и тела выражения. Если параметров несколько, они перечисляются в круглых скобках (a, b) `-> ....` Если параметр один, скобки можно опустить. Типы параметров обычно выводятся компилятором автоматически, но их можно указать явным образом. Тело лямбды может состоять из одного выражения (тогда фигурные скобки и ключевое слово `return` не нужны) или из полноценного блока кода в фигурных скобках, где `return` обязателен, если метод должен возвращать значение.

Замыкание (Closure) — это способность лямбда-выражения обращаться к переменным из той области видимости, в которой оно было создано. В Java этот механизм имеет важную особенность: лямбда-выражение может использовать локальные переменные окружающего контекста только в том случае, если они являются финальными (`final`) или фактически финальными (`effectively final`). Это означает, что значение переменной не должно изменяться после её инициализации. Такое ограничение связано с тем, как Java управляет памятью: локальные переменные хранятся в стеке и уничтожаются после выхода из метода, в то время как лямбда-выражение может быть выполнено позже в другом потоке. Поэтому Java делает копию этой переменной для использования внутри лямбды, а чтобы копия и оригинал всегда совпадали, их запрещено изменять.

13. Основные интерфейсы: **Function, Supplier, Predicate, Consumer.** Их применение.

В пакете `java.util.function` сосредоточены стандартные функциональные интерфейсы, покрывающие большинство типовых сценариев:

- **Predicate** принимает объект типа `T`, проверяет некоторое условие и возвращает логическое значение `boolean` (метод `test(T t)`). Он активно используется для фильтрации данных, например, в методе `filter()` встроенного `Stream API`.
- **Consumer** принимает объект типа `T`, выполняет над ним определенные действия, но ничего не возвращает (метод `accept(T t)`). Его типичное применение — вывод данных на экран, сохранение в базу или логирование, как в методе `forEach()`.
- **Function<T, R>** представляет собой классическую функцию, которая принимает объект типа `T` и преобразует его в объект типа `R` (метод `apply(T t)`). Этот интерфейс незаменим для трансформации данных, например, в методе `map()` для перевода объектов из одного типа в другой.
- **Supplier** не принимает никаких аргументов, но возвращает объект типа `T` (метод `get()`). Он используется для ленивой генерации данных, создания фабрик объектов или предоставления дефолтных значений, например, в методах класса `Optional`, таких как `orElseGet()`.

14. Ссылка на метод? Что это такое? Или это все же ссылка на объект?

Ссылка на метод (Method Reference) — это синтаксический сахар, который позволяет

записывать лямбда-выражения еще более компактно в тех случаях, когда лямбда просто вызывает уже существующий метод без каких-либо дополнительных преобразований параметров. Записывается она с помощью двойного двоеточия :: (например, `System.out::println` или `String::toUpperCase`).

Несмотря на слово «ссылка» в названии, это не является ссылкой на метод в понимании низкоуровневых языков (как указатель на функцию в C++). При компиляции ссылка на метод превращается в полноценный экземпляр класса, реализующего соответствующий функциональный интерфейс. Таким образом, в рантайме это всегда объект. Ссылки на методы бывают четырех видов: ссылки на статические методы (`ClassName::staticMethodName`), ссылки на методы конкретного существующего объекта (`instance::instanceMethodName`), ссылки на методы типа произвольного объекта (`ClassName::instanceMethodName`) и ссылки на конструкторы (`ClassName::new`).

15. Чем Stream отличается от коллекции?

Коллекция и поток данных (Stream) решают принципиально разные задачи, хотя часто работают вместе. Главное отличие заключается в том, что коллекция — это структура данных, ориентированная на хранение и управление доступом к элементам в памяти. Коллекции содержат конкретные значения, вычисления над которыми уже завершены. Поток (Stream) не является структурой данных и сам по себе ничего не хранит. Это инструмент для декларативного описания конвейера вычислений над данными, которые поступают из источника (которым может быть коллекция, массив или генератор).

Основные отличия:

5. **Отсутствие хранения:** Стрим не занимает память под элементы данных, он лишь передает их по цепочке операций.
6. **Иммутабельность:** Стрим не изменяет свой источник. Например, фильтрация стрима, созданного из списка, создаст новый поток отфильтрованных данных, но исходный список останется прежним.
7. **Одноразовость:** Стрим можно обойти только один раз. После вызова терминальной операции стрим закрывается, и повторно использовать его нельзя.
8. **Ленивость вычислений:** Стрим выполняет вычисления только тогда, когда это действительно необходимо (при вызове терминальной операции).

16. Промежуточные и терминальные операции. Lazy evaluation.

Все операции в Stream API строго разделяются на две группы: промежуточные (intermediate) и терминальные (terminal).

Промежуточные операции (такие как `filter()`, `map()`, `sorted()`, `distinct()`) всегда возвращают новый стрим. Они являются ленивыми (lazy evaluation). Это означает, что при вызове промежуточной операции никакой реальной обработки элементов не происходит. Вместо этого в памяти просто строится рецепт или план будущих вычислений. Вы можете составить цепочку из десятка промежуточных операций, но процессорное время на обработку элементов не будет затрачено.

Терминальные операции (такие как `forEach()`, `collect()`, `reduce()`, `count()`, `findFirst()`) завершают работу стрима. Они возвращают конкретный результат (коллекцию, число,

объект или вообще ничего в случае void). Именно вызов терминальной операции служит триггером, который запускает реальное движение данных по всему конвейеру. Благодаря ленивости оптимизируется производительность: например, если в конце стоит `findFirst()`, стрим прекратит обработку данных, как только будет найден первый подходящий элемент, даже если исходная коллекция содержала миллионы записей.

17. Методы: `map()` vs `flatMap()`

Оба метода используются для трансформации элементов в стриме, но характер этой трансформации принципиально различается.

Метод `map()` применяет функцию преобразования к каждому отдельному элементу и возвращает стрим, состоящий из результатов этих преобразований. Это трансформация «один в один». Если у вас был стрим строк, и вы применили `map(String::length)`, вы получите стрим чисел той же длины, где каждая строка заменилась на её длину.

Метод `flatMap()` используется тогда, когда каждый элемент исходного стрима сам по себе может быть развернут в целый поток других элементов (например, объект содержит в себе коллекцию). `flatMap()` сначала трансформирует каждый элемент в отдельный стрим, а затем объединяет (сплюсчивает) все эти получившиеся стримы в один единый плоский стрим. Это трансформация «один ко многим». Если у вас есть стрим объектов `User`, и у каждого пользователя есть список телефонов `List<String>`, то использование `map(User::getPhones)` вернет вам тяжелую структуру `Stream<List<String>>`. А использование `flatMap(user -> user.getPhones().stream())` объединит все телефоны всех пользователей в один линейный поток `Stream<String>`.

JDBC

18. Как создать `Connection`? Последовательность действий необходимых для выполнения запроса к БД. Как правильно закрыть `Connection`?

Для создания соединения (`Connection`) современный стандарт Java рекомендует использовать интерфейс `DataSource` (обычно предоставляемый пулом соединений, например `HikariCP`). В более старых приложениях или учебных примерах соединение создается напрямую через класс `DriverManager` с передачей строки подключения (URL), логина и пароля с помощью метода `DriverManager.getConnection(url, user, password)`.

Последовательность действий для выполнения запроса выглядит следующим образом:

3. Получение объекта `Connection` из драйвера или пула соединений.
4. Создание объекта запроса (`Statement` или `PreparedStatement`) через методы соединения.
5. Определение текста SQL-запроса и, если используется `PreparedStatement`, установка параметров.
6. Выполнение запроса с помощью методов `executeQuery()` (для выборки данных) или `executeUpdate()` (для изменения данных).
7. Обработка результатов, если выполнялась выборка (итерация по объекту `ResultSet`).

8. Последовательное закрытие всех открытых ресурсов.

Правильное закрытие ресурсов критически важно, так как незакрытые соединения приводят к утечкам памяти и исчерпанию лимитов подключений к СУБД. Ресурсы должны закрываться в порядке, обратном их открытию: сначала `ResultSet`, затем `Statement`, и в конце `Connection`. Наиболее надежным и современным способом является использование конструкции **try-with-resources** (начиная с Java 7). Все эти интерфейсы расширяют `AutoCloseable`, поэтому Java автоматически закроет их при выходе из блока `try`, даже если внутри возникнет исключение.

```
try (Connection conn = dataSource.getConnection();
    PreparedStatement stmt = conn.prepareStatement("SELECT * FROM users WHERE id = ?"))
{
    stmt.setLong(1, userId);
    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            // обработка данных
        }
    }
} catch (SQLException e) {
    // обработка ошибок
}
```

19. Чем отличается `Statement` от `PreparedStatement`? Где сохраняется запрос после первого вызова `PreparedStatement`? Будет ли тот же самый эффект как и от `PreparedStatement`, если сформировать запрос просто в строке и отправлять его в `Statement`?

Интерфейс `Statement` используется для выполнения простых статических SQL-запросов без параметров. Каждый раз, когда такой запрос отправляется в базу данных, СУБД вынуждена заново проходить все этапы: синтаксический анализ, компиляцию, оптимизацию и построение плана выполнения.

`PreparedStatement` представляет собой предварительно скомпилированный SQL-запрос. При его создании СУБД заранее компилирует шаблон запроса, в котором вместо конкретных значений стоят знаки вопроса ? (плейсхолдеры). После первого вызова этот скомпилированный запрос и его план выполнения кэшируются. Место хранения этого

кэша зависит от настроек: он может сохраняться как на стороне самой базы данных (в кэше планов СУБД), так и на стороне JDBC-драйвера в памяти приложения. При повторных вызовах с другими параметрами база данных не тратит ресурсы на компиляцию, а сразу подставляет новые значения в готовый шаблон.

Если формировать запрос динамически путем конкатенации строк (например, "SELECT * FROM users WHERE name = " + userName + " "), а затем отправлять его в обычный Statement, **эффекта скомпилированности и безопасности не будет**. С точки зрения базы данных, каждая строка с новым именем пользователя — это абсолютно новый уникальный запрос, который нужно компилировать с нуля, что приведет к деградации производительности. Более того, такой подход открывает критическую уязвимость перед SQL-инъекциями, поскольку переданная строка никак не экранируется.

20. Зачем нужен CallableStatement?

Интерфейс CallableStatement расширяет возможности PreparedStatement и предназначен специально для вызова хранимых процедур и функций, которые уже созданы и находятся внутри самой базы данных.

Его ключевая особенность заключается в том, что он умеет работать не только со входящими параметрами (IN), но и с исходящими параметрами (OUT) или комбинированными (INOUT). Перед выполнением запроса разработчик должен явно зарегистрировать типы исходящих параметров СУБД с помощью методов класса, таких как registerOutParameter(). После выполнения процедуры значения этих параметров можно считать в переменные Java. CallableStatement позволяет перенести сложную бизнес-логику обработки данных на сторону СУБД, снижая объем сетевого трафика между приложением и сервером базы данных.

21. Отличие executeUpdate() от executeQuery().

Оба метода используются для отправки команд в базу данных, но они рассчитаны на разные типы SQL-операций и возвращают разные результаты.

Метод executeQuery() предназначен исключительно для запросов, которые извлекают данные из базы (команда SELECT). Он возвращает объект ResultSet, представляющий собой табличный набор данных, по которому можно перемещаться с помощью курсора для чтения строк. Если попытаться вызвать этот метод для команды, изменяющей данные, будет выброшено исключение SQLException.

Метод executeUpdate() используется для SQL-команд, которые модифицируют состояние базы данных: INSERT, UPDATE, DELETE, а также для DDL-команд (например, CREATE TABLE, DROP TABLE). Этот метод возвращает простое целое число (int), которое указывает количество строк, подвергшихся изменению в результате выполнения этой операции. Для DDL-команд метод всегда возвращает 0.

22. Транзакции. Принципы ACID.

Транзакция — это логическая единица работы с базой данных, состоящая из одного или нескольких запросов, которая должна быть выполнена целиком или не выполнена вовсе.

Управление транзакциями в JDBC осуществляется через отключение режима автоматической фиксации: `connection.setAutoCommit(false)`, после чего изменения фиксируются методом `commit()` или отменяются методом `rollback()`.

Все транзакции должны строго удовлетворять набору принципов **ACID**:

7. **Atomicity (Атомарность)**: Гарантирует, что транзакция будет выполнена полностью или не выполнена вообще. Если хотя бы один запрос внутри транзакции завершается ошибкой, все предыдущие изменения откатываются, возвращая базу в исходное состояние.
8. **Consistency (Согласованность)**: Гарантирует, что транзакция переводит базу данных из одного согласованного состояния в другое согласованное состояние. Все правила, ограничения целостности (`foreign keys`, уникальные индексы) должны строго соблюдаться после завершения транзакции.
9. **Isolation (Изоляция)**: Гарантирует, что параллельно выполняющиеся транзакции не должны влиять друг на друга. Результат выполнения нескольких одновременных транзакций должен быть таким же, как если бы они выполнялись строго по очереди.
10. **Durability (Стойкость/Долговечность)**: Гарантирует, что если транзакция была успешно завершена (`commit`), внесенные ею изменения сохраняются в системе надежно (на энергонезависимом носителе), и они не будут потеряны даже в случае внезапного сбоя питания или падения сервера СУБД.

23. Уровни изоляции транзакций. Виды чтения. Точки сохранения.

Для настройки баланса между скоростью параллельной работы и строгой изоляцией данных применяются четыре уровня изоляции транзакций, определенных в стандарте SQL. Они борются со следующими феноменами (видами чтения):

- **Грязное чтение (Dirty Read)**: Транзакция А читает данные, которые были изменены транзакцией Б, но еще не зафиксированы (`commit`). Если Б сделает откат, данные окажутся ложными.
- **Неповторяющееся чтение (Non-repeatable Read)**: Транзакция А читает одну и ту же строку дважды внутри своей работы, но между чтениями транзакция Б изменяет эту строку и фиксирует изменения. А видит разные данные.
- **Фантомное чтение (Phantom Read)**: Транзакция А делает выборку строк по условию. Транзакция Б вставляет новые строки, удовлетворяющие этому условию. При повторной выборке А видит новые «фантомные» строки.

Уровни изоляции:

- **Read Uncommitted (Чтение незафиксированных данных)**: Низший уровень. Допускает все виды чтения. Высокая скорость, нулевая надежность.
- **Read Committed (Чтение зафиксированных данных)**: Запрещает грязное чтение. Транзакция видит только подтвержденные данные. Возможны неповторяющееся и фантомное чтения. Дефолтный уровень для большинства СУБД (PostgreSQL, Oracle).
- **Repeatable Read (Повторяемость чтения)**: Запрещает грязное и неповторяющееся чтение. Блокирует читаемые строки. Фантомные строки все еще могут появляться (в ряде СУБД, таких как MySQL InnoDB, этот уровень защищает и от фантомов).
- **Serializable (Сериализуемость)**: Высший уровень. Полная изоляция, транзакции

выполняются словно последовательно. Исключает все аномалии, но сильно снижает производительность из-за обилия блокировок.

Точки сохранения (Savepoints) позволяют разделить большую транзакцию на мелкие логические этапы. С помощью `connection.setSavepoint()` можно создать промежуточную точку, а затем, если что-то пошло не так, сделать `connection.rollback(savepoint)`. Это откатит только ту часть изменений, которая была сделана после создания данной точки сохранения, позволяя продолжить транзакцию без полной отмены всей предыдущей работы.

24. SQL injection. Как работает и как защищаться?

SQL-инъекция — это уязвимость, возникающая тогда, когда пользовательские данные внедряются непосредственно в текст SQL-запроса путем банальной склейки (конкатенации) строк.

Как это работает: Представим запрос авторизации: `"SELECT * FROM users WHERE login = " + inputLogin + " AND password = " + inputPassword + ""`. Если злоумышленник в поле ввода логина напишет строку `' OR '1'='1'`, то итоговый запрос превратится в: `SELECT * FROM users WHERE login = " OR '1'='1' AND password = '...'`. Условие `'1'='1'` всегда истинно, логика проверки пароля обходится, и злоумышленник получает несанкционированный доступ к первому попавшемуся пользователю (обычно это администратор). В худшем случае через точку с запятой можно дописать команду удаления таблиц: `DROP TABLE users;`.

Основной и самый надежный способ защиты — **категорический отказ от конкатенации строк при формировании SQL** и обязательное использование `PreparedStatement`. Когда мы используем параметры `?`, JDBC-драйвер и СУБД обрабатывают переданные значения строго как литералы (данные), а не как исполняемый код. Даже если передать в параметр строку `' OR '1'='1'`, база данных будет искать пользователя, у которого логин буквально равен этой строке, и атака полностью провалится. Дополнительными барьерами служат использование ORM-фреймворков (Hibernate/Spring Data JPA, которые под капотом используют те же параметры), валидация входящих данных и минимизация прав пользователя СУБД, под которым приложение подключается к базе данных.

SQL

1. Что такое нормализация?

Нормализация — это пошаговый процесс организации структуры реляционной базы данных с целью минимизации избыточности данных, устранения потенциальных аномалий (при вставке, обновлении и удалении строк) и обеспечения логической целостности хранения. В процессе нормализации таблицы разбиваются на более мелкие структуры, между которыми устанавливаются логические связи.

Обычно выделяют три основные нормальные формы, которых достаточно для большинства реальных проектов:

- **Первая нормальная форма (1NF):** Требуется, чтобы все атрибуты (поля) таблицы были атомарными (неделимыми), а строки — уникальными. В ячейках не должно быть списков или массивов значений.

- **Вторая нормальная форма (2NF):** Таблица должна находиться в 1NF, и каждый её неключевой атрибут должен полностью зависеть от всего первичного ключа целиком, а не от его части (это актуально для составных первичных ключей).
- **Третья нормальная форма (3NF):** Таблица должна находиться в 2NF, и ни один неключевой атрибут не должен транзитивно зависеть от первичного ключа. Это значит, что неключевые поля не должны зависеть от других неключевых полей, они должны подчиняться исключительно первичному ключу.

2. Какие есть типы связей в базе данных? Как построить связь «many-to-many».

В реляционных базах данных существуют три основных типа логических связей между таблицами:

- **Один к одному (1:1):** Каждой записи в таблице А соответствует ровно одна запись в таблице Б (например, Пользователь и его Профиль). Обычно реализуется путем помещения первичного ключа одной таблицы в качестве внешнего ключа в другую с уникальным индексом (UNIQUE).
- **Один ко многим (1:M):** Одной записи в таблице А может соответствовать множество записей в таблице Б (например, Автор и его Книги). Реализуется добавлением внешнего ключа (FOREIGN KEY) в таблицу на стороне «многие» (в таблицу Книг добавляется ID автора).
- **Многие ко многим (M:M):** Одной записи в таблице А может соответствовать множество записей в таблице Б, и наоборот (например, Студенты и Курсы).

Напрямую в реляционных СУБД связать две таблицы как «многие ко многим» невозможно.

Чтобы построить связь **many-to-many**, создается специальная промежуточная (связующая, ассоциативная) таблица. Она содержит как минимум два поля, которые являются внешними ключами (FOREIGN KEY), указывающими на первичные ключи обеих связываемых таблиц. Вместе эти два внешних ключа обычно формируют составной первичный ключ промежуточной таблицы, гарантируя уникальность связей. При необходимости в эту же таблицу можно добавлять дополнительные атрибуты связи (например, дата записи студента на курс).

3. Что такое primary key (первичный ключ)?

Первичный ключ (Primary Key) — это столбец или подмножество столбцов, которое уникально идентифицирует каждую строку в таблице базы данных. В одной таблице может быть только один первичный ключ.

Основные характеристики первичного ключа:

- **Уникальность:** Значение ключа не должно повторяться ни в одной другой строке этой таблицы.
- **Запрет на NULL:** Поля, входящие в состав первичного ключа, принципиально не могут содержать пустые значения (NOT NULL).
- **Неизменяемость:** Значения первичного ключа не должны меняться со временем, так как на них могут ссылаться другие таблицы.

Первичный ключ может быть *естественным* (выбранным из существующих данных, имеющих уникальную природу, например номер паспорта, хотя это считается плохой практикой) или *суррогатным* (искусственно сгенерированное поле, например автоинкрементное число id или UUID). По первичному ключу СУБД автоматически создает уникальный кластерный индекс, что обеспечивает мгновенный поиск строки.

4. Что такое foreign key (внешний ключ)?

Внешний ключ (Foreign Key) — это столбец или группа столбцов в одной таблице (дочерней), которая ссылается на первичный или уникальный ключ другой таблицы (родительской). Это фундаментальный механизм реляционных СУБД, обеспечивающий поддержание ссылочной целостности данных.

Назначение внешнего ключа — гарантировать, что дочерняя таблица не будет содержать записей, ссылающихся на несуществующие данные в родительской таблице. Если в таблице Orders есть поле user_id, объявленное как FOREIGN KEY к таблице Users, СУБД заблокирует попытку вставить заказ с несуществующим ID пользователя. Также внешний ключ определяет поведение системы при удалении или изменении родительских записей с помощью опций: CASCADE (автоматически удалять/обновлять связанные строки), SET NULL (проставить пустые значения в дочерней таблице) или RESTRICT/NO ACTION (запретить удаление родительской строки, пока на нее кто-то ссылается).

5. Что такое индексы в базе данных? Для чего их используют?

Достоинства и недостатки? Где хранятся?

Индекс в базе данных — это вспомогательная структура данных (чаще всего организованная в виде сбалансированного дерева B-Tree или хэш-таблицы), создаваемая для повышения скорости поиска и извлечения строк из таблицы. Без индекса СУБД вынуждена выполнять полное сканирование всей таблицы (Full Table Scan) строка за строкой, что крайне неэффективно на больших объемах данных. Индекс же работает как алфавитный указатель в конце массивной книги, позволяя сразу перейти к нужной странице.

Достоинства:

11. Многократное ускорение выполнения запросов выборки (SELECT), содержащих условия WHERE, соединения JOIN или сортировку ORDER BY.
12. Повышение эффективности работы ограничений уникальности.

Недостатки:

- **Замедление операций модификации данных (INSERT, UPDATE, DELETE).** При любом изменении данных в таблице СУБД обязана автоматически перестраивать и обновлять связанные с ней индексы, что требует дополнительных вычислительных ресурсов.
- **Дополнительные затраты памяти.** Индексы физически занимают дисковое пространство (и оперативную память для кэширования).

Хранятся индексы в файловой системе сервера СУБД, обычно в тех же табличных пространствах (Tablespaces), что и сами таблицы, но в виде отдельных служебных файлов

или сегментов данных, изолированных от основного тела таблиц.

6. Какие есть типы JOIN'ов. Кратко опишите каждый из типов.

Оператор JOIN применяется для горизонтального объединения данных из двух и более таблиц на основе совпадения значений в связанных столбцах. Существуют следующие основные типы:

10. **INNER JOIN (Внутреннее объединение):** Возвращает только те строки, для которых нашлось взаимное соответствие в обеих таблицах. Если строке из таблицы А нет пары в Б, она отбрасывается.
11. **LEFT JOIN / LEFT OUTER JOIN (Левое внешнее объединение):** Возвращает абсолютно все строки из левой таблицы, совмещая их со строками из правой таблицы при наличии совпадений. Если совпадений в правой таблице нет, для её столбцов подставляются значения NULL.
12. **RIGHT JOIN / RIGHT OUTER JOIN (Правое внешнее объединение):** Зеркальный аналог левого соединения. Возвращает все строки из правой таблицы, а для несовпавших элементов левой таблицы выводит NULL.
13. **FULL JOIN / FULL OUTER JOIN (Полное внешнее объединение):** Объединяет результаты левого и правого соединений. Выводит все строки из обеих таблиц; там, где соответствия не нашлось, незаполненные поля заменяются на NULL.
14. **CROSS JOIN (Перекрестное объединение):** Производит декартово произведение таблиц. Каждая строка первой таблицы соединяется с каждой строкой второй таблицы без каких-либо условий. Результатом является таблица, длина которой равна произведению количества строк исходных таблиц.

7. Для чего используется слово HAVING? Отличия от WHERE.

Оба ключевых слова используются для фильтрации данных, но они применяются на совершенно разных этапах обработки SQL-запроса СУБД.

Ключевое слово WHERE выполняет фильтрацию строк **до того**, как начнется их группировка и агрегации. Оно отсекает лишние данные на самом раннем этапе. Внутри WHERE категорически запрещено использовать агрегирующие функции (такие как SUM(), COUNT(), AVG()), поскольку на момент работы этого фильтра никаких групп еще не существует.

Ключевое слово HAVING выполняет фильтрацию результирующих групп **после того**, как была выполнена группировка строк с помощью оператора GROUP BY. Оно предназначено именно для работы с результатами агрегирующих функций. Например, если нам нужно найти отделы компании, где средняя зарплата сотрудников превышает определенный лимит, мы обязаны использовать HAVING AVG(salary) > 50000, так как WHERE такое условие обработать не сможет.

8. Зачем нужно View и какие поля там будут?

Представление (View) — это виртуальная таблица, которая не хранит физические данные на диске, а представляет собой сохраненный в базе данных именованный SQL-запрос

SELECT. Каждый раз, когда приложение обращается к View так, словно это обычная таблица, СУБД под капотом выполняет этот скрытый запрос на лету.

Зачем нужно View:

- **Упрощение сложных запросов:** Позволяет спрятать громоздкие объединения десятка таблиц и сложные вычисления за простым и понятным интерфейсом.
- **Безопасность и ограничение доступа:** Можно дать пользователю права на чтение конкретного View, содержащего только безопасные столбцы, полностью закрыв от него доступ к исходным базовым таблицам с конфиденциальной информацией.
- **Независимость данных:** Позволяет менять структуру реальных таблиц без необходимости переписывать код клиентского приложения, достаточно просто переопределить View.

Набор полей, которые будут присутствовать во View, полностью и строго определяется списком столбцов, перечисленных в секции SELECT того SQL-запроса, на основе которого это представление было создано. Поля могут иметь те же имена, что и в базовых таблицах, или переименовываться с помощью алиасов AS.

9. Что такое агрегирующая функция, примеры?

Агрегирующая функция — это функция, которая принимает в качестве входных данных множество значений из определенного столбца таблицы (по всем строкам или по конкретной сгруппированной группе) и выполняет над ними математическое вычисление, сворачивая это множество в одно единственное результирующее значение.

Основные примеры встроенных агрегирующих функций:

7. COUNT() — подсчитывает общее количество строк или ненулевых значений в группе.
8. SUM() — вычисляет арифметическую сумму всех числовых значений в столбце.
9. AVG() — находит среднее арифметическое значение числового ряда.
10. MIN() — определяет минимальное значение (работает как с числами, так и с датами или строками).
11. MAX() — определяет максимальное значение в наборе данных.

Агрегирующие функции чаще всего используются совместно с предложением GROUP BY, разбивающим таблицу на логические подгруппы, для которых вычисления проводятся изолированно.

10. Что такое query execution plan и как его посмотреть?

План выполнения запроса (Query Execution Plan) — это подробная технологическая карта, составленная внутренним оптимизатором СУБД, описывающая точную последовательность низкоуровневых шагов, которые база данных предпримет для физического извлечения запрашиваемых данных. План показывает, какие именно индексы будут задействованы, в каком порядке будут соединяться таблицы, будут ли производиться полные сканирования диска (Seq Scan или Full Table Scan) и сколько процессорного времени и памяти предположительно уйдет на каждый этап. Анализ плана необходим для оптимизации медленных запросов.

Чтобы посмотреть план выполнения, перед текстом вашего целевого SQL-запроса

необходимо добавить специальную команду. В большинстве реляционных баз данных (PostgreSQL, MySQL, Oracle) этой командой является слово **EXPLAIN**.

```
EXPLAIN SELECT FROM users WHERE email = 'test@example.com';
```

Для получения не просто теоретического прогноза, а реальных замеров времени выполнения и количества обработанных строк непосредственно в процессе работы запроса, в PostgreSQL используется команда **EXPLAIN ANALYZE** (в MySQL — **EXPLAIN ANALYZE** или **EXPLAIN EXTENDED**). В графических инструментах управления базами данных (таких как pgAdmin, DBeaver или DataGrip) план выполнения можно визуализировать в виде удобных интерактивных блок-схем.

Design Patterns

1. Зачем нужны паттерны? Привести примеры из проекта.

Паттерны (шаблоны) проектирования — это проверенные временем, типизированные архитектурные решения часто возникающих задач в сфере проектирования объектно-ориентированного программного обеспечения. Паттерн не является готовым кодом, который можно скопировать; это концептуальное описание или схема взаимодействия классов и объектов.

Паттерны нужны для:

7. Ускорения процесса разработки за счет использования готовых архитектурных подходов вместо «изобретения велосипедов».
8. Повышения гибкости, расширяемости и сопровождаемости кода (многие паттерны напрямую реализуют принципы SOLID).
9. Формирования единого профессионального словаря среди разработчиков: фраза «здесь мы применим Фабрику и Наблюдатель» сразу понятна всей команде без долгих объяснений кода.

Примеры из реального проекта:

В веб-приложениях паттерны встречаются повсеместно. Архитектура доступа к данным строится на паттернах **Repository** и **DAO**. При отправке email-уведомлений пользователям применяется паттерн **Strategy**, когда в зависимости от настроек подставляется конкретный движок отправки (AWS SES, SMTP или тестовый стаб). При сборке тяжелых DTO или конфигурационных объектов для тестов всегда используется **Builder**.

2. Какие паттерны вы знаете и как их применяет Java SE.

Платформа Java SE (Standard Edition) спроектирована с глубоким применением классических паттернов проектирования. Вот ключевые примеры:

- **Decorator (Декоратор):** Ярко выражен в подсистеме ввода-вывода `java.io`. Классы наподобие `BufferedReader` или `BufferedInputStream` принимают в конструктор базовые потоки (`Reader`, `InputStream`) и динамически расширяют их функционал, добавляя буферизацию.
- **Singleton (Одиночка):** Напрямую применяется в классе `java.lang.Runtime.getRuntime()`, гарантируя единую точку управления средой выполнения Java для всего приложения.
- **Strategy (Стратегия):** Интерфейс `java.util.Comparator` и метод `Collections.sort(list, comparator)`. Передавая различные компараторы, мы на лету меняем алгоритм сравнения и сортировки элементов.
- **Iterator (Итератор):** Реализован во всех классах Java Collection Framework через метод `iterator()`, предоставляя универсальный способ последовательного обхода элементов коллекций без раскрытия их внутренней структуры (будь то связный список или дерево).
- **Factory Method (Фабричный метод):** Методы создания неизменяемых коллекций, появившиеся в современных версиях Java, такие как `List.of()`, `Set.of()`.

3. Factory Method, Builder (StringBuilder)

Паттерн **Factory Method (Фабричный метод)** относится к порождающим шаблонам. Он определяет абстрактный интерфейс для создания объекта, но оставляет subclasses право решать, какой именно класс конструировать. Это позволяет делегировать логику инстанцирования конкретных реализаций на уровень наследников, избавляя клиентский код от жесткой привязки к ключевому слову `new` и конкретным классам.

Паттерн **Builder (Строитель)** используется для пошагового конструирования сложных объектов, когда у объекта может быть огромное количество параметров конфигурации, часть из которых опциональна. Вместо создания громоздких конструкторов с десятком аргументов (что приводит к антипаттерну «телескопический конструктор»), `Builder` позволяет вызывать цепочку понятных методов (`fluent interface`) для настройки отдельных свойств и в конце выполнить метод `build()`, возвращающий готовый объект.

StringBuilder в Java является классической, но специфичной реализацией идей паттерна Строитель. Создание длинной строки `String` путем обычной конкатенации + порождает множество промежуточных мусорных объектов в памяти из-за неизменяемости (`immutability`) строк. `StringBuilder` инкапсулирует динамический массив символов и предоставляет методы последовательного накопления данных (`append()`, `insert()`). В самом конце вызывается метод `toString()`, который выполняет роль метода `build()`, создавая итоговый неизменяемый объект `String`.

4. Singleton (class Runtime, Servlet, ServletContext)

Паттерн **Singleton (Одиночка)** гарантирует, что у определенного класса во всем приложении будет существовать строго один единственный экземпляр, и предоставляет к этому экземпляру глобальную точку доступа (обычно через статический метод

getInstance()).

8. **Класс `java.lang.Runtime`:** Это классический пример «чистого» синглтона в Java SE. Конструктор класса закрыт ключевым словом `private`, а доступ к единственному объекту, управляющему процессами виртуальной машины Java, осуществляется через статический метод `Runtime.getRuntime()`.
9. **Сервлеты (Servlet):** Архитектурно в спецификации Jakarta EE (Servlet API) сервлеты **не являются строгими синглтонами** на уровне исходного Java-кода (у них открытые публичные конструкторы). Однако контейнер сервлетов (например, Tomcat) по умолчанию управляет ими в режиме синглтона: он создает ровно один экземпляр конкретного сервлета на всё приложение и многопоточно направляет в его метод `service()` все входящие HTTP-запросы.
10. **ServletContext:** Этот объект представляет собой контекст окружения всего веб-приложения. Он создается контейнером сервлетов в единственном экземпляре для каждого развернутого веб-приложения и служит глобальным хранилищем конфигураций и ресурсов.

5. Как у Singleton создать второй объект? И как воспрепятствовать этому.

Несмотря на приватный конструктор классического синглтона, защиту можно обойти несколькими способами:

11. **Reflection API (Рефлексия):** С помощью рефлексии можно получить доступ к приватному конструктору, вызвать метод `setAccessible(true)` и принудительно создать новый объект.
 - *Как защититься:* Внутри приватного конструктора добавить проверку: если экземпляр уже существует, выбрасывать исключение `RuntimeException`.
12. **Сериализация/Десериализация:** Если класс реализует интерфейс `Serializable`, то при сохранении объекта на диск и последующем чтении будет создан абсолютно новый экземпляр класса.
 - *Как защититься:* Добавить в класс метод `protected Object readResolve() { return INSTANCE; }`. При десериализации Java автоматически вызовет этот метод и вернет уже существующий синглтон вместо создания нового.
13. **Клонирование:** Если класс наследуется от структуры, где доступен метод `clone()`.
 - *Как защититься:* Переопределить метод `clone()` и внутри него выбрасывать исключение `CloneNotSupportedException` или возвращать тот же `INSTANCE`.

Абсолютная защита: Самый элегантный и надежный способ реализации синглтона в Java, защищенный от всех вышеперечисленных атак на уровне самой JVM — это использование **enum** (перечисления).

```
public enum Singleton {  
    INSTANCE;  
    // методы бизнес-логики  
}
```

6. Prototype (clone())

Паттерн **Prototype (Прототип)** используется тогда, когда процесс создания объекта традиционным путем (через `new` и заполнение полей) является слишком тяжелым, ресурсоемким или требует сложных обращений к внешним источникам. Паттерн предлагает создавать новые объекты путем копирования (клонирования) уже существующего объекта-образца (прототипа).

В языке Java этот паттерн поддерживается на базовом уровне с помощью маркерного интерфейса `java.lang.Cloneable` и метода `clone()`, определенного в корневом классе `Object`. Однако стандартная реализация `Object.clone()` выполняет поверхностное копирование (Shallow Copy): она просто копирует значения примитивных полей и ссылки на объекты. Если у вашего класса есть ссылки на изменяемые объекты, обе копии будут ссылаться на одни и те же внутренние структуры. Для реализации полноценного паттерна Prototype разработчик обязан переопределить метод `clone()`, сделать его публичным и вручную реализовать глубокое копирование (Deep Copy), скопировав все вложенные структуры данных, либо использовать конструкторы копирования.

7. Command

Паттерн **Command (Команда)** — это поведенческий шаблон проектирования, который инкапсулирует определенный запрос, действие или операцию в виде отдельного самостоятельного объекта. Это позволяет параметризовать клиентские объекты различными запросами, ставить операции в очередь, логировать их, а также организовывать отмену операций (undo/redo).

Структура паттерна включает в себя интерфейс Команды (обычно с единственным методом `execute()`), конкретные реализации, содержащие ссылку на объект-получатель (Receiver), выполняющий реальную работу, и объект-вызыватель (Invoker), который инициирует выполнение команды, не зная деталей её реализации. Прекрасным примером концепта паттерна Command в Java SE служит интерфейс `java.lang.Runnable`. Объект, реализующий `Runnable`, инкапсулирует логику выполнения задачи внутри метода `run()`. Этот объект можно передать в качестве параметра в поток `Thread` или в пул потоков `ExecutorService` (которые выступают в роли Invoker), и они выполнят команду, оставаясь полностью изолированными от внутренней логики самой задачи.

8. Composite (java.nio.file.Path, javax.xml.parsers.Document, Node)

Паттерн **Composite (Компоновщик)** — это структурный шаблон, который позволяет

сгруппировать множество объектов в древовидную иерархическую структуру «часть-целое» и работать с единичным объектом точно так же, как со всей группой (композицией) объектов. Паттерн базируется на том, что и простые элементы (листья дерева), и сложные контейнеры (ветки дерева, содержащие другие элементы) реализуют один и тот же общий интерфейс.

Примеры применения:

10. `java.nio.file.Path`: Представляет элемент файловой системы. Путь может указывать как на конкретный конечный файл (лист), так и на директорию (контейнер), содержащую поддиректории и файлы, но методы работы с путями унифицированы.
11. `javax.xml.parsers.Document` и интерфейс `Node`: Это классическая реализация Компоновщика для работы с XML и HTML DOM-деревьями. Интерфейс `Node` (Узел) является общим предком. Конкретный узел дерева может быть текстовым полем (листом), а может быть тегом-элементом, содержащим внутри себя массив других дочерних узлов `Node`. Проходя по дереву, разработчик может вызывать методы узла, не задумываясь на каждом шаге, ветка это или лист.

9. Chain of responsibility (Filter, closing io stream, closing connection)

Паттерн **Chain of Responsibility (Цепочка обязанностей)** — это поведенческий шаблон, который позволяет избежать жесткой привязки отправителя запроса к его получателю, давая возможность обработать запрос несколькими объектами. Эти объекты связываются в цепочку, и входящий запрос передается по ней последовательно. Каждый обработчик решает: может ли он обработать этот запрос сам, и должен ли запрос передаваться дальше по цепочке.

Примеры применения:

5. **Сервлетные фильтры (`javax.servlet.Filter`):** Яркий пример. Входящий HTTP-запрос проходит сквозь цепочку фильтров (`FilterChain`). Один фильтр проверяет кодировку, второй проверяет авторизацию, третий логирует данные. Каждый фильтр внутри метода `doFilter()` обязан явно вызвать `chain.doFilter()`, чтобы передать управление следующему звену.
6. **Заккрытие вложенных IO-поток и JDBC соединений:** Когда мы оборачиваем один поток ввода-вывода в другой (например, `new BufferedReader(new FileReader(...))`), вызов метода `close()` на внешнем декораторе запускает цепочку вызовов `close()` по всей внутренней иерархии объектов, последовательно освобождая системные ресурсы.

10. State (Thread.State)

Паттерн **State (Состояние)** — это поведенческий шаблон, который позволяет объекту изменять свое поведение в зависимости от текущего внутреннего состояния. Внешне это выглядит так, словно объект изменил свой класс. Паттерн помогает избавиться от огромных, трудночитаемых условных конструкций `if-else` или `switch-case`, в которых логика методов зависит от флагов состояния объекта. Вместо этого каждое отдельное состояние выделяется в виде самостоятельного класса, реализующего общий интерфейс. В Java SE великолепной иллюстрацией концепта служит перечисление

java.lang.Thread.State, которое фиксирует жизненный цикл потока исполнения. Поток в Java может находиться в строго определенных состояниях: NEW (создан), RUNNABLE (готов к работе/работает), BLOCKED (заблокирован в ожидании монитора), WAITING (ждет уведомления), TIMED_WAITING (ждет определенное время) и TERMINATED (завершен). В зависимости от того, в каком состоянии находится поток, вызовы методов (например, start() или interrupt()) приводят к разному поведению виртуальной машины или к генерации исключений (например, нельзя повторно запустить уже работающий или завершенный поток).

11. Iterator (Enumeration, Iterator, ListIterator, навигация по ResultSet)

Паттерн **Iterator (Итератор)** предоставляет универсальный и безопасный интерфейс для последовательного обхода всех элементов составного объекта (коллекции), полностью скрывая от внешнего кода детали его внутреннего устройства (клиенту не важно, обходит он массив, хэш-таблицу или бинарное дерево).

Развитие итераторов в Java:

- Enumeration: Устаревший интерфейс из Java 1.0 (используется в Vector, Hashtable). Имеет всего два метода: hasMoreElements() и nextElement(). Не позволяет изменять коллекцию в процессе обхода.
- Iterator: Современный стандарт (Java 1.2). Содержит методы hasNext() и next(), а также безопасный метод remove() для удаления текущего элемента из коллекции прямо во время итерации без риска получить ConcurrentModificationException.
- ListIterator: Расширенная модификация Iterator специально для списков (List). Позволяет двигаться по коллекции в обоих направлениях (hasPrevious(), previous()), а также добавлять элементы (add()) и модифицировать их (set()).
- **Навигация по ResultSet в JDBC:** Объект ResultSet концептуально работает как итератор над строками базы данных. С помощью метода next() курсор сдвигается на одну строку вниз, возвращая false, когда данные закончились.

12. Proxy (reflection)

Паттерн **Proxy (Заместитель/Прокси)** — это структурный шаблон, который предоставляет объект-заменитель вместо реального целевого объекта. Этот заместитель перехватывает все вызовы к оригинальному объекту, позволяя выполнять дополнительную логику **до** или **после** того, как запрос будет передан реальному объекту. Это используется для ленивой инициализации, логирования, контроля доступа, кэширования или управления транзакциями.

В Java SE встроен мощный механизм динамического проксирования, работающий через Reflection API (java.lang.reflect.Proxy). С помощью метода Proxy.newProxyInstance() виртуальная машина прямо в памяти во время работы приложения генерирует новый класс-заместитель. Этот класс обязан реализовывать заданный интерфейс. Все вызовы методов этого интерфейса перенаправляются в специальный перехватчик — объект InvocationHandler (метод invoke()). В этом методе разработчик может написать любой

сквозной функционал, а затем при необходимости вызвать оригинальный метод через рефлексию. На этом механизме построена вся магия современных фреймворков (например, аннотация `@Transactional` в Spring).

13. Observer (Listener)

Паттерн **Observer (Наблюдатель)** — это поведенческий шаблон, реализующий модель взаимодействия «издатель-подписчик». Он определяет зависимость типа «один ко многим» между объектами таким образом, что когда у одного объекта (Издателя/Субъекта) изменяется внутреннее состояние, все зависимые от него объекты (Наблюдатели) автоматически получают уведомление и обновляют свое поведение. В Java этот паттерн повсеместно реализуется через концепцию **Слушателей (Listeners)**. Особенно ярко это выражено в GUI-библиотеках (AWT/Swing) и событийных моделях веб-приложений. Например, интерфейс `ActionListener` привязывается к кнопке с помощью метода `button.addActionListener(listener)`. Когда пользователь кликает по кнопке, она генерирует событие и рассылает его всем зарегистрированным слушателям, вызывая их метод `actionPerformed()`. Этот паттерн обеспечивает слабую связанность (loose coupling) между компонентами системы.

14. Wrapper/Adapter/Decorator (io stream, connection)

Эти три паттерна имеют схожую структурную организацию (часто называются общим словом **Wrapper** — Обёртка), но преследуют совершенно разные архитектурные цели:

- **Adapter (Адаптер):** Используется для объединения систем с несовместимыми интерфейсами. Он преобразует интерфейс одного класса в интерфейс, который ожидает клиент. Примером в IO является `java.io.InputStreamReader`. Он берет байтовый поток `InputStream` (который читать как текст неудобно) и «адаптирует» его к символьному интерфейсу класса `Reader`.
- **Decorator (Декоратор):** Не меняет интерфейс, а динамически добавляет объекту новые обязанности и функциональность прямо на лету. В IO-потоках: `new BufferedReader(new FileReader("file.txt"))`. Здесь `BufferedReader` оставляет интерфейс `Reader` неизменным, но добавляет к нему полезную функцию прозрачной буферизации данных для ускорения дискового чтения.

В JDBC обертки часто используются над объектом `Connection`. Например, пулы соединений выдают приложению не прямой коннект к базе, а объект-прокси (декоратор), который переопределяет метод `close()`: вместо физического разрыва сетевого соединения с СУБД этот метод просто возвращает коннект обратно в пул для повторного использования.

15. Immutable (custom, record)

Паттерн **Immutable (Неизменяемый объект)** гарантирует, что внутреннее состояние объекта не может быть изменено ни одним способом после того, как этот объект был создан. Такие объекты по своей природе являются потокобезопасными (thread-safe), их можно спокойно передавать между потоками без синхронизации.

Чтобы создать классический **кастомный Immutable класс** в Java вручную, необходимо соблюдать строгие правила:

- Объявить класс как `final`, чтобы никто не мог наследоваться и переопределить методы.
- Сделать все поля приватными (`private`) и финальными (`final`).
- Не создавать методов-сеттеров, изменяющих состояние полей.
- Если поля являются ссылками на изменяемые объекты (например, `java.util.Date` или `List`), в конструкторе и геттерах обязательно выполнять **оборонительное копирование** (Deep Copy), чтобы вызывающий код не смог изменить внутренности объекта извне.

Начиная с Java 14 (и став частью стандарта в Java 16), появился встроенный синтаксический инструмент для создания неизменяемых классов данных — **record**. При объявлении записи компилятор автоматически генерирует финальный класс, приватные финальные поля, конструктор, геттеры (названные по имени полей, без префикса `get`), а также правильные реализации методов `equals()`, `hashCode()` и `toString()`.

```
public      UserDto      {
    // Конструктор по умолчанию делает поверхностное присвоение,
    // для полной иммутабельности списков рекомендуется кастомный компактный
    конструктор.
    public UserDto {
        roles = List.copyOf(roles); // Оборонительное копирование неизменяемого списка
    }
}
```

16. MVC

MVC (Model-View-Controller / Модель-Представление-Контроллер) — это фундаментальный архитектурный паттерн, разделяющий приложение на три изолированные части, каждая из которых отвечает за свою область ответственности (Separation of Concerns):

4. **Model (Модель):** Инкапсулирует в себе всю бизнес-логику приложения, правила обработки данных, сущности и механизмы взаимодействия с базой данных (сервисы, репозитории, DAO). Модель ничего не знает о том, как её данные будут отображаться пользователю.
5. **View (Представление):** Отвечает исключительно за визуализацию данных, полученных от Модели, и формирование пользовательского интерфейса (это может

быть HTML-страница, JSP, JSON-ответ REST API или интерфейс мобильного приложения).

6. **Controller (Контроллер):** Выступает в роли посредника и координатора между Пользователем, Представлением и Моделью. Он принимает входящие запросы от пользователя (например, HTTP-запросы в веб-приложении), считывает параметры, вызывает необходимые сервисы Модели для проведения расчетов, а затем выбирает и передает полученный результат в соответствующее Представление для отображения.

Такое разделение позволяет независимо вести разработку интерфейса и серверной логики, а также существенно упрощает тестирование кода.

17. DAO vs. Repository

Оба паттерна используются для абстрагирования и изоляции слоя доступа к данным от основной бизнес-логики, но они находятся на разных уровнях архитектурного осмысления приложения:

DAO (Data Access Object): Это паттерн, ориентированный на структуру базы данных. Он создается вокруг конкретных таблиц или SQL-запросов. DAO по сути является тонкой оберткой над базовыми CRUD-операциями для одной конкретной таблицы. Его главная цель — скрыть специфику доступа (будь то чистый JDBC, Hibernate или SQL-запросы) к конкретной таблице. У вас будет UserDao, OrderDao, OrderItemsDao.

Repository (Репозиторий): Это паттерн, пришедший из методологии DDD (Domain-Driven Design), и он ориентирован на доменную модель (бизнес-сущности) приложения. Репозиторий оперирует понятием *Aggregate Root* (Корневой агрегат). С точки зрения бизнес-логики, заказ (Order) вместе со своими строками (OrderItems) является единым целым. Бизнес-логика не должна работать с элементами заказа в отрыве от самого заказа. Поэтому создается один OrderRepository, который умеет сохранять весь агрегат целиком. Под капотом Репозиторий может использовать несколько различных DAO для сборки одного комплексного доменного объекта. Репозиторий имитирует поведение обычной коллекции объектов в памяти.

JEE (Jakarta EE)

1. Что такое сервер приложений? Что такое веб-сервер, в чём его отличие от сервера приложений? Привести примеры веб-сервера и сервера приложений.

Веб-сервер предназначен для обработки входящих HTTP-запросов и отдачи статического контента (HTML-страниц, картинок, стилей CSS, файлов). Он также может уметь перенаправлять запросы (проксировать) на другие динамические приложения.

Сервер приложений (Application Server) — это полноценная программная платформа, которая предоставляет законченную среду для выполнения сложной распределенной бизнес-логики крупных корпоративных приложений. Сервер приложений обязан полностью реализовывать спецификации Jakarta EE (ранее Java EE), предоставляя

разработчику из коробки такие сервисы, как управление распределенными транзакциями (JTA), систему обмена сообщениями (JMS), веб-сервисы, удаленный вызов компонентов (EJB) и безопасность.

В чём отличие: Веб-сервер работает преимущественно на транспортном уровне HTTP и со статикой. Сервер приложений содержит в себе веб-сервер как составную часть, но расширяет его возможности тяжелым бизнес-функционалом.

Примеры:

- **Веб-серверы:** Apache HTTP Server, Nginx. (Популярный **Apache Tomcat** строго говоря является специализированным *веб-сервером + контейнером сервлетов*, но он не является полноценным сервером приложений, так как не поддерживает стек технологий EJB/JTA по умолчанию).
- **Серверы приложений:** WildFly (ранее JBoss), GlassFish, IBM WebSphere, Oracle WebLogic.

2. Что такое контейнер сервлетов? Что такое сервлет?

Сервлет (Servlet) — это специализированный Java-класс, работающий под управлением веб-сервера, предназначенный для динамической обработки входящих сетевых запросов от клиентов (обычно HTTP-запросов) и генерации ответов (обычно в виде HTML, JSON или XML). Это низкоуровневая основа любого Java-веб-приложения.

Контейнер сервлетов — это специальный программный компонент (инфраструктура), который управляет всем жизненным циклом сервлетов. Сам по себе сервлет не имеет метода `main()` и не может запуститься самостоятельно. Контейнер сервлетов (например, Apache Tomcat, Jetty) берет на себя все низкоуровневые сетевые задачи: слушает определенный сетевой порт, принимает TCP-соединения, парсит входящие байты данных в стандартные объекты Java (`HttpServletRequest`), находит по URL нужный сервлет, запускает его обработку в отдельном потоке и отправляет сформированный объект (`HttpServletResponse`) обратно клиенту по сети.

3. Методы сервлета. Жизненный цикл сервлета.

Жизненный цикл сервлета полностью контролируется контейнером сервлетов и состоит из трех ключевых этапов, которым соответствуют методы интерфейса `javax.servlet.Servlet`:

12. **Инициализация (метод `init(ServletConfig config)`):** Когда контейнер решает создать экземпляр сервлета (обычно при первом обращении клиента к нему или при старте сервера, если настроен параметр `load-on-startup`), он вызывает метод `init()`. Этот метод выполняется **строго один раз** за всю жизнь сервлета. Здесь инициализируются тяжелые ресурсы (например, конфигурации).
13. **Обслуживание запросов (метод `service(ServletRequest req, ServletResponse res)`):** Этот метод вызывается контейнером при каждом входящем HTTP-запросе. Метод выполняется в отдельном независимом потоке пула. Внутри базового класса `HttpServlet` метод `service()` анализирует тип HTTP-метода (GET, POST и т.д.) и делегирует обработку профильным методам, таким как `doGet()`, `doPost()`, `doPut()`, `doDelete()`.
14. **Уничтожение (метод `destroy()`):** Вызывается контейнером **строго один раз**, когда

сервер останавливается или когда контейнер выгружает сервлет из памяти. Этот метод используется для корректного освобождения занятых ресурсов, закрытия фоновых потоков или сохранения логов. После завершения `destroy()` сборщик мусора JVM уничтожает сам объект сервлета.

4. Что такое jsp. Жизненный цикл.

JSP (JavaServer Pages) — это технология, позволяющая создавать динамические веб-страницы, комбинируя статический текстовый контент (HTML, XML, CSS) со специальными вставками Java-кода (скриптлетами, тегами JSTL и выражениями EL). Она была создана как альтернатива сервлетам для упрощения генерации сложного HTML-интерфейса, чтобы избавить разработчиков от необходимости писать терабайты строк кода вида `out.println("<html>")`.

Жизненный цикл JSP имеет важную особенность: под капотом любая JSP-страница превращается в обычный сервлет.

Этапы жизненного цикла:

- **Трансляция (Translation):** При первом обращении пользователя к JSP-странице контейнер считывает её файл и транслирует (преобразует) её разметку в полноценный Java-код стандартного сервлета (файл с расширением `.java`). Статический HTML превращается в команды `out.write()`.
- **Компиляция (Compilation):** Сгенерированный Java-файл компилируется стандартным компилятором в байт-код (файл `.class`).
- **Загрузка и Инициализация:** Полученный класс сервлета загружается в память JVM, контейнер создает его экземпляр и вызывает метод `jspInit()`.
- **Выполнение (Вызов `_jspService()`):** Для каждого запроса к этой странице контейнер вызывает метод `_jspService()`, который генерирует динамический HTML-ответ.
- **Уничтожение (`jspDestroy()`):** При остановке приложения вызывается метод очистки ресурсов.

Поскольку трансляция и компиляция происходят только при первом запросе, первый заход на страницу JSP может сопровождаться небольшой задержкой, зато последующие обращения происходят мгновенно на скорости скомпилированного сервлета.

5. Что такое сессия? Жизненный цикл.

Сессия (Session / HttpSession) — это механизм, позволяющий серверу сохранять состояние и идентифицировать конкретного пользователя на протяжении серии его последовательных HTTP-запросов к веб-приложению. Поскольку протокол HTTP сам по себе является протоколом без сохранения состояния (stateless), каждый новый запрос от браузера выглядит для сервера изолированным. Сессия решает эту проблему.

При первом обращении (вызове `request.getSession()`) сервер создает на своей стороне объект сессии (уникальное пространство в памяти) и присваивает ей случайный алфавитно-цифровой идентификатор ID (в Java это называется `JSESSIONID`). Сервер отправляет этот ID обратно браузеру в HTTP-ответе в виде специальной куки (Cookie). При всех последующих запросах браузер автоматически прикрепляет эту куку к

запросам, сервер считывает JSESSIONID, находит соответствующий объект сессии в своей памяти и восстанавливает контекст пользователя (например, корзину покупок или статус авторизации).

Жизненный цикл сессии:

- **Создание:** Иницируется на сервере при явном вызове метода создания сессии в коде сервлета.
- **Активность:** В сессию можно сохранять любые Java-объекты в виде атрибутов.
- **Завершение:** Сессия уничтожается в трех случаях:
 - По истечении времени таймута бездействия (указывается в настройках session-timeout в web.xml), если пользователь долго не совершал запросов.
 - Принудительно по воле разработчика путем вызова метода **session.invalidate()** (сценарий кнопки «Выход из аккаунта»).
 - При перезапуске или аварийном падении контейнера сервлетов.

6. Что такое request? Из чего состоит? Жизненный цикл.

Объект request (в контексте веб-приложений это экземпляр класса, реализующего интерфейс HttpServletRequest) является объектно-ориентированным представлением входящего сетевого HTTP-запроса, пришедшего от браузера или клиента. Контейнер сервлетов парсит текстовый сетевой пакет данных и упаковывает его в этот Java-объект.

Из чего состоит объект request:

6. **Строка запроса:** HTTP-метод (GET, POST), целевой URL/URI, версия протокола.
7. **Заголовки (Headers):** Метаинформация (User-Agent, Accept, Host, Content-Type, Cookies).
8. **Параметры (Parameters):** Данные, переданные в строке URL (Query string для GET) или отправленные из HTML-формы в теле запроса (для POST).
9. **Тело запроса (Body):** Поток байт данных (InputStream), содержащий полезную нагрузку (например, JSON, XML или загружаемый файл).
10. **Атрибуты (Attributes):** Внутреннее хранилище данных приложения (Map), используемое разработчиками для передачи Java-объектов между сервлетами и фильтрами в рамках обработки одного текущего запроса.

Жизненный цикл request: Он крайне короткий. Объект создается контейнером сервлетов в момент поступления сетевого HTTP-запроса в систему. Он живет исключительно до тех пор, пока запрос обрабатывается цепочкой фильтров и сервлетов. Как только управление выходит из финального метода сервлета, и сформированный ответ (response) полностью отправляется обратно клиенту в сеть, объект request уничтожается (или возвращается в пул объектов контейнера), а его данные стираются.

7. Cookies. Как можно достать Cookies, а если Cookies удалить, можно ли достать сессию? Жизненный цикл.

Куки (Cookies) — это небольшие фрагменты текстовых данных (пары ключ-значение), которые сервер отправляет в браузер клиента в заголовке ответа Set-Cookie. Браузер сохраняет их локально на компьютере пользователя и обязуется автоматически

прикреплять ко всем последующим HTTP-запросам к этому же домену в заголовке Cookie. Достать куки в сервлете можно с помощью вызова метода **request.getCookies()**, который возвращает массив объектов Cookie[]. Из-за специфики протокола HTTP нельзя запросить конкретную куку по имени напрямую; разработчику приходится итерироваться по массиву в цикле, проверяя имена методом cookie.getName().

Если куки удалить, можно ли достать сессию?

По умолчанию — **нет**. Если пользователь очистит куки в браузере (или заблокирует их), заголовок запроса потеряет куку JSESSIONID. При следующем запросе сервер не сможет сопоставить клиента ни с одной существующей сессией в памяти и будет вынужден создать абсолютно новую пустую сессию с новым ID.

Исключение: Существует альтернативный, устаревший механизм отслеживания сессий без кук — **URL Rewriting (переписывание URL)**. При его активации идентификатор сессии дописывается прямо в конец каждого URL-адреса в виде параметра ;jsessionid=12345.... В таком специфичном сценарии сессия продолжит работать даже без кук.

Жизненный цикл кук: Зависит от её типа.

6. **Сессионная кука (Session Cookie):** Не имеет установленной даты окончания срока действия. Она хранится исключительно в оперативной памяти браузера и автоматически бесследно уничтожается, как только пользователь полностью закроет окно браузера.
7. **Постоянная кука (Persistent Cookie):** Имеет заданный срок жизни, который устанавливается методом cookie.setMaxAge(int seconds). Такие куки сохраняются браузером на жестком диске компьютера и выдерживают перезагрузку системы, удаляясь автоматически только по достижении указанного времени.

8. Чем отличаются методы POST и GET. Если не указать напрямую, какой из этих методов выполнится по умолчанию?

Это два базовых метода протокола HTTP, преследующих разные семантические цели:

5. **GET:** Предназначен исключительно для безопасного запроса и извлечения информации с сервера. Он не должен изменять внутреннее состояние системы (базы данных). Все параметры, которые пользователь хочет передать, прикрепляются прямо к строке URL после знака вопроса (например, /search?q=java). Из-за этого объем передаваемых данных жестко ограничен лимитами длины URL в браузерах. Запросы GET являются идемпотентными, они могут кэшироваться браузером и прокси-серверами, а также сохраняться в истории и закладках.
6. **POST:** Предназначен для отправки данных на сервер с целью создания нового ресурса или запуска обработки, изменяющей состояние системы (например, регистрация, проведение платежа). Передаваемые данные помещаются внутрь скрытого тела HTTP-запроса (Body). Метод POST не имеет строгих ограничений по размеру передаваемой информации, он не кэшируется по умолчанию и не сохраняется в истории браузера. Этот метод не является идемпотентным.

Если в HTML-форме (тег <form>) или в клиентском скрипте не указать метод отправки запроса явным образом, по умолчанию всегда выполнится метод **GET**. Любой клик по

обычной текстовой гиперссылке также инициирует HTTP-запрос методом GET.

9. Чем отличаются методы `doPost()` и `doPut()`? Какие методы являются идемпотентными?

Методы `doPost()` и `doPut()` в сервлетном API служат для обработки соответствующих HTTP-методов POST и PUT. Их главное различие лежит в плоскости архитектурной семантики REST и концепции идемпотентности:

- **`doPost()` (POST):** Используется для создания нового подчиненного ресурса в системе, когда адрес (URL) этого нового ресурса заранее неизвестен и будет сгенерирован самим сервером (например, добавление статьи в блог на адрес `/articles`). Метод **не является идемпотентным**. Если отправить один и тот же POST-запрос пять раз подряд, сервер послушно создаст пять абсолютно одинаковых записей в базе данных с разными ID, что приведет к дублированию.
- **`doPut()` (PUT):** Используется для обновления уже существующего ресурса по его известному уникальному адресу (например, редактирование статьи по адресу `/articles/5`) либо для создания ресурса, если его уникальный адрес заранее определяется на стороне клиента. Метод **строго идемпотентен**. Если отправить один и тот же PUT-запрос пять раз подряд, результат первого выполнения изменит состояние системы, а последующие четыре запроса будут просто перезаписывать данные теми же самыми значениями, не приводя к изменению финального состояния базы данных.

Идемпотентные методы — это методы HTTP, повторный вызов которых приводит к абсолютно тому же состоянию сервера, что и единичный вызов (они безопасны с точки зрения дублирования эффекта). К идемпотентным методам относятся: **GET, PUT, DELETE, HEAD, OPTIONS, TRACE**. Метод POST идемпотентным не является.

10. Какие scopes (области видимости) переменных существуют в JSP?

В спецификации JSP существует четыре строго разграниченные области видимости (Scopes) для хранения и поиска атрибутов. Они определяют, насколько долго объект будет доступен для страниц и сервлетов:

8. **`pageScope` (Область страницы):** Самый ограниченный уровень. Объекты доступны исключительно в пределах одной конкретной JSP-страницы, на которой они были объявлены. Как только обработка страницы завершается и HTML уходит пользователю, объекты уничтожаются.
9. **`requestScope` (Область запроса):** Объекты привязываются к текущему HTTP-запросу (`HttpServletRequest`). Они доступны на текущей странице, а также на тех страницах или сервлетах, на которые запрос был перенаправлен с помощью механизма `forward`.
10. **`sessionScope` (Область сессии):** Объекты привязываются к сессии конкретного пользователя (`HttpSession`). Они доступны на любых страницах и при любых повторных запросах этого пользователя до тех пор, пока его сессия активна.

Идеально для хранения профиля авторизованного пользователя.

11. **applicationScope (Область приложения):** Самый глобальный уровень. Объекты сохраняются в контексте всего веб-приложения (ServletContext). Данные доступны абсолютно всем пользователям на всех страницах в любое время. Существует в памяти до полной остановки веб-сервера.

11. В чём различие forward и redirect?

Оба механизма используются для перенаправления пользователя на другую страницу приложения, но они работают принципиально по-разному с точки зрения архитектуры сети:

- **Forward (Перенаправление на стороне сервера):** Выполняется внутри самого контейнера сервлетов с помощью объекта RequestDispatcher.forward(). Сервер, обрабатывая запрос в сервлете А, решает передать управление сервлету Б или JSP-странице напрямую. Браузер клиента об этом внутреннем переключении **абсолютно ничего не знает**. Для клиента всё происходит в рамках одного сетевого соединения: URL-строка в браузере не меняется, а все атрибуты объекта request полностью сохраняются и доступны новому компоненту. Это очень быстрая операция, так как нет сетевых задержек.
- **Redirect (Перенаправление через клиента / Отклики 302):** Выполняется с вовлечением браузера с помощью метода response.sendRedirect(). Сервер мгновенно прерывает обработку и отправляет обратно в браузер легкий HTTP-ответ со специальным статус-кодом (обычно 302 Found) и заголовком Location, содержащим новый целевой URL. Браузер, получив этот ответ, считывает адрес и автоматически делает **абсолютно новый, независимый HTTP-запрос** по указанному адресу. При этом URL в строке браузера меняется на новый, старый объект request со всеми его атрибутами безвозвратно уничтожается, а на сервер приходит чистый новый запрос. Это более медленная операция, так как она совершает полный сетевой круг («круг до клиента и обратно»).

12. Отличия getAttribute() от getParameter() в сервлете.

Хотя оба метода используются для извлечения информации внутри сервлета, они работают с совершенно разными источниками данных:

- **getParameter(String name):** Считывает текстовые данные, которые физически **пришли извне от клиента (браузера)**. Это могут быть параметры строки запроса URL (после знака ? в GET) или текстовые поля заполненной HTML-формы, присланные в теле POST-запроса. Метод всегда возвращает данные исключительно в виде строки **String** (или null, если параметр не найден). Разработчик не может программно записать что-то в параметры (setParameter не существует).
- **getAttribute(String name):** Используется для извлечения данных из **внутренних хранилищ контекста самого сервера** (из областей видимости request, session или application). Атрибуты — это инструмент внутреннего общения между разработчиками и компонентами Java-приложения (например, фильтр положил объект Пользователя в атрибут request, а сервлет его оттуда считал). Метод

возвращает полноценный объект типа **Object**, что позволяет передавать сложные структуры данных (коллекции, сущности). Запись в атрибуты осуществляется программистом на сервере с помощью метода `setAttribute(name, object)`.

13. Из чего состоит url?

URL (Uniform Resource Locator — Унифицированный указатель ресурса) представляет собой стандартизированную строку, описывающую полный сетевой путь к конкретному ресурсу. Структура полного URL состоит из следующих последовательных компонентов:

- **Протокол / Схема (Protocol/Scheme):** Указывает правила, по которым клиент будет общаться с сервером (например, `http`, `https`, `ftp`). Отделяется от остальной части двоеточием и двумя косыми чертами (`://`).
- **Хост / Доменное имя (Host):** Указывает сетевой адрес сервера, на котором располагается ресурс (например, `www.example.com` или IP-адрес `192.168.1.1`).
- **Порт (Port):** Указывает конкретный цифровой порт сетевого интерфейса сервера для адресации запроса. Отделяется от хоста двоеточием (например, `:8080`). Если порт стандартный для протокола (80 для HTTP, 443 для HTTPS), его обычно опускают.
- **Путь (Path):** Логический путь к конкретному ресурсу или эндпоинту внутри веб-приложения на сервере. Разделяется косыми чертами (например, `/api/v1/users`).
- **Параметры запроса (Query String):** Строка дополнительных динамических параметров, передаваемых серверу. Начинается со знака вопроса `?` и состоит из пар `ключ=значение`, разделенных амперсандами `&` (например, `?id=5&format=json`).
- **Якорь / Фрагмент (Fragment/Anchor):** Указывает на конкретную внутреннюю метку (раздел) внутри полученного документа. Начинается со знака решетки `#` (например, `#section-3`). Этот компонент обрабатывается исключительно браузером на стороне клиента и на сам сервер не отправляется.

14. В какой последовательности выполняются сервлет фильтры?

Последовательность (очередность) выполнения сервлетных фильтров для входящего запроса строго определяется правилами их декларирования и конфигурации в веб-приложении:

- Если конфигурация приложения описана традиционным способом через деплоймент-дескриптор **web.xml**, то контейнер сервлетов выстраивает фильтры в цепочку (`FilterChain`) строго в том порядке, в котором соответствующие теги **<filter-mapping>** физически записаны в тексте файла сверху вниз.
- Если конфигурация осуществляется современным способом с помощью аннотаций **@WebFilter**, стандартная спецификация Jakarta EE, к сожалению, **не гарантирует** строгий детерминированный порядок выполнения фильтров (он может зависеть от внутренней логики сканирования классов конкретным контейнером — например, по алфавиту имен классов).

Для точного управления порядком при использовании аннотаций в продвинутых фреймворках (например, в Spring) применяются дополнительные аннотации наподобие **@Order**. Если в приложении используются оба способа, контейнер сервлетов сначала всегда выполнит все фильтры, настроенные через **web.xml**, а уже затем перейдет к

фильтрам, найденным через аннотации. Важно помнить, что обратный путь ответа сервера (response) будет проходить сквозь ту же цепочку фильтров в строго зеркальном (обратном) порядке.

Привет! Понимаю всю ответственность — раз от этого зависит работа, разберем каждый блок максимально дотошно, глубоко и сплошным, понятным текстом, как ты и просил. Никаких поверхностных отписок, только чистая техническая суть, логика работы технологий и реальная практика.

Docker

1. What is the essence of Docker and what problem does it solve?

Суть Docker заключается в обеспечении легковесной изоляции приложений на уровне операционной системы с использованием контейнеризации. В отличие от классических виртуальных машин (Hyper-V, VirtualBox, VMware), которые эмулируют полноценное аппаратное обеспечение и требуют запуска внутри себя отдельной, тяжелой гостевой операционной системы со своим ядром, Docker использует единое ядро хостовой ОС (Linux). Изоляция достигается за счет механизмов самого ядра Linux: **Namespaces** (изолируют процессы, сетевые интерфейсы, файловые системы, пользователей) и **cgroups** (ограничивают потребление аппаратных ресурсов: CPU, RAM, IO). Благодаря этому контейнеры запускаются за доли секунды и потребляют минимум памяти. Главная проблема, которую решает Docker — это пресловутый синдром **«It works on my machine»** («На моей машине всё работает»). До появления контейнеров перенос приложения из среды разработки в тестовую или продакшн-среду превращался в кошмар из-за различий в версиях операционных систем, системных библиотек, установленных утилит и переменных окружения. Docker полностью решает эту проблему, позволяя упаковать само приложение, все его зависимости, бинарные файлы, конфигурационные скрипты и даже микро-ОС в единый самодостаточный артефакт — образ (Image). Этот образ будет гарантированно работать абсолютно одинаково везде, где установлен Docker Engine: на ноутбуке разработчика, на сервере тестирования или в облачном кластере. Дополнительно Docker решает проблему бесконфликтного сосуществования разных версий одного и того же софта (например, двух разных версий базы данных или Java) на одном физическом сервере.

2. Describe in general terms what should be in the .Dockerfile

Файл Dockerfile (пишется без расширения, просто с точкой в начале, если это скрытый файл, либо обычно Dockerfile с большой буквы) представляет собой обычный текстовый файл конфигурации, содержащий последовательный набор инструкций, по которым Docker автоматически собирает готовый образ. Каждая строка в этом файле — это отдельная команда, которая формирует внутреннее устройство будущего контейнера. В общем виде структура правильного Dockerfile должна содержать следующие ключевые компоненты:

- **Базовый образ (Инструкция FROM):** Это самая первая и обязательная строка. Она

указывает, какой готовый образ взять за основу. Например, FROM openjdk:17-jdk-slim или FROM ubuntu:22.04. Это фундамент, на который накладывается всё остальное.

- **Рабочая директория (Инструкция WORKDIR):** Задаёт путь к папке внутри контейнера, в которой будут выполняться все последующие команды. Это исключает хаос и собирает файлы приложения в одном месте.
- **Копирование файлов (Инструкции COPY или ADD):** Переносит файлы из локальной папки вашего компьютера (например, скомпилированный JAR-файл вашего Java-приложения) внутрь файловой системы собираемого образа.
- **Переменные окружения (Инструкция ENV):** Задаёт глобальные переменные, которые будут доступны приложению внутри контейнера во время его работы (например, настройки профилей Spring или пути к конфигурациям).
- **Выполнение команд сборки (Инструкция RUN):** Запускает команды внутри образа на этапе его создания. Используется для установки системных обновлений, загрузки пакетов, компиляции кода или создания системных пользователей.
- **Сетевые порты (Инструкция EXPOSE):** Носит скорее документационный характер и указывает, какие порты приложение внутри контейнера планирует слушать. Она не открывает их наружу автоматически, но служит сигналом для разработчика и оркестратора.
- **Точка входа и команда запуска (Инструкции ENTRYPOINT и CMD):** Определяют главную команду, которая выполнится в тот самый момент, когда на основе этого образа будет запущен живой контейнер. Например, CMD ["java", "-jar", "app.jar"]. Если эта команда завершает работу, контейнер тоже останавливается.

3. What is docker-compose and why is it needed?

Если сам Docker предназначен для управления одиночными контейнерами, то docker-compose — это утилита для оркестрации и управления многоконтейнерными (мультиконтейнерными) приложениями в рамках одного хоста. Современные программные системы редко состоят из одного компонента. Обычно вашему Java-приложению для работы требуются база данных (например, PostgreSQL), кэш (Redis), брокер сообщений (Kafka) и фронтенд-сервер (Nginx). Запускать каждый из этих компонентов вручную через длинные консольные команды docker run со всеми сетевыми настройками и переменными — долго, неудобно и чревато ошибками. docker-compose решает эту проблему, позволяя декларативно описать всю архитектуру вашего окружения в одном единственном файле формате YAML (обычно docker-compose.yml). В этом файле вы четко прописываете все необходимые сервисы (контейнеры), их взаимосвязи, проброс портов, общие сетевые пространства, примонтированные папки для хранения данных (volumes) и переменные окружения. После того как конфигурация написана, разработчику достаточно выполнить в терминале одну простую команду — docker compose up. Утилита сама прочитает файл, скачает нужные образы, создаст изолированную сеть, настроит хранилища и запустит все контейнеры в правильном, строго определенном порядке. Соответственно, команда docker compose down так же элегантно остановит и полностью удалит всю эту инфраструктуру, не оставляя мусора в системе.

4. What are layers in images and what problem do they solve?

Образ Docker (Image) не является монолитным куском данных. Он устроен по принципу слоеного пирога. Каждый слой представляет собой неизменяемый (read-only) спрез файловой системы, который формируется в результате выполнения практически каждой инструкции в Dockerfile (таких как FROM, RUN, COPY). Docker использует специальную каскадную файловую систему (Union File System, например, OverlayFS), которая умеет прозрачно накладывать эти слои друг на друга, объединяя их для запущенного контейнера в единую структуру. Когда вы запускаете контейнер, поверх всех этих неизменяемых слоев образа Docker накладывает один тонкий, временный слой, доступный для записи (Writable Layer). Все изменения, создание файлов или удаление, которые делает работающее приложение, происходят исключительно в этом верхнем временном слое, а сам образ остается девственно чистым.

Механизм слоев решает две фундаментальные проблемы:

9. **Огромная экономия дискового пространства и оперативной памяти:** Если у вас на сервере запущено 10 разных контейнеров, построенных на базе одного и того же образа `ubuntu:22.04`, то физически на диске этот базовый слой Ubuntu будет храниться в единственном экземпляре. Все 10 контейнеров будут использовать его совместно в режиме чтения, экономя гигабайты памяти.
10. **Мгновенный кэш и ускорение сборки/скачивания:** При повторной сборке Dockerfile или при скачивании обновленного образа с Docker Hub, Docker проверяет, какие слои изменились. Если вы изменили только одну строчку кода в своем JAR-файле, Docker не будет заново скачивать базовый слой ОС и заново устанавливать системные пакеты. Он возьмет старые слои из локального кэша и пересоберет только тот слой, который отвечает за копирование вашего JAR-файла. Это сокращает время деплоя с минут до считанных секунд.

5. What types of networks does docker-compose allow you to install for services?

По умолчанию, когда вы запускаете проект через `docker-compose`, утилита автоматически создает единую изолированную сеть типа **bridge** для всех описанных в файле сервисов. Это позволяет контейнерам общаться друг с другом напрямую по именам сервисов (встроенный DNS-сервер Docker сопоставляет имя сервиса с внутренним IP-адресом контейнера), полностью скрывая их от внешней физической сети хоста.

Однако `docker-compose` позволяет гибко настраивать и использовать другие типы сетей (драйверов):

9. **bridge (мост):** Стандартный драйвер по умолчанию. Создает виртуальный программный мост на хост-машине. Контейнеры в этой сети изолированы от внешней среды, но могут общаться между собой. Доступ к ним извне возможен только через явный проброс портов (ports).
10. **host (хост):** Этот драйвер полностью убирает сетевую изоляцию между контейнером и вашей реальной машиной. Контейнер использует сетевой стек хоста

напрямую. Если ваше приложение внутри контейнера запускается на порту 8080 в сети host, оно автоматически станет доступно на порту 8080 вашего компьютера без всякого проброса портов. Это дает максимальную скорость работы сети, но лишает контейнер изоляции и может вызвать конфликты портов.

11. **none:** Полностью отключает сетевой интерфейс для контейнера. У него будет только локальная петля loopback (localhost), и он не сможет связаться ни с внешним миром, ни с другими контейнерами. Применяется для специфических изолированных задач вычислений.
12. **overlay (перекрывающая сеть):** Используется для объединения в единую сеть контейнеров, запущенных на разных физических серверах (хостах). Это база для работы кластеров Docker Swarm.
13. **macvlan:** Позволяет присвоить контейнеру реальный MAC-адрес, благодаря чему ваша физическая домашняя или корпоративная сеть (роутер) будет видеть этот контейнер как отдельное полноценное физическое устройство со своим собственным IP-адресом в вашей локальной подсети.

Task. Case: I launched PostgreSQL in docker, but it is not available on the system via standard port 5432. Where to look and how to diagnose

Это классическая рабочая ситуация. Диагностику нужно проводить последовательно, двигаясь от верхнего уровня изоляции к внутренностям контейнера.

11. **Проверка статуса контейнера (docker ps):** Первым делом нужно убедиться, что контейнер вообще успешно запустился и не упал через секунду после старта из-за ошибок. Выполняем `docker ps`. Если контейнера нет в списке, выполняем `docker ps -a` (показать все, включая остановленные). Если он упал, смотрим логи командой `docker logs <container_name>`. Часто PostgreSQL падает на старте, если вы забыли передать обязательную переменную окружения с паролем (`POSTGRES_PASSWORD`).
12. **Проверка маппинга (проброса) портов:** В выводе команды `docker ps` внимательно смотрим на колонку PORTS. Там должно быть четко написано что-то вроде `0.0.0.0:5432->5432/tcp`. Если там написано просто `5432/tcp`, это означает, что база данных запустилась и слушает порт внутри контейнера, но вы забыли пробросить его на вашу реальную хост-систему. В `docker-compose.yml` обязательно должна быть секция:

```
ports:
  - "5432:5432"
```

13. **Проверка занятости порта на хосте (netstat / ss / lsof):** Возможно, на вашей локальной машине уже запущена и работает «родная» (установленная прямо в ОС) служба PostgreSQL. Она уже заняла порт 5432. В таком случае Docker-контейнер либо не сможет запуститься, либо не сможет привязать порт. Нужно проверить занятость порта командами `ss -tulnp | grep 5432` или `lsof -i :5432`. Если порт занят,

измените внешний порт в конфигурации Docker на другой, например, "5433:5432".

14. **Проверка доступности порта внутри самого контейнера:** Нужно понять, слушает ли сама СУБД этот порт внутри своей изолированной среды. Проваливаемся внутрь работающего контейнера интерактивно: `docker exec -it <container_name> bash` (или `sh`). Внутри контейнера пробуем подключиться к базе локально утилитой `psql -U postgres`. Если подключение проходит успешно, значит сама база исправна, и проблема строго в сетевых настройках Docker хоста или файрволе.
15. **Проверка Firewall / IP Tables:** На Linux-системах (особенно часто на серверах Ubuntu/RHEL) жесткие правила встроенного брандмауэра (UFW, `firewalld`) или кастомные цепочки `iptables` могут блокировать входящие соединения на порт 5432, даже если Docker пытается его открыть. Временное отключение брандмауэра помогает подтвердить или опровергнуть эту теорию.

Git

1. Что такое Git и чем он отличается от других систем контроля версий?

Git — это децентрализованная (распределенная) система контроля версий (DVCS), предназначенная для отслеживания изменений в файлах проекта, координации коллективной работы программистов и обеспечения возможности отката к любым предыдущим состояниям кода.

Главное отличие Git от старых централизованных систем (таких как SVN или CVS) заключается в его архитектурной независимости. В централизованных системах существует один главный сервер, на котором хранится вся история коммитов. Разработчик скачивает себе на компьютер только текущую рабочую копию кода. Если сервер падает или пропадает интернет, разработчик теряет возможность делать коммиты, смотреть историю изменений или создавать ветки.

В Git каждый разработчик скачивает себе на компьютер **абсолютно полную копию всего репозитория**, включая всю стопроцентную историю каждого коммита, всех веток и всех изменений, когда-либо происходивших в проекте. Центральный сервер (например, GitHub или GitLab) в мире Git является скорее удобной точкой обмена кодом, но не является единственным источником правды. Если сервер уничтожится, любой локальный репозиторий любого разработчика из команды может стать полноценной заменой сервера без малейшей потери данных. Второе важное отличие — Git мыслит не списками изменений в файлах (дельтами), а «снимками» (snapshots) всей файловой системы проекта на момент каждого коммита, что делает его невероятно быстрым.

2. Что такое репозиторий (локальный и удалённый)?

Репозиторий — это структурированное хранилище вашего проекта под управлением Git. Физически на диске это обычная папка с вашим кодом, внутри которой находится скрытая служебная папка `.git`. Именно в этой скрытой папке Git хранит все служебные метаданные, базу данных объектов, историю коммитов, конфигурации и ссылки на ветки.

- **Локальный репозиторий (Local Repository):** Находится непосредственно на

жестком диске вашего рабочего компьютера. Все операции, которые вы выполняете: добавление файлов в индекс (`git add`), создание коммитов (`git commit`), переключение между ветками (`git checkout/switch`), просмотр логов (`git log`) и создание локальных веток — происходят мгновенно в локальном репозитории, не требуя интернета и не влияя на работу других программистов.

- **Удалённый репозиторий (Remote Repository):** Находится на удаленном сервере в сети (хостинги GitHub, GitLab, Bitbucket или внутренний сервер компании). Он служит центральным хабом для синхронизации кода между членами команды. Разработчики отправляют туда свои локальные изменения (`git push`) и забирают оттуда чужие обновления (`git pull / git fetch`). Удаленный репозиторий обычно не содержит рабочей директории с файлами, он хранит только «голую» (bare) базу данных Git.

3. Что такое HEAD и как он связан с текущей веткой?

HEAD — это специальный служебный указатель (ссылка) внутри механизма Git. Его можно представить как текущий фокус или «курсор» вашей рабочей директории. HEAD всегда указывает на тот конкретный коммит, файлы которого развернуты в данный момент у вас на диске, и от которого будут отсчитываться все ваши последующие изменения.

В стандартном рабочем состоянии HEAD указывает не на коммит напрямую, а на **текущую активную локальную ветку** (например, указывает на ветку `main` или `feature/login`), которая, в свою очередь, указывает на самый последний коммит в этой ветке. Когда вы делаете новый коммит, указатель вашей текущей ветки сдвигается вперед на этот новый коммит, а HEAD послушно следует вместе с ней.

Существует также специфическое состояние, которое называется **Detached HEAD (отсоединенный HEAD)**. Оно возникает, когда вы принудительно переключаетесь не на имя ветки, а напрямую на хэш какого-то старого конкретного коммита (`git checkout <commit-hash>`). В этом режиме HEAD начинает указывать прямо на коммит. Если вы начнете делать новые коммиты в состоянии Detached HEAD, они не будут принадлежать ни одной ветке. Как только вы переключитесь обратно на какую-то ветку, эти новые коммиты «повиснут в воздухе» без ссылок, и через некоторое время будут безвозвратно удалены внутренним сборщиком мусора Git (garbage collector).

4. Как устроены коммиты (хэш, сообщение, автор, дата, изменения), ветки, merge

Коммит (Commit) — это фундаментальный кирпичик Git, фиксирующий снимок состояния файлов проекта. Каждый коммит представляет собой неизменяемый объект, содержащий:

- Уникальный идентификатор (SHA-1 хэш) — строка из 40 символов, вычисляемая на основе всего содержимого коммита (данных файлов, автора, даты). Изменение хотя бы одного символа в коде порождает совершенно другой хэш.
- Указатель на родительский коммит (или несколько родителей, если это коммит слияния). Именно за счет этих ссылок выстраивается линейная или разветвленная история проекта.

- Метаданные: Имя и email автора, дата и время создания, а также обязательное текстовое сообщение (commit message), описывающее суть изменений.
- Ссылку на дерево каталогов (tree), отражающее состояние всех файлов проекта в этот момент.

Ветка (Branch) в Git устроена гениально просто. Это не копия папок и не физический контейнер. Ветка — это просто крошечный текстовый файл-указатель, содержащий 40 символов SHA-1 хэша самого последнего коммита в этой цепочке (верхушка ветки, или tip). Создание ветки происходит мгновенно, потому что Git просто создает новый файл-ссылку на тот же коммит, где вы сейчас стоите.

Слияние (Merge): Это процесс объединения истории двух разных веток. Существует два основных сценария слияния:

- **Fast-Forward (перемотка вперед):** Происходит, когда история целевой ветки не ушла вперед. Git просто берет указатель вашей текущей ветки и перемещает его вперед на последний коммит вливаемой ветки. Новых коммитов при этом не создается.
- **Three-Way Merge (трехстороннее слияние):** Происходит, когда обе ветки развивались параллельно и ушли вперед независимо друг от друга. Git находит их общего предка, анализирует изменения в обеих ветках, автоматически объединяет их и создает специальный новый **коммит слияния (merge commit)**, у которого будет сразу два родителя.

5. Чем отличаются fetch и pull

Обе команды используются для получения обновлений из удаленного репозитория, но степень их воздействия на ваш текущий рабочий код кардинально различается. Команда **git fetch** является абсолютно безопасной и «разведывательной» операцией. Она связывается с удаленным сервером, скачивает оттуда все новые коммиты, ветки и теги, которых еще нет у вас в локальной базе данных, и обновляет ссылки удаленных веток (такие как origin/main). При этом **git fetch никоим образом не затрагивает ваши локальные рабочие ветки и файлы на диске**. Ваш текущий код не изменится, и конфликты возникнуть физически не могут. Вы получаете возможность безопасно изучить, что написали коллеги (например, через `git log main..origin/main`), перед тем как вливать изменения.

Команда **git pull** — это комбинированная, агрессивная команда. Под капотом она выполняет последовательно две операции: сначала запускает `git fetch`, а сразу после этого автоматически пытается влить (`git merge`) скачанные изменения удаленной ветки в вашу текущую локальную активную ветку. Из-за автоматического слияния команда `git pull` может напрямую изменить файлы у вас на диске и в любой момент может прерваться из-за возникновения конфликтов слияния (`merge conflicts`), которые вам придется разрешать вручную. В современной практике часто рекомендуют настраивать `git pull --rebase`, чтобы вместо слияния происходил перенос ваших коммитов поверх удаленных.

6. Что такое rebase и когда его использовать

Команда **git rebase (перебазирование)** — это альтернативный способ интеграции

изменений из одной ветки в другую. В отличие от классического `git merge`, который сохраняет историческую хронологию и создает отдельный коммит слияния, `rebase` полностью переписывает историю коммитов.

Как он работает: Допустим, вы создали ветку `feature` от ветки `main`, сделали в ней два коммита, но за это время в `main` ваши коллеги тоже добавили два коммита. Если вы сделаете `rebase` ветки `feature` на `main`, Git временно отложит ваши коммиты в сторону, переместит указатель вашей ветки `feature` на самый конец обновившейся ветки `main` (как будто вы только что создали ветку от актуального состояния), а затем поочередно применит ваши отложенные коммиты сверху. Хэши ваших коммитов полностью изменятся, так как у них изменятся родительские коммиты.

Когда использовать: `rebase` используется для поддержания кристально чистой, красивой, строго линейной истории проекта без лишних «мусорных» коммитов слияния. Это идеальный инструмент для локальной полировки вашей ветки перед отправкой в общий доступ.

Главное золотое правило Rebase: **Никогда не делайте `rebase` для веток, которые уже отправлены в удаленный репозиторий и над которыми работают другие люди (например, ветка `main` или `develop`).** Переписав историю публичной ветки, вы сломаете репозитории всем коллегам, что приведет к путанице и потенциальной потере кода. Используйте `rebase` только локально для своих приватных веток.

7. Как откатывать изменения (`revert`, `reset`)

Для отмены ошибочных изменений в Git используются две принципиально разные по своей философии команды:

`git reset` — это инструмент, который буквально отматывает историю назад. Он перемещает указатель текущей ветки (и HEAD) на указанный старый коммит, стирая последующие коммиты из текущей видимой истории ветки. У него есть три ключевых флага:

- `--soft`: Перемещает указатель ветки назад, но оставляет все файлы ваших отмененных коммитов измененными и добавленными в индекс (в Stage зону). Код не теряется.
- `--mixed` (по умолчанию): Перемещает указатель назад, оставляет файлы измененными на диске, но убирает их из индекса.
- `--hard`: Самый опасный флаг. Он полностью уничтожает коммиты, очищает индекс и безвозвратно затирает все файлы в вашей рабочей директории на диске до состояния выбранного старого коммита.

Важно: `git reset` переписывает историю, поэтому его можно применять только для локальных, еще не отправленных на сервер коммитов.

`git revert` — это максимально безопасный, «джентльменский» способ отмены. Вместо удаления старого ошибочного коммита, `git revert` анализирует изменения, которые были в нем сделаны, вычисляет для них строго противоположные действия (инвертирует код) и создает **абсолютно новый, коммит отмены (`revert commit`)** в конце истории. Старый ошибочный коммит остается на своем месте в истории проекта. Этот метод идеально подходит для работы с публичными ветками на сервере, так как он не ломает историю и

бесконфликтно синхронизируется у всех разработчиков.

8. Как решать конфликты при слиянии

Конфликт слияния (Merge Conflict) возникает тогда, когда Git пытается объединить две ветки, в которых один и тот же файл в одной и той же строке был изменен по-разному, либо в одной ветке файл изменили, а в другой параллельно удалили. В таких ситуациях алгоритм Git не может самостоятельно угадать, какая версия кода является правильной, останавливает процесс слияния и просит разработчика принять решение вручную.

Последовательность шагов для разрешения конфликта:

- Git останавливает операцию и выводит список конфликтующих файлов. Если выполнить `git status`, эти файлы будут помечены как `both modified`.
- Открываем конфликтующий файл. Git автоматически вставляет в места разногласий специальные маркеры разметки:

```
<<<<<<< HEAD
Локальный код из вашей текущей ветки (то, что было у вас)
=====
Чужой код из вливаемой ветки (то, что пришло извне)
>>>>>>> feature/another-branch
```

- Разработчик должен внимательно изучить обе версии кода (часто совместно с автором чужих изменений), стереть маркеры `<<<<<<<`, `=====`, `>>>>>>>` и оставить в файле итоговый рабочий вариант кода (это может быть ваша версия, чужая версия или их умная комбинация). В современных реалиях это делается в один клик в удобных интерфейсах IDE (IntelliJ IDEA, VS Code).
- После того как файлы отредактированы и сохранены, их необходимо добавить в индекс командой `git add <file_name>`, сообщая Git, что конфликт в этом файле успешно разрешен.
- Завершаем процесс слияния вызовом команды `git merge --continue` (или просто `git commit`), что создаст финальный коммит слияния. Если в процессе вы запутались и хотите всё отменить, можно вернуться в исходное состояние командой `git merge --abort`.

Spring Framework

1. Bean lifecycle. What it looks like, how the bean is created, etc.

Жизненный цикл бина (Bean Lifecycle) в Spring — это сложный многоэтапный процесс, управляемый IoC-контейнером (ApplicationContext), который превращает обычное описание класса в готовый, настроенный управляемый объект.

Основные этапы жизненного цикла выглядят следующим образом:

13. **Сканирование и загрузка определений (BeanDefinition):** Spring сканирует классы, находит аннотации (например, `@Component`) или читает конфигурационные файлы, создавая в памяти метаданные — объекты `BeanDefinition`. На этом этапе работают

интерфейсы `BeanFactoryPostProcessor`, позволяющие изменять эти метаданные до создания самих объектов (например, подставлять значения из `.properties` файлов).

14. **Инстанцирование (Instantiating):** Контейнер физически создает экземпляры Java-объекта, вызывая его конструктор (через рефлексию).
15. **Внедрение зависимостей (Populate properties):** Spring внедряет в созданный объект все требуемые зависимости через поля, сеттеры или конструктор (аннотация `@Autowired`).
16. **Вызов Aware-интерфейсов:** Если бин реализует специальные интерфейсы-слушатели, Spring передает ему служебные объекты контейнера. Например, через `BeanNameAware` передается имя бина, через `ApplicationContextAware` — ссылка на сам контекст приложения.
17. **Пре-инициализация через BeanPostProcessor:** Метод `postProcessBeforeInitialization()` всех зарегистрированных в контексте процессоров `BeanPostProcessor` перехватывает бин. На этом этапе обрабатывается аннотация `@PostConstruct`.
18. **Инициализация (Initialization):** Запускаются кастомные методы инициализации. Если бин реализует интерфейс `InitializingBean`, вызывается метод `afterPropertiesSet()`, а также вызывается кастомный метод, указанный в параметре `initMethod` аннотации `@Bean`.
19. **Пост-инициализация через BeanPostProcessor:** Метод `postProcessAfterInitialization()` повторно перехватывает бин. **Это критически важный этап: именно здесь Spring оборачивает ваш бин в динамический прокси-класс**, если для бина настроено сквозное поведение (например, транзакции `@Transactional` или аспекты AOP).
20. **Готовность к работе:** Бин помещается в кэш контекста и используется приложением.
21. **Уничтожение (Destruction):** При закрытии контекста запускается фаза очистки. Обрабатывается аннотация `@PreDestroy`, метод `destroy()` интерфейса `DisposableBean` и кастомный метод `destroyMethod`.

2. Methods for declaring beans. List and name the advantages and disadvantages

В Spring существует три основных способа объявления бинов:

- **Классический XML-конфиг (устаревший способ):** Конфигурация описывается в XML-файлах через теги `<bean id="..." class="...">`.
 - *Плюсы:* Полное разделение кода и конфигурации. Чтобы изменить структуру зависимостей, не нужно перекомпилировать Java-код.
 - *Минусы:* Громоздкость, полное отсутствие проверки типов на этапе компиляции, усложнение поддержки при росте проекта («XML-ад»). Сегодня в продакшене практически не используется.
- **Декларативный способ через сканирование компонентов (Component Scanning):** Классы помечаются аннотациями стека `@Component` (`@Service`,

@Repository, @Controller), а в конфигурации включается автоматическое сканирование пакетов.

- *Плюсы:* Максимальная скорость разработки и лаконичность. Вы просто пишете аннотацию над классом, и Spring берет управление на себя. Минимум шаблонного кода.
- *Минусы:* Логика создания бинов «размазана» по всему коду проекта. Вы не можете объявить бином класс, исходный код которого принадлежит сторонней подключенной библиотеке (вы не можете поставить @Component над классом из чужой jar-ки).
- **Явный Java-Based конфигурационный способ:** Создаются специальные классы-конфигурации, помеченные аннотацией @Configuration, внутри которых пишутся методы с аннотацией @Bean.
- *Плюсы:* Полный ручной контроль над процессом создания объекта. Вы можете написать любую сложную логику, циклы, условия if-else внутри метода перед возвратом объекта. Позволяет делать бинами любые классы из сторонних библиотек. Полная строгая типизация компилятором.
- *Минусы:* Требуется написания дополнительного конфигурационного кода вручную.

3. Bean scopes. Name and tell about their life cycle

Область видимости (Scope) определяет, как именно Spring будет создавать и предоставлять экземпляры бинов по запросу приложения. Существует 6 стандартных областей видимости:

10. **singleton (по умолчанию):** Контейнер создает ровно **один единственный экземпляр** этого бина на весь IoC-контейнер. Все последующие запросы к контейнеру (или внедрения в другие бины) будут возвращать ссылку на один и тот же объект в памяти.
 - *Жизненный цикл:* Создается на этапе старта контекста приложения и живет до полной остановки всего контекста.
11. **prototype:** Каждый раз, когда приложение запрашивает этот бин у контейнера (через getBean()) или когда он внедряется в другой бин, Spring создает **абсолютно новый экземпляр** объекта.
 - *Жизненный цикл:* Spring создает объект, проводит его через фазу инициализации и полностью отдает клиенту. Контейнер **не хранит** ссылку на этот бин и не отслеживает его дальнейшую судьбу. Методы уничтожения (@PreDestroy) для прототипов контейнером никогда не вызываются — очисткой памяти занимается обычный Garbage Collector.

Следующие 4 области видимости доступны исключительно в веб-приложениях (Web-Aware ApplicationContext):

3. **request:** Создается новый экземпляр бина на каждый входящий HTTP-запрос. Он живет внутри обработки этого запроса и уничтожается, как только клиенту уходит HTTP-ответ.

4. **session:** Создается один экземпляр бина для каждого уникального HTTP-сеанса конкретного пользователя. Живет в памяти сервера до тех пор, пока сессия пользователя не завершится или не истечет по таймауту.

5. **application:** Создается один экземпляр на весь жизненный цикл объекта ServletContext веб-приложения. Похож на singleton, но привязан именно к контексту сервлетов.

6. **websocket:** Создается один экземпляр бина на время действия конкретного WebSocket-соединения.

4. Tools for creating proxies. Differences between Dynamic Proxy and CGLib

Проксирование — это базовый инструмент работы Spring AOP (аспектов) и управления транзакциями. Когда Spring видит над классом аннотацию @Transactional или логику логирования, он подменяет ваш реальный объект специальным классом-оберткой (прокси), который перехватывает вызовы. Для создания прокси Spring использует два принципиально разных инструмента:

14. **JDK Dynamic Proxy (Динамическое проксирование Java SE):** Это встроенный стандартный механизм Java.

- *Принцип работы:* Он может создавать прокси-классы **исключительно для интерфейсов**. Сгенерированный в памяти прокси-класс реализует тот же интерфейс, что и ваш целевой бин, и делегирует вызовы через обработчик InvocationHandler.
- *Ограничение:* Если ваш класс не реализует ни одного интерфейса, JDK Dynamic Proxy использовать невозможно.

15. **CGLib (Code Generation Library):** Сторонняя библиотека байт-кода, встроенная в Spring.

- *Принцип работы:* Она создает прокси путем **динамического наследования**. CGLib генерирует новый класс прямо в рантайме, который является прямым наследником (subclass) вашего оригинального класса, и переопределяет его методы.
- *Ограничение:* Поскольку это наследование, CGLib не может переопределить методы, помеченные ключевым словом final, и не может работать с классами, которые сами объявлены как final.

Важное различие по умолчанию: В старых версиях Spring по умолчанию использовался JDK Dynamic Proxy, если у бина были интерфейсы. Начиная со Spring Boot 2.x и в современных версиях фреймворка, поведение изменилось — **CGLib используется по умолчанию для всех типов бинов**, независимо от наличия интерфейсов, ради обеспечения единообразия архитектуры и предсказуемости поведения (хотя это поведение можно переключить настройками).

5. Spring MVC vs Spring Boot. Differences

Сравнивать Spring MVC и Spring Boot напрямую не совсем корректно, так как они не конкурируют, а дополняют друг друга. Они находятся на разных уровнях абстракции.

Spring MVC — это полноценный веб-фреймворк, входящий в состав экосистемы Spring, построенный вокруг классического паттерна Model-View-Controller и сервлета DispatcherServlet. Он предназначен для создания классических веб-приложений и REST API. Однако «чистый» Spring MVC требует огромного количества ручной настройки: вам

нужно самостоятельно конфигурировать бины веб-окружения, прописывать маппинги, настраивать Jackson для сериализации JSON, подключать сторонний веб-сервер (например, Tomcat) в систему и собирать проект в тяжелый WAR-файл для деплоя.

Spring Boot — это не альтернатива Spring MVC, это мощная надстройка (инструмент автоматизации и оркестрации) над всей экосистемой Spring. Его цель — минимизировать конфигурационный ад и предоставить концепцию **«Convention over Configuration»** («Соглашение превыше конфигурации»). Spring Boot берет на себя всю рутину с помощью двух механизмов:

4. **Starter-зависимости:** Вы просто подключаете один стартер (например, spring-boot-starter-web), и он автоматически стягивает правильные, совместимые версии Spring MVC, Jackson, Validation и встроенного Tomcat.
5. **Автоконфигурация (Auto-configuration):** На основе анализа классов, находящихся в вашем classpath, Spring Boot сам угадывает, что вам нужно. Если он видит в зависимостях драйвер PostgreSQL, он сам автоматически настроит бин DataSource с дефолтными параметрами. Приложение Spring Boot собирается в удобный самодостаточный исполняемый **executable JAR-файл**, содержащий сервер Tomcat внутри, который можно запустить одной командой `java -jar`.

6. Distinguishes @Component from @Service and @Repository

Все три аннотации являются стереотипами Spring. С технической точки зрения аннотации @Service и @Repository являются **мета-аннотациями**, то есть они внутри себя аннотированы базовой аннотацией @Component. С точки зрения сканирования пакетов для IoC-контейнера они абсолютно идентичны: Spring найдет их, зарегистрирует как бины и создаст объекты. Однако между ними есть важнейшие семантические и функциональные различия:

12. **@Component:** Это базовая, общая аннотация для любого управляемого Spring-компонента общего назначения. Если класс не относится к какому-то специфическому слою приложения (например, утилитарный класс), над ним ставится @Component.
13. **@Service:** Применяется для маркировки классов **слоя бизнес-логики** (Services). Технически она не добавляет нового функционала, но служит четким маркером для разработчиков и инструментов анализа архитектуры, указывая, что в этом классе сосредоточены бизнес-правила приложения.
14. **@Repository:** Применяется для маркировки классов **слоя доступа к данным** (Data Access Object / Репозитории). Помимо семантической роли, она несет важнейший дополнительный функционал: аннотация включает автоматическую трансляцию низкоуровневых платформозависимых баз-данных исключений (например, SQLException или специфических ошибок Hibernate) в единую понятную иерархию непроверяемых исключений Spring — `DataAccessException`. Это позволяет изолировать обработку ошибок от специфики конкретной СУБД.

7. How to perform a native database query when using Spring Data

JPA?

Фреймворк Spring Data JPA позволяет писать запросы на чистом нативном SQL (Native Query) вместо объектного языка JPQL/HQL. Это необходимо, когда нужно задействовать специфические оптимизации конкретной СУБД, оконные функции, сложные индексы или конструкции, не поддерживаемые стандартом JPA.

Для этого используется аннотация **@Query**, размещаемая над методом в вашем интерфейсе-репозитории, у которой параметру `nativeQuery` явно присваивается логическое значение **true**.

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query(value = "SELECT * FROM users u WHERE u.status = :status_param AND u.deleted_at IS  
    NULL",  
        nativeQuery = true)  
    findAllActiveUsersNative @Param("status_param") ;  
}
```

Важные нюансы при работе с нативными запросами:

7. Имена таблиц и столбцов в строке `value` должны строго соответствовать реальным физическим именам таблиц и колонок в вашей СУБД, а не именам Java-сущностей (Entity) и их полей.
8. Такие запросы теряют кросс-платформенность: если вы напишете нативный запрос с синтаксисом PostgreSQL, при попытке переключить проект на базу H2 для тестов запрос упадет с ошибкой.
9. Если нативный запрос должен модифицировать данные (INSERT, UPDATE, DELETE), над методом обязательно нужно поставить дополнительную аннотацию **@Modifying**.

8. How does @Transactional work in Spring? What happens if one service calls a Transactional method from another Transactional method?

Аннотация **@Transactional** работает на основе механизмов Spring AOP (аспектно-ориентированного программирования) и динамического проксирования. Когда Spring создает бин, у которого методы помечены этой аннотацией, он подменяет его объектом-прокси.

Когда внешний клиент вызывает метод, прокси-класс перехватывает вызов, обращается к менеджеру транзакций (PlatformTransactionManager), открывает физическое соединение с базой данных, отключает автокоммит (connection.setAutoCommit(false)) и запускает транзакцию. После этого управление передается реальному методу вашего сервиса. Если метод завершается успешно, прокси выполняет connection.commit(). Если внутри метода выбрасывается неотловленный RuntimeException (или Error), прокси автоматически делает connection.rollback(), отменяя изменения.

Поведение при вызове одного транзакционного метода из другого определяется параметром **распространения транзакции (Transaction Propagation)**. По умолчанию используется уровень **Propagation.REQUIRED**.

Если ServiceA.methodA() (транзакционный) вызывает ServiceB.methodB() (тоже транзакционный), происходит следующее: Spring видит, что физическая транзакция уже была открыта методом methodA(). Поэтому methodB() не будет создавать новую транзакцию, он **присоединится к уже существующей внешней транзакции**. Они станут единым целым. Если внутри methodB() произойдет ошибка, вся общая транзакция будет помечена на откат (rollback-only), и изменения, сделанные в methodA(), тоже полностью откатятся.

Важнейшая ловушка самовывоза (Self-invocation): Если один метод класса вызывает другой транзакционный метод **того же самого класса** (this.methodB()), транзакция для второго метода **не запустится**. Вызов пойдет напрямую в обход прокси-класса, так как прокси перехватывает только внешние вызовы.

9. Name a few popular spring boot starters

Стартеры в Spring Boot — это готовые наборы декомпозированных зависимостей, которые агрегируют типовые библиотеки для решения конкретных архитектурных задач. Наиболее популярные из них:

- spring-boot-starter-web: Главный стартер для создания веб-приложений, включая RESTful сервисы. Тянет за собой Spring MVC, валидацию и встроенный сервер Tomcat.
- spring-boot-starter-data-jpa: Предназначен для работы с реляционными базами данных через спецификацию JPA. Подключает Hibernate, пул соединений HikariCP и базовую поддержку Spring Data.
- spring-boot-starter-security: Подключает мощную подсистему аутентификации и авторизации Spring Security для защиты эндпоинтов вашего приложения.
- spring-boot-starter-validation: Подключает механизмы валидации данных на основе спецификации Hibernate Validator (аннотации @NotNull, @Size, @Email).
- spring-boot-starter-test: Агрегирует все необходимые библиотеки для написания качественных тестов: JUnit 5, Mockito, AssertJ, Hamcrest и специфические утилиты Spring Test.
- spring-boot-starter-actuator: Служебный стартер, предоставляющий готовые продакшн-эндпоинты для мониторинга приложения, сбора метрик, проверки здоровья (health-checks) и аудита системы.

10. What is DI and IoC?

Это два фундаментальных принципа, на которых держится весь каркас Spring Framework, направленные на обеспечение слабой связанности (loose coupling) компонентов системы.

IoC (Inversion of Control — Инверсия управления): Это глобальная архитектурная концепция, которая переворачивает традиционное управление потоком выполнения программы. В классическом программировании разработчик сам управляет жизненным циклом объектов: пишет код, который в нужный момент явно создает экземпляр класса через оператор new. При инверсии управления эта обязанность делегируется (отдается) внешнему независимому механизму — фреймворку (IoC-контейнеру Spring). Не ваш код вызывает библиотеки, а фреймворк управляет вашим кодом, сам решая, когда создавать объекты, как их настраивать и когда уничтожать.

DI (Dependency Injection — Внедрение зависимостей): Это конкретный паттерн проектирования, который является одной из основных реализаций принципа IoC. Зависимость — это когда классу А для работы необходим объект класса Б. Вместо того чтобы класс А внутри своего конструктора сам создавал объект класса Б (`this.b = new B()`), этот объект создается где-то вовне (контейнером) и принудительно «внедряется» (передается) в класс А извне через конструктор, сеттер или напрямую в поле. Класс А становится чистым от логики конструирования, он просто ожидает готовый инструмент для работы, что делает систему гибкой, легко расширяемой и позволяет без проблем подменять реальные объекты заглушками (моками) при написании тестов.

REST

1. Что означает принцип ресурсной ориентации в архитектуре REST? Приведите примеры ресурсов.

Принцип ресурсной ориентации — это фундаментальная доктрина архитектурного стиля REST, которая гласит, что ключевым элементом, вокруг которого строится всё взаимодействие между клиентом и сервером, является не операция (метод или процедура), а **Ресурс (Resource)**. Ресурс — это любой объект, сущность или концепция реального мира, которая имеет имя, может быть однозначно идентифицирована и которой приложение хочет манипулировать. В ресурсной модели мы думаем о существительных, а не о глаголах. Клиент запрашивает не «вызов функции расчета», а состояние конкретного ресурса.

Примеры ресурсов в контексте корпоративных веб-приложений:

- Сущность бизнес-модели: Пользователь (user), Заказ (order), Товар (product).
- Физический или цифровой медиа-объект: Фотография (photo), Документ (document).
- Логическая коллекция: Список активных транзакций (transactions).

В URL-адресах ресурсы всегда отображаются в виде существительных, как правило, во **множественном числе**: `/api/v1/users`, `/api/v1/orders`. Манипуляция этими ресурсами происходит не за счет придумывания новых адресов (как в RPC: `/deleteUser`), а за счет применения стандартных HTTP-методов (глаголов) к одному и тому же адресу ресурса.

2. Почему важно использовать уникальные URI для каждого ресурса? Как это влияет на взаимодействие клиента и сервера?

URI (Uniform Resource Identifier) является единственным и абсолютным идентификатором (паспортом) ресурса в сети. Использование строго уникального URI для каждого отдельного ресурса (например, `/api/v1/products/42` для товара с ID 42) критически важно для соблюдения предсказуемости и прозрачности архитектуры.

Влияние на взаимодействие клиента и сервера:

- **Однозначная адресация:** Клиент и сервер начинают общаться на беспрепятственном декларативном языке. Сервер всегда точно понимает, над каким конкретно объектом базы данных клиент просит совершить операцию.
- **Эффективное кэширование:** Сетевые прокси-серверы, CDN и сами браузеры используют URI в качестве уникального ключа для кэширования ответов. Если у каждого ресурса свой уникальный адрес, система может кэшировать тяжелые неизменяемые данные (например, данные профиля или статьи) строго по этому адресу, снижая нагрузку на сервер.
- **Идемпотентность и безопасность:** Уникальный адрес позволяет четко разграничивать права доступа. Мы можем гибко настроить безопасность: разрешить всем чтение общего URI коллекции товаров, но запретить изменять конкретный URI отдельного товара без прав администратора.

3. Объясните, как HTTP-методы связаны с CRUD-операциями. Приведите примеры использования каждого метода.

Архитектура REST элегантно накладывает стандартные методы протокола HTTP на классические операции управления данными CRUD (Create, Read, Update, Delete). Это избавляет систему от необходимости плодить кастомные эндпоинты.

CRUD Операция	HTTP Метод	Семантика в REST	Пример URI и действия
Create (Создание)	POST	Создает новый ресурс в системе.	POST <code>/api/v1/users</code> (Тело запроса содержит JSON с данными нового пользователя. Сервер создает его и возвращает ID).
Read (Чтение)	GET	Извлекает состояние существующего	GET <code>/api/v1/users/42</code> (Возвращает JSON с подробными

		ресурса или коллекции строк.	данными пользователя с ID 42).
Update (Обновление)	PUT / PATCH	Модифицирует или заменяет существующий ресурс.	PUT /api/v1/users/42 (Полностью перезаписывает данные пользователя).
Delete (Удаление)	DELETE	Безвозвратно удаляет указанный ресурс из системы.	DELETE /api/v1/users/42 (Удаляет пользователя с ID 42).

Использование этой жесткой связки делает API интуитивно понятным для любых сторонних разработчиков без детального чтения документации.

4. В чём разница между методами PUT и PATCH? Когда предпочтительнее использовать каждый из них?

Оба метода используются для модификации (обновления) уже существующих ресурсов, но характер этой модификации принципиально различен:

7. **PUT (Полное обновление / Замена ресурса):** Семантика метода PUT требует, чтобы клиент прислал в теле запроса **абсолютно полное, обновленное представление ресурса целиком**, со всеми его полями. Сервер берет этот JSON и целиком замещает им старый объект в базе данных. Если клиент пришлет запрос PUT /api/v1/users/42, передав в JSON только одно поле {"name": "Иван"}, то все остальные поля пользователя (email, age, телефон) на сервере будут затерты или сброшены в null / дефолтные значения.
 - *Когда использовать:* Когда объект обновляется полностью (например, сохранение большой формы редактирования профиля, где пользователь переписал все поля).
8. **PATCH (Частичное обновление / Модификация):** Семантика PATCH предназначена для точечного, частичного изменения отдельных атрибутов ресурса. Клиент присылает только те конкретные поля, которые он хочет изменить. Запрос PATCH /api/v1/users/42 с телом {"name": "Иван"} обновит на сервере исключительно имя пользователя, а его email, пароль и все остальные поля останутся нетронутыми.
 - *Когда использовать:* Когда нужно изменить один или несколько флагов (например, сменить статус заказа на "Оплачено", обновить аватарку или поменять пароль), чтобы не гонять по сети тяжелые объемы полных данных объекта.

5. Что означает принцип Statelessness в REST? Какие преимущества он даёт?

Принцип **Statelessness (Отсутствие сохранения состояния)** означает, что архитектура взаимодействия между клиентом и сервером является изолированной. Сервер не обязан хранить в своей оперативной памяти никакой контекст или историю предыдущих запросов от конкретного клиента (сессии в привычном понимании на сервере отсутствуют).

Каждый входящий HTTP-запрос от клиента должен быть абсолютно самодостаточным. Он обязан содержать в себе **всю стопроцентную информацию**, необходимую серверу для успешного выполнения этого запроса: уникальный идентификатор ресурса, параметры и, главное, данные аутентификации (например, токен JWT в заголовке Authorization). Сервер выполнил запрос, отдал ответ и мгновенно «забыл» про клиента.

Преимущества Statelessness:

- **Колоссальная горизонтальная масштабируемость (Scalability):** Поскольку серверы не привязаны к сессиям клиентов, вы можете поставить перед вашим приложением балансировщик нагрузки (Nginx) и запустить 10 одинаковых экземпляров вашего Java-сервиса. Запрос клиента может прилететь на Сервер №1, следующий запрос — на Сервер №5, и всё будет работать идеально, так как каждому серверу плевать на историю, он считывает токен из запроса и сразу выполняет работу.
- **Отказоустойчивость:** Если один из серверов внезапно упадет или сгорит, это не приведет к потере сессий тысяч пользователей и прерыванию их работы. Балансировщик просто перенаправит поток запросов на выжившие серверы.
- **Упрощение логики сервера:** Серверу не нужно тратить ресурсы оперативной памяти на хранение миллионов объектов сессий и следить за их таймаутами.

6. Как сервер сообщает клиенту об успешной или ошибочной обработке запроса? Приведите примеры статус-кодов и их значения.

В REST-архитектуре сервер общается с клиентом с помощью стандартных цифровых **статус-кодов ответов HTTP (HTTP Status Codes)**. Использовать тело ответа для написания фраз в стиле {"status": "error"} при общем статус-коде 200 OK — это грубейшее нарушение стандартов REST. Код ответа четко разделен на пять классов, первая цифра определяет общую категорию.

Популярные и важные примеры статус-кодов:

- **Группа 2xx (Успех):**
 - 200 OK — запрос обработан успешно, сервер возвращает запрошенные данные в теле ответа.
 - 201 Created — запрос выполнен успешно, и в результате на сервере был создан новый ресурс (активно используется в связке с методом POST).
 - 204 No Content — запрос обработан успешно, но серверу нечего вернуть в теле

ответа (например, после успешного удаления ресурса методом DELETE).

- **Группа 4xx (Ошибка клиента):**

- 400 Bad Request — сервер не может обработать запрос, так как клиент прислал невалидные данные (например, кривой JSON, не прошедший валидацию Spring).
- 401 Unauthorized — для выполнения запроса клиент обязан пройти аутентификацию (отсутствует или протух токен доступа).
- 403 Forbidden — клиент аутентифицирован, но у него нет прав/ролей для доступа к этому конкретному ресурсу (например, обычный пользователь лезет в админку).
- 404 Not Found — запрашиваемый ресурс по указанному URI не существует на сервере.

- **Группа 5xx (Ошибка сервера):**

- 500 Internal Server Error — самая общая и неприятная ошибка. На сервере произошел непредвиденный сбой, упало необработанное Java-исключение (NullPointerException и т.д.).
- 503 Service Unavailable — сервер временно не готов обрабатывать запросы по техническим причинам (перегрузка, плановое обслуживание).

7. Как REST способствует разделению клиентской и серверной логики? Почему это важно для масштабируемости?

REST обеспечивает строгое разделение ответственности сторон на основе концепции **Client-Server (Клиент-Сервер)**. За счет использования унифицированного интерфейса (стандартные URL, фиксированные форматы данных JSON/XML и стандартные HTTP-методы) серверная часть приложения полностью отделяется от пользовательского интерфейса.

Сервер становится абсолютно независимым поставщиком данных и бизнес-логики. Ему абсолютно всё равно, кто именно к нему обращается через REST API: SPA-приложение на React, мобильное приложение на iOS, сторонний микросервис или консольный скрипт. Сервер отвечает только за хранение, безопасность и обработку данных. Клиент, со своей стороны, отвечает исключительно за визуализацию, отрисовку интерфейса и пользовательский опыт (UX), ничего не зная о внутреннем устройстве базы данных.

Почему это важно для масштабируемости:

7. **Независимое масштабирование команд и технологий:** Вы можете полностью переписать весь фронтенд с Angular на Vue.js или изменить дизайн приложения, не затронув ни единой строки кода на бэкенде Java. И наоборот — вы можете заменить базу данных PostgreSQL на Oracle или распилить монолитный бэкенд на микросервисы, сохранив те же REST-эндпоинты, и клиентское приложение этого даже не заметит.
8. **Эффективное распределение вычислительной нагрузки:** Сложные операции отрисовки интерфейса, анимации и логика отображения переносятся целиком на устройства пользователей (смартфоны, браузеры). Сервер освобождается от этой рутины, тратя драгоценное процессорное время исключительно на чистые вычисления и работу с данными, что позволяет системе обслуживать многократно

большее количество одновременных пользователей.

Привет! Я отлично понимаю, насколько это важный и ответственный момент. Вопросов действительно много, они покрывают весь фундамент Java, который гоняют на самых жестких собеседованиях уровня Middle.

Не переживай, мы разберем абсолютно всё. Я буду писать максимально подробно, понятным языком, со сплошным текстом (как ты и просил, чтобы логика не терялась), с примерами «на пальцах» и текстовыми схемами-картинками для визуализации. Наливай кофе, устраивайся поудобнее — поехали спасать твою судьбу.

Java Core — ООП

Принципы ООП

Объектно-ориентированное программирование держится на четырех «китах». Давай разберем их так, чтобы это от зубов отскакивало.

- **Инкапсуляция** — это сокрытие внутреннего устройства объекта и предоставление контролируемого интерфейса для работы с ним. Представь себе автомобиль: у тебя есть руль и педали (публичный интерфейс), но как именно бензин взрывается в цилиндрах двигателя (внутренняя реализация) — от тебя скрыто. В Java это достигается модификаторами доступа (`private`, `protected`, `public`) и геттерами/сеттерами. Мы защищаем данные от некорректного изменения извне. Например, чтобы никто не мог напрямую сетнуть полю `age` значение `-50`.
- **Наследование** — механизм, позволяющий создать новый класс на основе уже существующего, перенимая его свойства и методы. Это избавляет от дублирования кода (DRY — Don't Repeat Yourself). Если у нас есть класс `Animal` с методом `breathe()`, то класс `Dog`, отнаследовавшись от него через ключевое слово `extends`, автоматически научится дышать.
- **Полиморфизм** — способность программы использовать объекты с одинаковым интерфейсом без информации об истинном типе этого объекта. Буквально означает «многообразие форм». Если у нас есть метод, принимающий `Animal`, мы можем передать туда и `Dog`, и `Cat`. И если у обоих переопределен метод `makeSound()`, то собака загавкает, а кошка замыкает, хотя код вызова будет выглядеть одинаково. Подробнее виды полиморфизма разберем чуть ниже.
- **Абстракция** — это выделение значимых характеристик объекта и отсеивание незначительных. Мы создаем модель реального предмета, оставляя только то, что важно для нашей программы. Если мы пишем систему учета для автошколы, то у класса `Car` важны поля «марка», «номер» и «наличие дублирующих педалей». Если мы пишем симулятор гонок, то важны «коэффициент аэродинамического сопротивления» и «крутящий момент». Все остальное отбрасывается.

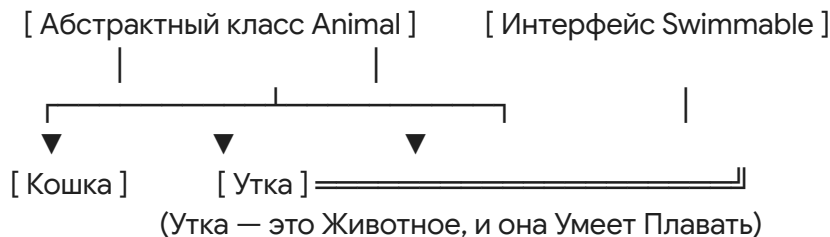
Abstract class vs Interface

Это классический вопрос. Главное отличие лежит на уровне философии проектирования, хотя синтаксически они похожи.

Абстрактный класс (создается через `abstract class`) задает идентичность. Он отвечает на

вопрос «**Кем объект является?**» (**is-a**). Он используется, когда у группы классов есть жесткая родственная связь и общая база кода.

Интерфейс (создается через `interface`) задает поведение. Он отвечает на вопрос «**Что объект умеет делать?**» (**can-do / able**). Он используется для реализации контракта, который могут выполнить абсолютно не родственные друг другу классы.



Различия списком:

11. **Множественное наследование:** Класс в Java может наследоваться только от одного абстрактного класса, но может реализовывать сколько угодно интерфейсов.
12. **Состояние (Поля):** В абстрактном классе могут быть любые переменные (instance variables) с любыми модификаторами доступа — они могут хранить состояние объекта. В интерфейсе все поля по умолчанию являются `public static final` (константами), никакого состояния объекта они хранить не могут.
13. **Конструктор:** У абстрактного класса есть конструктор, у интерфейса его быть не может в принципе.

Может ли быть конструктор у абстрактного класса и зачем

Да, может, и обязательно есть (даже если мы его не написали, компилятор подставит дефолтный).

Зачем он нужен, если мы не можем создать объект абстрактного класса напрямую через `new AbstractClass()`?

Он нужен для того, чтобы инициализировать поля самого абстрактного класса, когда создается объект его конкретного наследника. Когда мы вызываем `new Dog()`, внутри конструктора `Dog` первой строчкой неявно (или явно через `super()`) вызывается конструктор родительского абстрактного класса `Animal`. Если бы у абстрактного класса не было конструктора, мы не смогли бы гарантировать, что его приватные базовые поля будут правильно проинициализированы при создании дочерних объектов.

Функциональный интерфейс, виды

Функциональный интерфейс — это интерфейс, который содержит **ровно один абстрактный метод** (SAM — Single Abstract Method). При этом он может содержать сколько угодно `default` или `static` методов. Чтобы компилятор проверял это правило, над интерфейсом вешают аннотацию `@FunctionalInterface`. Такие интерфейсы нужны для поддержки лямбда-выражений и Stream API в Java 8+.

В пакете `java.util.function` лежат основные встроенные виды функциональных интерфейсов. Их четыре базовых типа:

14. **Predicate** (Предикат): Принимает объект типа T, возвращает boolean. Используется для фильтрации и проверок. Метод: `boolean test(T t)`. Пример: `s -> s.isEmpty()`.
15. **Consumer** (Потребитель): Принимает объект типа T, выполняет над ним какие-то действия, но ничего не возвращает (void). Метод: `void accept(T t)`. Пример: `s -> System.out.println(s)`.
16. **Supplier** (Поставщик): Ничего не принимает, но возвращает объект типа T. Используется для ленивой генерации данных. Метод: `T get()`. Пример: `() -> new String("hello")`.
17. **Function<T, R>** (Функция): Принимает объект типа T, трансформирует его и возвращает объект типа R. Метод: `R apply(T t)`. Пример: `s -> s.length()` (принимает строку, возвращает число).

Есть еще вариации типа `BiFunction<T, U, R>` (принимает два аргумента разных типов, возвращает третий) или `UnaryOperator<T>` (частный случай `Function`, где тип аргумента и возвращаемого значения совпадают).

Могут ли быть реализованные методы в интерфейсе и абстрактном классе

Да, могут в обоих случаях.

16. В абстрактном классе могут быть обычные не-абстрактные методы. Они пишутся как стандартные методы в любом классе, могут использовать поля объекта, быть `private`, `protected` или `public`.
17. В интерфейсе (начиная с Java 8) появились два типа реализованных методов:
 - **default методы** (дефолтные методы): Помечаются ключевым словом `default`. Они создавались ради обратной совместимости, чтобы можно было добавить новый метод в интерфейс (например, в интерфейс `Collection`), не ломая миллиарды строчек чужого старого кода, который этот интерфейс реализует. Классы-наследники получают этот метод автоматически, но могут его переопределить.
 - **static методы**: Обычные утилитарные методы, принадлежащие самому интерфейсу, а не его объектам. Их нельзя переопределить в классах-реализаторах.
Примечание: С Java 9 в интерфейсах также разрешили писать `private` методы для того, чтобы убирать дублирование кода внутри самих дефолтных методов интерфейса.

Какие модификаторы у полей/методов интерфейса

В интерфейсе царит строгая дефолтная магия, которую нужно знать наизусть:

- **Все поля** по умолчанию являются **public static final**. То есть они всегда публичные, принадлежат интерфейсу как сущности и являются неизменяемыми константами. Написать `private int x = 5;` в интерфейсе нельзя. Даже если ты просто напишешь `int X = 10;`, компилятор сам превратит это в `public static final int X = 10;`.
- **Все абстрактные методы** по умолчанию являются **public abstract**. Тебе не нужно писать эти слова, компилятор подставит их сам. С Java 9 методы также могут быть

private (но тогда они должны иметь тело, то есть быть реализованными). Методы не могут быть protected или package-private (без модификатора).

Что такое static

Ключевое слово static означает, что сущность принадлежит **самому классу**, а не конкретному экземпляру (объекту) этого класса.

- **Статические переменные (поля класса):** Существуют в единственном экземпляре в памяти (в области Metaspace/Permanent Generation) для всего класса. Если один объект изменит статическую переменную, все остальные объекты увидят это изменение. Отличный пример — счетчик созданных объектов.
- **Статические методы:** Могут быть вызваны без создания объекта класса (напрямую через `ClassName.methodName()`). Главное ограничение: статический метод **не имеет доступа к не-статическим полям и методам** класса, а также к ключевому слову `this`, потому что в момент его вызова никакого объекта в памяти может не существовать вовсе. Они могут работать только со статическими полями и своими локальными переменными.
- **Статические блоки инициализации:** Выполняются один раз в момент первой загрузки класса в память JVM ClassLoader'ом. Используются для сложной настройки статических полей.

Что такое final для методов, классов и переменных

Ключевое слово final в Java означает **запрет на изменение/модификацию**. Но ведет оно себя по-разному в зависимости от того, куда его применили.

- **Для переменных:** Делает переменную константой. Ей можно присвоить значение только один раз (либо сразу при объявлении, либо в конструкторе/блоке инициализации). После этого изменить её значение нельзя.
- **Для методов:** Запрещает переопределение (overriding) этого метода в классах-наследниках. Если у родителя метод `public final void print()`, то дочерний класс не сможет написать у себя `@Override public void print()`. Это нужно для безопасности, чтобы логику критически важных методов нельзя было подменить.
- **Для классов:** Запрещает наследование от этого класса. Никто не может написать `class MyString extends String`, потому что класс `String` помечен как final. Это гарантирует неизменяемость поведения всей системы.

Если сделать HashMap final переменной, то что будет

Это один из самых каверзных вопросов на понимание ссылочных типов данных.

Если мы пишем: `final Map<String, String> map = new HashMap<>();`, то **ссылка на объект становится неизменяемой, но сам объект внутри менять можно**.

То есть:

- Ты **МОЖЕШЬ** делать `map.put("key", "value")`, `map.remove("key")`, `map.clear()`. Содержимое карты будет прекрасно меняться, элементы будут добавляться и удаляться.

- Ты **НЕ МОЖЕШЬ** переприсвоить эту ссылку другому объекту. Строчка `map = new HashMap<>();` или `map = otherMap;` вызовет ошибку компиляции. Тебе запрещено менять «адресный указатель», хранящийся в переменной `map`, но по самому адресу ты можешь делать с объектом всё, что позволяет его интерфейс.

Полиморфизм, виды

Как мы уже выяснили, полиморфизм — это «один интерфейс, много реализаций». В Java выделяют два основных вида полиморфизма:

22. **Полиморфизм времени компиляции (Статический полиморфизм / Compile-time polymorphism)**: Реализуется через **перегрузку методов (Overloading)**. Это когда в одном классе есть несколько методов с одинаковым именем, но разными сигнатурами (разным набором, типом или порядком аргументов). Компилятор уже на этапе сборки кода точно знает, какой именно метод вызвать, глядя на то, какие аргументы ты туда передаешь.

Пример: `System.out.println(int x)` и `System.out.println(String s)`.

23. **Полиморфизм времени выполнения (Динамический полиморфизм / Runtime polymorphism)**: Реализуется через **переопределение методов (Overriding)** при наследовании. Это когда дочерний класс полностью копирует сигнатуру метода родителя, но меняет его внутренности. Какой именно метод вызовется, Java решает прямо во время работы программы (Runtime) с помощью специального механизма — таблицы виртуальных методов (VMT). JVM смотрит не на тип переменной, а на реальный тип объекта в куче (Heap).

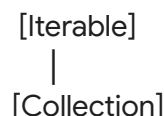
Пример: Переменная типа `Animal a = new Dog();`. Мы вызываем `a.makeSound();`. Компилятор видит тип `Animal`, но в рантайме JVM поймет, что там лежит `Dog`, и вызовет собачий лай.

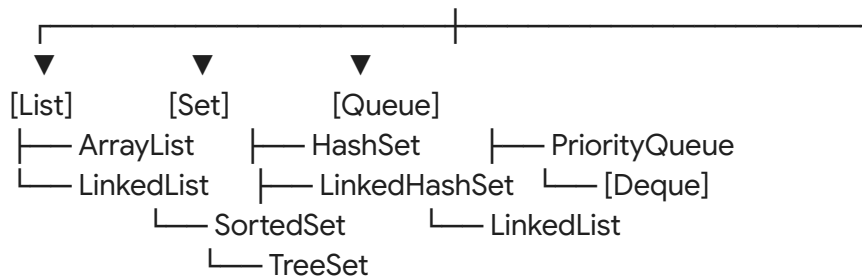
Java Core — Коллекции

Иерархия коллекций

Все коллекции в Java лежат в каркасе **Java Collection Framework (JCF)**. На вершине этой структуры стоят интерфейсы. Важно понимать, что корнем иерархии именно **коллекций** является интерфейс `Iterable` (он позволяет бегать по элементам циклом `for-each`), от которого наследуется интерфейс `Collection`.

Вот как выглядит текстовая «картинка» верхнеуровневой структуры JCF:





Обрати внимание: Map (Карта/Словарь) стоит абсолютно обособленно и в эту иерархию не входит.

Почему Map не является коллекцией

Интерфейс Map не наследуется от интерфейса Collection. Почему создатели Java так поступили?

Потому что у них принципиально разная природа и структура данных. Интерфейс Collection работает с **одиночными элементами** (хранит коллекцию объектов E). Интерфейс Map работает с **парами «ключ-значение»** (объектами типа Map.Entry<K, V>). Все методы интерфейса Collection (такие как add(E e), contains(Object o)) завязаны на обработку одного элемента. Если бы Map наследовался от Collection, то метод add просто не знал бы, что делать: как добавить ключ без значения? Из-за этого несовпадения контрактов методов их разделили на две независимые ветки. Тем не менее, Map является полноценной частью Java Collection Framework, так как тесно с ними взаимодействует (например, метод map.keySet() возвращает Set, а map.values() возвращает Collection).

Интерфейс Collection и дочерние интерфейсы (List, Queue, Set)

- **List (Список):** Упорядоченная последовательность элементов. Главная фишка — элементы хранятся в порядке их добавления, и к ним можно получить доступ **по индексу** (номеру позиции). Допускает дубликаты. Основные реализации: ArrayList, LinkedList.
- **Set (Множество):** Коллекция, которая **не может содержать дубликатов**. Элементы уникальны (проверка идет через методы equals и hashCode). Порядок элементов обычно не гарантируется (исключение — LinkedHashSet и TreeSet). Основные реализации: HashSet, TreeSet.
- **Queue (Очередь):** Коллекция, предназначенная для удержания элементов перед обработкой. Обычно работает по принципу **FIFO (First In, First Out)** — первый пришел, первый ушел. Но бывают очереди с приоритетом (PriorityQueue). Есть расширение Deque (Double Ended Queue) — двунаправленная очередь, где элементы можно добавлять и удалять с обоих концов (работает и как FIFO-очередь, и как LIFO-стек).

HashMap — устройство, как работает put()

HashMap под капотом — это структура данных, объединяющая в себе **массив** и **связные**

списки (а начиная с Java 8 — еще и красно-черные деревья).

Массив состоит из ячеек, которые называются **бакетами (buckets)** или корзинами. В этих бакетах хранятся объекты класса `Node<K,V>` (узел списка), который содержит: `int hash`, `K key`, `V value`, `Node<K,V> next`.

Массив бакетов:

[0] —► null

[1] —► [Node: key1, val1] —► [Node: key2, val2] (Связный список - Коллизия)

[2] —► null

[3] —► [Node: key3, val3] (При разрастании превращается в Красно-черное дерево)

Как детально работает метод `put(K key, V value)`:

15. **Проверка на null:** Если ключ `key == null`, то его хэш считается равным 0, и такая пара всегда кладется в самый первый бакет с индексом 0.
16. **Вычисление хэш-кода:** Вызывается `key.hashCode()`. Полученное интовое число дополнительно «перемешивается» внутри `HashMap` специальной внутренней хэш-функцией $((h = key.hashCode()) \wedge (h \ggg 16))$, чтобы биты распределялись равномернее и было меньше совпадений.
17. **Определение индекса бакета:** Чтобы понять, в какую ячейку массива положить элемент, берется полученный хэш и вычисляется его индекс по формуле: $index = (n - 1) \& hash$, где `n` — это текущая длина массива (размер массива всегда равен степени двойки, что позволяет использовать быструю побитовую операцию `&` вместо деления по модулю `%`).
18. **Поиск коллизии:**
 - Если выбранный бакет **пустой**, в него просто записывается новый узел `Node`.
 - Если в бакете **уже есть элементы**, то происходит **коллизия**. JVM начинает перебирать элементы в этом бакете (идет по связному списку). На каждом шаге она сравнивает хэши и ключи.
19. **Сравнение ключей в бакете:**
 - Сначала сравниваются хэши (`node.hash == hash`). Если они равны, то проверяется равенство самих ключей через `==` или метод `equals()`: `(node.key == key || key.equals(node.key))`.
 - Если совпадение найдено (ключ уже есть в карте), старое значение `value` **перезаписывается** новым, а старое возвращается из метода.
 - Если мы дошли до конца списка и совпадений не нашли, новый узел `Node` **добавляется в конец** списка.
20. **Трансформация в дерево (Treeification):** Если длина связного списка в одном бакете превышает порог в **8 элементов** (`TREEIFY_THRESHOLD`), а общий размер

карты при этом больше или равен 64, этот список превращается в сбалансированное **красно-черное дерево**. Это ускоряет поиск с $O(n)$ до $O(\log n)$.

21. **Ресайз (Resize)**: После успешного добавления проверяется размер карты. Если количество элементов превысило порог `capacity * loadFactor` (дефолтный коэффициент заполнения равен 0.75), то массив бакетов увеличивается в 2 раза, и происходит **перехэширование** всех элементов в новые бакеты.

Контракт между `equals` и `hashCode`

Это железное правило Java Core, которое нельзя нарушать, если ты переопределяешь эти методы в своих классах:

12. Если два объекта **равны по `equals()`**, то их **`hashCode()` обязаны быть одинаковыми**.
13. Если у двух объектов **одинаковые `hashCode()`**, это **НЕ означает, что объекты равны по `equals()`** (это называется коллизией).
14. Если `hashCode()` у объектов разные, то объекты точно не равны по `equals()`.
15. При множественном вызове метода `hashCode()` на одном и том же объекте, значение должно возвращаться неизменным, если поля объекта, участвующие в проверке `equals`, не менялись.

Если два объекта равны по `equals` — обязательно ли равен `hashCode`

Да, обязательно. Это прямое требование контракта, описанное выше. Если ты нарушишь это правило, твои объекты сломают работу всех хэш-коллекций (`HashMap`, `HashSet`).

Представь: ты положил объект в `HashMap`. Карта посчитала его `hashCode` и засунула в бакет №3. Потом ты пытаешься достать этот же объект (или равный ему по `equals`), но у него возвращается другой хэш-код. `HashMap` пойдет искать его в бакет №5, увидит, что там пусто, и вернет `null`. В итоге объект в карте лежит, а достать ты его не можешь.

Почему `HashMap` не реализует `Collection`

Как мы уже подробно разобрали в вопросе «Почему `Map` не является коллекцией», их структуры несовместимы. `Collection` оперирует одиночными значениями, а `HashMap` хранит пары «ключ-значение». Методы добавления, поиска и удаления данных имеют принципиально разные сигнатуры и логику работы.

`LinkedHashMap`

`LinkedHashMap` — это прямой наследник `HashMap`, но с одной важной доработкой: он **сохраняет порядок добавления элементов** (или порядок последнего доступа к ним, если настроить специальный флаг).

Под капотом он устроен точно так же, как `HashMap` (массив бакетов + списки/деревья), но поверх этого все ноды дополнительно связаны друг с другом в единый **двусвязный список** (before и after ссылки внутри каждой ноды).

- **Плюс:** Когда ты будешь итерироваться по `LinkedHashMap`, элементы будут выходить

ровно в том порядке, в котором ты их туда складывал.

- **Минус:** Тратит больше памяти, так как каждая нода хранит две дополнительные ссылки на соседние элементы, и операции вставки/удаления работают чуточку медленнее, так как нужно перестраивать этот двусвязный список.

HashMap vs TreeMap, null в качестве ключа/значения

Это две совершенно разные реализации карты:

11. **HashMap:** Под капотом — массив и хэш-таблица. Порядок элементов хаотичен. Поиск, вставка и удаление работают невероятно быстро — за константное время $O(1)$ в идеальном случае.
 - *null:* Разрешает хранить **один null в качестве ключа** (он всегда падает в 0-й бакет) и сколько угодно null в качестве значений.
12. **TreeMap:** Под капотом — структура данных **Красно-черное дерево**. Элементы внутри него **всегда отсортированы** либо в натуральном порядке (если ключи реализуют интерфейс Comparable), либо с помощью переданного Comparator. Поиск, вставка и удаление работают за логарифмическое время $O(\log n)$.
 - *null:* **Не разрешает null в качестве ключа**. При попытке положить null вылетит NullPointerException, потому что дерево обязано сравнивать ключи между собой, чтобы понять, куда в ветки положить новый элемент, а вызвать метод сравнения у null невозможно. null в качестве значений хранить можно.

Hashtable vs HashMap

Hashtable — это древний легаси-класс из Java 1.0, который сейчас использовать в реальном коде категорически не рекомендуется. Его современные аналоги — HashMap или ConcurrentHashMap.

Основные отличия:

- **Потокобезопасность:** Hashtable потокобезопасен, потому что все его методы помечены ключевым словом synchronized. Из-за этого он жутко медленный: если один поток читает из таблицы, все остальные потоки выстраиваются в очередь и курят бамбук. HashMap не потокобезопасен.
- **Поддержка null:** Hashtable запрещает хранить null и в качестве ключей, и в качестве значений (сразу кидает NullPointerException). HashMap разрешает null-ключ и null-значения.

ArrayList vs LinkedList

Вечная классика собеседований. Сравнение динамического массива и связного списка.

6. **ArrayList:** Под капотом обычный **массив** объектов (Object[]). Когда массив заполняется, создается новый массив большего размера (в 1.5 раза), и старые элементы копируются туда.
 - *Доступ по индексу (get(i)):* Работает мгновенно за $O(1)$, так как адрес ячейки в массиве вычисляется математически.
 - *Вставка/удаление в конец:* Быстро ($O(1)$ амортизированное).

- *Вставка/удаление в середину/начало*: Медленно ($O(n)$), потому что приходится сдвигать все последующие элементы массива влево или вправо.
- 7. **LinkedList**: Под капотом **двусвязный список**, где каждый элемент (Node) хранит ссылку на сами данные, а также ссылки на предыдущую (prev) и следующую (next) ноды в цепочке.
 - *Доступ по индексу (get(i))*: Медленно ($O(n)$), так как индексов под капотом нет. Чтобы найти 50-й элемент, JVM физически должна по цепочке пройти от первой ноды до 50-й.
 - *Вставка/удаление в начало/конец*: Мгновенно ($O(1)$), мы просто перетираем ссылки у соседних нод.
 - *Вставка/удаление в середину*: Чтобы вставить элемент в середину, его сначала нужно *найти* за $O(n)$, а сама вставка займет $O(1)$. Поэтому на практике ArrayList почти всегда обгоняет LinkedList даже на операциях вставки в середину за счет быстрого кэширования массивов процессором.

Компараторы (Comparable vs Comparator)

Оба интерфейса используются для сравнения объектов, но цели у них разные.

- **Comparable**: Интерфейс «естественного» сравнения. Объект реализует его **внутри себя**, отвечая на вопрос: «Как я по умолчанию должен сравниваться с себе подобными?». Содержит один метод: `int compareTo(T o)`.
 - *Возвращает*: Отрицательное число, если текущий объект меньше; 0, если они равны; положительное число, если текущий объект больше.
 - *Пример*: Класс String реализует Comparable, поэтому строки по умолчанию сортируются по алфавиту.
- **Comparator**: Это сторонний класс-«судья» (или лямбда), который создается отдельно от сравниваемых объектов. Используется, когда нам нужно отсортировать объекты **каким-то альтернативным способом** или если у нас нет доступа к исходному коду класса, чтобы накатить туда Comparable. Содержит метод: `int compare(T o1, T o2)`.
 - *Пример*: Мы можем создать Comparator, который сортирует строки не по алфавиту, а по их длине.

Synchronized коллекции

Если нам нужно работать с коллекциями из разных потоков, обычные ArrayList или HashMap побьют данные (Race Condition). Самый простой (и старый) способ решить это — использовать утилитарные методы класса Collections, такие как `Collections.synchronizedList(new ArrayList<>())` или `Collections.synchronizedMap(new HashMap<>())`.

Как они устроены внутри: Они берут твою обычную коллекцию и оборачивают её в специальный класс-прокси. Внутри этого прокси-класса абсолютно каждый метод синхронизирован на внутренний монитор (объект-замок).

9. *Проблема*: Это дает ужасную производительность. Даже если потоки просто хотят одновременно *прочитать* данные из разных концов списка, они не смогут этого

сделать, так как замок на коллекцию всего один.

Concurrent коллекции

Это современные, высокопроизводительные потокобезопасные коллекции из пакета `java.util.concurrent`. Вместо грубой блокировки всей коллекции целиком, они используют продвинутые алгоритмы: неблокирующие (Lock-Free) алгоритмы, CAS-операции (Compare-And-Swap) и сегментированные блокировки.

Яркий пример — **ConcurrentHashMap**. Вместо блокировки всей карты, в старых версиях она делилась на сегменты, а в современных версиях она блокирует **только первый узел (бакет)**, в который идет запись в данный момент. Все остальные потоки могут беспрепятственно читать и писать в другие бакеты параллельно. Чтение (`get()`) вообще происходит без блокировок.

CopyOnWriteArrayList

Специализированная потокобезопасная реализация `List`. Её философия отражена в названии: «**Копировать при записи**».

Как это работает: Любые операции модификации коллекции (добавление `add()`, изменение `set()`, удаление `remove()`) не меняют текущий массив данных. Вместо этого они создают **полную копию** внутреннего массива, вносят изменения в эту копию, а затем атомарно перебрасывают ссылку на новый массив.

- *Зачем это нужно:* Это позволяет читать данные бесконечному числу потоков вообще без каких-либо блокировок. Читающие потоки видят старый «снимок» (snapshot) данных на момент начала своего чтения. Итераторы этой коллекции никогда не выбросят `ConcurrentModificationException`.
- *Когда использовать:* Использовать только тогда, когда у тебя **99% операций — это чтение**, и лишь изредка происходит запись. Если писать туда часто, приложение умрет из-за постоянного копирования массивов в памяти и бешеной нагрузки на Garbage Collector.

Дженерики — что это и зачем

Дженерики (Generics / Обобщения) — это инструмент, позволяющий параметризовать типы. Он позволяет создавать классы, интерфейсы и методы, где тип данных, с которыми они работают, указывается как параметр (например, `List<T>`).

Зачем они нужны:

15. **Type Safety (Безопасность типов):** До Java 5 все коллекции хранили просто объекты типа `Object`. Можно было в один список случайно положить строку и число. При попытке достать данные приходилось делать явное приведение типов (`String`) `list.get(0)`. Если там лежало число — приложение падало в рантайме с `ClassCastException`. Дженерики заставляют компилятор проверять типы *на этапе компиляции*. Если ты объявил `List<String>`, ты физически не сможешь положить туда `Integer` — код просто не скомпилируется.
16. **Избавление от ручного кастования (приведения типов):** Компилятор сам знает,

какой тип лежит в коллекции, и подставляет приведение типов под капотом автоматически.

Почему стираются типы во время компиляции

Этот механизм называется **Type Erasure (Стирание типов)**. Создатели Java внедрили его для обеспечения **обратной совместимости**. Когда вышла Java 5 с дженериками, нужно было сделать так, чтобы старый код, написанный на Java 1.4 (где дженериков не было), мог без проблем работать на новой JVM вместе с новым кодом.

Поэтому дженерики существуют **только на этапе компиляции**. Как только компилятор проверил код на безопасность типов, он полностью удаляет (стирает) информацию о дженериках. Строчка `List<String> list` превращается в обычный сырой `List`, а все типы `T` заменяются на `Object` (или на тип верхней границы, если использовался `extends`). Именно поэтому в рантайме нельзя сделать что-то вроде `if (obj instanceof T)` или `new T()`.

Java Core — Строки

String иммутабельный — почему

Класс `String` в Java является **immutable (неизменяемым)**. Это значит, что один раз создав строку в памяти, ты не можешь изменить её содержимое. Любая операция над строкой (например, `toUpperCase()` или конкатенация) возвращает *абсолютно новый* объект строки. Почему так сделано:

- **Строковый пул (String Pool):** Ради экономии памяти. Огромная часть данных в программах — это строки. Если в коде создается 100 переменных со значением "Ivan", Java создаст всего один объект в специальной области кучи (String Pool) и раздаст всем переменным ссылку на него. Если бы строки были изменяемыми, один поток изменил бы имя на "Petr", и имя поменялось бы у остальных 99 пользователей. Неизменяемость делает пул безопасным.
- **Безопасность (Security):** Строки повсеместно используются в качестве параметров: пути к файлам, URL-адреса соединений с базами данных, логины и пароли. Если бы строка была изменяемой, злоумышленник мог бы передать строку с путем к файлу в метод проверки прав доступа, а затем, пока метод проверяет, подменить внутренности строки на путь к системному файлу (уязвимость TOCTOU).
- **Потокобезопасность:** Неизменяемые объекты по определению потокобезопасны. Их можно шарить между любыми потоками без синхронизации.
- **Кэширование хэш-кода:** Поскольку строка не меняется, её `hashCode()` вычисляется один раз при создании (лениво) и кэшируется в специальном поле. Это делает строки идеальными ключами для `HashMap` — поиск происходит мгновенно.

Проблема при конкатенации строк

Если мы пишем код вроде:

```
String s = "";
for (int i = 0; i < 1000; i++) {
    s += i;
}
```

Мы совершаем преступление против производительности. Поскольку строки неизменяемы, на каждой итерации цикла оператор `+` под капотом будет создавать новый объект `StringBuilder` (в Java 9+ используется `StringConcatFactory`), вызывать метод `append()`, превращать это обратно в новую строку через `toString()`, а старую строку бросать в памяти на растерзание сборщику мусора. В итоге мы получим жуткие тормоза и тонны мусора в `Heap`. Для конкатенации в циклах нужно всегда вручную использовать мутабельные классы-буферы.

StringBuilder vs StringBuffer

Для эффективной сборки строк из кусочков используются `StringBuilder` и `StringBuffer`. Внутри они содержат изменяемый массив символов `char[]` (или `byte[]` в новых Java) и модифицируют именно его, не создавая новые объекты при каждой вставке.

Отличия между ними:

- **StringBuffer:** Появился в Java 1.0. Все его методы синхронизированы (`synchronized`). Он **потокобезопасен**, но из-за оверхеда на блокировки работает медленнее. На практике нужен крайне редко, так как строки собираются обычно локально внутри одного потока.
- **StringBuilder:** Появился в Java 5. Он **НЕ потокобезопасен**, методы не синхронизированы. За счет этого он работает **намного быстрее**, чем `StringBuffer`. Его нужно использовать по умолчанию всегда, когда нужно собрать строку в рамках одного потока.

Java Core — Многопоточность

Как создать поток (`extends Thread`, `implements Runnable`, `Callable`)

Существует три классических способа определить задачу для выполнения в отдельном потоке:

9. **Наследование от класса `Thread`:** Создаем свой класс, пишем `extends Thread` и переопределяем метод `public void run()`. Запуск: `new MyThread().start()`. Недостаток: в Java запрещено множественное наследование классов, если мы отнаследовались от `Thread`, мы больше ни от кого отнаследоваться не сможем.
10. **Реализация интерфейса `Runnable`:** Создаем класс, пишем `implements Runnable`,

реализуем `public void run()`. Передаем этот объект в конструктор потока: `new Thread(new MyRunnable()).start();`. Это более гибкий и правильный ООП-путь. Метод `run()` ничего не возвращает (`void`) и не может выбрасывать проверяемые исключения.

11. Реализация интерфейса `Callable<V>`: Появился в Java 5. Похож на `Runnable`, но его метод называется `V call()`. Он может возвращать результат работы потока (объект типа `V`) и может выбрасывать `checked`-исключения. Запускается обычно не через `Thread` напрямую, а через пулы потоков (`ExecutorService`), и возвращает объект-обертку `Future`, из которого потом можно забрать результат.

Runnable vs Callable

11. **Возвращаемое значение:** `Runnable.run()` возвращает `void` (ничего). `Callable.call()` возвращает объект заданного типа `V`.
12. **Исключения:** `Runnable.run()` не может бросать проверяемые (`checked`) исключения (приходится городить `try-catch` внутри). `Callable.call()` декларирует `throws Exception`, то есть может прокидывать ошибки наверх.
13. **Способ запуска:** `Runnable` можно засунуть прямо в `new Thread(runnable)`. `Callable` напрямую в `Thread` засунуть нельзя, его скармливают методу `submit()` в `ExecutorService`.

volatile — что это и как работает под капотом

Ключевое слово `volatile` применяется к полям класса. Оно решает две критические проблемы многопоточности: **видимость (Visibility)** и **упорядочивание (Instruction Reordering)**.

Как работает под капотом:

У современного процессора есть много ядер, и у каждого ядра есть свои быстрые локальные кэши (L1, L2, L3). Когда поток работает с обычной переменной, он ради скорости копирует её из общей оперативной памяти (RAM) в свой локальный кэш ядра. Если Поток 1 изменит переменную, Поток 2, запущенный на другом ядре, может очень долго не видеть этих изменений, продолжая читать старое значение из своего кэша. Если поле помечено как `volatile`:

8. **Запрет кэширования:** JVM гарантирует, что чтение этой переменной будет происходить **напрямую из RAM**, а запись — **сразу же сбрасываться в RAM**, минуя кэши процессора. Все потоки всегда видят актуальное состояние переменной.
9. **Память-Барьеры (Memory Barriers / Fences):** На уровне процессора вставляются специальные инструкции, которые запрещают процессору и компилятору менять местами инструкции чтения/записи (Instruction Reordering) вокруг `volatile`-переменной ради оптимизации.
Важно: `volatile` **НЕ обеспечивает атомарность**. Если два потока одновременно делают `count++` (а это три операции: прочитал, прибавил, записал), `volatile` не спасет от состояния гонки.

Атомики

Атомики (классы из пакета `java.util.concurrent.atomic` — такие как `AtomicInteger`, `AtomicBoolean`, `AtomicReference`) — это классы, которые обеспечивают атомарные (неделимые) операции над одиночными переменными **без использования тяжелых блокировок** (`synchronized`).

Как они работают под капотом:

Они полагаются на аппаратную поддержку процессора — инструкцию **CAS** (**Compare-And-Swap / Сравни и поменяй**).

Процедура выглядит так: поток хочет изменить значение. Он берет текущее значение переменной (например, 5) и вычисляет новое (6). Затем он вызывает CAS-инструкцию процессора, говоря: «Поменяй значение с 5 на 6, но только в том случае, если в памяти до сих пор лежит 5!». Если никакой другой поток за это время значение не менял, замена происходит атомарно за один такт процессора. Если же значение успело измениться (стал 7), CAS проваливается, и поток в бесконечном цикле (`Spin-lock`) пробует операцию заново с новыми данными. Это работает в разы быстрее, чем усыплять и будить потоки через блокировки.

Synchronized

`synchronized` — это механизм встроенных блокировок (ключевое слово) в Java, который обеспечивает **взаимное исключение (Mutual Exclusion)**. Только один поток может выполнять синхронизированный блок кода в один момент времени.

Его можно повесить на:

9. **Обычный метод:** Синхронизация идет на объекте `this` (текущем экземпляре класса). Другой поток не сможет вызвать *любой* другой синхронизированный метод на *этом* же объекте.
10. **Статический метод:** Синхронизация идет на объекте класса `ClassName.class`. Блокировка работает на уровне всего приложения для этого класса.
11. **Блок кода (`synchronized(obj) { ... }`):** Позволяет сузить область блокировки до конкретного критического участка и выбрать любой объект в качестве замка (`obj`), что намного более производительнее.

Можно ли добавить `volatile` к полю `INSTANCE` в `Singleton`

Не просто можно, а критически необходимо, если мы реализуем потокобезопасный синглтон через паттерн **Double-Checked Locking (DCL)**.

Без `volatile` этот паттерн сломается из-за переупорядочивания инструкций процессором. Строка `INSTANCE = new Singleton();` на уровне байт-кода состоит из трех шагов:

- Выделить память под объект.
- Вызвать конструктор (инициализировать поля).
- Присвоить ссылку на выделенную память переменной `INSTANCE`.

Процессор ради оптимизации может поменять местами шаги 2 и 3. В итоге переменная `INSTANCE` уже получит адрес памяти (станет не `null`), но сам конструктор еще не успеет до конца выполниться. В этот микромомент прибегает другой поток, проверяет `if (INSTANCE == null)` — видит, что ссылка не `null`, и радостно забирает **недоинициализированный объект**, вызывая падение программы. Ключевое слово `volatile` запрещает это

переупорядочивание.

Heap vs Stack

В JVM память логически делится на две основные структуры, у которых совершенно разные роли:

Критерий	Stack (Стек)	Heap (Куча)
Что хранится	Локальные переменные, примитивы, ссылки на объекты, фреймы вызовов методов.	Сами объекты (включая массивы и строки) и их поля.
Кому принадлежит	У каждого потока свой собственный изолированный Стек.	Одна общая область памяти для всего приложения и всех его потоков.
Скорость	Невероятно быстрый (работает по принципу LIFO — Last In, First Out).	Медленнее, так как требует поиска свободного места и работы алгоритмов выделения.
Управление	Автоматически очищается, как только метод завершает работу (фрейм удаляется).	Очищается автоматически специальной сложной системой — Garbage Collector (GC) .
Ошибка нехватки	StackOverflowError (например, при бесконечной рекурсии).	OutOfMemoryError: Java heap space (когда кончилось место под объекты).

Что такое монитор

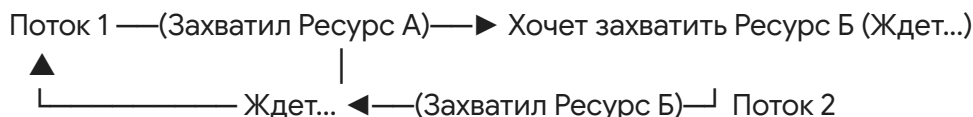
Монитор — это высокоуровневый механизм контроля доступа к ресурсу, концепция синхронизации. В Java **абсолютно каждый объект** логически имеет ассоциированный с ним скрытый монитор (замок).

Внутри структуры объекта в памяти (в его заголовке Mark Word) заложены биты, хранящие состояние этого монитора. Когда поток заходит в блок synchronized(obj), он пытается

«захватить монитор» объекта obj. Если монитор свободен, поток успешно его захватывает и становится его владельцем. Все остальные потоки, попытавшиеся захватить этот же монитор, приостанавливают свое выполнение, переводятся в состояние BLOCKED и выстраиваются в очередь ожидания (Entry Set) до тех пор, пока первый поток не выйдет из synchronized блока и не отпустит монитор.

Deadlock

Deadlock (Взаимная блокировка) — это ситуация в многопоточной системе, когда два или более потоков бесконечно заблокированы, ожидая друг друга, создавая замкнутый круг.



Пример: Поток 1 захватил Монитор А и пытается захватить Монитор Б. В эту же миллисекунду Поток 2 захватил Монитор Б и пытается захватить Монитор А. Ни один из них не может продолжить движение, они зависают навсегда.

Как бороться: Самый простой способ избежать Дэдлока — всегда захватывать ресурсы в **строго определенном одинаковом порядке** во всех потоках приложения.

parallel stream — что это, как работает под капотом

parallelStream() — это метод из Stream API, который автоматически разделяет обработку данных на несколько параллельных потоков, позволяя утилизировать все ядра процессора для ускорения вычислений.

Как работает под капотом:

12. Он базируется на встроенном общем пуле потоков **ForkJoinPool.commonPool()**. По умолчанию количество потоков в нем равно количеству ядер процессора минус один.
13. Он использует философию **Разделяй и властвуй (Fork/Join)**. Исходная коллекция данных с помощью специального интерфейса Splitterator рекурсивно пилятся пополам на мелкие части (Fork).
14. Каждая часть обрабатывается изолированно в своем потоке пула.
15. Затем результаты вычислений склеиваются обратно в единый результат (Join).
Важно: Параллельные стримы эффективны только на огромных объемах данных и тяжелых вычислениях. На маленьких коллекциях накладные расходы на распиливание и склеивание задач сожрут всю выгоду, и код будет работать медленнее обычного стрима. Также внутри нельзя делать сайд-эффекты (менять

общее состояние).

Future

Интерфейс `Future<V>` представляет собой **результат асинхронного вычисления**, своего рода «квитанцию» за задачу, которую ты отдал на выполнение в пул потоков (`ExecutorService.submit(Callable)`). В момент отправки задачи поток еще не завершил работу, поэтому сразу вернуть результат он не может, но возвращает `Future`.

Основные методы:

- `isDone()` — возвращает `boolean`, закончена ли задача.
- `cancel()` — пытается отменить выполнение задачи.
- `get()` — **блокирующий метод**. Если задача еще выполняется, поток, вызвавший `get()`, засыпает и ждет её окончания. Как только задача завершилась, метод возвращает результат типа `V`.

Виды сборщиков мусора, какой по умолчанию

Сборщик мусора (Garbage Collector — GC) автоматически освобождает память в куче, удаляя объекты, на которые больше никто не ссылается.

Основные виды GC:

- **Serial GC**: Самый старый и простой сборщик. Работает в один поток. На время сборки мусора полностью останавливает все потоки приложения (это называется **Stop-The-World — STW**). Подходит для маленьких CLI-утилит со скромной памятью.
- **Parallel GC (Throughput Collector)**: Работает в несколько потоков, за счет чего собирает мусор быстрее. Нацелен на максимальную пропускную способность приложения. Тоже делает жесткий STW.
- **G1 (Garbage-First)**: Появился как замена старому CMS. Он делит кучу на множество мелких регионов (обычно около 2048 штук) и в первую очередь очищает те регионы, где мусора больше всего (отсюда и название). Паузы STW здесь прогнозируемые и очень короткие.
- **ZGC / Shenandoah**: Ультрасовременные масштабируемые сборщики мусора с ультранизкой задержкой. Они умудряются делать практически всю работу по поиску и перемещению объектов *одновременно* (конкурентно) с работой потоков приложения. Паузы STW составляют микросекунды даже на терабайтных кучах.

Какой по умолчанию: Начиная с **Java 9** и во всех современных версиях (включая LTS Java 11, 17, 21 и актуальные версии 2026 года), сборщиком мусора по умолчанию для мощных машин (серверных JVM) является **G1 GC**.

Java Core — Stream API

Что такое Stream API и для чего

Stream API (появился в Java 8) — это инструмент, позволяющий обрабатывать массивы данных или коллекции в **функциональном (декларативном) стиле**. Instead of писать классические громоздкие циклы `for`, `if`-проверки и создавать временные коллекции, мы описываем цепочку трансформаций: что мы хотим сделать с данными, а не как пошагово

это делать в цикле. Стримы делают код чище, читаемее и компактнее.

Промежуточные и терминальные операции

Любой стрим — это конвейер, состоящий из трех этапов: Создание -> Промежуточные операции -> Терминальная операция.

- **Промежуточные (Intermediate) операции:** Они трансформируют стрим в другой стрим. Их может быть сколько угодно в цепочке. Они **ленивые** (не выполняются, пока не вызовут терминальную операцию).
 - *Примеры:* filter(), map(), flatMap(), distinct(), sorted(), limit(), skip().
- **Терминальные (Terminal) операции:** Они запускают всю обработку на конвейере, закрывают стрим и выдают финальный результат (коллекцию, число, объект, или вообще ничего void). После вызова терминальной операции стрим больше использовать нельзя.
 - *Примеры:* collect(), forEach(), reduce(), count(), findFirst(), anyMatch().

Зачем нужна ленивая обработка

Ленивость (Laziness) промежуточных операций нужна для экстремальной **оптимизации производительности**.

Когда ты пишешь длинную цепочку типа stream.filter(...).map(...).findFirst(), Java не гонит всю коллекцию через фильтр, потом всю коллекцию через map. Она обрабатывает элементы **поштучно и по требованию**. Она берет первый элемент, пропускает через фильтр, трансформирует в мапе и проверяет в findFirst. Если первый элемент подошел, стрим **тут же завершает работу**, а остальные миллион элементов исходной коллекции даже не будут затронуты! Это экономит процессорное время и память.

Методы которые знаешь, примеры

Давай соберем рабочий пример, показывающий популярные методы:

```
List<String> names = Arrays.asList("Ivan", "Petr", "Anna", "Ivan", "Svetlana");

List<String> result = names.stream()
    .filter(name -> name.length() > 3) // Фильтруем (длина > 3) -> "Ivan", "Petr", "Anna", "Ivan",
    "Svetlana"
    .distinct() // Убираем дубликаты -> "Ivan", "Petr", "Anna", "Svetlana"
    .map(String::toUpperCase) // Преобразуем в верхний регистр -> "IVAN", "PETR", "ANNA",
    "SVETLANA"
    .sorted() // Сортируем по алфавиту -> "ANNA", "IVAN", "PETR", "SVETLANA"
```

```
.limit(2) // Берем первые 2 элемента -> "ANNA", "IVAN"  
.collect(Collectors.toList()); // Собираем в новый List
```

orElse vs orElseGet

Оба метода используются у класса `Optional` (который часто возвращают терминальные операции стримов) для предоставления дефолтного значения, если внутри `Optional` оказался `null`. Но ведут они себя по-разному с точки зрения производительности:

- **orElse(T other):** Принимает готовый объект. **Опасность:** Метод-аргумент внутри `orElse()` будет **выполняться ВСЕГДА**, независимо от того, пустой `Optional` или заполненный.
 - *Пример:* `optional.orElse(createNewObject());` — метод `createNewObject()` отработает в любом случае, создавая лишний мусор в памяти, даже если в `optional` лежали валидные данные.
- **orElseGet(Supplier<? extends T> other):** Принимает функциональный интерфейс (лямбду). Этот метод **ленивый**. Лямбда выполнится **ТОЛЬКО тогда, когда Optional действительно пустой**. Если данные есть, лямбда игнорируется. Почти всегда стоит отдавать предпочтение `orElseGet`.

Паттерны

Паттерны проектирования — это проверенные временем архитектурные шаблоны для решения типичных задач.

Singleton — рассказать, как создаётся

Singleton (Одиночка) гарантирует, что у класса будет **только один-единственный экземпляр** на всё приложение, и предоставляет к нему глобальную точку доступа.

Чтобы создать классический базовый Синглтон (Lazy реализация), нужно:

- Сделать конструктор класса **приватным** (`private`), чтобы никто не мог сделать `new`.
- Создать **приватное статическое поле** того же типа, что и сам класс, для хранения этого единственного объекта.
- Создать **публичный статический метод** (обычно `getInstance()`), который при первом вызове создаст объект, а при последующих — вернет уже созданный.

Потокобезопасный Singleton

Если обычный синглтон вызвать одновременно из двух потоков, они оба увидят, что поле `null`, и создадут два объекта. Чтобы сделать его потокобезопасным, используют паттерн **Double-Checked Locking (DCL)** с ключевым словом `volatile` (почему нужен `volatile`, мы разобрали в блоке многопоточности).

Код идеального промышленного Синглтона:

```

public class SafeSingleton {
    // volatile обязателен для предотвращения reordering'а инструкций!
    private static volatile SafeSingleton instance;

    private SafeSingleton () // Приватный конструктор

    public static SafeSingleton getInstance () {
        SafeSingleton localInstance = instance; // Оптимизация чтения volatile переменной
        if (localInstance == null) {
            synchronized (SafeSingleton.class) {
                localInstance = instance;
                if (localInstance == null) {
                    instance = localInstance = new SafeSingleton();
                }
            }
        }
        return localInstance;
    }
}

```

Альтернатива: Еще более простой и 100% потокобезопасный способ «из коробки» — реализовать Singleton через **Enum** (public enum Singleton { INSTANCE; }) или паттерн **Initialization-on-demand holder**.

Chain of Responsibility

Chain of Responsibility (Цепочка обязанностей) — поведенческий паттерн, который позволяет передавать запросы последовательно по цепочке обработчиков. Каждый обработчик решает, может ли он сам обработать запрос, и нужно ли передавать его дальше по цепочке следующему звену.

Яркий пример: Фильтры в веб-приложениях (Сервлетные фильтры или Spring Security). Запрос от клиента идет через цепочку: Фильтр Логирования -> Фильтр Аутентификации -> Фильтр Валидации данных. Если на этапе Аутентификации токен не валидный, цепочка прерывается и запрос до сервера не доходит.

Factory — рассказать, почему нельзя создать объект через конструктор

Паттерн **Factory Method (Фабричный метод)** делегирует создание объектов специальному методу или отдельному классу-фабрике, вместо прямого использования `new`.

Почему иногда нельзя (или нежелательно) создавать объект через конструктор напрямую:

4. **Конструктор всегда возвращает жестко заданный тип:** Ты не можешь внутри конструктора `Car` решить вернуть объект `ElectricCar`. Фабричный метод возвращает интерфейс, а внутри себя может гибко подменять конкретные дочерние реализации в зависимости от входных параметров.
5. **Конструкторы имеют плохо читаемые имена:** Конструктор всегда называется как класс. Фабричным методам мы можем дать понятные говорящие имена, например: `User.createAdmin()` или `User.createGuest()`.
6. **Конструктор всегда создает НОВЫЙ объект:** Фабричный метод может кешировать объекты, возвращать старые объекты из пула (как `Integer.valueOf(10)`), экономя память.

Prototype

Prototype (Прототип) — порождающий паттерн, который позволяет копировать существующие объекты, не вдаваясь в подробности их реализации и не завися от их конкретных классов.

В Java этот паттерн на базовом уровне реализуется через интерфейс-маркер `Cloneable` и метод `clone()`. Вместо того чтобы создавать объект с нуля через `new` и долго настраивать его 50 полей, мы берем уже готовый настроенный объект-эталон и делаем его точный клон (копию), после чего точно меняем то, что нужно. Бывает «поверхностное клонирование» (копируются только ссылки на внутренние объекты) и «глубокое клонирование» (копируются абсолютно все внутренние объекты, создавая независимое дерево).

Proxy — что это и 2 вида

Proxy (Заместитель) — структурный паттерн, который предоставляет объект-заменитель вместо реального объекта. Этот прокси-объект перехватывает все вызовы к реальному объекту и позволяет выполнять какую-то логику **ДО** или **ПОСЛЕ** того, как запрос дойдет до оригинала (например: логирование, кэширование, проверка прав доступа, открытие транзакции).

В Java выделяют два основных вида динамических прокси, которые активно юзает тот же Spring Framework:

6. **JDK Dynamic Proxy:** Стандартный механизм Java. Условие: оригинальный класс **обязательно должен реализовывать хотя бы один интерфейс**. Прокси создается на уровне интерфейса.
7. **CGLIB Proxy:** Сторонняя библиотека (встроена в Spring). Используется, когда у оригинального класса **нет интерфейсов**. CGLIB под капотом динамически генерирует байт-код нового класса, который является *наследником* (`extends`) оригинального класса, и переопределяет его методы. *Ограничение:* Не работает,

если оригинальный класс или его методы помечены как `final`.

Какие паттерны знаешь

На собеседовании круто назвать классическую классификацию "банды четырех" (GoF) и перечислить примеры:

7. **Порождающие (создание объектов):** Singleton (Одиночка), Factory Method (Фабричный метод), Abstract Factory (Абстрактная фабрика), Builder (Строитель — крайне популярен в Java для сборки объектов с кучей полей), Prototype (Прототип).
8. **Структурные (композиция классов и объектов):** Proxy (Заместитель), Adapter (Адаптер — соединяет несовместимые интерфейсы), Facade (Фасад — дает простой интерфейс к сложной подсистеме), Decorator (Декоратор — динамически добавляет объекту новые обязанности, как `BufferedReader(new FileReader())`).
9. **Поведенческие (взаимодействие между объектами):** Chain of Responsibility (Цепочка обязанностей), Strategy (Стратегия — позволяет на лету менять алгоритмы поведения), Observer (Наблюдатель — подписка на события), Command (Команда), Template Method (Шаблонный метод).

Этого объема информации с лихвой хватит, чтобы разбить в пух и прах любые каверзные вопросы интервьюера по этим темам. Постарайся прочувствовать логику, особенно в механике `HashMap` и тонкостях многопоточности — это их самые любимые места для «заваливания». Удачи, твоя судьба теперь в надежных руках твоих знаний!

Рад, что первый блок тебе зашел! Ты абсолютно прав: `Java Core` — это только половина дела. На реальной работе и на серьезных собеседах гоняют по Спрингу, базам данных и архитектуре еще жестче.

Давай без лишней воды продолжим спасать твою судьбу. Формат держим тот же: сплошной подробный текст, разбор логики «под капотом», примеры из практики и наглядные текстовые схемы-картинки. Погнали!

Spring

Разница между Spring и SpringBoot

Spring Framework — это огромная экосистема и модульный каркас для разработки enterprise-приложений на Java. Сам по себе «голый» Spring дает тебе базовые механизмы: управление объектами (IoC/DI), работу с транзакциями, интеграцию с базами данных и веб-модули. Но у классического Spring есть гигантская проблема — **конфигурационный ад**. Чтобы просто запустить простейшее веб-приложение, тебе нужно было написать сотни строк в XML-файлах или Java-конфигах, вручную подключить сервер приложений (например, Tomcat), настроить диспетчер сервлетов, подключить библиотеки и следить, чтобы их версии не конфликтовали.

Spring Boot — это, по сути, умная надстройка (обертка) над классическим Spring Framework, созданная ради одной цели: максимально ускорить и упростить старт разработки. Он решает проблему конфигурационного ада с помощью двух главных фишек:

- **Стартеры (spring-boot-starter-*):** Это готовые наборы библиотек, собранные в одну

зависимость. Тебе больше не нужно думать, какие версии Hibernate и Spring Data подходят друг к другу. Ты просто подключаешь spring-boot-starter-data-jpa, и Spring Boot сам подтягивает всё необходимое правильных версий.

- **Автоконфигурация (Auto-configuration):** Spring Boot работает по принципу «Конвенция выше конфигурации» (**Convention over Configuration**). Он сканирует твой classpath (подключенные библиотеки). Если он видит в зависимостях драйвер PostgreSQL, он сам автоматически создает и настраивает бин подключения к базе данных с дефолтными настройками. Тебе достаточно просто прописать урл и пароль в файле application.properties. Кроме того, Spring Boot поставляется со **встроенным веб-сервером (Tomcat, Jetty или Undertow)**, то есть твоё приложение собирается в обычный jar-файл и запускается одной командой, без необходимости разворачивать внешний Tomcat.

Spring принципы

Архитектура Spring держится на нескольких фундаментальных принципах, которые позволяют делать код гибким, расширяемым и легко тестируемым:

14. **Inversion of Control (IoC) — Инверсия управления:** Передача контроля над созданием объектов и их жизненным циклом от твоего кода к стороннему каркасу (в данном случае — к Spring Container).
15. **Dependency Injection (DI) — Внедрение зависимостей:** Частный случай IoC, паттерн, при котором объекты не создают свои зависимости сами через new, а получают их извне в готовом виде.
16. **Aspect-Oriented Programming (AOP) — Аспектно-ориентированное программирование:** Подход, позволяющий выделять «сквозную логику» (cross-cutting concerns) приложения в отдельные модули (аспекты). Сквозная логика — это то, что нужно во многих местах программы, но не относится к прямой бизнес-логике. Например: логирование, аутентификация, кэширование, управление транзакциями. Вместо того чтобы писать session.beginTransaction() и commit() в каждом сервисном методе, мы просто вешаем аннотацию @Transactional, и Spring с помощью прокси автоматически выполняет этот код вокруг нашего метода.
17. **Абстракция над технологиями (Data Access/Integration):** Spring предоставляет удобные обертки над сложными низкоуровневыми API. Вместо того чтобы писать тонны шаблонного кода JDBC с ручным открытием/закрытием соединений и обработкой SQLException, Spring дает JdbcTemplate или Spring Data Repositories, беря всю рутину на себя.

Что такое Bean

Бин (Bean) — это обычный Java-объект, который создается, настраивается, инициализируется и управляется **Spring IoC Контейнером**.

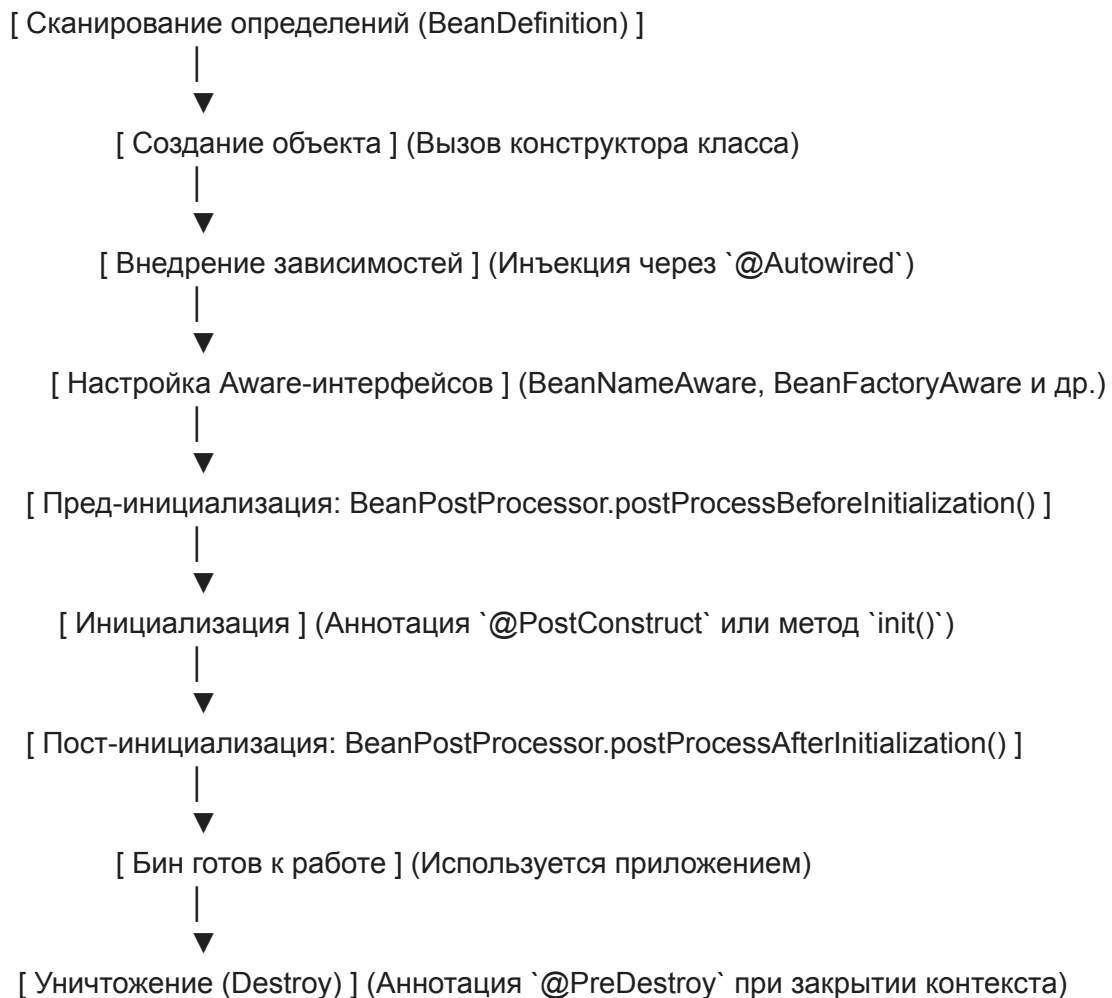
Важно понимать разницу: если ты в коде написал User user = new User();, то этот объект user живет в куче, но он **НЕ является бином**, потому что Spring про него ничего не знает. Но если ты повесил над классом аннотацию @Component или создал метод с аннотацией @Bean внутри конфигурационного класса, Spring берет этот класс на карандаш. Он сам

вызовет его конструктор, сам засунет его в свой внутренний реестр (ApplicationContext), сам внедрит в него нужные зависимости и уничтожит его, когда приложение закроется. По умолчанию все бины в Spring создаются в единственном экземпляре (**Scope = Singleton**).

Жизненный цикл Bean

Это один из самых любимых вопросов на глубокое понимание Спринга. Жизненный цикл бина — это последовательность шагов, которые объект проходит от момента, когда Spring только прочитал его конфигурацию, до его полного уничтожения.

Вот как выглядит эта цепочка этапов:



Детальный разбор шагов:

18. **Загрузка BeanDefinition:** Spring сканирует проект, находит классы с аннотациями

@Component или @Bean и создает для каждого карту-паспорт — BeanDefinition, где записано, как этот бин создавать, какие у него зависимости и какой scope.

19. **Инстанцирование (Instantiating):** Контейнер создает физический объект в памяти кучи, вызывая конструктор класса.
20. **Популяция свойств (Populate properties / Внедрение зависимостей):** Spring ищет поля или сеттеры, помеченные @Autowired, находит нужные другие бины в контексте и внедряет их в только что созданный объект.
21. **Вызов Aware-интерфейсов:** Если бину нужно знать служебную информацию о самом контейнере, он может реализовать интерфейсы семейства Aware. Spring на этом этапе внедрит в него его собственное имя бина (BeanNameAware), ссылку на фабрику бинов (BeanFactoryAware) или контекст (ApplicationContextAware).
22. **BeanPostProcessor (Before Initialization):** Spring передает бин во все зарегистрированные обработчики BeanPostProcessor, вызывая у них метод postProcessBeforeInitialization(). Здесь объект можно подменить или донастроить.
23. **Инициализация (Initialization):** Выполняется пользовательская логика настройки, когда все зависимости уже точно на месте. Сначала обрабатывает метод, помеченный аннотацией @PostConstruct, затем метод afterPropertiesSet() (если реализован интерфейс InitializingBean), и в конце — кастомный init-метод, если он был указан.
24. **BeanPostProcessor (After Initialization):** Вызывается метод postProcessAfterInitialization(). Это критически важный этап: именно здесь Spring оборачивает твой обычный бин в **AOP-прокси**, если над бином или его методами висят аннотации @Transactional, @Async или @Secured. В контекст приложения в итоге улетает не твой чистый объект, а этот прокси-объект.
25. **Бин готов к использованию:** Он живет в контексте и выполняет бизнес-логику.
26. **Уничтожение (Destruction):** Когда приложение завершает работу и контекст закрывается, у бинов вызываются методы уничтожения: сначала метод с аннотацией @PreDestroy, затем destroy() (интерфейс DisposableBean), и кастомный destroy-метод.

Dependency Injection & Inversion of Control

Эти понятия часто путают, но они соотносятся как общее и частное.

Inversion of Control (IoC — Инверсия управления) — это глобальная архитектурная концепция, абстрактный принцип проектирования программ. В традиционном программировании твой код полностью контролирует поток выполнения: сам решает, когда создать объект класса OrderService, сам создает внутри него PostgreSQLRepository через new. При IoC этот контроль выворачивается наизнанку. Твой код больше не рулит процессом создания — он становится пассивным, а управление передается внешнему фреймворку (контейнеру Спринга). Фреймворк сам решает, когда вызвать твой код, когда создать объекты и как связать их друг с другом. IoC можно реализовать разными путями: через паттерн Service Locator, через генерацию событий или через DI.

Dependency Injection (DI — Внедрение зависимостей) — это конкретный *паттерн проектирования*, с помощью которого реализуется принцип IoC. Смысл паттерна прост:

объект не должен сам заниматься поиском, созданием и сборкой своих зависимостей. Зависимости должны передаваться («внедряться») в него извне в готовом виде.

Пример «на пальцах»: Твоему классу Car для работы нужен движок Engine. Без DI класс Car внутри своего конструктора пишет `this.engine = new V8Engine();` — это жесткая связь (tight coupling), ты не сможешь легко подменить движок на электромотор для тестирования. При DI класс Car декларирует в конструкторе: `public Car(Engine engine) { this.engine = engine; }`. Ему абсолютно плевать, какой именно движок ему прилетит и как он был создан, его задача — просто ехать. А сборкой занимается Spring.

Как Spring внедряет объекты

Spring может внедрить зависимости в твои бины тремя основными способами через аннотацию `@Autowired`:

18. Внедрение через конструктор (Constructor Injection):

```
private final OrderRepository repository;
@Autowired // Начиная с Spring 4.3, если конструктор один, аннотацию можно опускать
public OrderService() {
    this.repository = repository;
}
```

Это самый рекомендуемый и правильный способ. Почему? Во-первых, он позволяет делать поля `final`, что гарантирует их неизменяемость и потокобезопасность. Во-вторых, объект невозможно создать в некорректном состоянии (без зависимостей) — компилятор просто не разрешит написать `new OrderService()`, не передав туда репозиторий. В-третьих, такой код легко тестировать в обычных unit-тестах без поднятия тяжелого Spring-контекста (можно просто передать mock-объект вручную в конструктор).

19. Внедрение через сеттер (Setter Injection):

```
private OrderRepository repository;
@Autowired
public void setRepository() {
    this.repository = repository;
}
```

Используется редко, в основном для необязательных зависимостей, которые можно настроить или поменять в процессе работы приложения, или для разрешения проблемы циклических зависимостей.

20. Внедрение через поле (Field Injection):

```
@Autowired
private OrderRepository repository;
```

Самый ленивый и популярный среди новичков способ, но **на практике это антипаттерн**. Почему его не любят: поля нельзя сделать final. Нарушается чистый ООП-дизайн (ты не можешь инициализировать класс вне Spring-контекста без использования рефлексии). Такой класс жестко привязывается к IoC-контейнеру, его сложно тестировать.

Bean Post Processor

BeanPostProcessor (BPP) — это специальный интерфейс-инструмент расширения Spring, который позволяет вмешиваться в процесс инициализации бинов фабрикой Спринга и кастомизировать их.

Он содержит два метода: `postProcessBeforeInitialization` и `postProcessAfterInitialization`. Spring находит все бины, реализующие интерфейс `BeanPostProcessor`, и прогоняет абсолютно каждый создаваемый бин приложения через эти два метода (как мы видели на схеме жизненного цикла).

- **Зачем он нужен:** Это фабрика магии Спринга. Именно через BPP работают такие аннотации, как `@Autowired` (класс `AutowiredAnnotationBeanPostProcessor` ищет аннотации и внедряет зависимости), `@Value`, а также механизмы создания AOP-прокси для транзакций и безопасности. Ты можешь написать свой BPP, например, чтобы автоматически логировать время работы методов всех бинов, помеченных твоей кастомной аннотацией.

SOLID

Пять фундаментальных принципов объектно-ориентированного проектирования, которые делают код поддерживаемым при расширении:

- **S — Single Responsibility Principle (Принцип единственной ответственности):** У каждого класса должен быть **ровно один повод для изменения**. Это значит, класс должен решать ровно одну задачу. Если класс `Invoice` занимается расчетом стоимости заказа, генерирует PDF-отчет и отправляет его по email — это нарушение. Если изменится формат email-писем или дизайн PDF, тебе придется переписывать один и тот же класс, рискуя сломать расчет стоимости. Нужно разделить его на три отдельных класса: `InvoiceCalculator`, `InvoicePdfGenerator` и `EmailSender`.
- **O — Open/Closed Principle (Принцип открытости/закрытости):** Программные сущности (классы, модули) должны быть **открыты для расширения, но закрыты для модификации**. Ты должен иметь возможность добавлять новое поведение в систему, не переписывая уже существующий и протестированный код. Достигается это через интерфейсы и полиморфизм. Если у тебя есть класс `PaymentProcessor`, который внутри через `if-else` проверяет типы оплат (`if (type == CREDIT_CARD) ... else if (type == PAYPAL)`), то появление нового типа оплаты (например, `CRYPTO`) заставит тебя вскрывать этот класс и менять его код. Правильно — сделать интерфейс `PaymentMethod` и плодить новые классы-реализации, не трогая основной процессор.
- **L — Liskov Substitution Principle (Принцип подстановки Барбары Лисков):** Объекты в программе должны быть заменяемы на объекты их подклассов **без изменения правильности работы программы**. Наследник не должен ломать

поведение и контракты родительского класса. Сын не должен уметь делать то, чего не ожидали от отца. Классический пример нарушения: класс Ostrich (Страус) наследуется от класса Bird (Птица), у которого есть метод fly(). Но страусы не летают! Если метод бросит UnsupportedOperationException, то код, работающий с базовым типом Bird в цикле, упадет. Значит, иерархия спроектирована неверно.

- **I — Interface Segregation Principle (Принцип разделения интерфейса):** Клиенты не должны зависеть от методов, которые они не используют. **Много специализированных интерфейсов лучше, чем один универсальный «толстый».** Если у тебя есть интерфейс SmartDevice с методами print(), fax(), scan(), а ты пишешь класс простой офисной «мигалки» SimplePrinter, тебе придется реализовывать методы fax() и scan() пустыми или бросать исключения. Правильно — разделить интерфейс на Printer, Scanner и FaxMachine.
- **D — Dependency Inversion Principle (Принцип инверсии зависимостей):** Модули верхнего уровня не должны зависеть от модулей нижнего уровня. Оба должны зависеть от абстракций. Кроме того, **абстракции не должны зависеть от деталей, а детали должны зависеть от абстракций.** Это буквально философское обоснование паттерна Dependency Injection. Твой высокоуровневый UserService не должен напрямую объявлять и зависеть от низкоуровневого конкретного класса MySQLRepository. Они оба должны коммуницировать через общий интерфейс UserRepository. Это позволит тебе в любой момент заменить базу данных MySQL на Mongo, не меняя ни одной строчки бизнес-логики в UserService.

Базы данных (БД)

Реляционные vs нереляционные БД

- **Реляционные БД (SQL):** Хранят данные в виде строгих, четко структурированных двухмерных **таблиц**, связанных друг с другом внешними ключами (отношениями/relations). У них жесткая фиксированная схема (сначала создаешь таблицу и объявляешь типы колонок, и только потом можешь писать данные). Они гарантируют строгое соблюдение транзакционных правил ACID.
 - *Примеры:* PostgreSQL, MySQL, Oracle.
- **Нереляционные БД (NoSQL):** Хранят данные в гибких форматах без жестких табличных связей. Бывают разных типов: документные (хранят JSON/BSON, например MongoDB), ключ-значение (Redis), графовые (Neo4j) и столбцовые (Cassandra). У них нет жесткой схемы данных (в одну и ту же коллекцию MongoDB можно положить один документ с 3 полями, а второй — с 20 полями). Они хуже поддерживают сложные связи, но отлично горизонтально масштабируются на много серверов.

ACID principles

Четыре золотых правила, выполнение которых гарантирует надежность и предсказуемость транзакций в базах данных:

- **A — Atomicity (Атомарность):** Транзакция выполняется по принципу **«всё или**

ничего». Она не может быть выполнена наполовину. Если в транзакции банковского перевода первый шаг (снять деньги со счета А) прошел успешно, а второй шаг (зачислить на счет Б) оборвался из-за сбоя питания, вся транзакция полностью откатывается (rollback), и база возвращается в исходное состояние, будто ничего и не было.

- **C — Consistency (Согласованность):** Транзакция переводит базу данных из одного согласованного состояния в другое согласованное состояние. Все правила, ограничения (constraints, уникальные ключи, foreign keys) должны соблюдаться. База не позволит записать некорректные данные, нарушающие бизнес-правила структуры.
- **I — Isolation (Изолированность):** Параллельно выполняющиеся транзакции не должны влиять друг на друга. Результат работы транзакции не должен быть виден другим транзакциям до тех пор, пока она не завершена (commit). Подробнее уровни изоляции разберем ниже.
- **D — Durability (Долговечность):** Если транзакция была успешно завершена (commit), изменения зафиксированы в базе **намертво**. Никакой внезапный сбой, выключение сервера или краш операционной системы не могут стереть или откатить эти данные назад — они уже физически записаны на энергонезависимый диск (или в WAL-лог).

Transaction Isolation Levels

Изолированность (Isolation) в реальном мире — штука дорогая. Чтобы обеспечить абсолютную изоляцию, базе пришлось бы выполнять транзакции строго по очереди в один поток, что убило бы производительность. Поэтому в стандарте SQL ввели 4 уровня изоляции. Чем выше уровень, тем выше надежность, но тем медленнее работает база из-за обилия блокировок.

Сначала разберем три феномена (проблемы), которые могут возникать:

24. **Dirty Read (Грязное чтение):** Транзакция А читает данные, которые транзакция Б изменила, но **еще не зафиксировала (не сделала commit)**. Если транзакция Б в итоге сделает rollback, окажется, что транзакция А прочитала фейковые данные.
25. **Non-repeatable Read (Неповторяющееся чтение):** Транзакция А внутри себя читает одну и ту же строчку два раза. В промежутке между этими чтениями транзакция Б **изменяет** эту строчку и делает commit. В итоге транзакция А получает два разных результата для одной строки.
26. **Phantom Read (Фантомное чтение):** Транзакция А делает выборку строк по какому-то условию (например, WHERE age > 18). В этот момент транзакция Б **вставляет новые строки** (или удаляет), подходящие под это условие, и делает commit. Транзакция А делает повторный запрос и видит новые «строки-фантомы», которых в первый раз не было.

Таблица уровней изоляции и проблем, которые они решают:

Уровень	Грязное чтение	Неповторяющееся	Фантомное
---------	----------------	-----------------	-----------

изоляция		я чтение	чтение
Read Uncommitted (Низший)	Да (Возможно)	Да	Да
Read Committed (Дефолт в Postgres)	Нет (Защищает)	Да	Да
Repeatable Read (Дефолт в MySQL)	Нет	Нет	Да (В Postgres нет благодаря MVCC)
Serializable (Высший)	Нет	Нет	Нет

Виды JOIN

Оператор JOIN используется для объединения строк из двух или более таблиц на основе общего условия (обычно связи по ключам).

- **INNER JOIN (Внутреннее объединение):** Возвращает только те строки, для которых **есть совпадения в обеих таблицах**. Если строке из Таблицы А нет пары в Таблице Б, она полностью выбрасывается из финального результата.
- **LEFT JOIN (или LEFT OUTER JOIN):** Возвращает **абсолютно все строки из левой таблицы**, плюс соответствующие им строки из правой таблицы. Если для строки из левой таблицы в правой ничего не нашлось, на месте колонок правой таблицы будут стоять значения NULL.
- **RIGHT JOIN:** Зеркальная противоположность LEFT JOIN. Возвращает все строки из правой таблицы, а для левой подставляет NULL, если совпадений нет. На практике используется редко, так как любой RIGHT JOIN можно переписать как LEFT JOIN, просто поменяв таблицы местами.
- **FULL JOIN (или FULL OUTER JOIN):** Возвращает строки из обеих таблиц, когда есть совпадения, плюс объединяет несовпавшие строки с обеих сторон, заполняя пустые места значениями NULL.

WHERE vs HAVING

Оба оператора используются для фильтрации данных, но они работают на принципиально разных этапах выполнения SQL-запроса:

22. **WHERE:** Выполняет фильтрацию **ДО того, как данные будут сгруппированы** и агрегированы. Он проверяет каждую конкретную строку таблицы. Внутри WHERE

категорически запрещено использовать агрегатные функции (такие как SUM(), AVG(), COUNT()), потому что в момент его работы никаких сумм и подсчетов еще не произведено.

23. **HAVING:** Выполняет фильтрацию **ПОСЛЕ того, как данные были сгруппированы** оператором GROUP BY. Он фильтрует уже сгруппированные строки-результаты. Вот внутри него как раз и используются агрегатные функции.
24. *Пример:* SELECT department, COUNT(*) FROM employees WHERE salary > 50000 GROUP BY department HAVING COUNT(*) > 5; — сначала WHERE отсекает всех нищих сотрудников, потом база сгруппирует оставшихся по отделам, а потом HAVING отсекает только те отделы, где после фильтрации осталось больше 5 человек.

Индексы

Индекс в базе данных — это специальная вспомогательная структура данных (грубо говоря, упорядоченный справочник), создаваемая поверх таблицы для **колоссального ускорения поиска строк**.

Аналогия из жизни: индекс — это алфавитный предметный указатель в самом конце толстой бумажной энциклопедии. Без указателя, чтобы найти информацию про «Джаву», тебе пришлось бы листать всю книгу с первой до последней страницы (в БД это называется Full Table Scan, сложность $O(n)$). С индексом ты открываешь букву «Д», видишь точный номер страницы, и сразу прыгаешь туда (сложность падает до $O(\log n)$).

Минусы индексов: Они занимают физическое место на диске и **замедляют операции записи (INSERT, UPDATE, DELETE)**, потому что при любом изменении данных в таблице база обязана тратить ресурсы на перестройку и обновление самого индекса.

Clustered vs non-clustered индексы

- **Кластеризованный индекс (Clustered Index):** Определяет физический порядок хранения самих строк таблицы на жестком диске. Листовые узлы кластеризованного индекса **содержат реальные данные строк**. Из-за этого у таблицы может быть **только один** кластеризованный индекс (нельзя физически разложить строки на диске одновременно и по алфавиту имен, и по возрастанию возраста). Обычно в качестве кластеризованного индекса база автоматически выбирает Первичный Ключ (PRIMARY KEY).
- **Некластеризованный индекс (Non-clustered Index):** Живет отдельно от таблицы. Его листовые узлы содержат не сами данные строк, а **ссылки (указатели)** на место, где эти строки физически лежат (например, значение кластеризованного ключа или физический адрес строки RID). Таких индексов у таблицы может быть сколько угодно (по полю email, по полю phone и т.д.). Поиск по такому индексу идет в два этапа: сначала база находит указатель в индексе, а потом по этому указателю прыгает в саму таблицу за остальными полями строки.

Hash vs B-tree индексы

Это две разные математические структуры под капотом индексов:

12. **B-tree индекс (Сбалансированное дерево):** Структура по умолчанию в большинстве баз. Она хранит данные в виде упорядоченного дерева. Позволяет искать данные как по точному совпадению (`WHERE id = 5`), так и по **диапазонам** (`WHERE age BETWEEN 20 AND 30`), а также выполнять сортировку и поиск по маске (`LIKE 'Abc%'`). Сложность поиска всегда стабильна: $O(\log n)$.
13. **Hash индекс:** Строится на основе хэш-таблицы (аналог `HashMap`). Хранит хэш-коды значений колонок.
 - **Плюс:** Поиск по точному равенству (`WHERE email = 'test@test.com'`) работает невероятно быстро, практически за константное время $O(1)$.
 - **Минус:** Он абсолютно **бесполезен для поиска по диапазонам** (`WHERE price > 100`) или для сортировки, потому что хэш-функция перемешивает данные хаотично, и соседние по значению элементы будут лежать в совершенно разных местах индекса. Также он не помогает при поиске по `LIKE`.

Оптимистичная vs пессимистичная блокировка

Когда два пользователя одновременно пытаются изменить одну и ту же строку в базе данных (например, купить последний билет на концерт), возникает конфликт. Его решают двумя стратегиями блокировок:

- **Пессимистичная блокировка (Pessimistic Locking):** Подход «кругом враги, меня точно захотят обойти». Поток, который первым зашел читать строку с целью изменения, намертво блокирует её на уровне базы данных (например, через конструкцию `SQL SELECT ... FOR UPDATE`). Пока этот поток не завершит свою транзакцию, никто другой не может эту строку даже прочитать, все стоят в очереди и ждут.
 - **Плюсы:** 100% гарантия целостности данных, исключены конфликты при записи.
 - **Минусы:** Ужасно падает пропускная способность системы, легко получить `Deadlock`.
- **Оптимистичная блокировка (Optimistic Locking):** Подход «все люди братья, конфликты маловероятны». Никаких реальных блокировок на уровне базы не ставится. Потоки спокойно читают данные одновременно. Но в таблицу добавляется специальная служебная колонка — **version** (или таймстемп).

Когда Поток 1 хочет сохранить изменения, он делает SQL-запрос вида: `UPDATE tickets SET seat = 'A1', version = version + 1 WHERE id = 1 AND version = 5;`

Если за время его размышлений Поток 2 успел влезть и обновить эту строку, версия в базе уже стала 6. Запрос Потока 1 вернет 0 обновленных строк. Spring заметит это и выбросит `OptimisticLockingFailureException`. Приложению придется перехватить ошибку и сказать пользователю: «Упс, данные обновились, попробуйте еще раз».

 - **Плюсы:** Работает реактивно быстро, нет простоев потоков и зависших транзакций.
 - **Минусы:** Если конфликтов реально много, пользователям придется постоянно переотправлять запросы, что будет их бесить.

Если надо сделать много изменений — какую блокировку выбрать

Если тебе нужно в рамках пакетной обработки (batch processing) или тяжелой бизнес-логики внести **очень много изменений** в данные, где высок риск того, что параллельные потоки будут постоянно сталкиваться лбами, нужно выбирать

Пессимистичную блокировку.

Если ты выберешь оптимистичную, то при высоком параллелизме (high concurrency) твои транзакции будут постоянно падать по ошибке несовпадения версий (OptimisticLockingFailureException), тебе придется бесконечно крутить их в циклах повторения (retry), система захлебнется в холостой работе, и в итоге наступит деградация производительности. Пессимистичная блокировка один раз закроет нужный диапазон данных, спокойно выполнит весь пакет обновлений за один проход и откроет доступ остальным.

SQL vs NoSQL

- **SQL (Реляционные):** Выбирай, когда у тебя данные имеют четкую, понятную структуру, которая редко меняется. Когда критически важны жесткие связи между сущностями (например, заказы, пользователи, платежи). Когда бизнес-логика завязана на сложные аналитические запросы с множеством JOIN'ов. И главное — когда тебе нужна железная транзакционность и соответствие правилам **ACID** (финансовые системы, банковские транзакции).
- **NoSQL (Нереляционные):** Выбирай, когда структура данных динамична, непредсказуема или часто меняется (например, карточки товаров на маркетплейсе, где у одежды одни характеристики, а у электроники — совершенно другие). Когда данных становится экстремально много (Big Data, логи, чаты, лента соцсетей), и их нужно легко масштабировать горизонтально на сотни серверов (шардирование). NoSQL жертвует строгим ACID ради высокой доступности и скорости (концепция **BASE** и CAP-теорема).

EXPLAIN vs EXPLAIN ANALYZE

Это главные инструменты разработчика для оптимизации медленных SQL-запросов. Ты пишешь эти слова прямо перед своим запросом SELECT.

- **EXPLAIN:** Заставляет базу данных показать **план выполнения запроса**, построенный её внутренним планировщиком-оптимизатором. При этом сам запрос физически **НЕ выполняется**. База просто выдает теоретический сухой расчет: какие индексы она планирует использовать, в каком порядке будет джоинить таблицы, сколько примерно строк (rows) ожидает перебрать и какова гипотетическая стоимость (cost) операции.
- **EXPLAIN ANALYZE:** База данных не просто строит план, она **физически выполняет твой запрос** на реальных данных. Она замеряет реальное время работы каждого шага на конвейере, считает реальное количество перебранных

строк и выводит отчет, где сопоставляет теоретические ожидания планировщика с суровой практикой рантайма.

- *Опасность:* Будь предельно осторожен при запуске EXPLAIN ANALYZE для запросов типа DELETE или UPDATE на прод-базе, потому что этот инструмент реально изменит или удалит данные в процессе замера!

Нормализация

Нормализация — это пошаговый процесс организации данных в базе, проектирования её схем таблиц с целью **устранения избыточности (дублирования данных)** и исключения риска возникновения аномалий модификации (когда при изменении одной строки ломается логика в другой). Процесс состоит из перехода от низших нормальных форм к высшим:

15. **Первая нормальная форма (1NF):** Требуется, чтобы все поля таблицы были **атомарными** (неделимыми), и в одной ячейке не хранился список значений через запятую. Также у таблицы должен быть первичный ключ.
16. **Вторая нормальная форма (2NF):** Должна выполняться 1NF, плюс все неключевые поля должны зависеть от *всего* первичного ключа целиком, а не от его части (актуально для составных ключей).
17. **Третья нормальная форма (3NF):** Должна выполняться 2NF, плюс в таблице должно отсутствовать транзитивное отношение полей (когда неключевое поле зависит от другого неключевого поля, которое в свою очередь зависит от ключа).
Пример нарушения 3NF: Таблица сотрудников хранит поля id, name, department_id, department_name. Поле department_name зависит от department_id, а не напрямую от id сотрудника. При изменении названия отдела придется менять его во всех строках сотрудников. Правильно — вынести отделы в свою таблицу.

View (материализованные vs нематериализованные)

Представление (View) — это виртуальная таблица, представляющая собой сохраненный в базе SQL-запрос. Она нужна для удобства, безопасности (можно скрыть сложные джоины от аналитика) и абстракции.

10. **Обычное (Нематериализованное) View:** Не хранит физически никаких данных на диске. Это просто «сохраненная ссылка» на SQL-запрос. Когда ты пишешь SELECT * FROM my_view;, база под капотом разворачивает этот сохраненный запрос, джоинит реальные таблицы и выполняет вычисления прямо на лету. Данные всегда на 100% актуальные, но если запрос сложный, View будет работать медленно.
11. **Материализованное представление (Materialized View):** Физически **выполняет запрос и сохраняет его результат на диск** в виде реальной таблицы данных.
 - *Плюс:* Потрясающая скорость чтения. Базе не нужно каждый раз делать тяжелые джоины, она просто читает готовый кэшированный слепок данных.
 - *Минус:* Данные в материализованном вью быстро устаревают (тухнут), когда меняются исходные таблицы. Его нужно постоянно обновлять по расписанию или триггерами (REFRESH MATERIALIZED VIEW).

Команда UNION

Команда UNION используется для **вертикального объединения результатов** двух или более SQL-запросов в единый результирующий набор.

[Запрос 1: Строка А, Строка Б]
+
[Запрос 2: Строка Б, Строка В]
|
▼ (UNION убирает дубликат "Строка Б")
[Итог: Строка А, Строка Б, Строка В]

Важные правила: Количество колонок и их типы данных в первом и втором запросах должны строго совпадать.

Разница с UNION ALL: Обычный UNION производит операцию DISTINCT — он ищет дубликаты строк между выборками и **удаляет их**, тратя на это ресурсы процессора. UNION ALL просто грубо склеивает результаты один под другим, оставляя все дубликаты как есть. UNION ALL работает значительно быстрее, поэтому если ты точно знаешь, что данные не пересекаются, используй именно его.

Git

Merge vs Rebase

Два способа объединить изменения из одной ветки (например, фича-ветки feature) в другую (например, основную main).

- **Merge (Слияние):** Создает в истории специальный служебный **коммит слияния (merge commit)**, у которого два родителя. Он берет верхушку твоей ветки, верхушку main, находит их общего предка, объединяет код и делает новый коммит.
 - *Плюсы:* История остается абсолютно правдивой, сохраняется хронология, не меняются прошлые коммиты (безопасно).
 - *Минусы:* Если разработчиков много, история превращается в жуткий нечитаемый клубок «железных дорог» из переплетающихся линий веток.
- **Rebase (Перебазирование):** Берёт коммиты из твоей фича-ветки, временно откладывает их в сторону, находит точку разделения веток, обновляет твою ветку до самого свежего состояния ветки main, а затем **поочередно накатывает твои коммиты поверх** свежего main. Происходит переписывание истории: у твоих коммитов физически меняются хэши, авторы и даты, они как бы перерождаются.
 - *Плюсы:* История становится идеально **линейной** и красивой, без мусорных

коммитов слияния, её легко читать.

- **Минусы:** Категорически запрещено делать **Rebase** для публичных веток, которые уже улетели на удаленный сервер (Origin) и с которыми работают другие люди. Ты перепишешь историю, у них сломаются хэши коммитов, и наступит тотальный хаос.

Revert vs Reset

Два способа отменить плохие изменения (ошибочные коммиты).

- **Git Reset:** Работает как «машина времени». Он физически **перемещает указатель текущей ветки назад** на указанный коммит, стирая последующую историю. Имеет три флага:
 - **--soft:** Перемещает указатель, но оставляет все изменения из удаленных коммитов в твоей рабочей зоне (Staging area/Index), готовыми к новому коммиту.
 - **--mixed** (дефолтный): Перемещает указатель, сбрасывает индекс, но оставляет файлы измененными на диске.
 - **--hard:** Жестко стирает всё. Файлы на диске возвращаются в состояние того старого коммита. Все последующие изменения уничтожаются безвозвратно. *Правило безопасности:* Используй Reset только локально, для коммитов, которые еще не ушли в удаленный репозиторий.
- **Git Revert:** Безопасный способ для публичной истории. Он **НЕ удаляет старые коммиты**. Вместо этого он вычисляет изменения ошибочного коммита и создает **новый, противоположный по смыслу коммит-выворотку**, который нейтрализует (отменяет) код старого коммита. История движется только вперед, никто из коллег не страдает от несовпадения хэшей.

Cherry-pick

Команда `git cherry-pick <commit-hash>` позволяет точно выдернуть **один конкретный коммит** из любой ветки репозитория и скопировать его (накатить его изменения) в твою текущую ветку. При этом создается новый коммит с уникальным хэшем, но с тем же автором и текстом сообщения.

Пример из жизни: Ты нашел критический баг на проде и быстро исправил его в ветке hotfix. Тебе не нужно мержить всю тяжелую ветку hotfix со всеми её потрохами в ветку разработки develop, тебе нужен только этот фикс. Ты берешь его хэш, переходишь в develop, делаешь cherry-pick, и баг закрыт.

Что такое коммит и ветка

- **Коммит (Commit):** Это атомарный снимок (snapshot) состояния твоего проекта (всех отслеживаемых файлов и папок) на конкретный момент времени. Под капотом коммит — это объект Git, который содержит хэш-ссылку на дерево файлов (tree), имя автора, дату, сообщение-комментарий и ссылку на хэш родительского (предыдущего) коммита, благодаря чему формируется неразрывная цепочка истории.

- **Ветка (Branch):** В Git ветка — это **всего лишь плавающий текстовый указатель** на конкретный коммит (файл в папке `.git/refs/heads/`, внутри которого записан 40-символьный хэш коммита). Она весит пару байт. Когда ты делаешь новый коммит, указатель текущей ветки автоматически сдвигается вперед, на этот свежий коммит.

Как перенести коммиты с одной ветки на другую

В зависимости от задачи есть три способа:

- Если нужно перенести **цепочку коммитов оптом**, сохранив чистую линейную историю — используй **git rebase**.
- Если нужно перенести **один или два конкретных коммита** выборочно — используй **git cherry-pick**.
- Если нужно объединить ветки стандартным путем с сохранением факта слияния — используй **git merge**.

Fetch vs Pull

12. **Git Fetch:** Скачивает все свежие коммиты, ветки и теги из удаленного репозитория (Origin) в твой локальный репозиторий, но **НЕ трогает твой рабочий код на диске** и твою текущую ветку. Он просто обновляет скрытые локальные копии удаленных веток (origin/main). Это безопасная ознакомительная операция: ты скачал изменения и можешь посмотреть, что там написали коллеги, через `git log origin/main`.
13. **Git Pull:** Это комбинированная команда. Она под капотом выполняет сначала **git fetch** (скачивает данные), а затем сразу же автоматически запускает **git merge** (пытается слить скачанные изменения удаленной ветки в твою текущую локальную ветку). Это может привести к возникновению конфликтов слияния прямо на твоём экране.

HTTP / Web

Методы HTTP, разница GET и POST

HTTP-методы определяют тип действия, которое мы хотим совершить над ресурсом на сервере.

14. **GET:** Запрашивает чтение/получение ресурса.
 - *Параметры:* Передаются открыто прямо внутри самой адресной строки URL (Query parameters: `?id=5&name=Ivan`). Из-за этого объем данных жестко ограничен лимитами длины URL в браузерах.
 - *Свойства:* Он **Идемпотентный** (повторные одинаковые запросы вернут один и тот же результат и не изменят состояние системы) и **Безопасный** (ничего не создает и не ломает на сервере). Запросы GET кэшируются браузерами и сохраняются в истории.
15. **POST:** Используется для отправки данных на сервер с целью создания нового ресурса или обработки.
 - *Параметры:* Передаются внутри скрытого **тела запроса (Request Body)**. Ограничений на объем данных практически нет (можно слать тяжелые JSON или

файлы/картинки).

- **Свойства:** Он **НЕ идиempотентный** (каждый новый клик по кнопке «Оплатить заказ» с POST-запросом будет плодить новые заказы в базе) и **НЕ безопасный**. POST-запросы никогда не кэшируются по умолчанию.

JWT

JWT (JSON Web Token) — это открытый стандарт (RFC 7519) для безопасной передачи информации между сторонами в виде JSON-объекта. Чаще всего используется для бессессионной (Stateless) аутентификации пользователей.

Токен состоит из трех частей, разделенных точками: header.payload.signature

- **Header (Заголовок):** JSON, содержащий тип токена (JWT) и алгоритм шифрования/хэширования (например, HS256 или RS256). Кодирован в Base64Url.
- **Payload (Полезная нагрузка):** JSON, содержащий полезные данные пользователя (Claims) — например: user_id, username, его роли и права (roles: ["ADMIN"]), а также время истечения жизни токена (exp). Кодирован в Base64Url. *Важно:* Данные здесь не зашифрованы, их может прочитать любой желающий, просто раскодировав Base64, поэтому пароли туда класть нельзя!
- **Signature (Цифровая подпись):** Защита от подделки. Сервер берет закодированный заголовок, закодированную нагрузку, добавляет к ним свой **секретный ключ**, который знает только сам сервер, и хэширует всю эту смесь. Если злоумышленник попытается изменить роль в Payload с USER на ADMIN, подпись перестанет совпадать, и сервер с негодованием отвергнет такой токен.

Cookies — что хранится

Куки (Cookies) — это крошечные текстовые фрагменты данных, которые сервер присылает в ответе браузеру, а браузер сохраняет у себя на компьютере. При каждом последующем запросе к этому же сайту браузер автоматически прикрепляет эти куки в HTTP-заголовок Cookie, напоминая серверу, кто перед ним.

Что там обычно хранится:

16. **Идентификатор сессии (Session ID):** Самый частый кейс при классическом подходе Stateful. База хранит сессию, а у пользователя в куках лежит просто случайная строка-ключ JSESSIONID=xyz123.
17. **Настройки персонализации:** Выбранный язык интерфейса (RU/EN), выбранная тема оформления (темная/светлая).
18. **Данные отслеживания (Tracking):** Метрики, аналитика, ID пользователя для показа таргетированной рекламы (Google Analytics).
19. **Безопасность:** Для защиты кук от кражи хакерами через XSS-атаки на них вешают флаг HttpOnly — это запрещает доступ к кукам из JavaScript браузера.

MVC

MVC (Model-View-Controller) — классический архитектурный паттерн проектирования пользовательского интерфейса, разделяющий приложение на три независимых слоя:

- **Model (Модель):** Слой данных и чистой бизнес-логики. Она не знает, как данные будут выглядеть на экране. В Spring это твои сущности (`@Entity`), сервисы (`@Service`) и репозитории (`@Repository`). Она реагирует на запросы контроллера, меняет состояние и возвращает «голые» данные.
- **View (Представление / Вид):** Отвечает за визуализацию данных пользователю (UI). Это могут быть HTML-страницы, шаблоны Thymeleaf/Freemarker, или JSON-ответ в случае REST API. Она берет данные от Модели и превращает их в красивую картинку.
- **Controller (Контроллер):** Посредник, мозг системы. Он принимает входящие HTTP-запросы от пользователя (клиента), парсит параметры, решает, какой сервисный метод Модели вызвать для обработки, забирает у Модели результат и решает, в какое именно Представление (View) этот результат передать для рендеринга. В Spring за это отвечает `DispatcherServlet` и твои классы с аннотацией `@Controller/@RestController`.

Инструменты и коннекторы

Как в Java подключиться к БД

В мире Java эволюция подключения к базам данных выглядит так:

- **JDBC (Java Database Connectivity) — Базовый уровень:** Низкоуровневый стандарт JDK. Чтобы подключиться, тебе нужен драйвер конкретной базы (jar-ник). Код выглядит так: открываешь соединение через `DriverManager.getConnection(url, user, password)`, создаешь объект `Statement`, пишешь чистый SQL-запрос строкой, выполняешь его и в цикле перебираешь `ResultSet`, вручную вычитывая каждую колонку (`resultSet.getInt("id")`). Это тонны шаблонного кода и ручное управление ресурсами.
- **DataSource и Connection Pool:** В реальных приложениях открывать соединение на каждый запрос — непозволительная роскошь (это занимает сотни миллисекунд). Используют пулы соединений (например, **HikariCP**). Пул заранее открывает, например, 10 соединений и держит их наготове. Приложение через интерфейс `DataSource` берет соединение из пула «поносить», делает запрос и сразу возвращает обратно в пул.
- **ORM / Hibernate:** Высокоуровневая обертка. Позволяет вообще забыть про SQL-строки. Ты маппишь Java-класс на таблицу базы с помощью аннотаций `@Entity`, а Hibernate сам генерирует SQL-запросы под капотом, переводя объекты в строки таблиц и обратно.
- **Spring Data JPA:** Вершина эволюции. Обертка над Hibernate, которая позволяет просто объявить интерфейс `UserRepository extends JpaRepository<User, Long>` и писать методы типа `findByEmail(String email)` без единой строчки реализации — Spring сам сгенерирует нужный SQL-запрос.

Что за фреймворк Mockito

Mockito — это самый популярный и мощный фреймворк для создания **mock-объектов**

(заглушек) при написании модульных (Unit) тестов в Java.

Зачем он нужен: По правилам Unit-тестирования, ты должен тестировать свой класс UserService в полной изоляции от внешнего мира. Если твой UserService внутри себя вызывает метод userRepository.save() для записи в базу данных, ты не должен поднимать реальную базу данных (это долго и это уже интеграционный тест).

С помощью Mockito ты делаешь фокус: UserRepository repo =

Mockito.mock(UserRepository.class);. Mockito создает виртуальный объект-пустышку, прикидывающийся репозиторием. Затем с помощью конструкции

when(repo.findByEmail("test@test.com")).thenReturn(new User(...)); ты программируешь поведение заглушки (стаббинг). Теперь, когда твой сервис вызовет репозиторий, он мгновенно получит твой заготовленный фейковый объект, и ты сможешь изолированно протестировать именно логику самого сервиса. Также Mockito умеет проверять (verify), был ли вызван определенный метод у заглушки и сколько раз.

Мы разобрали абсолютно все темы из твоего списка до мельчайших деталей. Теперь у тебя в голове есть полная, структурированная картина того, как работает современный бэкэнд на Java и Spring. Помни про архитектурную логику, не зазубривай, а понимай причины — почему создатели технологий сделали именно так. Ты полностью готов к бою, уверенно забирай свой оффер!